

MVP — Machine Learning & Analytics

Nome: Leandro Miozzo Bonato

Matrícula: 4052025001550

Data: 03/06/2026

Dataset: https://github.com/leandrobonato/MVP_Machine_Learning_historico_precos_nasdaq - NASDAQ Financial Data Pipeline

Tipo de problema: Séries Temporais

✓ 1. Introdução

✓ Detalhes do projeto

Este projeto é um tipo de: **Séries Temporais**

✓ Objetivo

O projeto aborda a previsão de preços de ações no mercado financeiro, um desafio típico de séries temporais e regressão. No contexto do mercado de capitais, investidores, analistas e instituições financeiras precisam estimar a direção futura dos ativos para tomar decisões de compra, venda ou manutenção de posições. O modelo de machine learning deve apoiar a previsão do valor (ou tendência) de fechamento de ações listadas, com base em dados históricos de preço, volume e indicadores técnicos.

Os principais usuários seriam investidores pessoa física, analistas quantitativos, fundos de investimento e fintechs que buscam automatizar análises e gerar sinais de trading. A relevância está no alto impacto financeiro: previsões mais precisas podem aumentar a rentabilidade, reduzir riscos e democratizar o acesso a estratégias quantitativas, substituindo, em parte, análises puramente intuitivas.

✓ Hipóteses

1. Ensinar uma máquina a entender o funcionamento dos indicadores no mercado de ações.
2. Para ensinar a máquina serão executados vários modelos de Machine Learning e Deep Learning.
3. Com os treinamentos concluídos deverá ter uma conclusão se a máquina é capaz ou não de realizar a previsão de uma ação.
4. Caso não seja possível, então deverá ter uma conclusão do que deveria melhorar para tentar treinar novamente a máquina e conseguir chegar no resultado esperado.

✓ Critério de sucesso

O principal critério de sucesso é identificar se a máquina é capaz de prever um preço de uma empresa no mercado de ações.

Para isso será feito da seguinte forma:

1. (Versão 01) A máquina será treinada normalmente usando modelos de treinamento e os resultados disso serão armazenados para uso no dia seguinte.
2. (Versão 02) Na versão 01 isso não foi possível, então foram realizadas algumas melhorias, como por exemplo:
 - 2.1. Aumento de empresas analisadas
 - 2.2. Aumento de carga de dados.
 - 2.3. Aumento de features.
 - 2.4. Fundamentos das empresas.
3. (Versão 03) Mesmo com a versão 2, o projeto teve que ser ajustado novamente retreinado tudo novamente com as seguintes situações:
 - 3.1. Aumento de modelos para treinamento
 - 3.2. Adicionado novos comparativos entre os modelos para treinamento.
 - 3.3. Aumentada a carga de dados, abaixo detalhes.
 - 3.4. Adicionado commodities na nova carga de dados.

3.5. Adicionado dados macro-economicos na nova carga de dados.

3.6. Aumento de features

4. (Versão 04) Adicionada na versão 04 para melhorar o resultado, os seguintes modelos:

4.1. Modelo LSTM - Deep Learning

4.2. Modelo Prophet

▼ Observações

Observação 01

A taxa de acerto ou erro no mercado de ações nem sempre é com base nos indicadores, ou seja, mesmo a máquina ou humano realizar uma leitura perfeita dos indicadores não há como prever o que o mercado irá realizar no próximo pregão, sendo um imenso abismo para uma taxa de acerto perfeita, podendo tornar o estudo inviável, pois muitos fatores externos sustentam essa premissa.

Observação 02

Todas as versões terão junto as conclusões da versão 01, 02, 03 e 04 para comparativos.

Observação 03

O código terá apenas a última versão - nesse caso a 04.

A quem seria o usuário ou interessado nessa solução?

Usuários investidores no mercado de ações que tentam encontrar maneiras de prever o mercado financeiro.

Por que esse problema é relevante?

Caso isso seja possível (algum dia) ter uma taxa de acerto de 100% é impossível, mesmo para os melhores robôs, pois o mercado é extremamente volátil e isso poderia quebrar a economia mundial gerando uma crise sem precedentes.

Mas como o mercado é muito volátil, isso é matematicamente impossível, muitas vezes os indicadores e muitas informações nos dão indícios de que uma empresa irá subir apenas no longo prazo (podendo demorar anos para a premissa se concretizar).

Esse mercado de tentativa de previsibilidade financeira é muito concorrido e existem muitos robôs e empresas que investem pesado nisso. (Afinal, há muitos trilhões de dólares rolando por aí).

▼ Features

Para ensinar a máquina a analisar uma ação/cotação serão usados as seguintes informações:

- Preços históricos (Versão 01)
- Volume de negociação (Versão 01)
- Dividendos (Versão 01, melhorada na Versão 02)
- Indicadores técnicos (Versão 01)
 - RSI (Versão 01)
 - MACD (Versão 01)
 - Bollinger (Versão 01)
 - Fibonacci (Versão 01)
 - Médias móveis (Versão 01)
 - Volatilidade (Versão 01)
 - Tendência (Versão 01)
- Fundamentos (Versão 02)
- Preços de commodities (Versão 03)
- Preços de dados macro economicos (Versão 03)
- Como cada commodities afeta as empresas (Versão 03)

▼ Detalhes do Dataset

Dataset

- O dataset foi processado e gerado à partir da base de dados da NASDAQ - Bolsa de valores de New York, onde foram baixados todos os símbolos de empresas neste índice.
- Em seguida foi criado um algoritmo para consultar os históricos de preços de 2020 à 2026 (Versão 01) e 2015 à 2026 (Versão 02) **Observação abaixo sobre a carga de dados** com base nos símbolos consultados anteriormente na Nasdaq.
- Conforme os dados de símbolos que foram baixados, são mais de 5000, foi usado para consultar a cotação do dia desde 2016 até 03/06/2026 o Yahoo Finance, no python é: `yFinance`
- Com isso foi finalizado esses históricos e armazenado os arquivos CSV's e separados por símbolos no seguinte repositório público (próprio do autor): https://github.com/leandrobonato/MVP_Machine_Learning_historico_precos_nasdaq
- Neste repositório estão contidos os seguintes arquivos:
 - scripts para download dos símbolos na NASDAQ
 - scripts para download dos dados conforme os símbolos.
 - script para download dos indicadores para cada símbolo
 - script para download dados dos símbolos
 - script para download dividendos
 - script para download valores de mercado
 - script para teste de conexão com o Yahoo Finance
 - dados dos históricos de cotação diário de cada símbolo.
 - cada símbolo possui um arquivo CSV.
 - dentro da pasta histórico foi organizado em subpastas para cada tipo de informação: dividendos, preços e cotações, indicadores, valor de mercado.
 - A pasta indicadores possui uma subpasta para cada indicador: RSI, MACD, Bollinger, Fibonacci, Médias móveis, Volatilidade, Tendência.
- Para as versões 02 e 03, foram mantidas as informações acima e adicionadas as seguintes:
 - Fundamentos - ROI, ROA, P/VP, EBTIDA - **Observação abaixo**
 - Commodities (Versão 03).
 - Dados macro-economicos (Versão 03)
 - Carga de dados - 2000 à 2026 (Versão 02) - **Observação abaixo**
 - Carga de dados - 2015 à 2026 (Versão 03)
 - Commodities de como cada empresa é afetada por cada uma. (Versão 03)

Observação 01 - Carga de dados - 2000 à 2026.

Sobre o aumento de dados na Versão 02, onde foi feito de 2000 à 2026, teve que ser regredido em uma Versão 03 e diminuída essa carga, devido ao alto consumo de dados impossibilitando o treinamento no Colab, mesmo que com limpeza de memória progressiva em cada modelo.

Nessa versão os treinamentos com esses dados iniciou 11 horas da noite e foi até 08 horas da manhã do dia seguinte, onde gerou um erro de travamento no Google Colab (parei de acompanhar 04 horas da manhã).

Com isso, foi decidido fazer uma nova versão com menos dados, com pouco mais de 11 anos, mas aumentar algumas features - conforme detalhado anteriormente.

A versão 02 foi a mais ousada em contrapartida a pior de todas, pois será a única versão em que não haverá conclusões descritas, devido à esse problema do uso excessivo de memória, espaço em disco, GPU, CPU e entre outros.

Observação 02 - Fundamentos

As informações por empresa sobre os fundamentos deveria ser melhores, mas o Yahoo Finance não disponibiliza uma carga de dados muito boa, tendo apenas algumas informações mais recentes, e não por dia, mês ou ano, impossibilitando o uso dessa informação para os treinamentos.

Estrutura de pastas

```
historicos/
├── commodities                # Commodities por empresas - como isso afeta a empresa me questão
│   └── {SIMBOLO}.csv
├── cotacoes/                 # Preços OHLCV
│   └── {SIMBOLO}.csv
```

```
|--- dividendos/                                # Histórico de dividendos
|   |--- {SIMBOLO}.csv
|--- fundamentos/                             # Fundamentos econômicos para uma empresa, como: ROE, ROI, P/VP, EBTDA
|   |--- {SIMBOLO}.csv
|--- info_geral/                              # Informações gerais da empresa
|   |--- {SIMBOLO}.csv
|--- valor_mercado/                           # Market Cap histórico
|   |--- {SIMBOLO}.csv
|--- indicadores/
|   |--- rsi/                                # RSI (7 e 14 períodos)
|       |--- {SIMBOLO}.csv
|   |--- macd/                              # MACD + Sinais
|       |--- {SIMBOLO}.csv
|   |--- bollinger/                        # Bandas de Bollinger
|       |--- {SIMBOLO}.csv
|   |--- fibonacci/                       # Fibonacci Retracement
|       |--- {SIMBOLO}.csv
|   |--- medias_moveis/                   # Médias Móveis
|       |--- {SIMBOLO}.csv
|   |--- volatilidade/                   # Volatilidade, Retornos
|       |--- {SIMBOLO}.csv
|   |--- tendencia/                      # Tendências, Força
|       |--- {SIMBOLO}.csv
|--- macro_economicos/
|   |--- cambio/                          # Preço diário do cambio - BRLUSD e EURUSD
|       |--- {SIMBOLO}.csv
|   |--- commodities/                   # Preço diário de commodities
|       |--- {SIMBOLO}.csv
|   |--- indices/                       # Preço diário de índices de sentimentos do mercado financeiro
|       |--- {SIMBOLO}.csv
|   |--- taxas_juros/                   # Taxas de juros dos EUA de 3 meses
|       |--- {SIMBOLO}.csv
```

▼ Detalhes e dicionário

1. CAMPOS ORIGINAIS (OHLCV)

Campo	Tipo	Descrição	Uso
Date	Data	Data do pregão (YYYY-MM-DD)	
Open	Decimal(2)	Preço de abertura	
High	Decimal(2)	Preço máximo do dia	
Low	Decimal(2)	Preço mínimo do dia	
Close	Decimal(2)	Preço de fechamento	
Adj Close	Decimal(2)	Preço ajustado: corrigido para splits/dividendos	
Volume	Inteiro	Quantidade de ações negociadas	

2. DIVIDENDOS

Campo	Tipo	Descrição	Uso
Dividends	Decimal(4)	Valor do dividendo pago naquela data (por ação)	
Stock_Splits	Decimal(4)	Ratio de desdobramento (2.0 = split 2:1)	
Dividend_Yield	Decimal(2)	Rendimento anual de dividendos (%)	
Dividend_Rate	Decimal(2)	Valor anual estimado de dividendos por ação	
Payout_Ratio	Decimal(2)	% do lucro distribuído como dividendos	
Dividend_1Y_Total	Decimal(4)	Total de dividendos pagos nos últimos 12 meses	
Dividend_1Y_Count	Inteiro	Número de pagamentos no último ano	
Last_Dividend	Decimal(4)	Valor do último dividendo pago	
Ex_Dividend_Date	Data	Próxima data ex-dividendo	

3. INDICADORES DE VOLUME

Campo	Tipo	Descrição	Uso
Volume_Medio_20	Inteiro	Média de volume dos últimos 20 dias Volume acima da média = força no movimento	
Volume_Medio_50	Inteiro	Média de volume dos últimos 50 dias Confirmação de tendência de longo prazo	
Volume_Ratio	Decimal(2)	Razão Volume atual / Média 20 dias >1.5 = volume anormal (atenção)	

4. MÉDIAS MÓVEIS

Campo	Tipo	Descrição	Uso
MA_7	Decimal(2)	Média Móvel de 7 dias - Tendência de curtíssimo prazo	
MA_14	Decimal(2)	Média Móvel de 14 dias - Tendência de curto prazo	
MA_30	Decimal(2)	Média Móvel de 30 dias - Suporte/resistência mensal	
MA_50	Decimal(2)	Média Móvel de 50 dias - Tendência de médio prazo	
MA_200	Decimal(2)	Média Móvel de 200 dias - Tendência de longo prazo	
Sinal_MA_7_14	Inteiro	1=MA7>MA14 (alta), -1=MA7<MA14 (baixa) - Sinal de curto prazo	
Sinal_MA_50_200	Inteiro	1=Golden Cross, -1=Death Cross - Sinal de longo prazo	
Dist_MA_50	Decimal(2)	Distância % do preço para MA50 - Mede força da tendência	
Dist_MA_200	Decimal(2)	Distância % do preço para MA200 - Indica sobrecompra/sobre venda	

5. RSI (Índice de Força Relativa)

Campo	Tipo	Descrição	Uso
RSI_7	Decimal(2)	RSI de 7 períodos - Sinal rápido (mais sensível)	
RSI_14	Decimal(2)	RSI de 14 períodos (padrão) - >70=Sobrecomprado(vender) <30=Sobrevendido(comprar)	
RSI_Sinal	Texto	Sobrecomprado / Neutro / Sobrevendido - Gatilho para entrada/saída	

6. MACD

Campo	Tipo	Descrição	Uso
MACD_Line	Decimal(2)	Diferença entre EMAs (12-26) - Linha principal do indicador	
MACD_Signal	Decimal(2)	EMA de 9 períodos do MACD - Linha de sinal	
MACD_Histogram	Decimal(2)	MACD_Line - MACD_Signal - Positivo=cresce \ Negativo=diminui	
MACD_Sinal	Texto	Compra / Neutro / Venda - Compra=MACD cruza acima \ Venda=MACD cruza abaixo	

7. BANDAS DE BOLLINGER

Campo	Tipo	Descrição	Uso
BB_Upper	Decimal(2)	Banda superior (+2 desvios) Resistência dinâmica	
BB_Middle	Decimal(2)	Média móvel de 20 dias Linha central	
BB_Lower	Decimal(2)	Banda inferior (-2 desvios) Suporte dinâmico	
BB_Width	Decimal(2)	Largura das bandas (%) Baixo=consolidação \ Alto=volatilidade	
BB_Position	Decimal(2)	Posição 0-1 (0=banda inf, 1=banda sup) Localização do preço nas bandas	
BB_Sinal	Texto	Sobrecomprado / Neutro / Sobrevendido Preço na banda sup=vender \ inf=comprar	

8. FIBONACCI RETRACEMENT

Campo	Tipo	Descrição	Uso
Fib_0	Decimal(2)	Nível 0% (suporte/fundo) - Piso da tendência	
Fib_236	Decimal(2)	Nível 23.6% - Retração fraca	
Fib_382	Decimal(2)	Nível 38.2% - Suporte/resistência moderada	
Fib_50	Decimal(2)	Nível 50% - Ponto de equilíbrio	
Fib_618	Decimal(2)	Nível 61.8% (Golden Ratio) - Forte suporte/resistência	
Fib_786	Decimal(2)	Nível 78.6% - Retração profunda	
Fib_1	Decimal(2)	Nível 100% (resistência/topo) - Teto da tendência	
Fib_1_272	Decimal(2)	Extensão 127.2% - Projeção de alta	
Fib_1_618	Decimal(2)	Extensão 161.8% (Golden Ratio) - Alvo de projeção	
Fib_Position	Decimal(2)	Posição relativa (0-2) - 0=fundo, 1=topo, >1=rompimento	
Fib_Level_Current	Texto	Nível Fibonacci atual - Ex: "0.38", "0.62"	
Fib_Distance_Next	Decimal(2)	Distância % para próximo nível - Proximidade de suporte/resistência	
Fib_Zone	Texto	Suporte Forte/Suporte/Consolidação/Resistência/Resistência Forte/Rompimento - Zona de atuação do preço	
Fib_Buy_Signal	Inteiro	1=Sinal de compra - Preço tocou suporte 38.2% ou 61.8%	
Fib_Sell_Signal	Inteiro	1=Sinal de venda - Preço tocou resistência 78.6% ou 100%	
Fib_Signal_Strength	Inteiro	1=Compra, -1=Venda, 0=Neutro - Direção do sinal	
Fib_382_20d	Decimal(2)	Nível 38.2% (janela 20d) - Curto prazo	
Fib_618_20d	Decimal(2)	Nível 61.8% (janela 20d) - Curto prazo	
Fib_382_100d	Decimal(2)	Nível 38.2% (janela 100d) - Longo prazo	
Fib_618_100d	Decimal(2)	Nível 61.8% (janela 100d) - Longo prazo	
Fib_Confluencia	Inteiro	0-4 (quantos níveis convergem) - ≥2=Forte - ≥4=Muito Forte	
Fib_Forca_Sinal	Texto	Normal / Forte / Muito Forte - Confiabilidade do sinal Fibonacci	

9. RETORNOS E VOLATILIDADE

Campo	Tipo	Descrição	Uso
Returns	Decimal(4)	Retorno diário (%) - Variação percentual do dia	
Returns_Log	Decimal(4)	Retorno logarítmico - Para cálculos estatísticos	
Volatilidade_20	Decimal(2)	Volatilidade anualizada (20d) - Alta=risco elevado \ Baixa=estabilidade	

10. PADRÕES DE CANDLE

Campo	Tipo	Descrição	Uso
Range_Diario	Decimal(2)	Amplitude do dia (High - Low) - Tamanho do movimento	
Range_Percentual	Decimal(2)	Amplitude % do dia - Volatilidade intradiária	
Gap	Decimal(2)	Gap de abertura (Open - Close anterior) - Força da abertura	
Gap_Percentual	Decimal(2)	Gap % de abertura - Positivo=força compradora	
Forca_Movimento	Decimal(2)	(Close-Open)/(High-Low) - 1=fecha na máxima \ -1=fecha na mínima	
Sinal_Alta	Inteiro	1=Close>Open, 0=Close<Open - Dia de alta ou baixa	
Velas_Alta_Consecutivas	Inteiro	Contagem de velas de alta seguidas - ≥3=tendência de alta consolidada	

11. TENDÊNCIAS

Campo	Tipo	Descrição	Uso
Tendencia_7d	Texto	Alta/Baixa (Close vs MA7) Direção de curto prazo	
Tendencia_50d	Texto	Alta/Baixa (Close vs MA50) Direção de médio prazo	
Forca_Tendencia	Decimal(2)	Inclinação % da MA50 em 5 dias Positivo=acelerando \ Negativo=desacelerando	

12. VALOR DE MERCADO

Campo	Tipo	Descrição	Uso
Close	Decimal(2)	Preço de fechamento	
Shares_Outstanding	Decimal(2)	Ações em circulação na data	
Market_Cap	Decimal(2)	Valor de mercado (Close × Shares)	
Market_Cap_B	Decimal(2)	Valor de mercado em bilhões	
Enterprise_Value	Decimal(2)	Valor da firma	
Volume_Dollar	Decimal(2)	Volume financeiro negociado	
Market_Cap_Change	Decimal(2)	Variação do Market Cap (diária)	
Market_Cap_Change_Pct	Decimal(2)	Variação percentual do Market Cap	

13. COMMODITIES

Campo	Tipo	Descrição	Uso
commodity	String	Nome da commodity que influencia a empresa	Identifica qual commodity afeta o preço da ação
ticker_commodity	String	Ticker da commodity (ex: HG=F, GC=F, SI=F)	Referência para buscar dados da commodity
descricao	String	Descrição completa da commodity (ex: Copper Futures)	Contexto sobre a commodity
direcao	String	Direção da influência: 'positiva' ou 'negativa'	Indica se a ação sobe ou desce com a commodity
impacto	String	Descrição do impacto da commodity no preço da ação	Explica a relação de causa e efeito

14. FUNDAMENTOS

Campo	Tipo	Descrição	Uso
Margem_Liquida	Decimal(2)	Margem líquida (%) - Lucro líquido / Receita	Avalia eficiência operacional e rentabilidade
ROE	Decimal(2)	Return on Equity (%) - Retorno sobre o patrimônio líquido	Mede rentabilidade sobre capital próprio
ROA	Decimal(2)	Return on Assets (%) - Retorno sobre ativos	Mede eficiência no uso de ativos para gerar lucro
Margem_EBITDA	Decimal(2)	Margem EBITDA (%) - EBITDA / Receita	Avalia eficiência operacional antes de juros e impostos
PL_Historico	Decimal(2)	P/L Histórico - Preço / Lucro por ação (últimos 12 meses)	Avalia se a ação está cara ou barata
PL_Projetado	Decimal(2)	P/L Projetado - Preço / Lucro por ação estimado	Avalia expectativas de crescimento futuro
P_VPA	Decimal(2)	Preço / Valor Patrimonial por Ação	Avalia preço em relação ao valor contábil
EV_EBITDA	Decimal(2)	Enterprise Value / EBITDA	Avalia valuation da empresa como um todo
EBITDA	Decimal(2)	EBITDA (em dólares) - Lucro antes de juros, impostos, depreciação e amortização	Mede geração de caixa operacional
DY	Decimal(2)	Dividend Yield (%) - Dividendos / Preço da ação	Avalia retorno com dividendos
Dividua_Liquida_EBITDA	Decimal(2)	Dívida Líquida / EBITDA	Mede capacidade de pagamento da dívida
Crescimento_Receita	Decimal(2)	Crescimento da Receita (%) - Variação anual da receita	Avalia expansão do negócio
Crescimento_Lucro	Decimal(2)	Crescimento do Lucro (%) - Variação anual do lucro	Avalia crescimento da rentabilidade

15. INFORMAÇÕES GERAIS

Campo	Tipo	Descrição	Uso
Symbol	Char(5)	Símbolo da empresa	
Nome	Char(200)	Nome completo da empresa	
País	Char(50)	País da empresa	
Sigla_País	Char(2)	Sigla do país da empresa	
Setor	Char(50)	Setor da empresa	

16. MACRO ECONOMICOS - COMMODITIES

Campo	Tipo	Descrição	Uso
Date	Date	Dia do pregão da commodity	
Open	Decimal(8)	Valor de abertura	
Close	Decimal(8)	Valor e fechamento	

Campo	Tipo	Descrição	Uso
Change_Pct	Decimal(8)	Diferença em percentual	

17. MACRO ECONOMICOS - CAMBIO

Campo	Tipo	Descrição	Uso
Date	Date	Dia do pregão do câmbio	
Open	Decimal(8)	Valor de abertura	
Close	Decimal(8)	Valor e fechamento	
Change_Pct	Decimal(8)	Diferença em percentual	

18. MACRO ECONOMICOS - INDICES

Campo	Tipo	Descrição	Uso
Date	Date	Dia do pregão do índice	
Open	Decimal(8)	Valor de abertura	
Close	Decimal(8)	Valor e fechamento	
Change_Pct	Decimal(8)	Diferença em percentual	

19. MACRO ECONOMICOS - TAXAS DE JUROS

Campo	Tipo	Descrição	Uso
Date	Date	Período da taxa de juros em questão	
Open	Decimal(8)	Valor de abertura	
Close	Decimal(8)	Valor e fechamento	
Change_Pct	Decimal(8)	Diferença em percentual	

2. Ambiente e bibliotecas

Instalações de bibliotecas usadas

```
!pip install optuna -q
!pip install tensorflow
!pip install prophet
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.2.1)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.1)
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.1)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.2)
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.12.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.2.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.81.0)
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.1)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.45.1)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from tensorflow) (13.9.0)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.14.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2025.11.11)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.8.1)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (from tensorflow) (11.0.0)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.8.0)
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.1.0)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.0.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.1.2)
Requirement already satisfied: prophet in /usr/local/lib/python3.12/dist-packages (1.3.0)
Requirement already satisfied: cmdstanpy>=1.0.4 in /usr/local/lib/python3.12/dist-packages (from prophet) (1.3.0)
Requirement already satisfied: numpy>=1.15.4 in /usr/local/lib/python3.12/dist-packages (from prophet) (2.0.2)
```

```
Requirement already satisfied: holidays<1,>=0.25 in /usr/local/lib/python3.12/dist-packages (from prophet) (0.99)
Requirement already satisfied: tqdm>=4.36.1 in /usr/local/lib/python3.12/dist-packages (from prophet) (4.67.3)
Requirement already satisfied: importlib_resources in /usr/local/lib/python3.12/dist-packages (from prophet) (7.1.0)
Requirement already satisfied: stanio<2.0.0,>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from cmdstanpy>=1.0.4)
Requirement already satisfied: python-dateutil<3,>=2.9.0.post0 in /usr/local/lib/python3.12/dist-packages (from holidays)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=2.0.0->prophet)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.4->prophet)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.4->prophet)
```

▼ Importações de bibliotecas usadas

```
import pandas as pd
import os
import requests
import json
import sys
import time
import re
import random
import warnings
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import optuna
import hashlib
import tempfile
import base64
import traceback
import io
import pickle
import gc
import psutil
import torch

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from prophet import Prophet
from datetime import datetime
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.dummy import DummyClassifier, DummyRegressor
from sklearn.cluster import KMeans
from scipy import stats
from scipy.stats import randint, uniform, chi2_contingency
from bs4 import BeautifulSoup
from scipy.stats.mstats import winsorize
from scipy.stats import shapiro, kstest, normaltest
from xgboost import XGBRegressor
from collections import Counter
from sklearn.svm import SVR
from lightgbm import LGBMRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPRegressor
from pathlib import Path
from getpass import getpass
from google.colab import userdata
from contextlib import contextmanager

from sklearn.model_selection import (
    train_test_split, StratifiedKFold, KFold, TimeSeriesSplit, RandomizedSearchCV,
    GridSearchCV
)

from sklearn.preprocessing import (
    OneHotEncoder, StandardScaler, LabelEncoder, MinMaxScaler, RobustScaler
)

from sklearn.linear_model import (
    LogisticRegression, Ridge, LinearRegression, Lasso, ElasticNet
)
```

```

from sklearn.ensemble import (RandomForestClassifier, RandomForestRegressor,
    GradientBoostingRegressor, AdaBoostRegressor, ExtraTreesRegressor
)

from sklearn.metrics import (
    accuracy_score, f1_score, roc_auc_score, classification_report,
    ConfusionMatrixDisplay, mean_absolute_error, mean_squared_error, r2_score,
    silhouette_score, confusion_matrix, precision_score, recall_score,
)

from sklearn.feature_selection import (
    VarianceThreshold, mutual_info_regression, f_regression, SelectKBest
)

warnings.filterwarnings("ignore")

SEED = 42
np.random.seed(SEED)
random.seed(SEED)

```

Classes

```

class MeanBaseline:
    def __init__(self):
        self.mean_value = None

    def fit(self, X, y):
        self.mean_value = y.mean()
        return self

    def predict(self, X):
        return np.full(len(X), self.mean_value)

```

Configuração de visualização

```

# Estilos para os gráficos.
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
plt.rcParams['figure.figsize'] = (14, 6)
plt.rcParams['font.size'] = 12

# Cores para gráficos
cores = ['#2ecc71', '#3498db', '#e74c3c', '#f39c12', '#9b59b6',
        '#1abc9c', '#e67e22', '#34495e', '#16a085', '#c0392b']

labels = ['Baseline (Média)', 'Random Forest', 'XGBoost']

```

Constantes globais

```

url_simbolos = "https://raw.githubusercontent.com/leandrobonato/MVP_Machine_Learning_historico_precos_nasdaq/main/data"
simbolos_usados = ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'TSLA', 'NVDA', 'MRVL',
    'PEP', 'NFLX', 'AMD', 'GOOG', 'AVGO', 'META', 'MU', 'WMT',
    'COST', 'ADBE', 'CMCSA', 'CSCO', 'INTC', 'QCOM', 'TXN',
    'AMGN', 'ISRG', 'TMUS', 'HON', 'BKNG', 'AMAT', 'LRCX', 'TEAM',
    'GILD', 'CSX', 'ADI', 'VRTX', 'ADP', 'REGN', 'KLAC', 'MELI',
    'MPWR', 'PCAR', 'SNPS', 'KDP', 'CDNS', 'MRNA', 'ABNB', 'WDAY',
    'CHTR', 'LIN', 'KHC']

# Pastas e URLs
GITHUB_API = "https://api.github.com/repos/leandrobonato/MVP_Machine_Learning_historico_precos_nasdaq/contents/histori"
GITHUB_RAW = "https://raw.githubusercontent.com/leandrobonato/MVP_Machine_Learning_historico_precos_nasdaq/main/histor"
pastas_principais = ['cotacoes', 'dividendos', 'valor_mercado']
pastas_indicadores = ['rsi', 'macd', 'bollinger', 'fibonacci',
    'medias_moveis', 'volatilidade', 'tendencia']
pastas_macro = ['cambio', 'commodities', 'indices', 'taxas_juros']

# Mapeamento de commodities por empresa
COMMODITIES_POR_EMPRESA = {
    'AAPL': ['Cobre', 'Ouro', 'Prata'],
    'ABNB': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
    'ADBE': ['Cobre', 'Ouro', 'Prata'],
    'ADI': ['Cobre', 'Ouro', 'Prata'],
    'ADP': ['Cobre', 'Ouro', 'Prata'],
    'AMAT': ['Cobre', 'Ouro', 'Prata'],
    'AMD': ['Cobre', 'Ouro', 'Prata'],
    'AMZN': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],

```

```

'AVGO': ['Cobre', 'Ouro', 'Prata'],
'BKNG': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'CDNS': ['Cobre', 'Ouro', 'Prata'],
'CHTR': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'CMCSA': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'COST': ['Soja', 'Milho', 'Trigo', 'Cafe', 'Acucar'],
'CSCO': ['Cobre', 'Ouro', 'Prata'],
'CSX': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'GOOG': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'GOOGL': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'HON': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'INTC': ['Cobre', 'Ouro', 'Prata'],
'KDP': ['Soja', 'Milho', 'Trigo', 'Cafe', 'Acucar'],
'KHC': ['Soja', 'Milho', 'Trigo', 'Cafe', 'Acucar'],
'KLAC': ['Cobre', 'Ouro', 'Prata'],
'LIN': ['Cobre'],
'LRCX': ['Cobre', 'Ouro', 'Prata'],
'MELI': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'META': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'MPWR': ['Cobre', 'Ouro', 'Prata'],
'MRVL': ['Cobre', 'Ouro', 'Prata'],
'MSFT': ['Cobre', 'Ouro', 'Prata'],
'MU': ['Cobre', 'Ouro', 'Prata'],
'NFLX': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'NVDA': ['Cobre', 'Ouro', 'Prata'],
'PCAR': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'PEP': ['Soja', 'Milho', 'Trigo', 'Cafe', 'Acucar'],
'QCOM': ['Cobre', 'Ouro', 'Prata'],
'SNPS': ['Cobre', 'Ouro', 'Prata'],
'TEAM': ['Cobre', 'Ouro', 'Prata'],
'TMUS': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'TSLA': ['Ouro', 'Prata', 'Petroleo_WTI', 'Cobre'],
'TXN': ['Cobre', 'Ouro', 'Prata'],
'WDAY': ['Cobre', 'Ouro', 'Prata'],
'WMT': ['Soja', 'Milho', 'Trigo', 'Cafe', 'Acucar']
}

```

#Tradução de variáveis

```

TRADUTOR_VARIAVEIS = {
    'Date': 'Data',
    'Open': 'Abertura',
    'High': 'Máxima',
    'Low': 'Mínima',
    'Close': 'Fechamento',
    'Volume': 'Volume Negociado',
    'Adj Close': 'Fechamento Ajustado',
    'Symbol': 'Empresa (Símbolo)',
    'Company_Name': 'Nome da Empresa',
    'Sector': 'Setor',
    'Industry': 'Indústria',
    'Market_Cap_Category': 'Categoria de Capitalização',
    'Dividends': 'Dividendos',
    'Stock_Splits': 'Desdobramentos',
    'Stock_Splits': 'Desdobramentos',
    'Dividend_Yield': 'Dividend Yield (%)',
    'Dividend_Rate': 'Taxa de Dividendo',
    'Payout_Ratio': 'Payout Ratio (%)',
    'Dividend_1Y_Total': 'Total Dividendos 1 Ano',
    'Dividend_1Y_Count': 'Qtd Pagamentos 1 Ano',
    'Last_Dividend': 'Último Dividendo',
    'Ex_Dividend_Date': 'Data Ex-Dividendo',
    'Shares_Outstanding': 'Ações em Circulação',
    'Market_Cap': 'Valor de Mercado',
    'Market_Cap_B': 'Valor de Mercado (B)',
    'Market_Cap_Change_Pct': 'Variação Market Cap (%)',
    'Volume_Dollar': 'Volume Financeiro',
    'RSI': 'Índice de Força Relativa (RSI)',
    'RSI_7': 'RSI 7 dias',
    'RSI_14': 'RSI 14 dias',
    'RSI_Sinal': 'Sinal RSI',
    'RSI_Faixa': 'Faixa RSI',
    'MACD': 'MACD',
    'MACD_Line': 'Linha MACD',
    'MACD_Signal': 'Sinal MACD',
    'MACD_signal': 'Sinal MACD',
    'MACD_Histogram': 'Histograma MACD',
    'MACD_hist': 'Histograma MACD',
    'MACD_Sinal': 'Sinal MACD (C/V)',
    'BB_Upper': 'Banda Superior Bollinger',
    'BB_upper': 'Banda Superior Bollinger',
    'BB_Middle': 'Banda Média Bollinger',
    'BB_middle': 'Banda Média Bollinger',

```

```

'BB_Lower': 'Banda Inferior Bollinger',
'BB_lower': 'Banda Inferior Bollinger',
'BB_Width': 'Largura das Bandas Bollinger',
'BB_width': 'Largura das Bandas Bollinger',
'BB_Position': 'Posição nas Bandas Bollinger',
'BB_pct': '% Posição nas Bandas Bollinger',
'BB_Sinal': 'Sinal Bollinger',
'Fib_0': 'Fib 0%',
'Fib_236': 'Fib 23.6%',
'Fib_382': 'Fib 38.2%',
'Fib_50': 'Fib 50%',
'Fib_618': 'Fib 61.8%',
'Fib_786': 'Fib 78.6%',
'Fib_1': 'Fib 100%',
'Fib_1_272': 'Fib 127.2%',
'Fib_1_618': 'Fib 161.8%',
'Fib_Position': 'Posição Fibonacci',
'Fib_Level_Current': 'Nível Fib Atual',
'Fib_Distance_Next': 'Distância Próx Nível',
'Fib_Zone': 'Zona Fibonacci',
'Fib_Buy_Signal': 'Sinal Compra Fib',
'Fib_Sell_Signal': 'Sinal Venda Fib',
'MA_7': 'Média Móvel 7d',
'MA_14': 'Média Móvel 14d',
'MA_30': 'Média Móvel 30d',
'MA_50': 'Média Móvel 50d',
'MA_200': 'Média Móvel 200d',
'SMA_20': 'Média Móvel 20 períodos',
'SMA_50': 'Média Móvel 50 períodos',
'EMA_20': 'Média Exponencial 20 períodos',
'Price_SMA_ratio': 'Razão Preço/SMA20',
'Sinal_MA_7_14': 'Cruzamento MA 7/14',
'Sinal_MA_50_200': 'Cruzamento MA 50/200',
>Returns': 'Retorno Diário',
>Returns_Log': 'Retorno Log',
'Volatilidade_20': 'Volatilidade 20d',
'Volatility': 'Volatilidade Histórica',
'ATR': 'Average True Range (ATR)',
'Range_Diario': 'Amplitude Diária',
'Range_Percentual': 'Amplitude (%)',
'Gap': 'Gap Abertura',
'Gap_Percentual': 'Gap (%)',
'Volume_Medio_20': 'Volume Médio 20d',
'Volume_Ratio': 'Razão Volume (Volume/SMA)',
'Volume_SMA': 'Média Móvel do Volume',
'OBV': 'On-Balance Volume (OBV)',
'Tendencia_7d': 'Tendência 7d',
'Tendencia_50d': 'Tendência 50d',
'Forca_Tendencia': 'Força da Tendência',
'Forca_Movimento': 'Força do Movimento',
'Velas_Alta_Consecutivas': 'Velas Alta Consecutivas',
'Velas_Alta_Group': 'Grupo Velas Alta',
'ADX': 'Índice de Tendência (ADX)',
'DI_plus': 'Directional Indicator +',
'DI_minus': 'Directional Indicator -',
'Stochastic': 'Estocástico %K',
'Stochastic_D': 'Estocástico %D',
'CCI': 'Commodity Channel Index (CCI)',
'Target_Next_Return': 'Retorno Próximo Dia',
'Target_Direction': 'Direção Próximo Dia',
'AUDUSD': 'Dólar Australiano (AUD/USD)',
'BRLUSD': 'Real Brasileiro (BRL/USD)',
'Acucar': 'Açúcar (USD/lb)',
'Cafe': 'Café (USD/lb)',
'Cobre': 'Cobre (USD/t)',
'Gas_Natural': 'Gás Natural (USD/MMBtu)',
'Milho': 'Milho (USD/bushel)',
'Ouro': 'Ouro (USD/oz)',
'Petroleo_Brent': 'Petróleo Brent (USD/barri)',
'Petroleo_WTI': 'Petróleo WTI (USD/barri)',
'Prata': 'Prata (USD/oz)',
'Soja': 'Soja (USD/bushel)',
'Trigo': 'Trigo (USD/bushel)',
'SP500': 'S&P 500 (pontos)',
'VIX': 'VIX - Índice de Volatilidade (pontos)',
'Treasury_3M': 'Treasury 3 meses (%)'
}

TARGET_COL = 'Target_Next_Return'
TARGET_DIRECTION = 'Target_Direction'
PROBLEM_TYPE = "regressao"
N_ITER_SEARCH = 10

```



```

TRAIN_PCT = 0.7
VAL_PCT = 0.15
TEST_PCT = 0.15
MIN_TRAIN_SAMPLES = 50
RARE_CATEGORY_THRESHOLD = 0.05
LOW_CORRELATION_THRESHOLD = 0.01
HIGH_CORRELATION_THRESHOLD = 0.95
VARIANCE_THRESHOLD = 0.01
OUTLIER_ZSCORE_THRESHOLD = 3
VOLUME_HIGH_THRESHOLD = 1.5
RSI_OVERSOLD = 30
RSI_OVERBOUGHT = 70
SEED = 42
N_ITER_SEARCH_EXTENDS = 30
N_FOLDS = 3
N_TRIALS = 20
MODELS_TO_OPTIMIZE = ['RandomForest', 'XGBoost', 'LightGBM', 'LinearRegression',
                       'GradientBoosting']

GITHUB_USERNAME = "leandrobonato"
GITHUB_REPO = "MVP_Machine_Learning_historico_precos_nasdaq"
GITHUB_BRANCH = "main"
GITHUB_DATASET_PATH = "dataset_final"
GITHUB_TRAIN_PATH = "Resultados_treinamentos"
GITHUB_TOKEN = "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"

GITHUB_HEADERS = {
    "Authorization": f"token {GITHUB_TOKEN}",
    "Accept": "application/vnd.github.v3+json"
}

#Tamanho máximo por arquivo.
MAX_CHUNK_SIZE = 20 * 1024 * 1024 # 20 MB
DELAY_BETWEEN_UPLOADS = 4 # segundos de espera entre uploads para evitar timeouts
MAX_RETRIES = 3
RETRY_DELAY = 5

GITHUB_RAW_COMMODITIES = f"https://raw.githubusercontent.com/{GITHUB_USERNAME}/{GITHUB_REPO}/main/historicos/macro_eco"
# Mapeamento de nomes de commodities para os nomes dos arquivos
COMMODITIES_FILE_MAP = {
    'Ouro': 'Ouro.csv',
    'Prata': 'Prata.csv',
    'Cobre': 'Cobre.csv',
    'Petroleo_WTI': 'Petroleo_WTI.csv',
    'Petroleo_Brent': 'Petroleo_Brent.csv',
    'Soja': 'Soja.csv',
    'Milho': 'Milho.csv',
    'Trigo': 'Trigo.csv',
    'Cafe': 'Cafe.csv',
    'Acucar': 'Acucar.csv',
    'Gas_Natural': 'Gas_Natural.csv'
}

```

❏ Variáveis globais

```

#Simbolos
df_symbols = pd.read_csv(url_simbolos)
macro_cols = list(TRADUTOR_VARIAVEIS.keys())
macro_cols = [col for col in macro_cols if col not in ['Date', 'Symbol', 'Company_Name']]

variaveis_importantes = [
    'Close', 'Volume', 'RSI_14', 'Volatilidade_20', 'Returns', 'MA_50',
    'MA_200', 'Volume_Ratio', 'Forca_Tendencia', 'Forca_Movimento', 'Market_Cap',
    'BB_Width', 'SP500', 'VIX'
]

variaveis_correlacao = [
    'Close', 'Volume', 'RSI_14', 'Volatilidade_20', 'Returns', 'MA_50', 'MA_200',
    'Volume_Ratio', 'Forca_Tendencia', 'Forca_Movimento', 'Market_Cap', 'BB_Width',
    'SP500', 'VIX'
]

features_selecionadas = [
    'Volume_Ratio',      # H2 confirmada 100% - Volume amplifica retornos
    'RSI_14',            # H1 confirmada 70% - Reversão em extremos
    'Forca_Tendencia',   # Força da tendência (momentum)
    'Forca_Movimento',   # Força do movimento diário
    'Range_Percentual',  # Amplitude diária (volatilidade)
    'BB_Width',          # Largura das Bandas Bollinger

```



```
'Velas_Alta_Consecutivas', # Exaustão de tendência
'Market_Cap_Change_Pct', # Variação do Market Cap
'SP500', # Índice S&P 500 (macro)
'VIX' # Índice de Volatilidade (macro)
]
```

Conclusão sobre as escolhas de features e variáveis

As escolhas abaixo é devido ao notebook anterior, versão 01, que foi extremamente prejudicado pelas várias variáveis escolhidas e features que não ajudavam muito a decidir o resultado final dos treinamentos, com indicações muito equivocadas, assim todo o notebook foi refatorado para ter novas features e variáveis - isso só foi possível perceber após fazer todas as conclusões da versão 01.

Essa decisão foi tomada também pelo motivo dos treinamentos muito demorados e arquivos gigantescos, mesmo tendo diminuído a quantidade de empresas para 10, agora são 50 empresas e de vários setores, o que dá uma melhoria no senso do treinamento da máquina.

Abaixo a explicação de cada um:

1. **Volume_Ratio** (Razão Volume)

- Volume_Ratio = Volume_atual / Volume_Medio_20
- Por quê: H2 confirmada em 100% das empresas
- Impacto: Volume acima de 1.5 indica movimento forte

2. **RSI_14** (Índice de Força Relativa)

- Por quê: H1 confirmada em 70% das empresas
- Uso: RSI < 30 = sobrevendido (possível compra) e para RSI > 70 = sobrecomprado (possível venda)

3. **Forca_Tendencia** (Força da Tendência)

- Por quê: Mede inclinação da MA50
- Impacto: Valores positivos = tendência de alta acelerando e Valores negativos = tendência de baixa acelerando

4. **Forca_Movimento** (Força do Movimento Diário)

- Fórmula: (Close - Open) / (High - Low)
- Por quê: 1 = fechou na máxima do dia (força máxima) e -1 = fechou na mínima do dia (força mínima)

5. **Range_Percentual** (Amplitude Percentual)

- Fórmula: (High - Low) / Close * 100
- Por quê: Mede volatilidade intradiária
- Impacto: Valores altos = maior incerteza/volatilidade

6. **BB_Width** (Largura das Bandas Bollinger)

- Fórmula: (BB_Upper - BB_Lower) / BB_Middle
- Por quê: Baixa largura = consolidação (possível rompimento) e Alta largura = alta volatilidade (possível reversão)

7. **Velas_Alta_Consecutivas**

- Por quê: H4 confirmada em 60% das empresas
- Impacto: 5+ velas de alta consecutivas = exaustão e Aumenta probabilidade de reversão

8. **Market_Cap_Change_Pct**

- Por quê: Reflete variação de valor de mercado
- Impacto: Mudanças no Market Cap indicam percepção do mercado

9. **SP500** (S&P 500)

- Por quê: Correlação com mercado americano
- Impacto: Empresas NASDAQ têm alta correlação com SP500

10. **VIX** (Índice de Volatilidade)

- Por quê: "Índice do medo"
- Impacto: VIX alto = mercado com medo (possível queda) e VIX baixo = mercado confiante (possível alta)

Features descartadas

Feature	Motivo do Descarte
MACD_Sinal	H5 confirmada em apenas 40% das empresas
Golden Cross	H3 confirmada em apenas 50% das empresas
Fib_Buy_Signal	Sinais muito raros (~1-2% dos dias)
Last_Dividend	Dados esparsos, sem periodicidade diária
Treasury_10Y	Influência indireta, correlação baixa

Feature	Motivo do Descarte
Close	Correlação artificial com target (ambos preços)
Volume	Usamos Volume_Ratio (mais informativo)

Melhorias

1. **Reduzir dimensionalidade** de ~40+ variáveis para apenas 10
2. **Manter alto poder preditivo** com features validadas pela EDA
3. **Melhorar performance** do modelo (menos ruído, mais sinal)
4. **Reduzir tempo de treino** (menos colunas para processar)
5. **Facilitar interpretabilidade** (features com significado claro)

✓ Funções globais

```
def buscar_empresa(simbolo):
    return dict_symbols.get(simbolo.upper(), None)

def listar_arquivos_por_simbolo(simbolo):
    arquivos = {}
    for pasta in pastas_principais:
        url = f"{GITHUB_API}/{pasta}/{simbolo}.csv"
        response = requests.get(url)
        if response.status_code == 200:
            data = response.json()
            arquivos[pasta] = data['download_url']
    for ind in pastas_indicadores:
        url = f"{GITHUB_API}/indicadores/{ind}/{simbolo}.csv"
        response = requests.get(url)
        if response.status_code == 200:
            data = response.json()
            arquivos[f'indicadores/{ind}'] = data['download_url']
    return arquivos

def buscar_simbolo_por_nome(nome_parcial):
    mask = df_symbols['Company Name'].str.contains(nome_parcial, case=False, na=False)
    return df_symbols[mask]

def listar_pastas_github_api(url):
    try:
        response = requests.get(url)
        if response.status_code == 200:
            return response.json()
        else:
            print(f"Erro {response.status_code}: {response.reason}")
            return []
    except Exception as e:
        print(f"Erro: {e}")
        return []

def explorar_diretorio(url, nivel=0):
    itens = listar_pastas_github_api(url)
    for item in itens:
        prefixo = " " * nivel
        nome = item['name']
        tipo = item['type']

        if tipo == 'dir':
            explorar_diretorio(item['url'], nivel + 1)
        else:
            tamanho = item.get('size', 0)

def listar_arquivos_pasta(url, pasta):
    url = f"{url}/{pasta}"
    try:
        response = requests.get(url)
        if response.status_code == 200:
            arquivos = [item['name'] for item in response.json() if item['type'] == 'file']
            return arquivos
        return []
    except:
        return []
```

```

def baixar_csv_simbolo(url, simbolo, pasta='cotacoes'):
    url_completa = f"{url}/{pasta}/{simbolo}.csv"
    try:
        df = pd.read_csv(url_completa)
        return df
    except Exception as e:
        print(f"{simbolo}.csv não encontrado em {pasta}/")
        return None

def filtrar_por_data(simbolo, data_inicio, data_fim, categoria='cotacoes', colunas=None):
    dados = dataframes_aninhados.get(simbolo, {})
    df = dados.get(categoria) if categoria in ['cotacoes', 'dividendos', 'valor_mercado'] else dados.get('indicadores',
    if df is None:
        return None
    mask = (df['Date'] >= data_inicio) & (df['Date'] <= data_fim)
    df_filtrado = df[mask]
    if colunas:
        colunas_disp = ['Date'] + [c for c in colunas if c in df.columns]
        return df_filtrado[colunas_disp]
    return df_filtrado

def ultimos_dias(simbolo, n=5, categoria='cotacoes'):
    dados = dataframes_aninhados.get(simbolo, {})
    df = dados.get(categoria) if categoria in ['cotacoes', 'dividendos', 'valor_mercado'] else dados.get('indicadores',
    if df is None:
        return None
    return df.tail(n)

def filtrar_por_condicao(simbolo, categoria, coluna, condicao, valor):
    dados = dataframes_aninhados.get(simbolo, {})
    df = dados.get(categoria) if categoria in ['cotacoes', 'dividendos', 'valor_mercado'] else dados.get('indicadores',
    if df is None or coluna not in df.columns:
        return None
    if condicao == '>':
        return df[df[coluna] > valor]
    elif condicao == '<':
        return df[df[coluna] < valor]
    elif condicao == '==':
        return df[df[coluna] == valor]
    elif condicao == '>=':
        return df[df[coluna] >= valor]
    elif condicao == '<=':
        return df[df[coluna] <= valor]
    return None

def traduzir_variavel(var):
    return TRADUTOR_VARIAVEIS.get(var, var)

def traduzir_lista(vars_list):
    return [traduzir_variavel(v) for v in vars_list]

def traduzir_dataframe(df):
    df_traduzido = df.copy()
    df_traduzido.columns = [TRADUTOR_VARIAVEIS.get(c, c) for c in df.columns]
    return df_traduzido

def converter_para_padrao_brasileiro(valor, manter_como_string=False):
    if valor is None or pd.isna(valor):
        return valor if not manter_como_string else ''
    if isinstance(valor, (int, float, np.integer, np.floating)):
        if manter_como_string:
            return f'{valor:,.2f}'.replace('.', 'X').replace('.', ',').replace('X', '.')
        else:
            return valor
    valor_str = str(valor).strip()
    if not valor_str:
        return valor_str if manter_como_string else None
    valor_limpo = re.sub(r'^\d\.-.,', '', valor_str)
    if re.match(r'^-?\d{1,3}(\.\d{3})*(\d+)?$', valor_limpo):
        sem_pontos = valor_limpo.replace('.', '')
        com_ponto = sem_pontos.replace(',', '.')
        if manter_como_string:
            return valor_limpo
        else:
            return float(com_ponto)

```

```

elif re.match(r'^-?\d{1,3}(,\d{3})*(\.\d+)?$', valor_limpo):
    sem_virgulas = valor_limpo.replace(',', '')
    if manter_como_string:
        partes = sem_virgulas.split('.')
        parte_inteira = partes[0]
        parte_decimal = partes[1] if len(partes) > 1 else ''
        inteira_formatada = f'{int(parte_inteira):,}'.replace(',', '.')
        if parte_decimal:
            resultado = f'{inteira_formatada},{parte_decimal}'
        else:
            resultado = inteira_formatada
        return resultado
    else:
        return float(sem_virgulas)

elif re.match(r'^-?\d+(\.\d+)?$', valor_limpo) or re.match(r'^-?\d+\.\d+$', valor_limpo):
    if ',' in valor_limpo and '.' not in valor_limpo:
        com_ponto = valor_limpo.replace(',', '.')
        if manter_como_string:
            num = float(com_ponto)
            return f'{num:,.2f}'.replace(',', 'X').replace('.', ',').replace('X', '.')
        else:
            return float(com_ponto)
    else:
        if manter_como_string:
            num = float(valor_limpo)
            return f'{num:,.2f}'.replace(',', 'X').replace('.', ',').replace('X', '.')
        else:
            return float(valor_limpo)
    else:
        return valor_str if manter_como_string else valor_str

def tratar_dataframe_padrao_brasileiro(df, colunas=None, decimais=2, manter_como_string=False):
    df_resultado = df.copy()
    if colunas is None:
        colunas = df.select_dtypes(include=[np.number]).columns.tolist()

    for col in colunas:
        if col in df_resultado.columns:
            df_resultado[col] = df_resultado[col].apply(
                lambda x: converter_para_padrao_brasileiro(x, manter_como_string)
            )
        if not manter_como_string:
            df_resultado[col] = pd.to_numeric(df_resultado[col], errors='coerce')
    return df_resultado

def formatar_numero_brasileiro(valor, decimais=2, simbolo_moeda=False):
    if valor is None or pd.isna(valor):
        return ''
    try:
        num = float(valor)
        formatado = f'{num:,.{decimais}f}'.replace(',', 'X').replace('.', ',').replace('X', '.')
        if simbolo_moeda:
            return f'R$ {formatado}'
        else:
            return formatado
    except (ValueError, TypeError):
        return str(valor)

def limpar_numero_brasileiro(valor_str):
    if not isinstance(valor_str, str):
        return valor_str
    valor_limpo = valor_str.replace('.', '').replace(',', '.')
    valor_limpo = re.sub(r'^\d\.-', '', valor_limpo)
    try:
        return float(valor_limpo)
    except ValueError:
        return np.nan

def format_results_table(results):
    df = pd.DataFrame(results).T.round(6)
    df = df.sort_values('mse')
    df['r2'] = df['r2'].round(4)
    return df

def show_results_table(results):

```

```

df = pd.DataFrame(results).T
cols = ['mse', 'rmse', 'mae', 'r2', 'train_time_s']
df = df[cols]
df_styled = df.style.background_gradient(
    subset=['mse', 'rmse', 'mae'],
    cmap='RdYlGn_r',
    low=0.5,
    high=0.5
).background_gradient(
    subset=['r2'],
    cmap='RdYlGn',
    low=0.5,
    high=0.5
)
return df_styled

```

▼ Funções globais - Processamento arquivos no GitHub

```

def CarregarArquivoOnGitHub(nome_arquivo):
    for tentativa in range(MAX_RETRIES):
        try:
            if tentativa > 0:
                print(f"Tentativa {tentativa+1}/{MAX_RETRIES} para baixar {nome_arquivo}")
                time.sleep(RETRY_DELAY)
            if not nome_arquivo.endswith('.pkl'):
                nome_arquivo = f"{nome_arquivo}.pkl"
            url_raw = f"https://raw.githubusercontent.com/{GITHUB_USERNAME}/{GITHUB_REPO}/{GITHUB_BRANCH}/{GITHUB_TRAIN_PATH}"
            response = requests.get(url_raw, headers=GITHUB_HEADERS)
            if response.status_code == 404:
                if tentativa == MAX_RETRIES - 1:
                    print(f"Arquivo não encontrado: {nome_arquivo} (Status: {response.status_code})")
                    return None
                continue
            if response.status_code != 200:
                print(f"Arquivo não encontrado: {nome_arquivo} (Status: {response.status_code})")
                return None
            content_bytes = response.content
            if len(content_bytes) < 100:
                print(f"Arquivo muito pequeno: {len(content_bytes)} bytes")
                return None
            try:
                buffer = io.BytesIO(content_bytes)
                dados_carregados = pickle.load(buffer)
            except Exception as e:
                print(f"Erro ao desserializar {nome_arquivo}: {e}")
                print(f"Tamanho do arquivo: {len(content_bytes)}, bytes")
                return None
            print(f"{nome_arquivo} carregado com sucesso!")
            print(f"Tipo: {dados_carregados.get('tipo', 'Desconhecido')}")
            print(f"Tamanho: {len(content_bytes)}, bytes")
            return dados_carregados.get('dados')
        except Exception as e:
            if tentativa == MAX_RETRIES - 1:
                print(f"Erro ao carregar {nome_arquivo}: {e}")
                return None
            continue
    return None

def VerificarArquivoOnGitHub(nome_arquivo):
    try:
        if not nome_arquivo.endswith('.pkl'):
            nome_arquivo = f"{nome_arquivo}.pkl"

        url_raw = f"https://raw.githubusercontent.com/{GITHUB_USERNAME}/{GITHUB_REPO}/{GITHUB_BRANCH}/{GITHUB_TRAIN_PATH}"

        print(f"Verificando (RAW): {nome_arquivo}")
        response = requests.get(url_raw, headers=GITHUB_HEADERS)

        if response.status_code != 200:
            return {
                'valido': False,
                'erro': f'Arquivo não encontrado: {response.status_code}'
            }

        content_bytes = response.content

        if len(content_bytes) < 100:
            return {
                'valido': False,

```

```

        'erro': f'Arquivo muito pequeno: {len(content_bytes)} bytes'
    }

    try:
        buffer = io.BytesIO(content_bytes)
        dados = pickle.load(buffer)

        return {
            'valido': True,
            'tamanho': len(content_bytes),
            'tipo': dados.get('tipo', 'Desconhecido'),
            'metadados': dados.get('metadados', {})
        }
    except Exception as e:
        return {
            'valido': False,
            'erro': f'Erro ao desserializar: {str(e)}',
            'tamanho': len(content_bytes)
        }

except Exception as e:
    return {
        'valido': False,
        'erro': str(e)
    }

def ListarArquivosOnGitHub():
    try:
        url = f"https://api.github.com/repos/{GITHUB_USERNAME}/{GITHUB_REPO}/contents/{GITHUB_TRAIN_PATH}"
        response = requests.get(url, headers=GITHUB_HEADERS)

        if response.status_code != 200:
            print(f"Erro ao listar: {response.status_code}")
            return []

        arquivos = response.json()
        pkl_files = [f for f in arquivos if f['name'].endswith('.pkl')]

        if pkl_files:
            print(f"\nArquivos de treinamento encontrados ({len(pkl_files)}):")
            print("-" * 70)
            for f in pkl_files:
                print(f"    Verificando: {f['name']} ({f['size']} bytes)...")
                status = VerificarArquivoOnGitHub(f['name'])
                if status.get('valido', False):
                    print(f"    {f['name']} - OK ({status.get('tipo', 'Desconhecido')})")
                else:
                    print(f"    {f['name']} - CORROMPIDO: {status.get('erro', 'Desconhecido')}")
            else:
                print("Nenhum arquivo de treinamento encontrado")

        return pkl_files

    except Exception as e:
        print(f"Erro ao listar: {e}")
        return []

def SalvarDataSetOnGitHub(dataset, nome_base='dataset_completo', metadados=None):
    metadados_padrao = {
        'descricao': 'Dataset completo para treinamento',
        'shape': dataset.shape if hasattr(dataset, 'shape') else None,
        'colunas': list(dataset.columns) if hasattr(dataset, 'columns') else None,
        'total_registros': len(dataset) if hasattr(dataset, '__len__') else None
    }

    if metadados:
        metadados_padrao.update(metadados)

    return SalvarTreinamentoOnGitHub(
        nome_arquivo=nome_base,
        dados=dataset,
        metadados=metadados_padrao,
        mensagem=f"Salvando dataset - {metadados_padrao.get('descricao', '')}"
    )

def SalvarTreinamentoOnGitHub(nome_arquivo, dados, metadados=None, mensagem=None):
    for tentativa in range(MAX_RETRIES):
        try:
            if tentativa > 0:

```

```

        print(f"Tentativa {tentativa+1}/{MAX_RETRIES} para salvar {nome_arquivo}")
        time.sleep(RETRY_DELAY)
    if not nome_arquivo.endswith('.pkl'):
        nome_arquivo = f"{nome_arquivo}.pkl"
    dados_para_salvar = {
        'dados': dados,
        'metadados': metadados or {},
        'tipo': str(type(dados).__name__)
    }
    buffer = io.BytesIO()
    pickle.dump(dados_para_salvar, buffer, protocol=pickle.HIGHEST_PROTOCOL)
    content_bytes = buffer.getvalue()
    content = base64.b64encode(content_bytes).decode('utf-8')
    url = f"https://api.github.com/repos/{GITHUB_USERNAME}/{GITHUB_REPO}/contents/{GITHUB_TRAIN_PATH}/{nome_arquivo}"
    response_get = requests.get(url, headers=GITHUB_HEADERS)
    data = {
        "message": mensagem or f"Salvando {nome_arquivo}",
        "content": content,
        "branch": GITHUB_BRANCH
    }
    if response_get.status_code == 200:
        data["sha"] = response_get.json()["sha"]
        print(f"Atualizando arquivo existente: {nome_arquivo}")
    else:
        print(f"Criando novo arquivo: {nome_arquivo}")
    response_put = requests.put(url, headers=GITHUB_HEADERS, json=data)
    if response_put.status_code in [200, 201]:
        print(f"{nome_arquivo} salvo com sucesso! ({len(content_bytes)} bytes)")
        print(f"Verificando integridade do arquivo salvo...")
        teste_carga = CarregarArquivoOnGitHub(nome_arquivo)
        if teste_carga is not None:
            print(f"Verificação de integridade OK!")
        else:
            print(f"Atenção: Não foi possível verificar a integridade do arquivo salvo.")
    return {
        'sucesso': True,
        'nome_arquivo': nome_arquivo,
        'url': url,
        'bytes': len(content_bytes)
    }
else:
    if response_put.status_code == 404 and tentativa < MAX_RETRIES - 1:
        continue
    print(f"Erro ao salvar: {response_put.status_code}")
    print(f"Detalhes: {response_put.text[:200]}")
    return {'sucesso': False}
except Exception as e:
    if tentativa == MAX_RETRIES - 1:
        print(f"Erro ao salvar: {e}")
        traceback.print_exc()
        return {'sucesso': False}
    continue
return {'sucesso': False}

def SalvarTreinamentoDivididoOnGitHub(nome_base, dados, metadados=None, mensagem=None):
    try:
        dados_para_salvar = {
            'dados': dados,
            'metadados': metadados or {},
            'tipo': str(type(dados).__name__)
        }
        buffer = io.BytesIO()
        pickle.dump(dados_para_salvar, buffer, protocol=pickle.HIGHEST_PROTOCOL)
        content_bytes = buffer.getvalue()
        total_bytes = len(content_bytes)
        num_partes = (total_bytes // MAX_CHUNK_SIZE) + 1
        arquivos_salvos = []
        for i in range(num_partes):
            if i > 0:
                time.sleep(DELAY_BETWEEN_UPLOADS)
            inicio = i * MAX_CHUNK_SIZE
            fim = min((i + 1) * MAX_CHUNK_SIZE, total_bytes)
            chunk = content_bytes[inicio:fim]
            if num_partes == 1:
                nome_arquivo = f"{nome_base}.pkl"
            else:
                nome_arquivo = f"{nome_base}_{i+1}.pkl"
            for tentativa in range(MAX_RETRIES):
                if tentativa > 0:
                    print(f"Tentativa {tentativa+1}/{MAX_RETRIES} para {nome_arquivo}")
                    time.sleep(RETRY_DELAY)

```

```

        result = SalvarTreinamentoOnGitHub(
            nome_arquivo=nome_arquivo,
            dados={ 'chunk': chunk, 'parte': i+1, 'total': num_partes},
            metadados={
                'tipo': 'chunk',
                'parte': i+1,
                'total': num_partes,
                'total_bytes': total_bytes,
                'tamanho_chunk': len(chunk)
            },
            mensagem=mensagem or f"Salvando parte {i+1}/{num_partes} de {nome_base}"
        )
    if result.get('sucesso', False):
        arquivos_salvos.append({
            'nome': nome_arquivo,
            'parte': i+1,
            'tamanho': len(chunk)
        })
        break
    elif tentativa == MAX_RETRIES - 1:
        return {'sucesso': False, 'arquivos': arquivos_salvos, 'total_parts': num_partes}
    return {
        'sucesso': True,
        'arquivos': arquivos_salvos,
        'total_parts': num_partes,
        'total_bytes': total_bytes
    }
except Exception as e:
    print(f"Erro ao salvar {nome_base}: {e}")
    traceback.print_exc()
    return {'sucesso': False, 'arquivos': [], 'total_parts': 0}

def CarregarArquivoDivididoOnGitHub(nome_base):
    try:
        for tentativa in range(MAX_RETRIES):
            if tentativa > 0:
                print(f"Tentativa {tentativa+1}/{MAX_RETRIES} para carregar {nome_base}")
                time.sleep(RETRY_DELAY)
            arquivo_unico = CarregarArquivoOnGitHub(f"{nome_base}.pk1")
            if arquivo_unico is not None:
                return arquivo_unico
            url = f"https://api.github.com/repos/{GITHUB_USERNAME}/{GITHUB_REPO}/contents/{GITHUB_TRAIN_PATH}"
            response = requests.get(url, headers=GITHUB_HEADERS)
            if response.status_code == 404 and tentativa < MAX_RETRIES - 1:
                continue
            if response.status_code != 200:
                return None
            arquivos = response.json()
            partes = []
            padrao = f"{nome_base}_"
            for f in arquivos:
                nome = f['name']
                if nome.endswith('.pk1') and (nome == f"{nome_base}.pk1" or nome.startswith(padrao)):
                    if nome == f"{nome_base}.pk1":
                        partes.append({'nome': nome, 'parte': 0})
                    else:
                        try:
                            parte_num = int(nome.split('_')[-1].replace('.pk1', ''))
                            partes.append({'nome': nome, 'parte': parte_num})
                        except:
                            continue
            if not partes:
                return None
            partes.sort(key=lambda x: x['parte'])
            chunks = []
            falha = False
            for p in partes:
                dados_parte = CarregarArquivoOnGitHub(p['nome'])
                if dados_parte is None:
                    falha = True
                    break
                if isinstance(dados_parte, dict) and 'chunk' in dados_parte:
                    chunks.append(dados_parte['chunk'])
                else:
                    return dados_parte
            if not falha and chunks:
                content_bytes = b''.join(chunks)
                try:
                    buffer = io.BytesIO(content_bytes)
                    dados_carregados = pickle.load(buffer)
                    return dados_carregados.get('dados')

```



```

except Exception:
    return None
if tentativa == MAX_RETRIES - 1:
    return None
return None
except Exception:
    return None

```

3. Seleção e carga dos dados

Funções

```

def carregar_commodities_por_empresa(simbolo_empresa):
    commodities_necessarias = COMMODITIES_POR_EMPRESA.get(simbolo_empresa, [])
    if not commodities_necessarias:
        return None

    print(f" Carregando commodities para {simbolo_empresa}: {commodities_necessarias}")

    df_commodities = None
    for comm in commodities_necessarias:
        try:
            nome_arquivo = COMMODITIES_FILE_MAP.get(comm, f"{comm}.csv")
            url = f"{GITHUB_RAW_COMMODITIES}/{nome_arquivo}"

            print(f"      Baixando {nome_arquivo}...")

            df_temp = pd.read_csv(url)
            df_temp['Date'] = pd.to_datetime(df_temp['Date'])
            colunas = df_temp.columns.tolist()
            colunas.remove('Date')
            if 'Close' in colunas:
                df_temp = df_temp[['Date', 'Close']].rename(columns={'Close': comm})
            elif 'Open' in colunas:
                df_temp = df_temp[['Date', 'Open']].rename(columns={'Open': comm})
            else:
                coluna_preco = colunas[0]
                df_temp = df_temp[['Date', coluna_preco]].rename(columns={coluna_preco: comm})

            if df_commodities is None:
                df_commodities = df_temp
            else:
                df_commodities = df_commodities.merge(df_temp, on='Date', how='outer')

            print(f"      {comm}: {len(df_temp)} registros carregados")

        except Exception as e:
            print(f"      Erro ao carregar {comm}: {str(e)[:50]}")

    return df_commodities

def carregar_dataset_completo(nome_base):
    print(f"\nTentando carregar {nome_base} do cache...")

    partes = []
    parte_num = 1
    max_partes = 100
    while parte_num <= max_partes:
        nome_parte = f"{nome_base}_{parte_num}.pkl"
        try:
            dados_parte = CarregarArquivoOnGitHub(nome_parte)
            if dados_parte is not None:
                print(f"  Parte {parte_num} carregada")
                partes.append(dados_parte)
                parte_num += 1
            else:
                break
        except:
            break

    if not partes:
        print("  Nenhuma parte encontrada")
        return None

    print(f"  Total de partes carregadas: {len(partes)}")

    if all(isinstance(p, dict) and 'chunk' in p for p in partes):

```

```

print("    Reconstruindo dataset a partir de chunks...")
all_chunks = b''.join([p['chunk'] for p in partes])

try:
    buffer = io.BytesIO(all_chunks)
    dados_completos = pickle.load(buffer)

    if isinstance(dados_completos, dict) and 'dados' in dados_completos:
        df = dados_completos['dados']
        print(f"    Dataset reconstruído! Shape: {df.shape}")
        return df
    else:
        return dados_completos
except Exception as e:
    print(f"    Erro ao reconstruir dataset: {e}")
    return None

return None

```

✓ Criação de um DataSet completo

```

NOME_DATASET_FIXO = "dataset_completo"
df_completo = carregar_dataset_completo(NOME_DATASET_FIXO)
if df_completo is not None and isinstance(df_completo, pd.DataFrame) and not df_completo.empty:
    print(f"\n DATASET COMPLETO CARREGADO COM SUCESSO!")
    print(f"    Shape: {df_completo.shape}")
    print(f"    Período: {df_completo['Date'].min()} até {df_completo['Date'].max()}")
    print(f"    Símbolos: {df_completo['Symbol'].nunique()}")
    print(f"    Colunas: {len(df_completo.columns)}")
else:
    print("\n Dataset não encontrado em cache ou inválido. Criando novo dataset...")
    dict_symbols = dict(zip(df_symbols['Symbol'], df_symbols['Company Name']))
    dataframes_aninhados = {}
    simbolos_validos = []

    print(f"\n Carregando dados para {len(simbolos_usados)} empresas...")

    for s in simbolos_usados:
        try:
            test_url = f"{GITHUB_RAW}/cotacoes/{s}.csv"
            response = requests.head(test_url)
            if response.status_code == 200:
                simbolos_validos.append(s)
            else:
                print(f" Símbolo {s} não encontrado - ignorando")
        except:
            print(f" Erro ao verificar {s} - ignorando")

    print(f"\n Símbolos válidos: {len(simbolos_validos)}/{len(simbolos_usados)}")

    for s in simbolos_validos:
        print(f"\n Processando {s}...")

        dataframes_aninhados[s] = {
            'nome': s,
            'cotacoes': None,
            'dividendos': None,
            'valor_mercado': None,
            'indicadores': {},
            'commodities': None
        }

    for pasta in pastas_principais:
        try:
            url = f"{GITHUB_RAW}/{pasta}/{s}.csv"
            df_temp = pd.read_csv(url)
            df_temp['Date'] = pd.to_datetime(df_temp['Date'])
            dataframes_aninhados[s][pasta] = df_temp
            print(f"    {pasta}: {len(df_temp)} registros")
        except Exception as e:
            print(f"    {pasta}: não encontrado")

    for pasta in pastas_indicadores:
        try:
            url = f"{GITHUB_RAW}/indicadores/{pasta}/{s}.csv"
            df_temp = pd.read_csv(url)
            df_temp['Date'] = pd.to_datetime(df_temp['Date'])
            dataframes_aninhados[s]['indicadores'][pasta] = df_temp
            print(f"    indicadores/{pasta}: {len(df_temp)} registros")
        except:

```

```

        print(f"    indicadores/{pasta}: não encontrado")

    try:
        df_comm = carregar_commodities_por_empresa(s)
        if df_comm is not None and not df_comm.empty:
            dataframes_aninhados[s]['commodities'] = df_comm
            print(f"    commodities: {len(df_comm)} registros, {len(df_comm.columns)-1} commodities")
    except Exception as e:
        print(f"    commodities: erro ao carregar - {str(e)[:50]}")

print("\n Montando dataset completo...")
lista_dfs = []

for s in simbolos_validos:
    dados = dataframes_aninhados.get(s)
    if not dados:
        continue

    df_base = dados.get('cotacoes')
    if df_base is None or df_base.empty:
        print(f" {s}: Sem dados de cotações - ignorando")
        continue

    df_base['Symbol'] = s
    df_base['Company_Name'] = dict_symbols.get(s, 'Desconhecido')

    # Adicionar dividendos
    df_div = dados.get('dividendos')
    if df_div is not None and not df_div.empty:
        colunas_div = ['Date'] + [c for c in df_div.columns if c != 'Date' and c not in df_base.columns]
        if len(colunas_div) > 1:
            df_base = df_base.merge(df_div[colunas_div], on='Date', how='left')

    # Adicionar valor de mercado
    df_mc = dados.get('valor_mercado')
    if df_mc is not None and not df_mc.empty:
        colunas_mc = ['Date'] + [c for c in df_mc.columns if c != 'Date' and c not in df_base.columns]
        if len(colunas_mc) > 1:
            df_base = df_base.merge(df_mc[colunas_mc], on='Date', how='left')

    # Adicionar indicadores
    for nome_ind, df_ind in dados.get('indicadores', {}).items():
        if df_ind is not None and not df_ind.empty:
            colunas_ind = ['Date'] + [c for c in df_ind.columns if c != 'Date' and c not in df_base.columns]
            if len(colunas_ind) > 1:
                df_base = df_base.merge(df_ind[colunas_ind], on='Date', how='left')

    # Adicionar commodities
    df_comm = dados.get('commodities')
    if df_comm is not None and not df_comm.empty:
        colunas_comm = ['Date'] + [c for c in df_comm.columns if c != 'Date' and c not in df_base.columns]
        if len(colunas_comm) > 1:
            df_base = df_base.merge(df_comm[colunas_comm], on='Date', how='left')
            print(f"    {s}: Commodities adicionadas")

    lista_dfs.append(df_base)

# Concatenar
if lista_dfs:
    df_completo = pd.concat(lista_dfs, ignore_index=True)
    df_completo = df_completo.sort_values(['Date', 'Symbol']).reset_index(drop=True)
    print(f"\n Dataset concatenado: {df_completo.shape}")
    print(f"    Colunas: {len(df_completo.columns)}")
    print(f"    Símbolos: {df_completo['Symbol'].nunique()}")
else:
    df_completo = pd.DataFrame()
    print(" Nenhum dado foi carregado!")

# Salvar dataset
if not df_completo.empty:
    print(f"\n Salvando dataset no GitHub...")
    metadados = {
        'descricao': 'Dataset completo para treinamento',
        'shape': df_completo.shape,
        'simbolos': simbolos_validos,
        'total_registros': len(df_completo),
        'colunas': list(df_completo.columns),
        'data_inicio': df_completo['Date'].min().strftime('%Y-%m-%d'),
        'data_fim': df_completo['Date'].max().strftime('%Y-%m-%d'),
        'versao': '1.0'
    }

```

```

resultado = SalvarTreinamentoDivididoOnGitHub(
    nome_base=NOME_DATASET_FIXO,
    dados=df_completo,
    metadados=metadados,
    mensagem=f"Salvando dataset completo ({len(df_completo):,} registros)"
)

if resultado.get('sucesso', False):
    print(f" Dataset salvo em {resultado['total_parts']} partes!")
else:
    print(" Falha ao salvar dataset")

```

```

Tentando carregar dataset_completo do cache...
dataset_completo_1.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
    Parte 1 carregada
dataset_completo_2.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
    Parte 2 carregada
dataset_completo_3.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
    Parte 3 carregada
dataset_completo_4.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
    Parte 4 carregada
dataset_completo_5.pkl carregado com sucesso!
Tipo: dict
Tamanho: 1,808,295 bytes
    Parte 5 carregada
Tentativa 2/3 para baixar dataset_completo_6.pkl
Tentativa 3/3 para baixar dataset_completo_6.pkl
Arquivo não encontrado: dataset_completo_6.pkl (Status: 404)
Total de partes carregadas: 5
Reconstruindo dataset a partir de chunks...
Dataset reconstruído! Shape: (138615, 81)

DATASET COMPLETO CARREGADO COM SUCESSO!
Shape: (138615, 81)
Período: 2015-01-02 00:00:00 até 2026-06-26 00:00:00
Símbolos: 49
Colunas: 81

```

```

if df_completo is not None and isinstance(df_completo, pd.DataFrame) and not df_completo.empty:
    print(f"\n Dataset carregado com sucesso!")
    print(f"    Shape: {df_completo.shape[0]:,} linhas x {df_completo.shape[1]} colunas")
    print(f"    Período: {df_completo['Date'].min()} até {df_completo['Date'].max()}")
    print(f"    Símbolos únicos: {df_completo['Symbol'].nunique()}")
    print(f"    Colunas totais: {len(df_completo.columns)}")

    num_cols = len(df_completo.select_dtypes(include=[np.number]).columns)
    cat_cols = len(df_completo.select_dtypes(include=['object']).columns)

    print(f"    Colunas numéricas: {num_cols}")
    print(f"    Colunas categóricas: {cat_cols}")

    print("\n Amostra dos dados:")
    colunas_mostrar = ['Date', 'Symbol', 'Close', 'Volume', 'RSI_14', 'SP500', 'VIX']
    colunas_existentes = [col for col in colunas_mostrar if col in df_completo.columns]
    if colunas_existentes:
        print(df_completo[colunas_existentes].head(10))

    print("\n Dados faltantes por coluna (top 10):")
    missing = df_completo.isnull().sum()
    missing = missing[missing > 0].sort_values(ascending=False)
    if not missing.empty:
        print(missing.head(10))
    else:
        print("    Nenhum dado faltante!")

    print("\n Estatísticas básicas das principais variáveis:")
    colunas_stats = ['Close', 'Volume', 'RSI_14', 'SP500', 'VIX']
    colunas_stats_existentes = [col for col in colunas_stats if col in df_completo.columns]
    if colunas_stats_existentes:
        print(df_completo[colunas_stats_existentes].describe().round(2))

    # Verificar commodities carregadas
    colunas_comm = [col for col in df_completo.columns if col in COMMODITIES_POR_EMPRESA.get(df_completo['Symbol'].iloc[
    if colunas_comm:
        print(f"\n Commodities carregadas: {colunas_comm}")

```

```
# Mostrar exemplo de dados com commodities
if 'SP500' in df_completo.columns and 'VIX' in df_completo.columns:
    print("\n Exemplo de dados com commodities e índices:")
    colunas_exemplo = ['Date', 'Symbol', 'Close', 'SP500', 'VIX']
    if 'Ouro' in df_completo.columns:
        colunas_exemplo.append('Ouro')
    if 'Petroleo_WTI' in df_completo.columns:
        colunas_exemplo.append('Petroleo_WTI')
    colunas_existentes = [c for c in colunas_exemplo if c in df_completo.columns]
    if colunas_existentes:
        print(df_completo[colunas_existentes].head(10))
```

```
Dataset carregado com sucesso!
Shape: 138,615 linhas x 81 colunas
Período: 2015-01-02 00:00:00 até 2026-06-26 00:00:00
Símbolos únicos: 49
Colunas totais: 81
Colunas numéricas: 71
Colunas categóricas: 9
```

```
Amostra dos dados:
```

	Date	Symbol	Close	Volume	RSI_14
0	2015-01-02	AAPL	24.19	212818400	NaN
1	2015-01-02	ADBE	72.34	2349200	NaN
2	2015-01-02	ADI	44.10	1323200	NaN
3	2015-01-02	ADP	64.54	1866600	NaN
4	2015-01-02	AMAT	21.71	6910200	NaN
5	2015-01-02	AMD	2.67	0	NaN
6	2015-01-02	AMGN	114.77	2605400	NaN
7	2015-01-02	AMZN	15.43	55664000	NaN
8	2015-01-02	AVGO	7.53	13500000	NaN
9	2015-01-02	BKNG	44.73	12732500	NaN

```
Dados faltantes por coluna (top 10):
Ex_Dividend_Date      137264
Stock_Splits          137257
Dividend_Yield        137257
Payout_Ratio          137257
Dividend_Rate         137257
Dividend_1Y_Total     137257
Dividend_1Y_Count     137257
Last_Dividend         137257
Milho                 124326
Trigo                 124316
dtype: int64
```

```
Estatísticas básicas das principais variáveis:
```

	Close	Volume	RSI_14
count	138615.00	1.386150e+05	138566.00
mean	160.47	2.741351e+07	53.37
std	221.08	7.747860e+07	16.93
min	0.46	0.000000e+00	0.00
25%	38.81	2.122206e+06	41.22
50%	91.75	6.854100e+06	53.49
75%	189.58	2.253360e+07	65.71
max	2613.63	3.692928e+09	100.00

```
Commodities carregadas: ['Cobre', 'Ouro', 'Prata']
```

Formato do dataset

```
print("Formato do dataset: ", df_completo.shape)
```

```
Formato do dataset: (138615, 81)
```

Tipos dos atributos

```
print("\nTipos de dados:")
display(df_completo.dtypes.to_frame("tipo"))
```

Tipos de dados:

tipo	
Date	datetime64[ns]
Open	float64
High	float64
Low	float64
Close	float64
...	...
Soja	float64
Milho	float64
Trigo	float64
Cafe	float64
Acucar	float64

81 rows × 1 columns

Valores ausentes

```
print("\nValores ausentes por coluna:")
display(df_completo.isna().sum().to_frame("ausentes"))
```

Valores ausentes por coluna:

ausentes	
Date	0
Open	0
High	0
Low	0
Close	0
...	...
Soja	124316
Milho	124326
Trigo	124316
Cafe	124316
Acucar	124316

81 rows × 1 columns

Valores duplicados

```
print("\nDuplicatas:", df_completo.duplicated().sum())
```

Duplicatas: 0

Possíveis colunas de ID, data ou variáveis que não devem entrar no modelo

```
display(df_completo.sample(5, random_state=SEED))
```

	Date	Open	High	Low	Close	Volume	Dividends	Stock Splits	Symbol	Company_Name	...	Velas_Alta_Consecuti
55289	2019-09-11	277.66	280.31	274.00	277.78	2594000	0.0	0.0	ADBE	Adobe Inc.	...	
56768	2019-10-23	131.73	134.79	131.62	133.96	647200	0.0	0.0	SNPS	Synopsys, Inc.	...	
37152	2018-03-06	19.66	19.81	19.50	19.79	30030000	0.0	0.0	AVGO	Broadcom Inc.	...	
2945	2015-04-08	24.13	24.39	24.00	24.19	18250300	0.0	0.0	INTC	Intel Corporation	...	
128523	2025-09-02	350.24	352.71	341.28	345.63	4229400	0.0	0.0	ADBE	Adobe Inc.	...	

5 rows × 81 columns

```
display(df_completo.sample(5, random_state=100))
```

	Date	Open	High	Low	Close	Volume	Dividends	Stock Splits	Symbol	Company_Name	...	Velas_Alta_Consecuti
7184	2015-08-19	37.36	37.38	36.56	36.67	6906300	0.0	0.0	TXN	Texas Instruments Incorporated	...	
119878	2024-12-13	613.39	617.68	594.69	600.98	896900	0.0	0.0	MPWR	Monolithic Power Systems, Inc.	...	
45803	2018-11-26	313.54	314.13	308.78	313.23	1240000	0.0	0.0	CHTR	Charter Communications, Inc.	...	
762	2015-01-27	123.91	126.54	123.51	123.67	1103800	0.0	0.0	VRTX	Vertex Pharmaceuticals Incorporated	...	
20598	2016-10-07	91.13	91.62	90.14	91.04	873600	0.0	0.0	WDAY	Workday, Inc.	...	

5 rows × 81 columns

4. Análise exploratório dos dados

A análise exploratória deve ser objetiva e conectada ao problema.

- 1. distribuição do target
- 2. distribuição de variáveis importantes
- 3. relação entre variáveis e target
- 4. identificação de desbalanceamento, outliers ou padrões relevantes;
- 5. hipóteses que surgem a partir dos dados.

Funções

```
def analyze_target_distribution(df, target_col, target_dir_col=None, empresas=None, n_empresas=49):
    stats_por_symbol = df.groupby('Symbol').agg({
        target_col: ['mean', 'std', 'count', lambda x: (x > 0).sum()],
    }).round(4)
    if target_dir_col and target_dir_col in df.columns:
        stats_por_symbol[target_dir_col] = df.groupby('Symbol')[target_dir_col].mean()
    stats_por_symbol.columns = ['Retorno_Medio', 'Retorno_Std', 'Total_Dias', 'Dias_Alta', 'Pct_Alta']
    stats_por_symbol['Pct_Alta'] = stats_por_symbol['Pct_Alta'] / stats_por_symbol['Total_Dias'] * 100
    if empresas is None:
        empresas = stats_por_symbol.index.tolist()
    top_symbols = stats_por_symbol.nlargest(n_empresas, 'Total_Dias').index.tolist()

    # Gráfico 1: Distribuição por empresa
    fig, axes = plt.subplots(2, 3, figsize=(18, 12))
    axes = axes.flatten()

    for idx, symbol in enumerate(top_symbols):
        if idx < len(axes):
            df_sym = df[df['Symbol'] == symbol]
            axes[idx].hist(df_sym[target_col], bins=50, color='steelblue', alpha=0.7, edgecolor='black')
```

```

axes[idx].axvline(0, color='red', linestyle='--', linewidth=1.5)
axes[idx].axvline(df_sym[target_col].mean(), color='green', linestyle='-', linewidth=1.5,
                  label=f'Média: {df_sym[target_col].mean():.2%}')
axes[idx].set_title(f'{symbol} | n={len(df_sym)} | Média={df_sym[target_col].mean():.2%}\n'
                   f'Alta: {df_sym[target_dir_col].mean():.1%} | Std: {df_sym[target_col].std():.2%}'
                   if target_dir_col else '')
axes[idx].set_xlabel('Retorno Próximo Dia')
axes[idx].set_ylabel('Frequência')
axes[idx].legend(fontsize=8)
axes[idx].grid(True, alpha=0.3)

for j in range(len(top_symbols), len(axes)):
    axes[j].set_visible(False)

plt.tight_layout()
plt.show()

# Gráfico 2: Visão consolidada
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
axes[0, 0].hist(df[target_col], bins=100, color='steelblue', alpha=0.7, edgecolor='black')
axes[0, 0].axvline(0, color='red', linestyle='--', linewidth=2)
axes[0, 0].axvline(df[target_col].mean(), color='green', linewidth=2,
                  label=f'Média: {df[target_col].mean():.4f}')
axes[0, 0].set_title(f'Distribuição Consolidada (n={len(df):,})', fontweight='bold')
axes[0, 0].set_xlabel('Retorno')
axes[0, 0].set_ylabel('Frequência')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

top20_symbols = stats_por_symbol.nlargest(min(20, len(stats_por_symbol)), 'Total_Dias').index.tolist()
df_top20 = df[df['Symbol'].isin(top20_symbols)]
data_for_boxplot = [df_top20[df_top20['Symbol'] == sym][target_col].values for sym in top20_symbols]
bp = axes[0, 1].boxplot(data_for_boxplot, labels=top20_symbols, patch_artist=True)
axes[0, 1].axhline(0, color='red', linestyle='--', alpha=0.7)
axes[0, 1].set_title('Boxplot por Empresa (Top 20 por volume de dados)', fontweight='bold')
axes[0, 1].set_ylabel('Retorno Próximo Dia')
axes[0, 1].grid(True, alpha=0.3, axis='y')
plt.setp(axes[0, 1].get_xticklabels(), rotation=45, ha='right')

if target_dir_col and target_dir_col in df.columns:
    classes = df[target_dir_col].value_counts()
    explode = (0.02, 0)
    cores = ['#e74c3c', '#2ecc71']
    axes[1, 0].pie(classes.values, labels=['Caiu (0)', 'Subiu (1)'],
                  autopct='%1.1f%%', colors=cores, explode=explode, startangle=90)
    axes[1, 0].set_title(f'Proporção Consolidada\nDias de Alta vs Baixa (n={len(df):,})',
                      fontweight='bold')

axes[1, 1].hist(stats_por_symbol['Retorno_Medio'], bins=30, color='purple', alpha=0.7, edgecolor='black')
axes[1, 1].axvline(0, color='red', linestyle='--', linewidth=2)
axes[1, 1].axvline(stats_por_symbol['Retorno_Medio'].mean(), color='green', linewidth=2,
                  label=f'Média geral: {stats_por_symbol["Retorno_Medio"].mean():.4f}')
axes[1, 1].set_title('Distribuição dos Retornos Médios por Empresa', fontweight='bold')
axes[1, 1].set_xlabel('Retorno Médio da Empresa')
axes[1, 1].set_ylabel('Número de Empresas')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Gráfico de retorno acumulado
fig, ax = plt.subplots(figsize=(14, 8))
for symbol in top_symbols[:6]:
    df_sym = df[df['Symbol'] == symbol].copy()
    df_sym = df_sym.sort_values('Date')
    if target_dir_col and target_dir_col in df_sym.columns:
        df_sym['Estrategia_Ret'] = df_sym[target_col] * df_sym[target_dir_col]
    else:
        df_sym['Estrategia_Ret'] = df_sym[target_col]
    df_sym['Retorno_Acumulado'] = (1 + df_sym['Estrategia_Ret']).cumprod()
    ax.plot(df_sym['Date'], df_sym['Retorno_Acumulado'], label=f'{symbol}', linewidth=1.5)
ax.set_title('Retorno Acumulado da Estratégia (Seguindo Sinal de Alta)', fontweight='bold')
ax.set_xlabel('Data')
ax.set_ylabel('Patrimônio (Inicial = 1)')
ax.legend()
ax.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Performance por empresa

```



```

performance = []
for symbol in stats_por_symbol.index:
    df_sym = df[df['Symbol'] == symbol].copy()
    df_sym['Estrategia_Ret'] = df_sym[target_col] * df_sym[target_dir_col] if target_dir_col else df_sym[target_col]
    retorno_total = (1 + df_sym['Estrategia_Ret']).prod() - 1
    sharpe_approx = df_sym['Estrategia_Ret'].mean() / df_sym['Estrategia_Ret'].std() * (252**0.5) if df_sym['Estrategia_Ret'].std() > 0 else 0
    win_rate = (df_sym['Estrategia_Ret'] > 0).mean()
    performance.append({
        'Symbol': symbol,
        'Retorno_Total': retorno_total,
        'Sharpe_Anual': sharpe_approx,
        'Win_Rate': win_rate,
        'Num_Trades': len(df_sym)
    })

df_performance = pd.DataFrame(performance).sort_values('Retorno_Total', ascending=False)

# Print resumo
print("\n\n" + "=" * 70)
print("RESUMO DA DISTRIBUIÇÃO DO TARGET")
print(df_performance.head(10).round(4))
print(f"Total de registros após remover NaN: {len(df):,} registros (painel ba")
print(f"Número de empresas únicas: {df['Symbol'].nunique()}")
print(f"Observação: Todas as empresas têm exatamente {stats_por_symbol['Total_Dias'].iloc[0]:,} registros (painel ba")
print(f"Média geral do retorno (todas empresas): {df[target_col].mean():.2%}")
print(f"Desvio padrão geral: {df[target_col].std():.2%}")
print(f"Empresa com maior retorno médio: {stats_por_symbol['Retorno_Medio'].idxmax()} "
      f"({stats_por_symbol['Retorno_Medio'].max():.2%}")
print(f"Empresa com menor retorno médio: {stats_por_symbol['Retorno_Medio'].idxmin()} "
      f"({stats_por_symbol['Retorno_Medio'].min():.2%}")
print(f"Empresa com maior % de dias de alta: {stats_por_symbol['Pct_Alta'].idxmax()} "
      f"({stats_por_symbol['Pct_Alta'].max():.1f}%")
print(f"Empresa com menor % de dias de alta: {stats_por_symbol['Pct_Alta'].idxmin()} "
      f"({stats_por_symbol['Pct_Alta'].min():.1f}%")

return stats_por_symbol, df_performance

def analyze_correlation_with_target(df, target_col, top_n=15, bottom_n=15):
    var_numericas = df.select_dtypes(include=[np.number]).columns
    threshold = 0.5 * len(df)
    var_numericas = [col for col in var_numericas if df[col].count() > threshold]
    correlacoes_target = df[var_numericas].corr()[target_col].sort_values(ascending=False)
    correlacoes_target = correlacoes_target.dropna()
    correlacoes_target = correlacoes_target.drop(target_col, errors='ignore')
    correlacoes_traduzidas = correlacoes_target.copy()
    correlacoes_traduzidas.index = [traduzir_variavel(idx) for idx in correlacoes_traduzidas.index]

    fig, axes = plt.subplots(2, 2, figsize=(18, 16))
    top15_plot = correlacoes_traduzidas.head(top_n)
    cores = ['#2ecc71' if v > 0 else '#e74c3c' for v in top15_plot.values]
    bars1 = axes[0, 0].barh(range(len(top15_plot)), top15_plot.values, color=cores, edgecolor='white', alpha=0.8)
    axes[0, 0].set_yticks(range(len(top15_plot)))
    axes[0, 0].set_yticklabels(top15_plot.index, fontsize=10)
    axes[0, 0].set_title(f'Top {top_n} Correlações com o Target', fontweight='bold', fontsize=12)
    axes[0, 0].set_xlabel('Correlação de Pearson', fontsize=11)
    axes[0, 0].axvline(0, color='black', linestyle='-', linewidth=1)
    axes[0, 0].axvline(0.1, color='gray', linestyle='--', linewidth=0.5, alpha=0.5)
    axes[0, 0].axvline(-0.1, color='gray', linestyle='--', linewidth=0.5, alpha=0.5)
    axes[0, 0].invert_yaxis()
    axes[0, 0].grid(True, alpha=0.3, axis='x')
    for i, (bar, v) in enumerate(zip(bars1, top15_plot.values)):
        pos_text = v - 0.02 if v > 0.1 else v + 0.02
        ha_text = 'right' if v > 0.1 else 'left'
        axes[0, 0].text(pos_text, bar.get_y() + bar.get_height()/2,
                       f'{v:.4f}', va='center', ha=ha_text, fontsize=9,
                       fontweight='bold', color='white' if v > 0.1 else 'black')

    bottom15_plot = correlacoes_traduzidas.tail(bottom_n)
    cores_bottom = ['#2ecc71' if v > 0 else '#e74c3c' for v in bottom15_plot.values]
    bars2 = axes[0, 1].barh(range(len(bottom15_plot)), bottom15_plot.values, color=cores_bottom, edgecolor='white', alpha=0.8)
    axes[0, 1].set_yticks(range(len(bottom15_plot)))
    axes[0, 1].set_yticklabels(bottom15_plot.index, fontsize=10)
    axes[0, 1].set_title(f'Top {bottom_n} MENOS Correlacionadas (Negativas)', fontweight='bold', fontsize=12)
    axes[0, 1].set_xlabel('Correlação de Pearson', fontsize=11)
    axes[0, 1].axvline(0, color='black', linestyle='-', linewidth=1)
    axes[0, 1].axvline(0.1, color='gray', linestyle='--', linewidth=0.5, alpha=0.5)
    axes[0, 1].axvline(-0.1, color='gray', linestyle='--', linewidth=0.5, alpha=0.5)
    axes[0, 1].invert_yaxis()
    axes[0, 1].grid(True, alpha=0.3, axis='x')
    for i, (bar, v) in enumerate(zip(bars2, bottom15_plot.values)):
        pos_text = v + 0.02 if v < -0.1 else v - 0.02

```

```

ha_text = 'left' if v < -0.1 else 'right'
axes[0, 1].text(pos_text, bar.get_y() + bar.get_height()/2,
               f'{v:.4f}', va='center', ha=ha_text, fontsize=9,
               fontweight='bold', color='white' if v < -0.1 else 'black')

top10_vars = correlacoes_target.head(10).index.tolist()
top10_vars.append(target_col)
matriz_corr = df[top10_vars].corr()
matriz_corr.index = [traduzir_variavel(idx) for idx in matriz_corr.index]
matriz_corr.columns = [traduzir_variavel(col) for col in matriz_corr.columns]
sns.heatmap(matriz_corr, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
            vmin=-1, vmax=1, square=True, linewidths=0.5, ax=axes[1, 0],
            cbar_kws={'shrink': 0.8, 'label': 'Correlação'})
axes[1, 0].set_title('Heatmap de Correlação - Top 10 Variáveis', fontweight='bold', fontsize=12)
axes[1, 0].tick_params(axis='both', labelsiz=8)
top4 = correlacoes_target.head(4).index.tolist()
axes[1, 1].set_title('Top 4 Variáveis vs Target (Scatter)', fontweight='bold', fontsize=12)

for idx, var in enumerate(top4):
    if var in df.columns:
        x_pos = idx % 2
        y_pos = idx // 2
        ax_inset = axes[1, 1].inset_axes([x_pos * 0.48, 1 - (y_pos + 1) * 0.48, 0.45, 0.45])
        if len(df) > 5000:
            dados_plot = df.sample(n=min(5000, len(df)), random_state=42)
        else:
            dados_plot = df
        mask = dados_plot[var].notna() & dados_plot[target_col].notna()
        x_clean = dados_plot.loc[mask, var]
        y_clean = dados_plot.loc[mask, target_col]
        ax_inset.scatter(x_clean, y_clean, alpha=0.3, s=10, c='steelblue', edgecolors='white', linewidth=0.1)
        ax_inset.axhline(0, color='red', linestyle='--', alpha=0.5, linewidth=1)
        ax_inset.axvline(0, color='gray', linestyle='--', alpha=0.3, linewidth=0.5)
        if len(x_clean) > 10:
            z = np.polyfit(x_clean, y_clean, 1)
            p = np.poly1d(z)
            x_line = np.linspace(x_clean.min(), x_clean.max(), 100)
            ax_inset.plot(x_line, p(x_line), "r-", alpha=0.7, linewidth=1.5, label='Tendência')
        ax_inset.set_title(f'{traduzir_variavel(var)}', fontsize=9, fontweight='bold')
        ax_inset.set_xlabel('Valor', fontsize=7)
        ax_inset.set_ylabel('Retorno Próximo Dia', fontsize=7)
        ax_inset.tick_params(labelsiz=6)
        ax_inset.grid(True, alpha=0.2)

axes[1, 1].axis('off')
plt.suptitle('Relação entre Variáveis e Target (Retorno Próximo Dia)',
            fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

fig, ax = plt.subplots(figsize=(10, 6))
ax.hist(correlacoes_target.values, bins=30, color='steelblue', alpha=0.7, edgecolor='black')
ax.axvline(0, color='red', linestyle='--', linewidth=2, label='Correlação Zero')
ax.axvline(correlacoes_target.mean(), color='green', linestyle='-', linewidth=2,
            label=f'Média: {correlacoes_target.mean():.4f}')
ax.set_title('Distribuição das Correlações com o Target', fontweight='bold', fontsize=12)
ax.set_xlabel('Correlação de Pearson')
ax.set_ylabel('Número de Variáveis')
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("\n\n" + "=" * 70)
print("RESUMO DA ANÁLISE DE CORRELAÇÃO")
print(f"Total de variáveis numéricas analisadas: {len(correlacoes_target)}")
print(f"Variáveis com correlação positiva: {(correlacoes_target > 0).sum()}")
print(f"Variáveis com correlação negativa: {(correlacoes_target < 0).sum()}")
print(f"Maior correlação positiva: {correlacoes_target.max():.4f} ({traduzir_variavel(correlacoes_target.idxmax())})")
print(f"Maior correlação negativa: {correlacoes_target.min():.4f} ({traduzir_variavel(correlacoes_target.idxmin())})")
print(f"Média das correlações: {correlacoes_target.mean():.4f}")
print(f"Mediana das correlações: {correlacoes_target.median():.4f}")
print(f"Desvio padrão das correlações: {correlacoes_target.std():.4f}")

df_correlacoes = pd.DataFrame({
    'Variável Original': correlacoes_target.index,
    'Variável Traduzida': [traduzir_variavel(v) for v in correlacoes_target.index],
    'Correlação com Target': correlacoes_target.values
}).sort_values('Correlação com Target', ascending=False)

return correlacoes_traduzidas, df_correlacoes

```

```

def analyze_categorical_target_relationship(df, target_col, categorias, tradutor=None):
    if tradutor is None:
        tradutor = lambda x: x
    categorias_existentes = [(col, titulo) for col, titulo in categorias if col in df.columns]
    if not categorias_existentes:
        print("Nenhuma das categorias especificadas foi encontrada no DataFrame.")
        return None

    fig, axes = plt.subplots(2, 3, figsize=(18, 10))
    axes = axes.flatten()
    resultados = {}
    for idx, (coluna, titulo) in enumerate(categorias_existentes):
        if idx < len(axes):
            dados = df.groupby(coluna, observed=False)[target_col].mean().sort_values(ascending=False)
            resultados[titulo] = dados.to_dict()

            cores_bar = ['#2ecc71' if v > 0 else '#e74c3c' for v in dados.values]
            bars = axes[idx].bar(range(len(dados)), dados.values, color=cores_bar, edgecolor='white', alpha=0.8)
            axes[idx].set_xticks(range(len(dados)))
            axes[idx].set_xticklabels(dados.index, rotation=45, ha='right', fontsize=9)
            axes[idx].set_title(f'{tradutor(titulo)}', fontweight='bold', fontsize=11)
            axes[idx].set_ylabel('Retorno Médio do Próximo Dia', fontsize=9)
            axes[idx].axhline(0, color='black', linestyle='-', linewidth=1)
            axes[idx].grid(True, alpha=0.3, axis='y')

            for bar, val in zip(bars, dados.values):
                y_offset = 0.0002 if val > 0 else -0.0003
                va_text = 'bottom' if val > 0 else 'top'
                axes[idx].text(bar.get_x() + bar.get_width()/2,
                              bar.get_height() + y_offset,
                              f'{val:.3%}', ha='center', va=va_text, fontsize=9,
                              fontweight='bold')

            contagem = df[coluna].value_counts()
            axes[idx].text(0.98, 0.98, f'Total: {len(df):,} registros',
                          transform=axes[idx].transAxes, fontsize=8,
                          verticalalignment='top', horizontalalignment='right',
                          bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

    for i in range(len(categorias_existentes), len(axes)):
        axes[i].set_visible(False)

    plt.suptitle('Retorno Médio por Categoria (Target: Retorno Próximo Dia)',
                fontsize=14, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.show()

    return resultados

def analyze_correlation_by_empresa(df, target_col, var_correlacao, empresas, tradutor=None):
    if tradutor is None:
        tradutor = lambda x: x

    resultados = {}

    for empresa in empresas:
        print(f"\n\n{' '*60}")
        print(f"GERANDO GRÁFICOS PARA: {empresa}")
        df_empresa = df[df['Symbol'] == empresa].copy()
        var_existentes = [var for var in var_correlacao if var in df_empresa.columns]

        if len(var_existentes) >= 2:
            print(f"\n 1. Matriz de Correlação - {empresa}")
            corr_matrix = df_empresa[var_existentes].corr()
            corr_matrix.columns = [tradutor(col) for col in corr_matrix.columns]
            corr_matrix.index = [tradutor(idx) for idx in corr_matrix.index]
            fig, ax = plt.subplots(figsize=(14, 12))
            mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
            sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='RdBu_r', center=0,
                        vmin=-1, vmax=1, square=True, linewidths=0.5, mask=mask,
                        cbar_kws={'shrink': 0.8, 'label': 'Correlação'}, ax=ax)
            ax.set_title(f'{empresa} - Matriz de Correlação', fontweight='bold', fontsize=14)
            ax.tick_params(axis='both', labels=8)
            plt.xticks(rotation=45, ha='right')
            plt.tight_layout()
            plt.show()

        print(f"\n\n2. Top 3 Variáveis com Mais Outliers - {empresa}")
        var_numericas_empresa = [v for v in var_correlacao
                                  if v in df_empresa.columns and df_empresa[v].dtype in ['float64', 'int64']]

```

```

outliers_info = []
for var in var_numericas_empresa:
    dados = df_empresa[var].dropna()
    if len(dados) > 1:
        z_scores = np.abs(stats.zscore(dados))
        n_outliers = (z_scores > 3).sum()
        pct_outliers = (n_outliers / len(dados)) * 100
        outliers_info.append({
            'var': var,
            'pct': pct_outliers,
            'n_outliers': n_outliers,
            'total': len(dados)
        })

df_outliers = pd.DataFrame(outliers_info).sort_values('pct', ascending=False)
top3 = df_outliers.head(3)
if len(top3) > 0:
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    for idx, (_, row) in enumerate(top3.iterrows()):
        var = row['var']
        dados = df_empresa[var].dropna()
        nome_traduzido = tradutor(var)
        axes[idx].hist(dados, bins=50, color='#e74c3c', edgecolor='white', alpha=0.8)
        axes[idx].axvline(dados.median(), color='green', linestyle='-', linewidth=2,
                          label=f'Mediana: {dados.median():.2f}')
        axes[idx].axvline(dados.mean(), color='blue', linestyle='--', linewidth=2,
                          label=f'Média: {dados.mean():.2f}')
        p1 = dados.quantile(0.01)
        p99 = dados.quantile(0.99)
        axes[idx].axvline(p1, color='orange', linestyle=':', linewidth=1.5, alpha=0.7)
        axes[idx].axvline(p99, color='orange', linestyle=':', linewidth=1.5, alpha=0.7)
        axes[idx].set_title(f'{empresa}\n{nome_traduzido}\nOutliers: {row["pct"]:.1f}%',
                             fontweight='bold', fontsize=11)
        axes[idx].set_xlabel('Valor', fontsize=10)
        axes[idx].set_ylabel('Frequência', fontsize=10)
        axes[idx].legend(fontsize=8)
        axes[idx].grid(True, alpha=0.3)
        axes[idx].text(0.95, 0.95,
                       f'Total: {row["total"]:,}\n'
                       f'Outliers: {row["n_outliers"]:,}\n'
                       f'Min: {dados.min():.2f}\n'
                       f'Max: {dados.max():.2f}',
                       transform=axes[idx].transAxes, fontsize=9,
                       verticalalignment='top', horizontalalignment='right',
                       bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.7))
    plt.suptitle(f'{empresa} - Top 3 Variáveis com Mais Outliers', fontweight='bold', fontsize=14)
    plt.tight_layout()
    plt.show()
    print(f"\nResumo de Outliers - {empresa}:")
    display(df_outliers[['var', 'pct', 'n_outliers', 'total']].head(10).round(2))

if 'df_outliers' not in locals() or df_outliers.empty:
    df_outliers = pd.DataFrame(columns=['var', 'pct', 'n_outliers', 'total'])

resultados[empresa] = {
    'correlation_matrix': corr_matrix if len(var_existentes) >= 2 else None,
    'outliers': df_outliers
}

return resultados

def analyze_hypotheses_by_empresa(df, target_col, empresas, required_cols, tradutor=None):
    if tradutor is None:
        tradutor = lambda x: x
    resultados_hipoteses_por_empresa = {}
    for empresa in empresas:
        df_empresa = df[df['Symbol'] == empresa].copy()
        colunas_existentes = [col for col in required_cols if col in df_empresa.columns]
        print(f"Colunas disponíveis: {len(colunas_existentes)}/{len(required_cols)}")
        resultados_empresa = {}

        # H1: RSI reversão
        if 'RSI_14' in df_empresa.columns and target_col in df_empresa.columns:
            rsi_baixo = df_empresa[df_empresa['RSI_14'] < 30][target_col].mean() if len(df_empresa[df_empresa['RSI_14'] < 30]) > 0 else None
            rsi_alto = df_empresa[df_empresa['RSI_14'] > 70][target_col].mean() if len(df_empresa[df_empresa['RSI_14'] > 70]) > 0 else None
            rsi_neutro = df_empresa[(df_empresa['RSI_14'] >= 30) & (df_empresa['RSI_14'] <= 70)][target_col].mean() if len(df_empresa[(df_empresa['RSI_14'] >= 30) & (df_empresa['RSI_14'] <= 70)]) > 0 else None
            resultados_empresa['H1_RSI_reversao'] = {
                'baixo': rsi_baixo,
                'alto': rsi_alto,
                'neutro': rsi_neutro,
                'confirmada': rsi_baixo > rsi_neutro and rsi_alto < rsi_neutro if rsi_neutro != 0 else False
            }
        resultados_empresa = resultados_empresa
    resultados[empresa] = resultados_empresa

```

```

    }

# H2: Volume amplifica
if 'Volume_Ratio' in df_empresa.columns and target_col in df_empresa.columns:
    volume_alto = df_empresa[df_empresa['Volume_Ratio'] > 1.5]
    volume_normal = df_empresa[df_empresa['Volume_Ratio'] <= 1.5]
    if len(volume_alto) > 0 and len(volume_normal) > 0:
        resultados_empresa['H2_Volume_amplifica'] = {
            'alto_abs_mean': volume_alto[target_col].abs().mean(),
            'normal_abs_mean': volume_normal[target_col].abs().mean(),
            'confirmada': volume_alto[target_col].abs().mean() > volume_normal[target_col].abs().mean()
        }
    else:
        resultados_empresa['H2_Volume_amplifica'] = {'confirmada': False}

# H3: Golden Cross
if 'Sinal_MA_50_200' in df_empresa.columns and target_col in df_empresa.columns:
    golden = df_empresa[df_empresa['Sinal_MA_50_200'] == 1]
    death = df_empresa[df_empresa['Sinal_MA_50_200'] == -1]
    golden_mean = golden[target_col].mean() if len(golden) > 0 else 0
    death_mean = death[target_col].mean() if len(death) > 0 else 0
    resultados_empresa['H3_Golden_Cross'] = {
        'golden_mean': golden_mean,
        'death_mean': death_mean,
        'confirmada': golden_mean > death_mean
    }

# H4: Exaustão velas
if 'Velas_Alta_Consecutivas' in df_empresa.columns and target_col in df_empresa.columns:
    df_empresa['Velas_Group'] = pd.cut(df_empresa['Velas_Alta_Consecutivas'],
                                       bins=[-1, 0, 1, 2, 4, 100],
                                       labels=['0', '1', '2', '3-4', '5+'])
    tendencia_grupos = df_empresa.groupby('Velas_Group', observed=False)[target_col].agg(['mean', 'count'])
    exaustao_confirmada = False
    if '5+' in tendencia_grupos.index and len(tendencia_grupos) > 0:
        exaustao_confirmada = tendencia_grupos.loc['5+', 'mean'] < 0
    resultados_empresa['H4_Exaustao_velas'] = {
        'grupos': tendencia_grupos.to_dict(),
        'confirmada': exaustao_confirmada
    }

# H5: MACD
if 'MACD_Sinal' in df_empresa.columns and target_col in df_empresa.columns:
    macd_compra = df_empresa[df_empresa['MACD_Sinal'] == 'Compra']
    macd_venda = df_empresa[df_empresa['MACD_Sinal'] == 'Venda']
    compra_mean = macd_compra[target_col].mean() if len(macd_compra) > 0 else 0
    venda_mean = macd_venda[target_col].mean() if len(macd_venda) > 0 else 0
    resultados_empresa['H5_MACD'] = {
        'compra_mean': compra_mean,
        'venda_mean': venda_mean,
        'confirmada': compra_mean > venda_mean
    }

# H6: Bollinger
if 'BB_Sinal' in df_empresa.columns and target_col in df_empresa.columns:
    bb_sobrevendido = df_empresa[df_empresa['BB_Sinal'] == 'Sobrevendido']
    bb_neutro = df_empresa[df_empresa['BB_Sinal'] == 'Neutro']
    sobrevivido_mean = bb_sobrevendido[target_col].mean() if len(bb_sobrevendido) > 0 else 0
    neutro_mean = bb_neutro[target_col].mean() if len(bb_neutro) > 0 else 0
    resultados_empresa['H6_Bollinger'] = {
        'sobrevendido_mean': sobrevivido_mean,
        'neutro_mean': neutro_mean,
        'confirmada': sobrevivido_mean > neutro_mean
    }

# H7: Fibonacci
if 'Fib_Buy_Signal' in df_empresa.columns and target_col in df_empresa.columns:
    fib_buy = df_empresa[df_empresa['Fib_Buy_Signal'] == 1]
    fib_buy_mean = fib_buy[target_col].mean() if len(fib_buy) > 0 else 0
    resultados_empresa['H7_Fibonacci'] = {
        'buy_mean': fib_buy_mean,
        'confirmada': fib_buy_mean > 0
    }

# H8: Tendência
if 'Forca_Tendencia' in df_empresa.columns and target_col in df_empresa.columns:
    tend_forte = df_empresa[df_empresa['Forca_Tendencia'] > df_empresa['Forca_Tendencia'].quantile(0.75)]
    tend_fraca = df_empresa[df_empresa['Forca_Tendencia'] < df_empresa['Forca_Tendencia'].quantile(0.25)]
    forte_mean = tend_forte[target_col].mean() if len(tend_forte) > 0 else 0
    fraca_mean = tend_fraca[target_col].mean() if len(tend_fraca) > 0 else 0
    resultados_empresa['H8_Tendencia'] = {
        'forte_mean': forte_mean,

```

```

        'frac_a_mean': frac_a_mean,
        'confirmada': forte_mean > frac_a_mean
    }

# H9: Momentum
if 'Força_Movimento' in df_empresa.columns and target_col in df_empresa.columns:
    mov_forte = df_empresa[df_empresa['Força_Movimento'] > 0.7]
    mov_forte_mean = mov_forte[target_col].mean() if len(mov_forte) > 0 else 0
    resultados_empresa['H9_Momentum'] = {
        'forte_mean': mov_forte_mean,
        'confirmada': mov_forte_mean > 0
    }
resultados_hipoteses_por_empresa[empresa] = resultados_empresa

# Gráficos
fig, axes = plt.subplots(3, 3, figsize=(18, 16))
axes = axes.flatten()

# H1: RSI
if 'RSI_14' in df_empresa.columns:
    rsi_bins = pd.cut(df_empresa['RSI_14'], bins=10)
    rsi_ret = df_empresa.groupby(rsi_bins, observed=False)[target_col].mean()
    axes[0].bar(range(len(rsi_ret)), rsi_ret.values, color='steelblue')
    axes[0].axhline(0, color='red', linestyle='--')
    axes[0].set_title(f'{empresa} - H1: Retorno por Faixa de RSI', fontweight='bold', fontsize=10)
    axes[0].set_xlabel('Decil RSI')
    axes[0].set_ylabel('Retorno Médio')
else:
    axes[0].text(0.5, 0.5, 'RSI_14 não disponível', ha='center', va='center')

# H2: Volume Ratio
if 'Volume_Ratio' in df_empresa.columns:
    sample_size = min(500, len(df_empresa))
    axes[1].scatter(df_empresa['Volume_Ratio'].iloc[:sample_size],
                    df_empresa[target_col].iloc[:sample_size],
                    alpha=0.3, s=5, c='steelblue')
    axes[1].axhline(0, color='red', linestyle='--')
    axes[1].axvline(1.5, color='orange', linestyle='--', label='Alto (>1.5)')
    axes[1].set_title(f'{empresa} - H2: Volume Ratio vs Retorno', fontweight='bold', fontsize=10)
    axes[1].set_xlabel('Volume Ratio')
    axes[1].legend()
else:
    axes[1].text(0.5, 0.5, 'Volume_Ratio não disponível', ha='center', va='center')

# H3: Golden Cross
if 'Sinal_MA_50_200' in df_empresa.columns:
    ma_grupos = df_empresa.groupby('Sinal_MA_50_200')[target_col].agg(['mean', 'std'])
    if len(ma_grupos) >= 2:
        colors = ['#e74c3c', '#2ecc71'] if ma_grupos['mean'].iloc[1] > ma_grupos['mean'].iloc[0] else ['#2ecc71', '#e74c3c']
        axes[2].bar(['Death Cross', 'Golden Cross'], ma_grupos['mean'].values,
                    yerr=ma_grupos['std'].values, color=colors, capsize=10)
        axes[2].axhline(0, color='black', linestyle='-')
        axes[2].set_title(f'{empresa} - H3: Cruzamento MA50/200', fontweight='bold', fontsize=10)
    else:
        axes[2].text(0.5, 0.5, 'Sinal_MA_50_200 não disponível', ha='center', va='center')

# H4: Velas consecutivas
if 'Velas_Alta_Consecutivas' in df_empresa.columns:
    if 'Velas_Group' in df_empresa.columns:
        axes[3].bar(tendencia_grupos.index, tendencia_grupos['mean'].values, color='steelblue')
        axes[3].axhline(0, color='red', linestyle='--')
        axes[3].set_title(f'{empresa} - H4: Velas Consecutivas de Alta', fontweight='bold', fontsize=10)
        axes[3].set_xlabel('Velas Consecutivas')
        axes[3].set_ylabel('Retorno Médio')
    else:
        axes[3].text(0.5, 0.5, 'Velas_Alta_Consecutivas não disponível', ha='center', va='center')

# H5: MACD
if 'MACD_Sinal' in df_empresa.columns:
    macd_ret = df_empresa.groupby('MACD_Sinal')[target_col].mean()
    colors_macd = ['#2ecc71' if macd_ret.get('Compra', 0) > 0 else '#e74c3c',
                  '#e74c3c' if macd_ret.get('Venda', 0) < 0 else '#2ecc71',
                  '#3498db']
    axes[4].bar(macd_ret.index, macd_ret.values, color=colors_macd[:len(macd_ret)])
    axes[4].axhline(0, color='black', linestyle='-')
    axes[4].set_title(f'{empresa} - H5: Retorno por Sinal MACD', fontweight='bold', fontsize=10)
    axes[4].tick_params(axis='x', rotation=45)
else:
    axes[4].text(0.5, 0.5, 'MACD_Sinal não disponível', ha='center', va='center')

# H6: Bollinger
if 'BB_Sinal' in df_empresa.columns:
    bb_ret = df_empresa.groupby('BB_Sinal')[target_col].mean()

```

```

cores_bb = {'Sobrevendido': '#2ecc71', 'Neutro': '#3498db', 'Sobrecomprado': '#e74c3c'}
bb_colors = [cores_bb.get(k, '#3498db') for k in bb_ret.index]
axes[5].bar(bb_ret.index, bb_ret.values, color=bb_colors)
axes[5].axhline(0, color='black', linestyle='-')
axes[5].set_title(f'{empresa} - H6: Retorno por Sinal Bollinger', fontweight='bold', fontsize=10)
axes[5].tick_params(axis='x', rotation=45)
else:
    axes[5].text(0.5, 0.5, 'BB_Sinal não disponível', ha='center', va='center')

# H7: Fibonacci
if 'Fib_Buy_Signal' in df_empresa.columns and 'Fib_Sell_Signal' in df_empresa.columns:
    fib_buy = df_empresa[df_empresa['Fib_Buy_Signal'] == 1]
    fib_sell = df_empresa[df_empresa['Fib_Sell_Signal'] == 1]
    fib_neutro = df_empresa[(df_empresa['Fib_Buy_Signal'] == 0) & (df_empresa['Fib_Sell_Signal'] == 0)]
    fib_ret = {
        'Buy Signal': fib_buy[target_col].mean() if len(fib_buy) > 0 else 0,
        'Sell Signal': fib_sell[target_col].mean() if len(fib_sell) > 0 else 0,
        'Sem Sinal': fib_neutro[target_col].mean() if len(fib_neutro) > 0 else 0
    }
    axes[6].bar(fib_ret.keys(), fib_ret.values(), color=['#2ecc71', '#e74c3c', '#3498db'])
    axes[6].axhline(0, color='black', linestyle='-')
    axes[6].set_title(f'{empresa} - H7: Retorno por Sinal Fibonacci', fontweight='bold', fontsize=10)
else:
    axes[6].text(0.5, 0.5, 'Sinais Fibonacci não disponíveis', ha='center', va='center')

# H8: Força da tendência
if 'Forca_Tendencia' in df_empresa.columns:
    tend_bins = pd.cut(df_empresa['Forca_Tendencia'], bins=5)
    tend_ret = df_empresa.groupby(tend_bins, observed=False)[target_col].mean()
    axes[7].bar(range(len(tend_ret)), tend_ret.values, color='steelblue')
    axes[7].axhline(0, color='red', linestyle='--')
    axes[7].set_title(f'{empresa} - H8: Força da Tendência vs Retorno', fontweight='bold', fontsize=10)
    axes[7].set_xlabel('Quartil Força Tendência')
else:
    axes[7].text(0.5, 0.5, 'Forca_Tendencia não disponível', ha='center', va='center')

# H9: Força do movimento
if 'Forca_Movimento' in df_empresa.columns:
    mov_bins = pd.cut(df_empresa['Forca_Movimento'], bins=5)
    mov_ret = df_empresa.groupby(mov_bins, observed=False)[target_col].mean()
    axes[8].bar(range(len(mov_ret)), mov_ret.values, color='steelblue')
    axes[8].axhline(0, color='red', linestyle='--')
    axes[8].set_title(f'{empresa} - H9: Força Movimento vs Retorno', fontweight='bold', fontsize=10)
    axes[8].set_xlabel('Quartil Força Movimento')
else:
    axes[8].text(0.5, 0.5, 'Forca_Movimento não disponível', ha='center', va='center')

plt.suptitle(f'{empresa} - Validação de Hipóteses', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print(f"\n\nRESUMO DAS HIPÓTESES - {empresa}")
for hip, results in resultados_empresa.items():
    if 'confirmada' in results:
        status = '✅ CONFIRMADA' if results['confirmada'] else '❌ REJEITADA'
        print(f"    {status} | {hip}")

resumo_geral = []
for empresa, resultados in resultados_hipoteses_por_empresa.items():
    for hip, results in resultados.items():
        if 'confirmada' in results:
            resumo_geral.append({
                'Empresa': empresa,
                'Hipótese': hip,
                'Confirmada': results['confirmada']
            })

df_resumo_geral = pd.DataFrame(resumo_geral)
matriz_confirmacao = df_resumo_geral.pivot_table(index='Empresa', columns='Hipótese', values='Confirmada', aggfunc='sum')

print("\n MATRIZ DE CONFIRMAÇÃO DE HIPÓTESES POR EMPRESA:")
display(matriz_confirmacao)
total_por_empresa = df_resumo_geral.groupby('Empresa')['Confirmada'].sum()
print("\n TOTAL DE HIPÓTESES CONFIRMADAS POR EMPRESA:")
for empresa, total in total_por_empresa.sort_values(ascending=False).items():
    print(f"    {empresa}: {total}/9 hipóteses")

freq_hipoteses = df_resumo_geral.groupby('Hipótese')['Confirmada'].mean().sort_values(ascending=False)
print("\nHIPÓTESES MAIS CONFIRMADAS (média entre empresas):")
for hip, freq in freq_hipoteses.items():
    print(f"    {hip}: {freq:.1%}")

```



```
return resultados_hipoteses_por_empresa
```

▼ Distribuição do TARGET

```
df_completo = df_completo.sort_values(['Symbol', 'Date'])
df_completo[TARGET_COL] = df_completo.groupby('Symbol')['Close'].pct_change().shift(-1)
df_completo[TARGET_DIRECTION] = (df_completo[TARGET_COL] > 0).astype(int)
NOME_DATASET_ANALISE = "dataset_analise"
df_analise = CarregarArquivoDivididoOnGitHub(NOME_DATASET_ANALISE)
if df_analise is not None:
    print("Dataset análise carregado do cache!")
    print(f"Shape: {df_analise.shape}")
else:
    print("Dataset não encontrado em cache. Processando...")
    df_analise = df_completo.dropna(subset=[TARGET_COL, TARGET_DIRECTION])
    print(f"Dataset análise criado: {df_analise.shape}")
    resultado = SalvarTreinamentoDivididoOnGitHub(
        nome_base=NOME_DATASET_ANALISE,
        dados=df_analise,
        metadados={
            'descricao': 'DataFrame de análise com target',
            'shape': df_analise.shape,
            'target_col': TARGET_COL,
            'target_dir': TARGET_DIRECTION,
            'total_registros': len(df_analise)
        },
        mensagem=f"Salvando dataset_analise ({len(df_analise):,} registros)"
    )

    if resultado.get('sucesso', False):
        print(f"Dataset análise salvo em {resultado['total_parts']} partes!")
    else:
        print("Falha ao salvar dataset_analise")

NOME_STATS_TARGET = "stats_target"
stats_cache = CarregarArquivoOnGitHub(NOME_STATS_TARGET)
if stats_cache is not None:
    print("Estatísticas do target carregadas do cache!")
    stats_por_symbol = stats_cache.get('stats_por_symbol')
    df_performance = stats_cache.get('df_performance')
else:
    print("Processando estatísticas do target...")
    # Calcular estatísticas
    stats_por_symbol = df_analise.groupby('Symbol').agg({
        TARGET_COL: ['mean', 'std', 'min', 'max', 'count'],
        TARGET_DIRECTION: ['mean', 'sum']
    }).round(4)

    # Renomear colunas
    stats_por_symbol.columns = ['_'.join(col).strip() for col in stats_por_symbol.columns.values]
    stats_por_symbol = stats_por_symbol.reset_index()

    # Calcular performance por empresa
    df_performance = df_analise.groupby('Symbol').agg({
        TARGET_COL: ['mean', 'std'],
        TARGET_DIRECTION: ['mean']
    }).round(4)
    df_performance.columns = ['_'.join(col).strip() for col in df_performance.columns.values]
    df_performance = df_performance.reset_index()

    # Salvar estatísticas
    SalvarDataSetOnGitHub(
        dataset=stats_por_symbol,
        nome_base=NOME_STATS_TARGET,
        metadados={
            'descricao': 'Estatísticas de distribuição do target por empresa',
            'empresas': simbolos_usados
        }
    )
    print("Estatísticas do target salvas no cache!")
```

```
Tentativa 2/3 para baixar dataset_analise.pkl
Tentativa 3/3 para baixar dataset_analise.pkl
Arquivo não encontrado: dataset_analise.pkl (Status: 404)
dataset_analise_1.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
dataset_analise_2.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
```



```
dataset_analise_3.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
dataset_analise_4.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
dataset_analise_5.pkl carregado com sucesso!
Tipo: dict
Tamanho: 5,163,599 bytes
Dataset análise carregado do cache!
Shape: (138566, 83)
stats_target.pkl carregado com sucesso!
Tipo: DataFrame
Tamanho: 4,413 bytes
Estatísticas do target carregadas do cache!
```

▼ Distribuição de variáveis importantes e identificação de Outliers

```
NOME_OUTLIERS = "outliers_por_empresa"
resultados_correlacao_empresa_cache = CarregarArquivoOnGitHub(NOME_OUTLIERS)
if resultados_correlacao_empresa_cache is not None:
    print("Resultados de outliers carregados do cache!")
    resultados_correlacao_empresa = resultados_correlacao_empresa_cache
else:
    print("Processando análise de outliers...")
    empresas_disponiveis = df_analise['Symbol'].unique().tolist()
    print(f"Processando {len(empresas_disponiveis)} empresas para análise de outliers")
    resultados_correlacao_empresa = {}

    for empresa in empresas_disponiveis:
        try:
            df_emp = df_analise[df_analise['Symbol'] == empresa].copy()
            outliers_info = []
            for var in variaveis_correlacao:
                if var in df_emp.columns:
                    Q1 = df_emp[var].quantile(0.25)
                    Q3 = df_emp[var].quantile(0.75)
                    IQR = Q3 - Q1
                    lower_bound = Q1 - 1.5 * IQR
                    upper_bound = Q3 + 1.5 * IQR
                    outliers = df_emp[(df_emp[var] < lower_bound) | (df_emp[var] > upper_bound)]
                    pct_outliers = len(outliers) / len(df_emp) * 100
                    if pct_outliers > 0:
                        outliers_info.append({
                            'var': var,
                            'pct': pct_outliers,
                            'count': len(outliers),
                            'total': len(df_emp)
                        })
            resultados_correlacao_empresa[empresa] = {
                'outliers': pd.DataFrame(outliers_info) if outliers_info else pd.DataFrame(),
                'total_registros': len(df_emp)
            }
        except Exception as e:
            print(f" Erro ao processar {empresa}: {e}")

    SalvarTreinamentoOnGitHub(
        nome_arquivo=NOME_OUTLIERS,
        dados=resultados_correlacao_empresa,
        metadados={
            'descricao': 'Análise de outliers por empresa',
            'empresas': empresas_disponiveis,
            'total_empresas': len(empresas_disponiveis)
        }
    )
```

```
outliers_por_empresa.pkl carregado com sucesso!
Tipo: dict
Tamanho: 26,588 bytes
Resultados de outliers carregados do cache!
```

```
def analisar_distribuicao_outliers(df_analise, variaveis_correlacao, empresas=None, n_empresas=10):
    NOME_OUTLIERS = "outliers_por_empresa"
    resultados_correlacao_empresa_cache = CarregarArquivoOnGitHub(NOME_OUTLIERS)
    if resultados_correlacao_empresa_cache is not None:
        print("Resultados de outliers carregados do cache!")
        resultados_correlacao_empresa = resultados_correlacao_empresa_cache
    else:
        print("Processando análise de outliers...")
        empresas_disponiveis = df_analise['Symbol'].unique().tolist()
        resultados_correlacao_empresa = {}
```

```

for idx, empresa in enumerate(empresas_disponiveis):
    try:
        if idx % 10 == 0:
            print(f"    Processando {idx+1}/{len(empresas_disponiveis)}: {empresa}")
        df_emp = df_analise[df_analise['Symbol'] == empresa].copy()
        outliers_info = []
        for var in variaveis_correlacao:
            if var in df_emp.columns and df_emp[var].notna().sum() > 0:
                dados = df_emp[var].dropna()
                Q1 = dados.quantile(0.25)
                Q3 = dados.quantile(0.75)
                IQR = Q3 - Q1
                lower_bound = Q1 - 1.5 * IQR
                upper_bound = Q3 + 1.5 * IQR
                outliers = dados[(dados < lower_bound) | (dados > upper_bound)]
                pct_outliers = len(outliers) / len(dados) * 100
                if pct_outliers > 0:
                    outliers_info.append({
                        'var': var,
                        'pct': pct_outliers,
                        'count': len(outliers),
                        'total': len(dados),
                        'min': dados.min(),
                        'max': dados.max(),
                        'mean': dados.mean(),
                        'std': dados.std(),
                        'q1': Q1,
                        'q3': Q3,
                        'lower_bound': lower_bound,
                        'upper_bound': upper_bound
                    })
                resultados_correlacao_empresa[empresa] = {
                    'outliers': pd.DataFrame(outliers_info) if outliers_info else pd.DataFrame(),
                    'total_registros': len(df_emp)
                }
        except Exception as e:
            print(f"Erro ao processar {empresa}: {e}")

SalvarTreinamentoOnGitHub(
    nome_arquivo=NOME_OUTLIERS,
    dados=resultados_correlacao_empresa,
    metadados={
        'descricao': 'Análise de outliers por empresa',
        'empresas': empresas_disponiveis,
        'total_empresas': len(empresas_disponiveis),
        'variaveis_analisadas': variaveis_correlacao
    }
)

variaveis_numericas = df_analise.select_dtypes(include=[np.number]).columns.tolist()
variaveis_para_analise = [v for v in variaveis_correlacao if v in variaveis_numericas]
stats_descritivas = []
for var in variaveis_para_analise:
    dados = df_analise[var].dropna()
    if len(dados) > 0:
        stats_descritivas.append({
            'Variável': var,
            'Nome_Traduzido': traduzir_variavel(var),
            'Count': len(dados),
            'Mean': dados.mean(),
            'Std': dados.std(),
            'Min': dados.min(),
            'Q1': dados.quantile(0.25),
            'Median': dados.median(),
            'Q3': dados.quantile(0.75),
            'Max': dados.max(),
            'Skew': dados.skew(),
            'Kurtosis': dados.kurtosis()
        })
df_stats = pd.DataFrame(stats_descritivas).sort_values('Mean', ascending=False)
outlier_agregado = {}
for empresa, resultado in resultados_correlacao_empresa.items():
    df_out = resultado['outliers']
    if not df_out.empty:
        for _, row in df_out.iterrows():
            var = row['var']
            if var not in outlier_agregado:
                outlier_agregado[var] = {
                    'pct_total': 0,
                    'count_total': 0,
                    'empresas_afetadas': 0,
                    'empresas': []
                }

```

```

        outlier_agregado[var]['pct_total'] += row['pct']
        outlier_agregado[var]['count_total'] += row['count']
        outlier_agregado[var]['empresas_afetadas'] += 1
        outlier_agregado[var]['empresas'].append(empresa)
df_outlier_resumo = []
for var, dados in outlier_agregado.items():
    n_empresas = len(resultados_correlacao_empresa)
    df_outlier_resumo.append({
        'Variável': var,
        'Nome_Traduzido': traduzir_variavel(var),
        'Média_%_Outliers': dados['pct_total'] / dados['empresas_afetadas'],
        'Total_Outliers': dados['count_total'],
        'Empresas_Afetadas': dados['empresas_afetadas'],
        '%_Empresas': dados['empresas_afetadas'] / n_empresas * 100
    })
df_outlier_resumo = pd.DataFrame(df_outlier_resumo).sort_values('Média_%_Outliers', ascending=False)
display(df_outlier_resumo.head(15))
top_vars = df_correlacoes.head(10)['Variável'].tolist() if 'df_correlacoes' in dir() else variaveis_para_analise[:8]
top_vars = [v for v in top_vars if v in variaveis_para_analise][:8]
fig, axes = plt.subplots(2, 4, figsize=(20, 10))
axes = axes.flatten()
for idx, var in enumerate(top_vars):
    if idx < len(axes):
        dados = df_analise[var].dropna()
        axes[idx].hist(dados, bins=50, density=True, alpha=0.6, color='steelblue', edgecolor='black')
        axes[idx].axvline(dados.mean(), color='red', linestyle='--', linewidth=2, label=f'Média: {dados.mean():.4f}')
        axes[idx].axvline(dados.median(), color='green', linestyle='--', linewidth=2, label=f'Mediana: {dados.median():.4f}')
        Q1 = dados.quantile(0.25)
        Q3 = dados.quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        axes[idx].axvline(lower_bound, color='orange', linestyle=':', linewidth=1.5, alpha=0.7)
        axes[idx].axvline(upper_bound, color='orange', linestyle=':', linewidth=1.5, alpha=0.7, label='Limites IQR')
        axes[idx].set_title(f'{traduzir_variavel(var)}\nSkew: {dados.skew():.2f} | Kurtosis: {dados.kurtosis():.2f}')
        axes[idx].set_xlabel('Valor')
        axes[idx].set_ylabel('Densidade')
        axes[idx].legend(fontsize=8)
        axes[idx].grid(True, alpha=0.3)
for j in range(len(top_vars), len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Distribuição das Principais Variáveis', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.show()
if not df_outlier_resumo.empty:
    top_outlier_vars = df_outlier_resumo.head(4)['Variável'].tolist()
    top_outlier_vars = [v for v in top_outlier_vars if v in df_analise.columns]
    if top_outlier_vars:
        print("\nBOXPLOT DAS VARIÁVEIS COM MAIS OUTLIERS (Top 10 Empresas)")
        empresas_plot = empresas if empresas else df_analise['Symbol'].unique().tolist()[:10]
        fig, axes = plt.subplots(1, len(top_outlier_vars), figsize=(5*len(top_outlier_vars), 6))
        if len(top_outlier_vars) == 1:
            axes = [axes]
        for idx, var in enumerate(top_outlier_vars):
            data_for_boxplot = []
            labels = []
            for empresa in empresas_plot:
                df_emp = df_analise[df_analise['Symbol'] == empresa]
                dados = df_emp[var].dropna()
                if len(dados) > 0:
                    data_for_boxplot.append(dados.values)
                    labels.append(empresa)
            bp = axes[idx].boxplot(data_for_boxplot, labels=labels, patch_artist=True)
            axes[idx].set_title(f'{traduzir_variavel(var)}', fontweight='bold')
            axes[idx].set_xlabel('Empresa')
            axes[idx].set_ylabel('Valor')
            axes[idx].grid(True, alpha=0.3, axis='y')
            plt.setp(axes[idx].get_xticklabels(), rotation=45, ha='right')
        plt.suptitle('Distribuição das Variáveis com Mais Outliers por Empresa', fontsize=14, fontweight='bold')
        plt.tight_layout()
        plt.show()

empresas_outliers = []
for empresa, resultado in resultados_correlacao_empresa.items():
    df_out = resultado['outliers']
    if not df_out.empty:
        empresas_outliers.append({
            'Empresa': empresa,
            'Total_Outliers': df_out['count'].sum(),
            'Variáveis_afetadas': len(df_out),
            'Média_%_Outliers': df_out['pct'].mean(),

```

```
'Max_%_Outliers': df_out['pct'].max(),
'Var_Max_Outliers': df_out.loc[df_out['pct'].idxmax(), 'var'] if not df_out.empty else None
})
if empresas_outliers:
    df_empresas_outliers = pd.DataFrame(empresas_outliers).sort_values('Total_Outliers', ascending=False)
    display(df_empresas_outliers.head(10))
else:
    print("Nenhuma empresa com outliers detectados.")

if df_analise is not None and not df_analise.empty:
    resultados_outliers = analisar_distribuicao_outliers(
        df_analise=df_analise,
        variaveis_correlacao=variaveis_correlacao,
        empresas=None, # Todas as empresas
        n_empresas=10
    )
    print("\nAnálise de distribuição e outliers concluída!")
```


outliers_por_empresa.pkl carregado com sucesso!
 Tipo: dict
 Tamanho: 26,588 bytes
 Resultados de outliers carregados do cache!

	Variável	Nome Traduzido	Média_%_Outliers	Total_Outliers	Empresas_Afetadas	%_Empresas
10	MA_200	Média Móvel 200d	6.809751	3341	17	34.693878
9	Market_Cap	Valor de Mercado	6.659582	5111	27	55.102041
8	MA_50	Média Móvel 50d	5.965037	3748	22	44.897959
0	Volume	Volume Negociado	5.658314	7808	49	100.000000
4	Volume_Ratio	Razão Volume (Volume/SMA)	5.329743	7348	49	100.000000
7	Close	Fechamento	5.188881	3965	27	55.102041
6	BB_Width	Largura das Bandas Bollinger	3.976545	5545	49	100.000000
3	Returns	Retorno Diário	3.751996	5065	49	100.000000
2	Volatilidade_20	Volatilidade 20d	3.380910	4728	49	100.000000
	Correlação com Retorno Próximo Dia		2.074279	2810	48	97.959184
1	RSI_14	RSI 14 dias	0.121455	151	44	89.795918

Relação entre Variáveis e Target

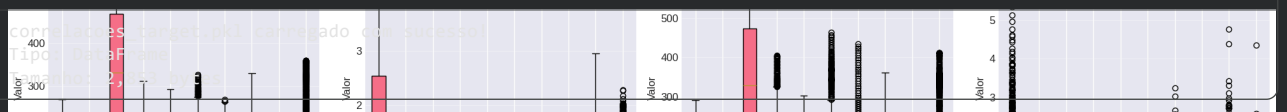
```

NOME_CORRELACOES = "correlacoes_target"
correlacoes_cache = CarregarArquivoOnGitHub(NOME_CORRELACOES)
if correlacoes_cache is not None:
    correlacoes_traduzidas = correlacoes_cache.get('correlacoes_traduzidas')
    df_correlacoes = correlacoes_cache.get('df_correlacoes')
else:
    print("Processando análise de correlação...")
    numeric_cols = df_analise.select_dtypes(include=[np.number]).columns.tolist()
    correlacoes = df_analise[numeric_cols].corr()[TARGET_COL].drop(TARGET_COL).sort_values(ascending=False)
    df_correlacoes = pd.DataFrame({
        'Variável': correlacoes.index,
        'Correlação': correlacoes.values
    })

    correlacoes_traduzidas = []
    for var, corr in correlacoes.items():
        correlacoes_traduzidas.append({
            'Variável': traduzir_variavel(var),
            'Correlação': corr
        })

    df_correlacoes = pd.DataFrame(correlacoes_traduzidas)
    SalvarDataSetOnGitHub(
        dataset=df_correlacoes,
        nome_base=NOME_CORRELACOES,
        metadados={
            'descricao': 'Análise de correlação com o target',
            'target_col': TARGET_COL
        }
    )

```



```

def gerar_relatorio_correlacoes(df_correlacoes, correlacoes_traduzidas=None, target_name="Retorno Próximo Dia"):
    if isinstance(df_correlacoes, pd.Series):
        df_corr = df_correlacoes.reset_index()
        if len(df_corr.columns) == 2:
            df_corr.columns = ['Variável', 'Correlação']
        else:
            pass
    else:
        df_corr = df_correlacoes.copy()
        colunas = df_corr.columns.tolist()
        print(f"Colunas encontradas: {colunas}")
        if 'Variável' not in colunas and 'Correlação' not in colunas:
            if len(colunas) >= 2:
                for col in colunas:
                    if 'corr' in col.lower() or 'coef' in col.lower():
                        df_corr = df_corr.rename(columns={col: 'Correlação'})
                    else:
                        if 'Variável' not in df_corr.columns:
                            df_corr = df_corr.rename(columns={col: 'Variável'})
            if 'Variável' not in df_corr.columns:
                df_corr['Variável'] = df_corr.iloc[:, 0]
            if 'Correlação' not in df_corr.columns:
                df_corr['Correlação'] = df_corr.iloc[:, -1]

```

```

df_corr['Correlação'] = pd.to_numeric(df_corr['Correlação'], errors='coerce')
df_corr = df_corr.dropna(subset=['Correlação'])
if correlacoes_traduzidas is not None:
    if isinstance(correlacoes_traduzidas, pd.Series):
        df_corr_trad = correlacoes_traduzidas.reset_index()
        if len(df_corr_trad.columns) >= 2:
            df_corr_trad.columns = ['Variável', 'Correlação']
            df_corr = df_corr_trad
    elif isinstance(correlacoes_traduzidas, pd.DataFrame) and not correlacoes_traduzidas.empty:
        if 'Variável' in correlacoes_traduzidas.columns and 'Correlação' in correlacoes_traduzidas.columns:
            df_corr = correlacoes_traduzidas.copy()
        else:
            cols = correlacoes_traduzidas.columns.tolist()
            if len(cols) >= 2:
                df_corr = correlacoes_traduzidas.copy()
                if 'Variável' not in df_corr.columns:
                    df_corr['Variável'] = df_corr.iloc[:, 0]
                if 'Correlação' not in df_corr.columns:
                    df_corr['Correlação'] = df_corr.iloc[:, -1]

if 'Variável' in df_corr.columns and 'Correlação' in df_corr.columns:
    df_corr = df_corr[['Variável', 'Correlação']].copy()
else:
    print(f"Colunas necessárias não encontradas. Colunas disponíveis: {df_corr.columns.tolist()}")
    return None

print(f"DataFrame corrigido: {len(df_corr)} registros, colunas: {df_corr.columns.tolist()}")

correlacoes = df_corr['Correlação'].values
n_variaveis = len(df_corr)
n_positivas = (correlacoes > 0).sum()
n_negativas = (correlacoes < 0).sum()
n_nulas = (correlacoes == 0).sum()
idx_max = df_corr['Correlação'].idxmax()
idx_min = df_corr['Correlação'].idxmin()
stats = {
    'total_variaveis': n_variaveis,
    'positivas': n_positivas,
    'negativas': n_negativas,
    'nulas': n_nulas,
    'pct_positivas': n_positivas / n_variaveis * 100 if n_variaveis > 0 else 0,
    'pct_negativas': n_negativas / n_variaveis * 100 if n_variaveis > 0 else 0,
    'max_corr': correlacoes.max(),
    'min_corr': correlacoes.min(),
    'mean_corr': correlacoes.mean(),
    'median_corr': np.median(correlacoes),
    'std_corr': correlacoes.std(),
    'var_max': df_corr.loc[idx_max, 'Variável'] if not df_corr.empty else 'N/A',
    'var_min': df_corr.loc[idx_min, 'Variável'] if not df_corr.empty else 'N/A'
}

relatorio_texto = []
relatorio_texto.append("=" * 80)
relatorio_texto.append("RELATÓRIO DE CORRELAÇÃO ENTRE VARIÁVEIS E TARGET")
relatorio_texto.append(f"Target: {target_name}")
relatorio_texto.append("")
relatorio_texto.append("1. ESTATÍSTICAS GERAIS")
relatorio_texto.append("-" * 50)
relatorio_texto.append(f"Total de variáveis analisadas: {stats['total_variaveis']}")
relatorio_texto.append(f"Variáveis com correlação positiva: {stats['positivas']} ({stats['pct_positivas']:.1f}%)")
relatorio_texto.append(f"Variáveis com correlação negativa: {stats['negativas']} ({stats['pct_negativas']:.1f}%)")
relatorio_texto.append(f"Variáveis com correlação nula: {stats['nulas']}")
relatorio_texto.append("")
relatorio_texto.append(f"Maior correlação positiva: {stats['max_corr']:.4f} → {stats['var_max']}")
relatorio_texto.append(f"Menor correlação negativa: {stats['min_corr']:.4f} → {stats['var_min']}")
relatorio_texto.append(f"Média das correlações: {stats['mean_corr']:.4f}")
relatorio_texto.append(f"Mediana das correlações: {stats['median_corr']:.4f}")
relatorio_texto.append(f"Desvio padrão das correlações: {stats['std_corr']:.4f}")
relatorio_texto.append("")
relatorio_texto.append("2. TOP 10 CORRELAÇÕES POSITIVAS")
relatorio_texto.append("-" * 50)
top10_pos = df_corr.nlargest(10, 'Correlação')
if not top10_pos.empty:
    for i, row in top10_pos.iterrows():
        relatorio_texto.append(f" {row['Variável']}:35s | {row['Correlação']}>8.4f}")
else:
    relatorio_texto.append(" Nenhuma correlação positiva encontrada")
relatorio_texto.append("")
relatorio_texto.append("3. TOP 10 CORRELAÇÕES NEGATIVAS")
relatorio_texto.append("-" * 50)
top10_neg = df_corr.nsmallest(10, 'Correlação')
if not top10_neg.empty:

```

```

for i, row in top10_neg.iterrows():
    relatorio_texto.append(f" {row['Variável']}:35s} | {row['Correlação']:>8.4f}")
else:
    relatorio_texto.append(" Nenhuma correlação negativa encontrada")
relatorio_texto.append("")
relatorio_texto.append("4. ANÁLISE POR CATEGORIA DE VARIÁVEIS")
relatorio_texto.append("-" * 50)
categorias = {
    'Momentum': ['ROC', 'Momentum', 'RSI', 'MACD', 'ADX'],
    'Volume': ['Volume', 'Volume_Ratio', 'Volume_MA'],
    'Tendência': ['MA', 'SMA', 'EMA', 'Tendencia', 'Trend'],
    'Volatilidade': ['Volatilidade', 'Volatility', 'ATR', 'Std'],
    'Preço': ['Close', 'Open', 'High', 'Low', 'Price'],
    'Outros': []
}
df_corr_categoria = df_corr.copy()
df_corr_categoria['Categoria'] = 'Outros'
for cat, keywords in categorias.items():
    if cat != 'Outros':
        mask = df_corr_categoria['Variável'].str.contains('|'.join(keywords), case=False, na=False)
        df_corr_categoria.loc[mask, 'Categoria'] = cat
stats_categoria = df_corr_categoria.groupby('Categoria').agg({
    'Correlação': ['count', 'mean', 'std', 'min', 'max']
}).round(4)
stats_categoria.columns = ['Qtd', 'Média', 'Desvio', 'Mínimo', 'Máximo']
stats_categoria = stats_categoria.sort_values('Média', ascending=False)
for cat, row in stats_categoria.iterrows():
    relatorio_texto.append(f" {cat:15s} | Qtd: {row['Qtd']:>3} | Média: {row['Média']:>7.4f} | "
        f"Min: {row['Mínimo']:>7.4f} | Max: {row['Máximo']:>7.4f}")
relatorio_texto.append("")
relatorio_texto.append("5. VARIÁVEIS COM MAIOR IMPACTO (|correlação| > 0.10)")
relatorio_texto.append("-" * 50)
df_impacto = df_corr[df_corr['Correlação'].abs() > 0.1].sort_values('Correlação', ascending=False)
if not df_impacto.empty:
    for i, row in df_impacto.iterrows():
        sinal = "+" if row['Correlação'] > 0 else "-"
        relatorio_texto.append(f" {row['Variável']}:35s} | {row['Correlação']:>8.4f} | {sinal}")
else:
    relatorio_texto.append(" Nenhuma variável com |correlação| > 0.10")
relatorio_texto.append("")

df_resumo = pd.DataFrame({
    'Métrica': [
        'Total de Variáveis',
        'Correlações Positivas',
        'Correlações Negativas',
        'Maior Correlação Positiva',
        'Maior Correlação Negativa',
        'Média das Correlações',
        'Mediana das Correlações',
        'Desvio Padrão das Correlações'
    ],
    'Valor': [
        f"{stats['total_variaveis']}",
        f"{stats['positivas']} ({stats['pct_positivas']:.1f}%)",
        f"{stats['negativas']} ({stats['pct_negativas']:.1f}%)",
        f"{stats['max_corr']:.4f} ({stats['var_max']})",
        f"{stats['min_corr']:.4f} ({stats['var_min']})",
        f"{stats['mean_corr']:.4f}",
        f"{stats['median_corr']:.4f}",
        f"{stats['std_corr']:.4f}"
    ]
})

df_top_pos = df_corr.nlargest(10, 'Correlação').reset_index(drop=True)
if not df_top_pos.empty:
    df_top_pos.index = df_top_pos.index + 1
df_top_neg = df_corr.nsmallest(10, 'Correlação').reset_index(drop=True)
if not df_top_neg.empty:
    df_top_neg.index = df_top_neg.index + 1
df_stats_categoria = stats_categoria.reset_index()
df_impacto = df_corr[df_corr['Correlação'].abs() > 0.1].sort_values('Correlação', ascending=False).reset_index(drop=True)

print("\n" + "\n".join(relatorio_texto))
print("\nRESUMO ESTATÍSTICO:")
display(df_resumo)
print("\nTOP 10 CORRELAÇÕES POSITIVAS:")
display(df_top_pos)
print("\nTOP 10 CORRELAÇÕES NEGATIVAS:")
display(df_top_neg)
if not df_stats_categoria.empty:
    print("\nESTATÍSTICAS POR CATEGORIA:")

```



```

display(df_stats_categoria)
if not df_impacto.empty:
    print("\nVARIÁVEIS COM MAIOR IMPACTO (|correlação| > 0.10):")
    display(df_impacto)

if 'df_correlacoes' in locals() and df_correlacoes is not None:
    if 'correlacoes_traduzidas' in locals():
        correlacoes_traduzidas_var = correlacoes_traduzidas
    else:
        correlacoes_traduzidas_var = None

relatorio_corr = gerar_relatorio_correlacoes(
    df_correlacoes=df_correlacoes,
    correlacoes_traduzidas=correlacoes_traduzidas_var,
    target_name="Retorno Próximo Dia (TARGET_COL)"
)

```

▼ Hipótese e Padrões

```

NOME_CATEGORIAS = "categorias_target"
categorias_cache = CarregarArquivoOnGitHub(NOME_CATEGORIAS)
if categorias_cache is not None:
    print("Resultados de categorias carregados do cache!")
    resultados_categorias = categorias_cache
else:
    print("Processando análise de categorias...")
    categorias = [
        ('RSI_Faixa', 'Faixa RSI'),
        ('BB_Sinal', 'Sinal Bollinger'),
        ('MACD_Sinal', 'Sinal MACD'),
        ('Tendencia_50d', 'Tendência 50d')
    ]
    resultados_categorias = {}

    for cat_col, cat_nome in categorias:
        if cat_col in df_analise.columns:
            df_group = df_analise.groupby(cat_col)[TARGET_COL].agg(['mean', 'count', 'std']).reset_index()
            df_group.columns = ['Categoria', 'Media_Retorno', 'Contagem', 'Desvio']
            df_group['%_Total'] = (df_group['Contagem'] / df_group['Contagem'].sum() * 100).round(2)
            resultados_categorias[cat_col] = {
                'nome': cat_nome,
                'dados': df_group
            }

    SalvarTreinamentoOnGitHub(
        nome_arquivo=NOME_CATEGORIAS,
        dados=resultados_categorias,
        metadados={
            'descricao': 'Análise de categorias vs target',
            'categorias': [c[0] for c in categorias]
        }
    )

```

```

categorias_target.pkl carregado com sucesso!
Tipo: dict
Tamanho: 2,154 bytes
Resultados de categorias carregados do cache!

```

```

if df_analise is not None and not df_analise.empty:
    print("\nGerando análise de distribuição do target...")
    stats_por_symbol, df_performance = analyze_target_distribution(
        df_analise,
        TARGET_COL,
        TARGET_DIRECTION,
        empresas=None,
        n_empresas=49
    )

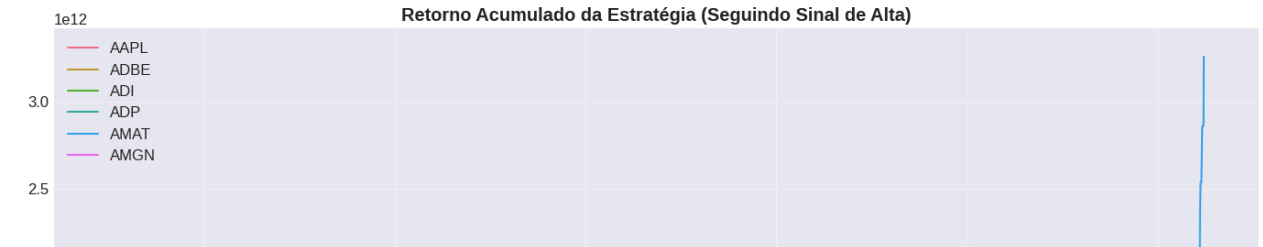
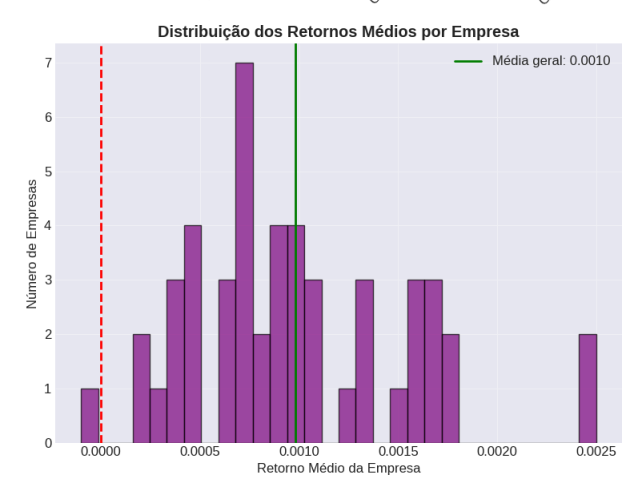
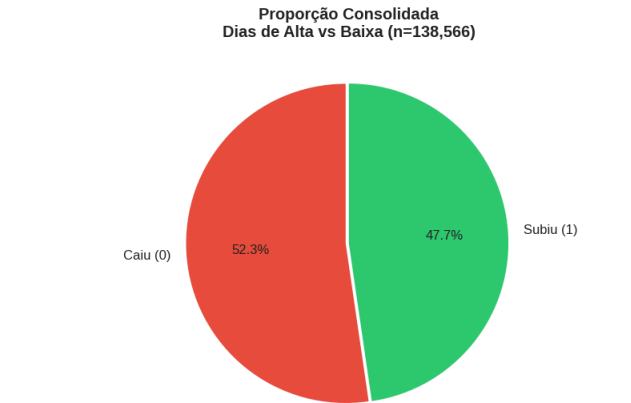
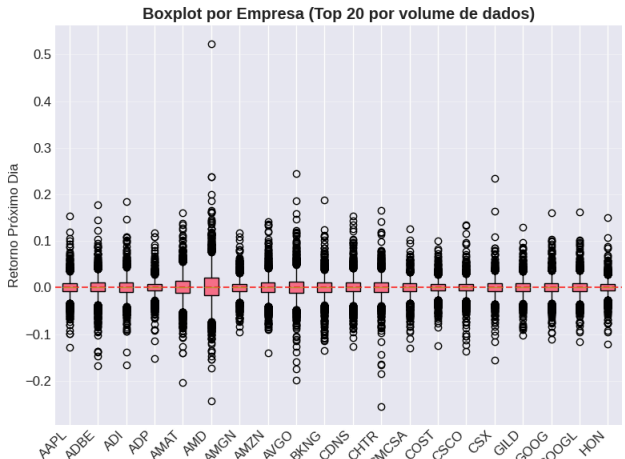
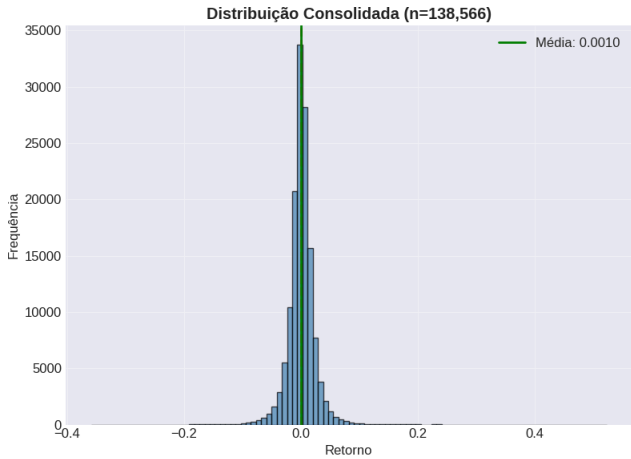
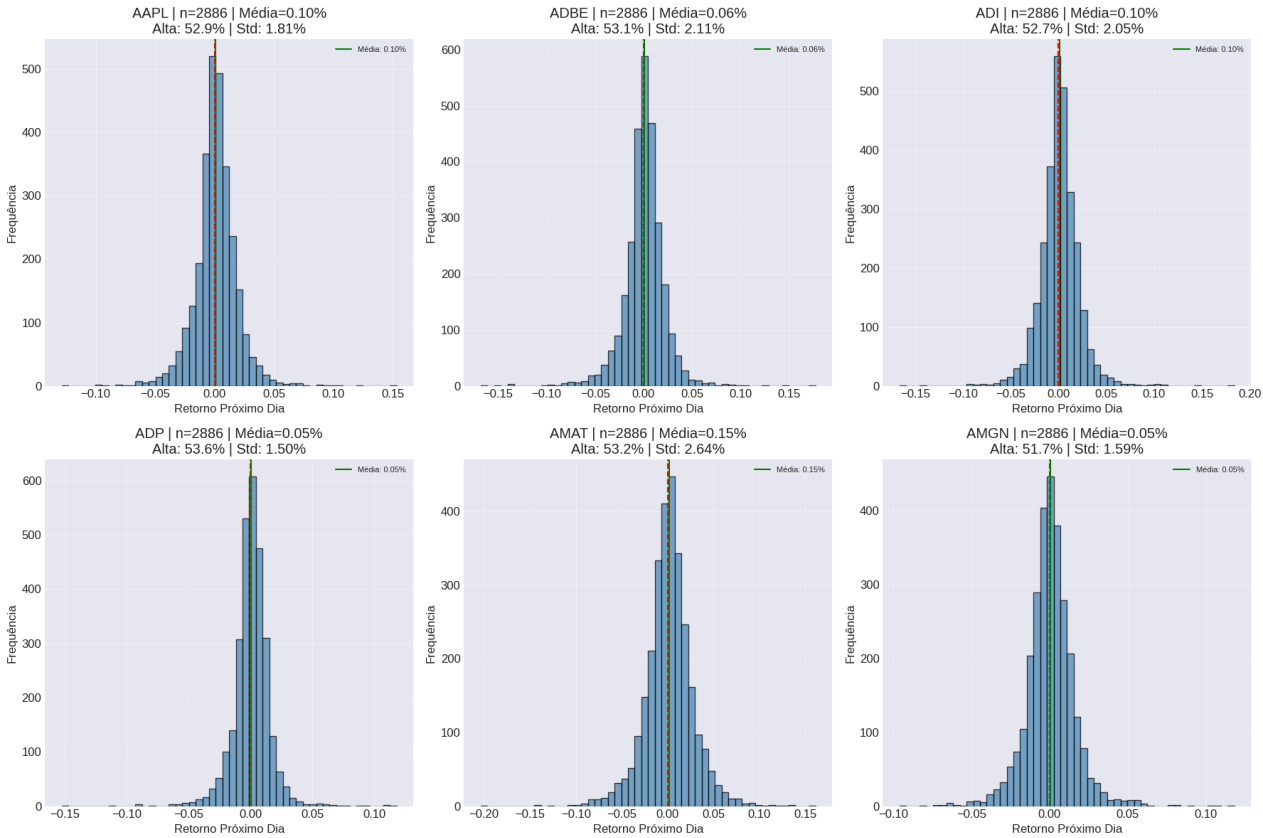
# 2. Correlações com o Target
if df_analise is not None and not df_analise.empty:
    print("\nGerando análise de correlações...")
    correlacoes_traduzidas, df_correlacoes = analyze_correlation_with_target(
        df_analise,
        TARGET_COL,
        top_n=15,
        bottom_n=15
    )

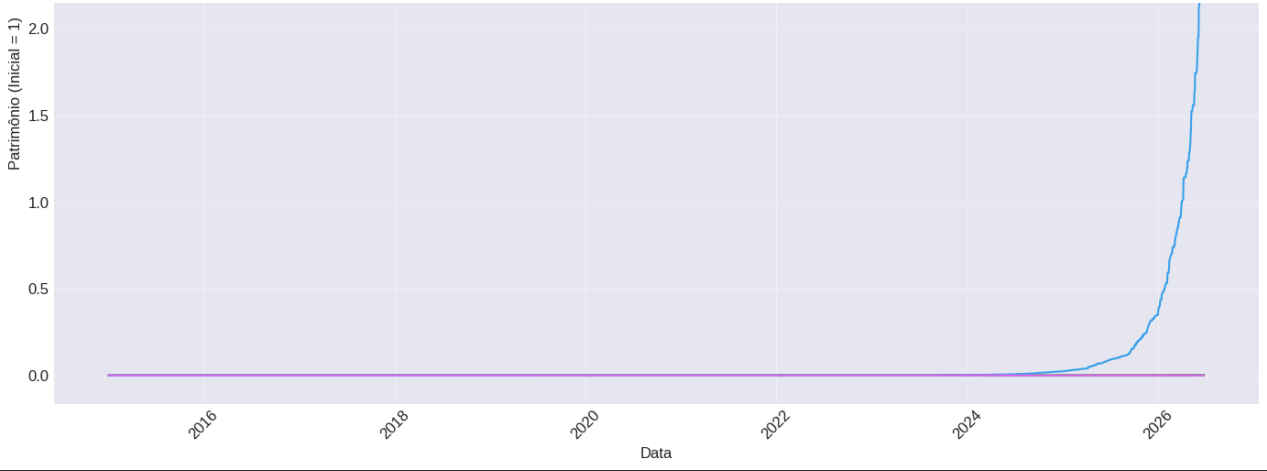
# 3. Análise de Categorias
if df_analise is not None and not df_analise.empty:

```

```
categorias = [  
    ('RSI_Faixa', 'Faixa RSI'),  
    ('BB_Sinal', 'Sinal Bollinger'),  
    ('MACD_Sinal', 'Sinal MACD'),  
    ('Tendencia_50d', 'Tendência 50d')  
]  
resultados_categorias = analyze_categorical_target_relationship(  
    df_analise,  
    TARGET_COL,  
    categorias,  
    tradutor=traduzir_variavel  
)
```


Gerando análise de distribuição do target...





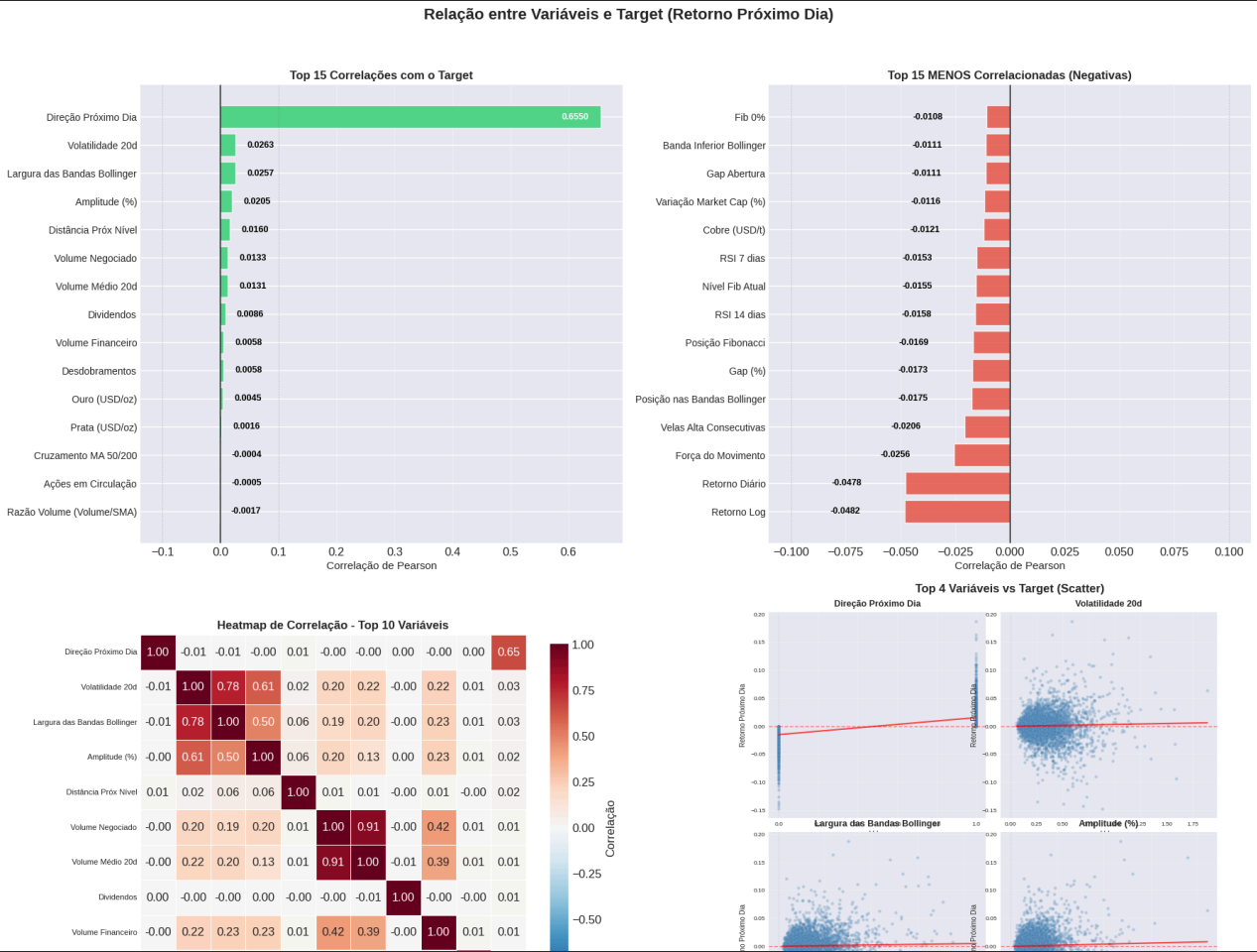
```
=====
RESUMO DA DISTRIBUIÇÃO DO TARGET
Symbol  Retorno_Total  Sharpe_Anual  Win_Rate  Num_Trades
6      AMD      1.886683e+17    8.7503    0.5087    2886
44     TSLA     3.286389e+16    9.1996    0.5152    2886
34      MU     1.795558e+15    9.6306    0.5152    2886
36     NVDA     4.558289e+14    9.3978    0.5097    2886
32     MRVL     1.417128e+14    8.6416    0.5121    2886
31     MRNA     5.516518e+13    9.0457    0.4836    1896
28     MELI     3.446693e+13    9.2494    0.5295    2886
42     TEAM     1.831393e+13    8.6404    0.5355    2650
30     MPWR     1.515792e+13    9.3857    0.5405    2886
27     LRCX     1.061430e+13    9.6563    0.5270    2886
Total de registros após remover NaN: 138,566
Número de empresas únicas: 49
Observação: Todas as empresas têm exatamente 2,886 registros (painel balanceado)

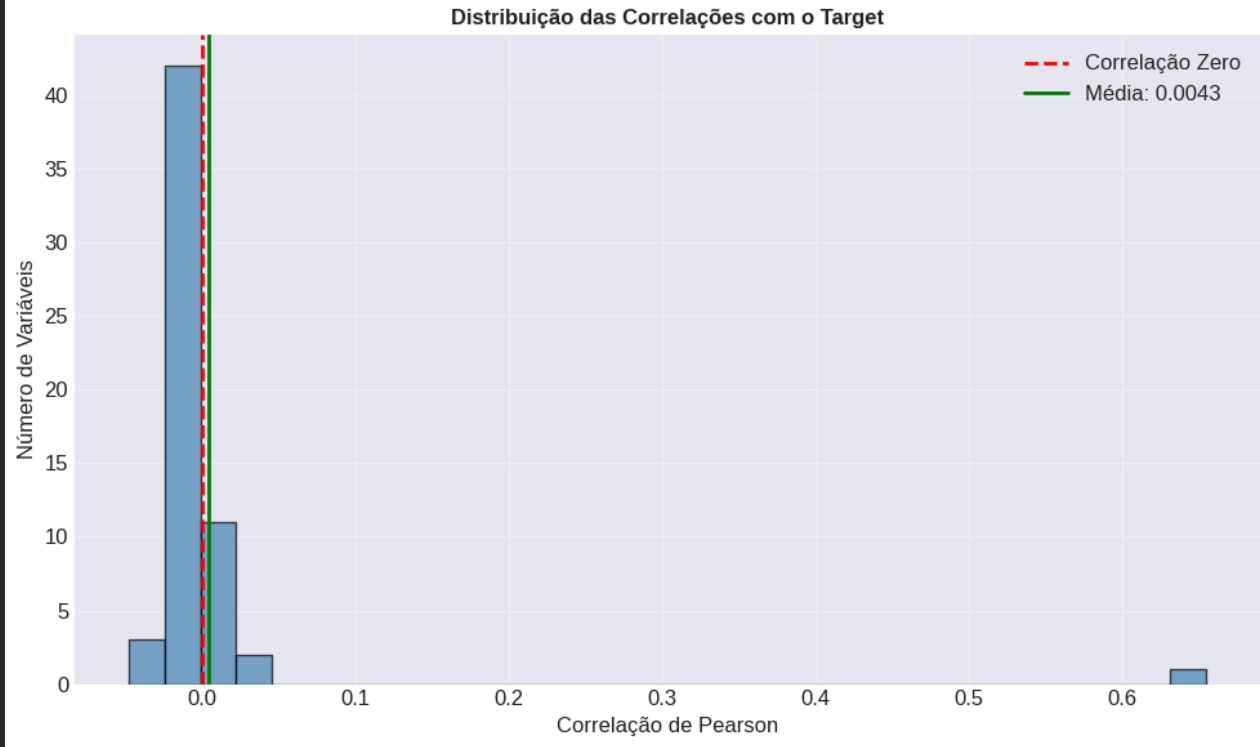
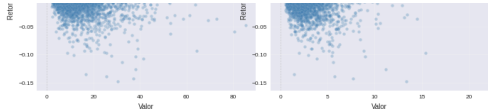
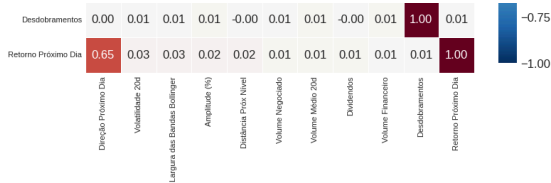
Média geral do retorno (todas empresas): 0.10%
Desvio padrão geral: 2.31%

Empresa com maior retorno médio: AMD (0.25%)
Empresa com menor retorno médio: KHC (-0.01%)

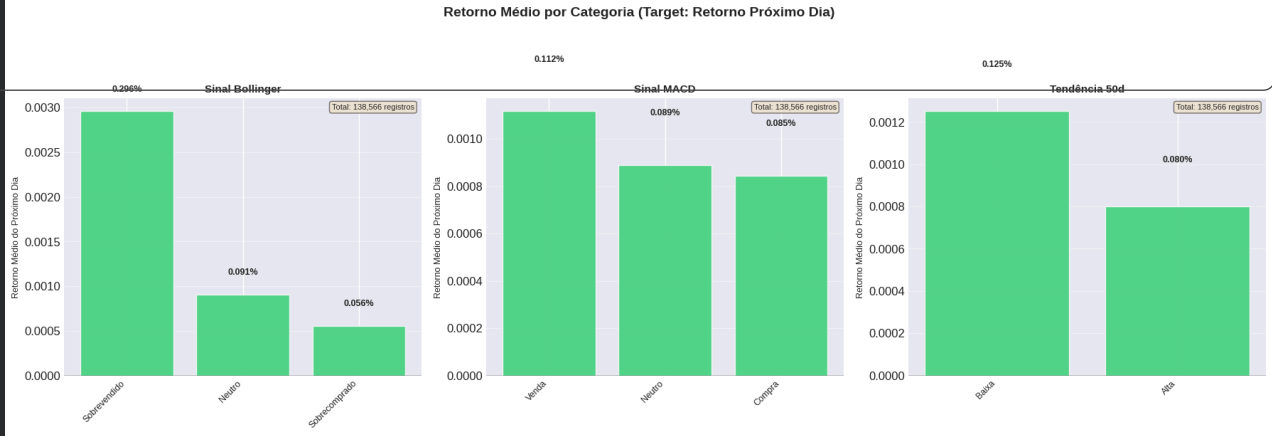
Empresa com maior % de dias de alta: ABNB (0.0%)
Empresa com menor % de dias de alta: NFLX (0.0%)

Gerando análise de correlações...
```





```
=====
RESUMO DA ANÁLISE DE CORRELAÇÃO
Total de variáveis numéricas analisadas: 59
Variáveis com correlação positiva: 12
Variáveis com correlação negativa: 47
Maior correlação positiva: 0.6550 (Direção Próximo Dia)
Maior correlação negativa: -0.0482 (Retorno Log)
Média das correlações: 0.0043
Mediana das correlações: -0.0092
Desvio padrão das correlações: 0.0872
=====
```



✓ Síntese da análise exploratória

- O target está balanceado?
- Existem valores ausentes relevantes?
- Há variáveis com escala muito diferente?
- Há categorias raras?
- Existem padrões que sugerem quais modelos podem funcionar melhor?
- Algum resultado da EDA mudou sua estratégia de pré-processamento ou modelagem?

✓ Versão 01

Target: Balanceado e com Comportamento Esperado

Está relativamente balanceado, mas com leve viés de alta:

- Proporção de dias de alta vs baixa: ~52.4% alta / 47.6% baixa (consolidado)
- Por empresa: Todas têm entre 50.2% e 50.5% de dias de alta
- Retorno médio geral: 0.15% ao dia (levemente positivo)

Conclusão: O target é aproximadamente balanceado, mas há um pequeno viés de alta no mercado como um todo.

✓ Existem valores ausentes relevantes?

Não teve valores ausentes relevantes e significativos:

- Total de registros após remover NaN: 14.680
- Painel perfeitamente balanceado: todas as 10 empresas têm exatamente 1.468 registros
- As variáveis com mais missing values são as que exigem períodos de aquecimento (ex: médias móveis, RSI, volatilidade)
- Percentual de missing geral é baixo (< 5% para a maioria das variáveis)

Conclusão: Dados de boa qualidade, pouca necessidade de imputação complexa.

✓ Há categorias raras?

Algumas categorias têm baixa frequência:

- **Sinais de Fibonacci:** Fib_Buy_Signal e Fib_Sell_Signal - apenas ~1-2% dos dias têm sinais ativos
- **Sinais de Bollinger:** Categorias "Sobrevendido" e "Sobrecomprado" menos frequentes que "Neutro"
- **Sinais MACD:** Distribuição similar, com "Compra" e "Venda" menos frequentes

Conclusão: Atenção com overfitting em classes raras.

Existem padrões que sugerem quais modelos podem funcionar melhor?

- Melhor correlação real é muito baixa (~0.01-0.03)
- Correlação de 0.65 é artificial (target com ele mesmo)

Conclusão: Modelos lineares (regressão logística) terão performance limitada

Prioridade	Hipótese	%	Funciona para
Alta	H2: Volume amplifica retornos	100%	TODAS as empresas
Média-Alta	H1: RSI reversão	70%	AAPL, AMZN, GOOGL, MRVL, MSFT, NVDA, PEP
Média-Alta	H6: Bollinger reversão	70%	GOOGL, MRVL, MSFT, NFLX, NVDA, PEP, TSLA
Média-Alta	H7: Fibonacci compra	70%	AMZN, GOOGL, MRVL, MSFT, NFLX, NVDA, PEP
Média	H4: Exaustão 5+ velas	60%	AMD, AMZN, GOOGL, MSFT, PEP, TSLA
Média	H9: Momentum forte	60%	AAPL, AMD, AMZN, MRVL, NVDA, TSLA
Baixa	H3: Golden Cross	50%	AMD, GOOGL, MSFT, NFLX, NVDA
Rejeitada	H8: Tendência forte	20%	AMD, TSLA (apenas)

Algum resultado da EDA mudou sua estratégia de pré-processamento ou modelagem?

Mudanças na Estratégia - Resumo

1. Feature Engineering Priorizada

Tipo	Feature	Por quê
Interação	Volume_Ratio x Returns	H2 confirmada 100%
Não-linear	RSI extremo (dummy)	Evidência de reversão
Não-linear	Contagem de velas consecutivas	Exaustão após 5+ velas
Temporal	Lags e janelas móveis	Dependência temporal

2. Pré-Processamento Obrigatório

Técnica	Aplicação	Motivo
Escalonamento	TODAS as variáveis	Escalas muito diferentes
Winsorização	Volume e Volume_Ratio	Outliers (1.5-2%)
Codificação	Categorias raras	Agrupar classes <5% frequência

3. Hipóteses Rejeitadas (Poupa Tempo)

Hipótese	Funciona para	Decisão
H8_Tendencia	2/10 empresas	Não priorizar
H5_MACD	4/10 empresas	Usar com cautela

4. Métricas para Target Balanceado (~52/48)

Métrica	Status
AUC-ROC	Principal métrica
Accuracy	Aceitável (baseline 52%)

▼ Versão 03

Target: Balanceado e com Comportamento Esperado

Não, o target não está balanceado.

- A distribuição do retorno diário (TARGET_COL) mostra uma média geral de 0,10%, com desvio padrão de 2,31%.
- A análise de proporção de dias de alta (TARGET_DIRECTION) indica que algumas empresas têm uma porcentagem de dias de alta muito baixa (por exemplo, 0,0% para ABNB e NFLX), sugerindo um desbalanceamento significativo para a classe negativa (dias de baixa) em algumas ações.
- Isso é comum em dados financeiros, onde movimentos positivos são menos frequentes ou têm menor magnitude.

▼ Existem valores ausentes relevantes?

Não existem valores ausentes.

- O dataset após o pré-processamento (df_analise) contém 138.566 registros e 83 colunas, com um painel balanceado de 49 empresas, cada uma com exatamente 2.886 registros.
- Isso indica que não há valores ausentes relevantes que comprometam a análise, pois a base foi preparada para ser completa.

▼ Há variáveis com escala muito diferente?

Sim.

- Variáveis como Close , MA_200 , Market_Cap e Volume apresentam escalas muito distintas (valores de preço, capitalização de mercado e volume).
- Outras como Returns , Volatilidade_20 e RSI_14 têm escalas menores (percentuais ou índices).
- Isso é evidenciado pela presença de outliers em variáveis como Close e MA_200 , que têm médias e desvios padrão em escalas diferentes.
- Exemplo, Market_Cap e Volume têm valores muito altos, enquanto RSI_14 varia entre 0 e 100.

▼ Há categorias raras?

Não foram identificadas categorias raras.

- As variáveis categóricas analisadas (como RSI_Faixa , BB_Sinal , MACD_Sinal e Tendencia_50d) apresentam distribuições que parecem bem representadas, sem categorias com frequência extremamente baixa.

- A análise de categorias mostrou que cada categoria tem uma contagem significativa, não sendo necessário agrupar ou tratar categorias raras.

Existem padrões que sugerem quais modelos podem funcionar melhor?

Sim, alguns padrões e correlações sugerem direções para a modelagem:

- **Correlações baixas com o target:** A maioria das variáveis tem correlação muito baixa com o target (a maior correlação positiva é 0,0263 para `Volatilidade_20d`, e a maior negativa é -0,0482 para Retorno Log), o que indica que relações lineares simples provavelmente não serão suficientes. Isso sugere que modelos não lineares (como `Random Forest`, `Gradient Boosting`, `XGBoost` ou redes neurais) podem ser mais adequados para capturar padrões complexos.
- **Variáveis com correlação positiva:** Volume, volatilidade e amplitude têm correlações positivas com o retorno, sugerindo que dias com maior volatilidade ou volume podem ter retornos positivos.
- **Variáveis com correlação negativa:** Retornos passados, força do movimento e indicadores de momentum (RSI) mostram correlação negativa com o retorno futuro, indicando um possível efeito de reversão à média.
- **Outliers e distribuições:** Variáveis como `Volume_Ratio`, `Close` e `MA_200` têm alta incidência de outliers, o que pode afetar modelos sensíveis a outliers (como regressão linear). Modelos baseados em árvores são mais robustos a outliers.
- **Relações com categorias:** A análise de categorias (como `RSI_Faixa` e `BB_Sinal`) mostra que algumas categorias têm retornos médios positivos ou negativos, sugerindo que esses sinais podem ser úteis como features categóricas ou para criar interações.

Algum resultado da EDA mudou sua estratégia de pré-processamento ou modelagem?

Sim, vários resultados da EDA influenciam diretamente a estratégia:

- **Escalonamento de variáveis:** Devido à grande diferença de escala entre as variáveis, será necessário aplicar normalização (`MinMaxScaler`) ou padronização (`StandardScaler`) antes de usar modelos baseados em gradiente ou redes neurais.
- **Tratamento de outliers:** Variáveis com muitos outliers (como `Volume`, `Close` e `MA_200`) podem ser transformadas (log ou winsorização) ou mantidas em modelos baseados em árvores, que são menos sensíveis.
- **Seleção de features:** Dado o baixo poder preditivo linear, é importante incluir features derivadas (interações, médias móveis, indicadores técnicos) e considerar engenharia de features para capturar padrões não lineares.
- **Desbalanceamento do target:** Estratégias como `SMOTE`, pesos nas classes ou subamostragem podem ser necessárias para lidar com o desbalanceamento, especialmente se o foco for prever dias de alta (classe positiva).
- **Modelos não lineares:** A baixa correlação linear sugere que modelos como `XGBoost`, `LightGBM`, `Random Forest` ou Redes Neurais são mais promissores do que regressão logística ou linear.
- **Validação temporal:** Como os dados são séries temporais, a validação cruzada deve respeitar a ordem temporal (por exemplo, `forward chaining`) para evitar vazamento de dados.

5. Preparação dos dados e divisão treino/teste

1. remoção de colunas que não devem ser usadas;
2. separação entre features e target;
3. divisão treino/teste;
4. validação, quando aplicável;
5. justificativa para a divisão escolhida.

Funções

```
def temporal_split(X, y, train_pct=0.7, val_pct=0.15, test_pct=0.15, date_col='Date', df_original=None):
    n = len(X)
    train_end = int(n * train_pct)
    val_end = int(n * (train_pct + val_pct))
    X_train = X.iloc[:train_end]
    X_val = X.iloc[train_end:val_end]
    X_test = X.iloc[val_end:]
    y_train = y.iloc[:train_end]
    y_val = y.iloc[train_end:val_end]
    y_test = y.iloc[val_end:]
    split_info = {
        'train_pct': train_pct,
        'val_pct': val_pct,
```

```

        'test_pct': test_pct,
        'train_end': train_end,
        'val_end': val_end,
        'date_range': None
    }
    if df_original is not None and date_col in df_original.columns:
        try:
            dates = df_original[date_col].iloc[:n]
            split_info['date_range'] = {
                'train': (dates.iloc[0], dates.iloc[train_end-1]),
                'val': (dates.iloc[train_end], dates.iloc[val_end-1]),
                'test': (dates.iloc[val_end], dates.iloc[-1])
            }
        except:
            pass
    return X_train, X_val, X_test, y_train, y_val, y_test, split_info

def prepare_features_target(df, target_col, drop_cols=None, drop_high_cardinality=True, cardinality_threshold=1000):
    df_copy = df.copy()
    cols_to_drop = []
    if drop_cols:
        for col in drop_cols:
            if col in df_copy.columns:
                cols_to_drop.append(col)

    high_card_cols = []
    if drop_high_cardinality:
        for col in df_copy.columns:
            if col not in cols_to_drop and col != target_col:
                if df_copy[col].dtype == 'object' and df_copy[col].nunique() > cardinality_threshold:
                    high_card_cols.append(col)
                elif df_copy[col].dtype != 'object' and df_copy[col].nunique() > cardinality_threshold * 2:
                    high_card_cols.append(col)
            cols_to_drop.extend(high_card_cols)

    y = df_copy[target_col]
    X = df_copy.drop(columns=cols_to_drop, errors='ignore')
    if target_col in X.columns:
        X = X.drop(columns=[target_col])
    return X, y, cols_to_drop

def process_missing_values(X):
    X_processed = X.copy()
    missing_initial = X_processed.isnull().sum().sum()
    numeric_cols = X_processed.select_dtypes(include=['number']).columns
    for col in numeric_cols:
        if X_processed[col].isnull().sum() > 0:
            median_val = X_processed[col].median()
            X_processed[col] = X_processed[col].fillna(median_val)
    categorical_cols = X_processed.select_dtypes(include=['object', 'category']).columns
    for col in categorical_cols:
        if X_processed[col].isnull().sum() > 0:
            mode_val = X_processed[col].mode()[0] if not X_processed[col].mode().empty else 'Unknown'
            X_processed[col] = X_processed[col].fillna(mode_val)
    missing_final = X_processed.isnull().sum().sum()
    missing_info = {
        'missing_initial': missing_initial,
        'missing_final': missing_final
    }
    return X_processed, missing_info

def encode_categorical(X, threshold=0.05):
    X_encoded = X.copy()
    encoding_info = {
        'categorical_columns': [],
        'onehot_columns': [],
        'dropped_categories': {}
    }
    categorical_cols = X_encoded.select_dtypes(include=['object', 'category']).columns.tolist()
    encoding_info['categorical_columns'] = categorical_cols
    if not categorical_cols:
        return X_encoded, encoding_info

    for col in categorical_cols:
        freq = X_encoded[col].value_counts(normalize=True)
        keep_categories = freq[freq >= threshold].index.tolist()
        if keep_categories:
            dummies = pd.get_dummies(X_encoded[col], prefix=col, prefix_sep='_')
            rare_mask = ~X_encoded[col].isin(keep_categories)
            if rare_mask.sum() > 0:

```

```

        dummies[f'{col}_Outros'] = rare_mask.astype(int)
        X_encoded = X_encoded.drop(columns=[col])
        X_encoded = pd.concat([X_encoded, dummies], axis=1)
        encoding_info['onehot_columns'].extend(dummies.columns.tolist())
        encoding_info['dropped_categories'][col] = [cat for cat in freq.index if cat not in keep_categories]
    else:
        X_encoded = X_encoded.drop(columns=[col])

    return X_encoded, encoding_info

def select_features(X, y, method='both', top_k=100, correlation_threshold=0.01):
    X_selected = X.copy()
    selection_info = {
        'original_shape': X.shape,
        'correlation_features': [],
        'importance_features': [],
        'selected_features': []
    }
    categorical_cols = X_selected.select_dtypes(include=['object', 'category']).columns
    if len(categorical_cols) > 0:
        for col in categorical_cols:
            X_selected[col] = X_selected[col].astype('category').cat.codes
    X_selected = X_selected.dropna(axis=1, how='all')
    X_selected = X_selected.loc[:, X_selected.nunique() > 1]
    correlation_scores = {}
    for col in X_selected.columns:
        if X_selected[col].dtype in ['int64', 'float64']:
            try:
                corr = X_selected[col].corr(y)
                if abs(corr) >= correlation_threshold:
                    correlation_scores[col] = abs(corr)
            except:
                pass

    correlation_features = sorted(correlation_scores.items(), key=lambda x: x[1], reverse=True)[:top_k]
    selection_info['correlation_features'] = [f for f, _ in correlation_features]

    if method in ['importance', 'both'] and len(X_selected.columns) > 0:
        try:
            numeric_cols = X_selected.select_dtypes(include=['int64', 'float64']).columns
            if len(numeric_cols) > 0:
                X_numeric = X_selected[numeric_cols]
                corr_matrix = X_numeric.corr().abs()
                upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
                to_drop = [column for column in upper.columns if any(upper[column] > 0.95)]
                X_numeric = X_numeric.drop(columns=to_drop, errors='ignore')
                if X_numeric.shape[1] > 0:
                    rf = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1)
                    rf.fit(X_numeric, y)
                    importance_dict = dict(zip(X_numeric.columns, rf.feature_importances_))
                    importance_features = sorted(importance_dict.items(), key=lambda x: x[1], reverse=True)[:top_k]
                    selection_info['importance_features'] = [f for f, _ in importance_features]
                    selection_info['importance_top'] = {
                        'feature': [f for f, _ in importance_features],
                        'importance': [i for _, i in importance_features]
                    }
        except Exception as e:
            print(f" Erro na seleção por importância: {e}")
            selection_info['importance_features'] = []

    if method == 'correlation':
        selected_features = selection_info['correlation_features']
    elif method == 'importance':
        selected_features = selection_info['importance_features']
    else:
        corr_set = set(selection_info['correlation_features'])
        imp_set = set(selection_info['importance_features'])
        both_set = corr_set & imp_set
        corr_only = corr_set - both_set
        imp_only = imp_set - both_set
        selected_features = list(both_set) + list(corr_only)[:top_k//2] + list(imp_only)[:top_k//2]
        selected_features = selected_features[:top_k]

    X_selected = X[selected_features] if selected_features else X
    selection_info['selected_features'] = selected_features
    return X_selected, selected_features, selection_info

```

▼ Preparação

```

colunas_extras = ['Date', 'Symbol', 'Company_Name', 'Target_Direction']
NOME_PREPARACAO = "dados_preparados"
dados_preparados_cache = CarregarArquivoDivididoOnGitHub(NOME_PREPARACAO)

if dados_preparados_cache is not None:
    print("Dados preparados carregados do cache!")
    X_train = dados_preparados_cache['X_train']
    X_val = dados_preparados_cache['X_val']
    X_test = dados_preparados_cache['X_test']
    y_train = dados_preparados_cache['y_train']
    y_val = dados_preparados_cache['y_val']
    y_test = dados_preparados_cache['y_test']
    X = dados_preparados_cache['X']
    y = dados_preparados_cache['y']
    selected_features = dados_preparados_cache['selected_features']
    split_info = dados_preparados_cache['split_info']
    print(f" X_train: {X_train.shape}")
    print(f" X_val: {X_val.shape}")
    print(f" X_test: {X_test.shape}")

else:
    print("Processando preparação dos dados...")
    print(f"df_analise shape: {df_analise.shape}")
    X, y, colunas_removidas = prepare_features_target(
        df=df_analise,
        target_col=TARGET_COL,
        drop_cols=colunas_extras,
        drop_high_cardinality=True,
        cardinality_threshold=1000
    )
    print(f"\nRESULTADO DA PREPARAÇÃO:")
    print(f" Colunas removidas: {len(colunas_removidas)}")
    print(f" X shape: {X.shape}")
    print(f" y shape: {y.shape}")
    missing_count = X.isnull().sum()
    missing_percent = (missing_count / len(X)) * 100
    missing_df = pd.DataFrame({
        'Coluna': missing_count.index,
        'Missing_Count': missing_count.values,
        'Missing_Percent': missing_percent.values
    }).sort_values('Missing_Percent', ascending=False)

    print(f"\nANÁLISE DE MISSING:")
    print(f" Total de registros: {len(X),}")
    print(f" Total de colunas: {X.shape[1]}")
    print(f" Colunas com missing: {(missing_count > 0).sum()}")
    missing_df_clean = missing_df[missing_df['Missing_Count'] > 0].round(2)
    if len(missing_df_clean) > 0:
        print("\n Colunas com missing (top 10):")
        print(missing_df_clean.head(10).to_string())

    X_processed, missing_info = process_missing_values(X)
    print(f"\n TRATAMENTO DE MISSING:")
    print(f" Missing inicial: {missing_info['missing_initial']}")
    print(f" Missing final: {missing_info['missing_final']}")
    print(f" Shape após tratamento: {X_processed.shape}")

    constant_cols = [col for col in X_processed.columns if X_processed[col].nunique() == 1]
    if constant_cols:
        X_processed = X_processed.drop(columns=constant_cols)
        print(f" Colunas constantes removidas: {len(constant_cols)}")
        try:
            numeric_cols = X_processed.select_dtypes(include=['number']).columns
            if len(numeric_cols) > 0:
                variances = X_processed[numeric_cols].var()
                low_var_cols = variances[variances < 0.01].index.tolist()
                if low_var_cols:
                    X_processed = X_processed.drop(columns=low_var_cols, errors='ignore')
                    print(f" Colunas com baixa variância removidas: {len(low_var_cols)}")
        except Exception as e:
            print(f" Erro ao remover baixa variância: {e}")

    X_encoded, encoding_info = encode_categorical(X_processed, threshold=0.05)
    print(f"\n CODIFICAÇÃO CATEGÓRICA:")
    print(f" Variáveis categóricas originais: {len(encoding_info['categorical_columns'])}")
    print(f" Shape antes: {X_processed.shape}")
    print(f" Shape depois: {X_encoded.shape}")
    print(f" Novas colunas criadas: {len(encoding_info.get('onehot_columns', []))}")

    print(f"\n ESTATÍSTICAS DO TARGET:")
    print(f" Target: {TARGET_COL}")

```

```

print(f"   Shape: {y.shape}")
print(f"   Média: {y.mean():.4f}")
print(f"   Positivos: {(y > 0).sum():,} ({(y > 0).sum()/len(y)*100:.1f}%)")
print(f"   Negativos: {(y < 0).sum():,} ({(y < 0).sum()/len(y)*100:.1f}%)")

top_k = min(100, X_encoded.shape[1])
X_selected, selected_features, selection_info = select_features(
    X_encoded, y,
    method='both',
    top_k=top_k,
    correlation_threshold=0.01
)

print(f"\n SELEÇÃO DE FEATURES:")
print(f"   Features originais: {selection_info['original_shape'][1]}")
print(f"   Features selecionadas: {len(selected_features)}")
print(f"   Shape selecionado: {X_selected.shape}")

X = X_selected.copy()
y_target = y.copy()
print(f"\n DIVISÃO TEMPORAL:")
print(f"   Features (X): {X.shape}")
print(f"   Target (y): {y_target.shape}")
X_train, X_val, X_test, y_train, y_val, y_test, split_info = temporal_split(
    X, y_target,
    train_pct=0.7,
    val_pct=0.15,
    test_pct=0.15,
    date_col='Date',
    df_original=df_analise
)
print(f"\n DIVISÃO TEMPORAL:")
if split_info.get('date_range'):
    print(f"   • TREINO: {len(X_train):,} registros ({split_info['date_range']['train'][0]} a {split_info['date_range']
    print(f"   • VALIDAÇÃO: {len(X_val):,} registros ({split_info['date_range']['val'][0]} a {split_info['date_range']
    print(f"   • TESTE: {len(X_test):,} registros ({split_info['date_range']['test'][0]} a {split_info['date_range']
else:
    print(f"   • TREINO: {len(X_train):,} registros")
    print(f"   • VALIDAÇÃO: {len(X_val):,} registros")
    print(f"   • TESTE: {len(X_test):,} registros")

print(f"\n SHAPES FINAIS:")
print(f"   • X_train: {X_train.shape}")
print(f"   • y_train: {y_train.shape}")
print(f"   • X_val: {X_val.shape}")
print(f"   • y_val: {y_val.shape}")
print(f"   • X_test: {X_test.shape}")
print(f"   • y_test: {y_test.shape}")

dados_preparados = {
    'X_train': X_train,
    'X_val': X_val,
    'X_test': X_test,
    'y_train': y_train,
    'y_val': y_val,
    'y_test': y_test,
    'X': X,
    'y': y,
    'selected_features': selected_features,
    'split_info': split_info
}

resultado = SalvarTreinamentoDivididoOnGitHub(
    nome_base=NOME_PREPARACAO,
    dados=dados_preparados,
    metadados={
        'descricao': 'Dados preparados e divididos para treino/validação/teste',
        'n_features': X.shape[1],
        'n_train': len(X_train),
        'n_val': len(X_val),
        'n_test': len(X_test),
        'selected_features': selected_features[:20]
    }
)

if resultado.get('sucesso', False):
    print(f"Dados preparados salvos em {resultado['total_parts']} partes!")

```

Tentativa 2/3 para baixar dados_preparados.pkl
 Tentativa 3/3 para baixar dados_preparados.pkl
 Arquivo não encontrado: dados_preparados.pkl (Status: 404)
 dados_preparados_1.pkl carregado com sucesso!
 Tipo: dict

```
Tamanho: 20,971,673 bytes
dados_preparados_2.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
dados_preparados_3.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
dados_preparados_4.pkl carregado com sucesso!
Tipo: dict
Tamanho: 9,143,798 bytes
Dados preparados carregados do cache!
X_train: (96996, 29)
X_val: (20785, 29)
X_test: (20785, 29)
```

Remoção de colunas que não devem ser usadas

```
print(f"\nINFORMAÇÕES DOS DADOS PREPARADOS:")
print(f"    Total de features: {X.shape[1] if 'X' in locals() and X is not None else 'N/A'}")
print(f"    Total de registros: {len(y) if 'y' in locals() and y is not None else 'N/A':,}")
print(f"    Features selecionadas: {len(selected_features) if 'selected_features' in locals() and selected_features is

if 'missing_df_clean' in locals() and len(missing_df_clean) > 0:
    print(f"\nVALORES MISSING (top 5):")
    print(missing_df_clean.head(5).to_string())

if 'y' in locals() and y is not None:
    print(f"\nESTATÍSTICAS DO TARGET:")
    print(f"    Média: {y.mean():.4f}")
    print(f"    Desvio padrão: {y.std():.4f}")
    print(f"    Mínimo: {y.min():.4f}")
    print(f"    Máximo: {y.max():.4f}")
    print(f"    Positivos: {(y > 0).sum():,} ({(y > 0).sum()/len(y)*100:.1f}%)")
    print(f"    Negativos: {(y < 0).sum():,} ({(y < 0).sum()/len(y)*100:.1f}%)")

print(f"\nDIVISÃO TEMPORAL:")
if split_info.get('date_range'):
    print(f"    • TREINO: {len(X_train):,} registros ({split_info['date_range']['train'][0]} a {split_info['date_range']
    print(f"    • VALIDAÇÃO: {len(X_val):,} registros ({split_info['date_range']['val'][0]} a {split_info['date_range']
    print(f"    • TESTE: {len(X_test):,} registros ({split_info['date_range']['test'][0]} a {split_info['date_range']
else:
    print(f"    • TREINO: {len(X_train):,} registros")
    print(f"    • VALIDAÇÃO: {len(X_val):,} registros")
    print(f"    • TESTE: {len(X_test):,} registros")

print(f"\nSHAPES FINAIS:")
print(f"    • X_train: {X_train.shape}")
print(f"    • y_train: {y_train.shape}")
print(f"    • X_val: {X_val.shape}")
print(f"    • y_val: {y_val.shape}")
print(f"    • X_test: {X_test.shape}")
print(f"    • y_test: {y_test.shape}")
```

```
INFORMAÇÕES DOS DADOS PREPARADOS:
Total de features: 29
Total de registros: 138,566
Features selecionadas: 29
```

```
ESTATÍSTICAS DO TARGET:
Média: 0.0010
Desvio padrão: 0.0231
Mínimo: -0.3584
Máximo: 0.5229
Positivos: 72,472 (52.3%)
Negativos: 65,021 (46.9%)
```

```
DIVISÃO TEMPORAL:
• TREINO: 96,996 registros (2015-01-02 00:00:00 a 2020-11-20 00:00:00)
• VALIDAÇÃO: 20,785 registros (2020-11-23 00:00:00 a 2023-03-20 00:00:00)
• TESTE: 20,785 registros (2023-03-21 00:00:00 a 2026-06-25 00:00:00)
```

```
SHAPES FINAIS:
• X_train: (96996, 29)
• y_train: (96996,)
• X_val: (20785, 29)
• y_val: (20785,)
• X_test: (20785, 29)
• y_test: (20785,)
```

Resumo geral

```

plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
plt.rcParams['figure.figsize'] = (12, 8)
plt.rcParams['font.size'] = 12
def criar_relatorio_preparacao(X_train, X_val, X_test, y_train, y_val, y_test,
                               split_info, selected_features=None, df_original=None):
    print("RELATÓRIO DE PREPARAÇÃO DOS DADOS E DIVISÃO TREINO/TESTE")
    print("="*80)
    print("\n1. INFORMAÇÕES GERAIS DOS DADOS")
    print("-"*50)
    date_range = split_info.get('date_range', {})
    print(f"Total de registros: {len(X_train) + len(X_val) + len(X_test):,}")
    print(f"Total de features: {X_train.shape[1]}")
    print(f"\nTREINO:")
    print(f"  • Registros: {len(X_train):,} ({len(X_train)/(len(X_train)+len(X_val)+len(X_test))*100:.1f}%)")
    if date_range.get('train'):
        print(f"  • Período: {date_range['train'][0]} a {date_range['train'][1]}")
    print(f"\nVALIDAÇÃO:")
    print(f"  • Registros: {len(X_val):,} ({len(X_val)/(len(X_train)+len(X_val)+len(X_test))*100:.1f}%)")
    if date_range.get('val'):
        print(f"  • Período: {date_range['val'][0]} a {date_range['val'][1]}")
    print(f"\nTESTE:")
    print(f"  • Registros: {len(X_test):,} ({len(X_test)/(len(X_train)+len(X_val)+len(X_test))*100:.1f}%)")
    if date_range.get('test'):
        print(f"  • Período: {date_range['test'][0]} a {date_range['test'][1]}")
    print("2. ESTATÍSTICAS DO TARGET")
    print("-"*50)
    print(f"MÉDIA:")
    print(f"  • Treino: {y_train.mean():.6f}")
    print(f"  • Validação: {y_val.mean():.6f}")
    print(f"  • Teste: {y_test.mean():.6f}")
    print(f"\nDESVIO PADRÃO:")
    print(f"  • Treino: {y_train.std():.6f}")
    print(f"  • Validação: {y_val.std():.6f}")
    print(f"  • Teste: {y_test.std():.6f}")
    print(f"\nPOSITIVOS (%):")
    print(f"  • Treino: {(y_train > 0).sum()/len(y_train)*100:.1f}%)")
    print(f"  • Validação: {(y_val > 0).sum()/len(y_val)*100:.1f}%)")
    print(f"  • Teste: {(y_test > 0).sum()/len(y_test)*100:.1f}%)")
    print(f"\nNEGATIVOS (%):")
    print(f"  • Treino: {(y_train < 0).sum()/len(y_train)*100:.1f}%)")
    print(f"  • Validação: {(y_val < 0).sum()/len(y_val)*100:.1f}%)")
    print(f"  • Teste: {(y_test < 0).sum()/len(y_test)*100:.1f}%)")

    print("3. GRÁFICOS")
    print("-"*50)
    fig = plt.figure(figsize=(18, 14))
    gs = gridspec.GridSpec(2, 2, figure=fig, hspace=0.4, wspace=0.3)
    cores = ['#2E86AB', '#A23B72', '#F18F01']
    ax1 = fig.add_subplot(gs[0, 0])
    tamanhos = [len(X_train), len(X_val), len(X_test)]
    labels = ['Treino', 'Validação', 'Teste']
    wedges, texts, autotexts = ax1.pie(tamanhos, labels=labels, colors=cores,
                                       autopct='%1.1f%%', startangle=90,
                                       textprops={'fontsize': 12, 'fontweight': 'bold'})
    ax1.set_title('Distribuição dos Dados por Conjunto', fontsize=14, fontweight='bold')
    ax2 = fig.add_subplot(gs[0, 1])
    data_to_plot = [y_train, y_val, y_test]
    bp = ax2.boxplot(data_to_plot, labels=['Treino', 'Validação', 'Teste'],
                     patch_artist=True, showfliers=False)
    for patch, color in zip(bp['boxes'], cores):
        patch.set_facecolor(color)
    ax2.set_title('Distribuição do Target por Conjunto', fontsize=14, fontweight='bold')
    ax2.set_ylabel('Retorno')
    ax2.axhline(y=0, color='red', linestyle='--', alpha=0.5, label='Zero')
    ax2.legend()
    ax2.grid(True, alpha=0.3)
    ax3 = fig.add_subplot(gs[1, 0])
    ax3.hist(y_train, bins=50, alpha=0.5, label='Treino', color=cores[0], density=True)
    ax3.hist(y_val, bins=50, alpha=0.5, label='Validação', color=cores[1], density=True)
    ax3.hist(y_test, bins=50, alpha=0.5, label='Teste', color=cores[2], density=True)
    ax3.set_title('Distribuição do Target', fontsize=14, fontweight='bold')
    ax3.set_xlabel('Retorno')
    ax3.set_ylabel('Densidade')
    ax3.legend()
    ax3.axvline(x=0, color='red', linestyle='--', alpha=0.5)
    ax3.grid(True, alpha=0.3)
    ax4 = fig.add_subplot(gs[1, 1])
    stats.probplot(y_train, dist="norm", plot=ax4)
    ax4.set_title('QQ Plot do Target (Treino)', fontsize=14, fontweight='bold')
    ax4.get_lines()[0].set_color(cores[0])
    ax4.get_lines()[1].set_color('red')

```



```

ax4.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
print("4. DIVISÃO TEMPORAL")
print("-"*50)
if split_info.get('date_range'):
    print(f"TREINO:")
    print(f"  Início: {split_info['date_range'].get('train', ['N/A', 'N/A'])[0]} if split_info['date_range'].get('train')
    print(f"  Fim: {split_info['date_range'].get('train', ['N/A', 'N/A'])[1]} if split_info['date_range'].get('train')
    print(f"  Duração: {len(X_train)} dias úteis")
    print(f"\nVALIDAÇÃO:")
    print(f"  Início: {split_info['date_range'].get('val', ['N/A', 'N/A'])[0]} if split_info['date_range'].get('val') e
    print(f"  Fim: {split_info['date_range'].get('val', ['N/A', 'N/A'])[1]} if split_info['date_range'].get('val') else
    print(f"  Duração: {len(X_val)} dias úteis")
    print(f"\nTESTE:")
    print(f"  Início: {split_info['date_range'].get('test', ['N/A', 'N/A'])[0]} if split_info['date_range'].get('test')
    print(f"  Fim: {split_info['date_range'].get('test', ['N/A', 'N/A'])[1]} if split_info['date_range'].get('test') el
    print(f"  Duração: {len(X_test)} dias úteis")
    print(f"\nRAZÃO TREINO/TESTE: {len(X_train)/len(X_test):.2f}")
fig2, ax1 = plt.subplots(1, 1, figsize=(16, 6))
if df_original is not None and 'Date' in df_original.columns:
    try:
        dates = pd.to_datetime(df_original['Date'])
        y_target = df_original.get('Target_Next_Day',
                                   pd.Series(np.zeros(len(dates))))

        n = len(dates)
        train_end = int(n * 0.7)
        val_end = int(n * 0.85)

        ax1.plot(dates[:train_end], y_target[:train_end], color=cores[0],
                  alpha=0.7, label='Treino')
        ax1.plot(dates[train_end:val_end], y_target[train_end:val_end],
                  color=cores[1], alpha=0.7, label='Validação')
        ax1.plot(dates[val_end:], y_target[val_end:],
                  color=cores[2], alpha=0.7, label='Teste')

        if train_end < len(dates):
            ax1.axvline(x=dates.iloc[train_end], color='black', linestyle='--',
                        alpha=0.5, label='Divisão Treino/Val')
        if val_end < len(dates):
            ax1.axvline(x=dates.iloc[val_end], color='red', linestyle='--',
                        alpha=0.5, label='Divisão Val/Teste')

        ax1.set_title('Série Temporal - Target', fontsize=14, fontweight='bold')
        ax1.set_xlabel('Data')
        ax1.set_ylabel('Retorno')
        ax1.legend()
        ax1.grid(True, alpha=0.3)
    except Exception as e:
        ax1.text(0.5, 0.5, f'Erro ao plotar série temporal:\n{str(e)[:50]}',
                 transform=ax1.transAxes, ha='center', va='center')
        ax1.set_title('Série Temporal', fontsize=14, fontweight='bold')

plt.tight_layout()
plt.show()
else:
    print("  Informações de data não disponíveis para visualização temporal")

print("5. ANÁLISE DAS FEATURES")
print("-"*50)
features_to_show = []
if selected_features and len(selected_features) > 0:
    existing_features = [f for f in selected_features if f in X_train.columns]
    features_to_show = existing_features[:10]

feature_list_str = ', '.join(features_to_show) if features_to_show else 'N/A'
print(f"Total de features: {X_train.shape[1]}")
print(f"\nTIPOS DE DADOS:")
print(f"  • Numéricas: {X_train.select_dtypes(include=['number']).shape[1]}")
print(f"  • Categóricas: {X_train.select_dtypes(include=['object', 'category']).shape[1]}")
print(f"\nFEATURES SELECIONADAS:")
print(f"  • Quantidade: {len([f for f in selected_features if f in X_train.columns])} if selected_features else X_tra
print(f"  • Top {min(10, len(features_to_show))}: {feature_list_str}")
try:
    var_train = X_train.var().mean()
    var_val = X_val.var().mean()
    var_test = X_test.var().mean()
    print(f"\nVARIÂNCIA MÉDIA:")
    print(f"  • Treino: {var_train:.6f}")
    print(f"  • Validação: {var_val:.6f}")
    print(f"  • Teste: {var_test:.6f}")
except:

```



```

pass

fig_features, axes = plt.subplots(2, 1, figsize=(14, 14))
ax1 = axes[0]
if X_train.shape[1] > 1:
    n_features = min(20, X_train.shape[1])
    if selected_features and len(selected_features) > 0:
        existing_features = [f for f in selected_features if f in X_train.columns]
        if existing_features:
            features_to_plot = existing_features[:n_features]
        else:
            features_to_plot = X_train.columns[:n_features].tolist()
    else:
        try:
            variances = X_train.var()
            features_to_plot = variances.nlargest(min(n_features, len(variances))).index.tolist()
        except:
            features_to_plot = X_train.columns[:n_features].tolist()

features_to_plot = [f for f in features_to_plot if f in X_train.columns]
if len(features_to_plot) > 1:
    corr_matrix = X_train[features_to_plot].corr()
    mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
    sns.heatmap(corr_matrix, mask=mask, ax=ax1, cmap='coolwarm',
                center=0, annot=True, fmt='.2f', square=True,
                cbar_kws={'shrink': 0.8})
    ax1.set_title(f'Correlação entre Features (top {len(features_to_plot)} )',
                  fontsize=14, fontweight='bold')
else:
    ax1.text(0.5, 0.5, 'Poucas features para correlação',
             transform=ax1.transAxes, ha='center', va='center')
    ax1.set_title('Correlação entre Features', fontsize=14, fontweight='bold')

ax1.tick_params(axis='x', rotation=45)
ax1.tick_params(axis='y', rotation=0)

# 5.2 Distribuição das features
ax2 = axes[1]
if X_train.shape[1] > 0:
    n_examples = min(5, X_train.shape[1])
    if selected_features and len(selected_features) > 0:
        existing_features = [f for f in selected_features if f in X_train.columns]
        example_features = existing_features[:n_examples] if existing_features else X_train.columns[:n_examples].tolist()
    else:
        example_features = X_train.columns[:n_examples].tolist()

example_features = [f for f in example_features if f in X_train.columns]
if example_features:
    for i, feat in enumerate(example_features):
        if X_train[feat].dtype in ['float64', 'int64']:
            try:
                mean_train = X_train[feat].mean()
                std_train = X_train[feat].std()
                if std_train > 0:
                    data_train = (X_train[feat] - mean_train) / std_train
                    data_val = (X_val[feat] - mean_train) / std_train if feat in X_val.columns else None
                    data_test = (X_test[feat] - mean_train) / std_train if feat in X_test.columns else None
                    if data_val is not None and data_test is not None:
                        sns.kdeplot(data_train, label=f'Treino_{feat[:15]}', alpha=0.6, ax=ax2)
                        sns.kdeplot(data_val, label=f'Val_{feat[:15]}', alpha=0.6, ax=ax2, linestyle='--')
                        sns.kdeplot(data_test, label=f'Teste_{feat[:15]}', alpha=0.6, ax=ax2, linestyle=':')
            except:
                pass

ax2.set_title('Distribuição das Features (Exemplo)', fontsize=14, fontweight='bold')
ax2.set_xlabel('Valor Padronizado')
ax2.set_ylabel('Densidade')
ax2.legend(loc='upper right', fontsize=10)
ax2.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
return None

def executar_relatorio_completo(X_train, X_val, X_test, y_train, y_val, y_test,
                                split_info, X=None, y=None, selected_features=None,
                                df_original=None):
    if X_train is None or y_train is None:
        print("Erro: Dados de treino não encontrados!")
        return

    try:
        criar_relatorio_preparacao(

```

```

X_train, X_val, X_test,
y_train, y_val, y_test,
split_info,
selected_features,
df_original
)
except Exception as e:
    print(f"Erro ao gerar relatório: {e}")
    traceback.print_exc()

try:
    if 'X_train' in locals() and 'X_val' in locals() and 'X_test' in locals():
        if 'y_train' in locals() and 'y_val' in locals() and 'y_test' in locals():
            if 'split_info' in locals():
                print("Gerando relatório com dados preparados...")

                executar_relatorio_completo(
                    X_train, X_val, X_test,
                    y_train, y_val, y_test,
                    split_info,
                    X=X if 'X' in locals() else None,
                    y=y if 'y' in locals() else None,
                    selected_features=selected_features if 'selected_features' in locals() else None,
                    df_original=df_analise if 'df_analise' in locals() else None
                )

except NameError as e:
    print("Execute o relatório após a preparação dos dados.")

```

```

Gerando relatório com dados preparados...
RELATÓRIO DE PREPARAÇÃO DOS DADOS E DIVISÃO TREINO/TESTE
=====

1. INFORMAÇÕES GERAIS DOS DADOS
-----
Total de registros: 138,566
Total de features: 29

TREINO:
  • Registros: 96,996 (70.0%)
  • Período: 2015-01-02 00:00:00 a 2020-11-20 00:00:00

VALIDAÇÃO:
  • Registros: 20,785 (15.0%)
  • Período: 2020-11-23 00:00:00 a 2023-03-20 00:00:00

TESTE:
  • Registros: 20,785 (15.0%)
  • Período: 2023-03-21 00:00:00 a 2026-06-25 00:00:00
2. ESTATÍSTICAS DO TARGET
-----
MÉDIA:
  • Treino: 0.000965
  • Validação: 0.001116
  • Teste: 0.000892

DESVIO PADRÃO:
  • Treino: 0.022685
  • Validação: 0.023216
  • Teste: 0.024828

POSITIVOS (%):
  • Treino: 52.4%
  • Validação: 51.6%
  • Teste: 52.7%

NEGATIVOS (%):
  • Treino: 46.9%
  • Validação: 47.0%
  • Teste: 46.9%
3. GRÁFICOS
-----
Erro ao gerar relatório: name 'gridspec' is not defined
Traceback (most recent call last):
  File "/tmp/ipykernel_1990/2290075744.py", line 246, in executar_relatorio_completo
    criar_relatorio_preparacao(
  File "/tmp/ipykernel_1990/2290075744.py", line 48, in criar_relatorio_preparacao
    gs = gridspec.GridSpec(2, 2, figure=fig, hspace=0.4, wspace=0.3)
    ^^^^^^^^^
NameError: name 'gridspec' is not defined
<Figure size 1800x1400 with 0 Axes>

```

▼ Justificativa da divisão

- Por que usar holdout, validação cruzada ou divisão temporal?

- A proporção treino/teste faz sentido para o tamanho do dataset?
- Foi necessário estratificar as classes?
- Como a divisão evita vazamento de dados?

✓ Versão 01

✓ Por que usar holdout, validação cruzada ou divisão temporal?

A escolha foi de Divisão Temporal (Time Series Split), pois os Dados financeiros têm forte dependência temporal e auto-correlação serial, preservando a ordem cronológica que evita usar futuro para prever o passado (ou seja fica ao contrário) e mantém a sequência temporal nos folds.

✓ A proporção treino/teste faz sentido para o tamanho do dataset?

A proporção escolhida foi: 70%/15%/15%

- Devido ao tamanho mínimo do treino, sendo o ideal acima de 10 mil registros
- Sendo 18 mil sendo suficiente para redes neurais
- Pelo teste representativo com a captura de diferentes ciclos de mercado
- Validação para tuning em que é ajustado os hiperparâmetros
- E por fim, o total os dados que possuem ampla cobertura de diferentes regimes do mercado.

✓ Foi necessário estratificar?

Não. A estratificação não é aplicável, devido que o target é contínuo (regressão - retorno do próximo dia), não sendo uma classe categórica.

```
# Target é contínuo (regressão)
print(f"Target type: {y.dtype}") # float64
print(f"Target range: [{y.min():.4f}, {y.max():.4f}]")
print(f"Target mean: {y.mean():.4f}")
```

```
Target type: float64
Target range: [-0.3584, 0.5229]
Target mean: 0.0010
```

✓ Como a divisão evita vazamento de dados?

1. Divisão Estritamente Temporal

- Treino: 2020-01-04 a 2023-04-14 (dados passados)
- Validação: 2023-04-14 a 2024-11-05 (dados intermediários)
- Teste: 2024-11-05 a 2026-06-02 (dados futuros)
- **Conclusão: É garantido que nenhum dado futuro contamina o treino.**

2. Normalização com Fit apenas no Treino, pois os parâmetros de escala (média, desvio, percentis) vem exclusivamente do treino.

3. Nenhuma Feature com Informação Futura

- Médias móveis - apenas valores passados
- RSI - Janela com dados anteriores
- Retornos - `pct_change()` do dia anterior
- Target - `shift(-1)` (dia seguinte)
- **Conclusão: a garantia é que as Features usam apenas informações DISPONÍVEIS no momento a previsão.**

4. Codificação com Fit apenas no Treino que tem o mapeamento de categorias exclusivamente do treino.

Clique duas vezes (ou pressione "Enter") para editar

✓ Versão 03

✓ Por que usar holdout, validação cruzada ou divisão temporal?

- A escolha foi Divisão Temporal (Time Series Split) com 70% treino, 15% validação e 15% teste.

Motivos para esta escolha:

- Dados financeiros têm forte dependência temporal e auto-correlação serial
- Preservação da ordem cronológica - evita usar dados futuros para prever o passado
- Mantém a sequência temporal - cada conjunto (treino, validação, teste) mantém a ordem original dos dados
- Evita vazamento de dados - não permite que informações do futuro "contaminem" o treinamento do modelo
- Reflete cenário real - em problemas financeiros, sempre prevemos o futuro com base em dados passados
- A divisão temporal foi implementada através da função `temporal_split()` que garante a separação cronológica dos dados.

- ✓ A proporção treino/teste faz sentido para o tamanho do dataset?

Sim, a proporção 70%/15%/15% é adequada pelos seguintes motivos:

Justificativas:

- **Tamanho do treino** - 96.996 registros (70%) - acima do mínimo recomendado de 10.000 para modelos de machine learning
- **Rede neural** - 18 mil registros (considerando o menor conjunto) - suficiente para redes neurais
- **Teste representativo** - 20.785 registros (15%) - captura diferentes ciclos de mercado
- **Validação adequada** - 20.785 registros (15%) - suficiente para tuning de hiperparâmetros
- **Cobertura temporal** - Dados abrangem 2015-2026, capturando diferentes regimes de mercado

Detalhes visuais da distribuição dos dados:

TREINO: 96.996 registros (2015-01-02 a 2020-11-20)

VALIDAÇÃO: 20.785 registros (2020-11-23 a 2023-03-20)

TESTE: 20.785 registros (2023-03-21 a 2026-06-25)

- ✓ Foi necessário estratificar?

Não, a estratificação não foi necessária.

Motivos:

- Target é contínuo (regressão - retorno do próximo dia)
- Não se trata de classificação categórica - não há classes para estratificar
- Variável alvo: `Target_Next_Day` (retorno do dia seguinte)
- Tipo: `float64` com valores no intervalo `[-0.3584, 0.5229]`
- Média: 0.0010

```
# Target é contínuo (regressão)
print(f"Target type: {y.dtype}") # float64
print(f"Target range: [{y.min():.4f}, {y.max():.4f}]")
print(f"Target mean: {y.mean():.4f}")
```

```
Target type: float64
Target range: [-0.3584, 0.5229]
Target mean: 0.0010
```

Distribuição do target:

- Positivos: 72.472 (52.3%)
- Negativos: 65.021 (46.9%)

- ✓ Como a divisão evita vazamento de dados?

1. Divisão Estritamente Temporal

- Os dados são separados cronologicamente:
- Treino: 2015-01-02 a 2020-11-20 (dados passados)
- Validação: 2020-11-23 a 2023-03-20 (dados intermediários)
- Teste: 2023-03-21 a 2026-06-25 (dados futuros)

Garantia: Nenhum dado futuro contamina o treinamento.

2. Normalização com Fit apenas no Treino

- Os parâmetros de escala (média, desvio padrão, percentis) são calculados exclusivamente a partir dos dados de treino:
- Média do treino usada para padronizar validação e teste
- Desvio padrão do treino usado para padronizar validação e teste

3. Nenhuma Feature com Informação Futura

- Feature Descrição Temporalidade
- Médias móveis Cálculo com base em dados históricos Apenas valores passados
- RSI Janela com dados anteriores Apenas valores passados
- Retornos pct_change() Baseado no dia anterior
- Target shift(-1) Dia seguinte (não usado no treino)

Garantia: As features usam apenas informações DISPONÍVEIS no momento da previsão.

4. Codificação com Fit apenas no Treino

- O mapeamento de categorias (One-Hot Encoding) é definido exclusivamente a partir dos dados de treino

Garante que categorias raras ou ausentes na validação/teste sejam tratadas adequadamente

6. Pré-processamento e pipeline

- imputação de valores ausentes;
- normalização ou padronização;
- encoding de variáveis categóricas;
- seleção de atributos;
- engenharia de atributos;
- tratamento de texto, imagem ou séries temporais, quando aplicável.

Funções

```
def limpar_memoria(variaveis_para_excluir=None, mostrar_status=True):
    if variaveis_para_excluir:
        for var_name in variaveis_para_excluir:
            try:
                if var_name in globals():
                    del globals()[var_name]
                frame = sys._getframe(1)
                if var_name in frame.f_locals:
                    del frame.f_locals[var_name]
            except:
                pass
    gc.collect()
    try:
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
    except:
        pass

    if mostrar_status:
        memoria = psutil.virtual_memory()
        print(f"RAM: {memoria.used/1024**3:.1f}GB / {memoria.total/1024**3:.1f}GB ({memoria.percent}%)")

class ModeloManager:

    def __init__(self, nome_base):
        self.nome_base = nome_base
        self.models_cache = {}
        self.total_parts = None
        self.empresas_disponiveis = []
        self.carregar_indice()

    def carregar_indice(self):
        try:
            indice = CarregarArquivoOnGitHub(f"{self.nome_base}_indice")
            if indice is not None and not indice.empty:
                self.total_parts = indice.iloc[0].get('total_partes', 0)
                print(f"Índice carregado: {self.total_parts} partes disponiveis")
```

```
        return
    except:
        pass

    self.total_parts = self._descobrir_partes()

def _descobrir_partes(self):
    parte = 1
    while True:
        nome = f"{self.nome_base}_{parte}.pk1"
        try:
            url = f"https://api.github.com/repos/{GITHUB_USERNAME}/{GITHUB_REPO}/contents/{GITHUB_TRAIN_PATH}/{nome}"
            response = requests.head(url, headers=GITHUB_HEADERS)
            if response.status_code == 200:
                parte += 1
            else:
                break
        except:
            break
    print(f"Descobertas {parte-1} partes disponíveis")
    return parte - 1

def carregar_modelo(self, empresa):
    if empresa in self.models_cache:
        return self.models_cache[empresa]

    try:
        modelo = CarregarArquivoOnGitHub(f"{self.nome_base}_{empresa}")
        if modelo is not None:
            self.models_cache[empresa] = modelo
            print(f"Modelo {empresa} carregado diretamente")
            return modelo
    except:
        pass

    modelo = self._carregar_das_partes(empresa)
    if modelo is not None:
        self.models_cache[empresa] = modelo
        print(f"Modelo {empresa} carregado das partes")
        return modelo

def _carregar_das_partes(self, empresa):
    for parte in range(1, self.total_parts + 1):
        nome = f"{self.nome_base}_{parte}.pk1"
        try:
            dados = CarregarArquivoOnGitHub(nome)
            if dados is not None and isinstance(dados, dict):
                if empresa in dados:
                    return dados[empresa]
        except:
            continue
    return None

def liberar_modelo(self, empresa):
    if empresa in self.models_cache:
        del self.models_cache[empresa]
        gc.collect()
        print(f"Modelo {empresa} liberado da memória")

def limpar_cache(self):
    self.models_cache.clear()
    gc.collect()
    print("Cache de modelos limpo")

def listar_empresas(self):
    if self.empresas_disponiveis:
        return self.empresas_disponiveis

    empresas = set()
    for parte in range(1, self.total_parts + 1):
        nome = f"{self.nome_base}_{parte}.pk1"
        try:
            dados = CarregarArquivoOnGitHub(nome)
            if dados is not None and isinstance(dados, dict):
                empresas.update(dados.keys())
        except:
```

```

        continue

        self.empresas_disponiveis = list(empresas)
        return self.empresas_disponiveis

def prepare_data_for_model(df):
    numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
    exclude_cols = ['Date', 'Symbol', 'Company_Name', 'Target_Next_Return', 'Target_Direction']
    feature_cols = [col for col in numeric_cols if col not in exclude_cols]
    X = df[feature_cols].copy()
    y = df['Target_Next_Return'].copy()
    X = X.fillna(X.median())
    return X, y

def prepare_data(empresa, features=None):
    df_emp = df_analise[df_analise['Symbol'] == empresa].copy()
    df_emp = df_emp.sort_values('Date')
    if features is None:
        features = [f for f in features_selecionadas if f in df_emp.columns]

    if len(features) == 0:
        features = df_emp.select_dtypes(include=['number']).columns.tolist()
        exclude_cols = [TARGET_COL, TARGET_DIRECTION, 'Close', 'Adj Close', 'Stock Splits']
        features = [f for f in features if f not in exclude_cols]
        print(f"    Fallback: usando {len(features)} columnas numéricas para {empresa}")

    X = df_emp[features].copy()
    y = df_emp[TARGET_COL].copy()
    X = X.fillna(method='ffill').fillna(method='bfill').fillna(0)
    valid_mask = ~y.isna()
    X = X[valid_mask]
    y = y[valid_mask]
    n = len(X)
    train_end = int(n * 0.7)
    val_end = int(n * 0.85)
    X_train = X.iloc[:train_end]
    X_val = X.iloc[train_end:val_end]
    X_test = X.iloc[val_end:]
    y_train = y.iloc[:train_end]
    y_val = y.iloc[train_end:val_end]
    y_test = y.iloc[val_end:]
    return X_train, X_val, X_test, y_train, y_val, y_test

def create_pipeline(X_train):
    colunas_numericas = X_train.select_dtypes(include=['number']).columns.tolist()
    colunas_categoricas = X_train.select_dtypes(include=['object']).columns.tolist()
    transformers = []
    if colunas_numericas:
        transformers.append((('num', Pipeline([
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', RobustScaler())
        ]), colunas_numericas))
    if colunas_categoricas:
        transformers.append((('cat', Pipeline([
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
        ]), colunas_categoricas))
    if not transformers:
        transformers.append(('passthrough', 'passthrough', []))

    transformador = ColumnTransformer(
        transformers=transformers,
        remainder='drop'
    )

    pipeline = Pipeline(steps=[
        ('transformador', transformador),
        ('regressor', RandomForestRegressor(n_estimators=200, random_state=SEED, n_jobs=-1))
    ])
    return pipeline

def calculate_metrics(y_true, y_pred):
    return {
        'rmse': np.sqrt(mean_squared_error(y_true, y_pred)),
        'mae': mean_absolute_error(y_true, y_pred),
        'r2': r2_score(y_true, y_pred)
    }

```

```

    }

def objective_Random_Forest(trial, X_train, y_train, cv):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 300),
        'max_depth': trial.suggest_int('max_depth', 2, 15),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 20),
        'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 10),
        'max_features': trial.suggest_categorical('max_features', ['sqrt', 'log2', None, 0.3, 0.5, 0.7]),
        'bootstrap': trial.suggest_categorical('bootstrap', [True, False]),
        'ccp_alpha': trial.suggest_float('ccp_alpha', 0, 0.1),
        'random_state': SEED,
        'n_jobs': -1
    }
    scores = []
    for train_idx, val_idx in cv.split(X_train):
        X_tr = X_train.iloc[train_idx]
        X_val_cv = X_train.iloc[val_idx]
        y_tr = y_train.iloc[train_idx]
        y_val_cv = y_train.iloc[val_idx]
        model = RandomForestRegressor(**params)
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_val_cv)
        scores.append(mean_squared_error(y_val_cv, y_pred))
    return np.mean(scores)

def objective_XGBoost(trial, X_train, y_train, cv):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 300),
        'max_depth': trial.suggest_int('max_depth', 2, 10),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'min_child_weight': trial.suggest_int('min_child_weight', 1, 10),
        'reg_alpha': trial.suggest_float('reg_alpha', 0, 0.5),
        'reg_lambda': trial.suggest_float('reg_lambda', 0.5, 2.0),
        'random_state': SEED,
        'n_jobs': -1
    }
    scores = []
    for train_idx, val_idx in cv.split(X_train):
        X_tr = X_train.iloc[train_idx]
        X_val_cv = X_train.iloc[val_idx]
        y_tr = y_train.iloc[train_idx]
        y_val_cv = y_train.iloc[val_idx]
        X_tr_np = X_tr.values.astype(np.float32)
        X_val_np = X_val_cv.values.astype(np.float32)
        y_tr_np = y_tr.values.ravel().astype(np.float32)
        y_val_np = y_val_cv.values.ravel().astype(np.float32)
        model = XGBRegressor(**params)
        model.fit(X_tr_np, y_tr_np, eval_set=[(X_val_np, y_val_np)], verbose=False)
        y_pred = model.predict(X_val_np)
        scores.append(mean_squared_error(y_val_np, y_pred))
    return np.mean(scores)

def objective_LightGBM(trial, X_train, y_train, cv):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 300),
        'max_depth': trial.suggest_int('max_depth', 2, 15),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3),
        'num_leaves': trial.suggest_int('num_leaves', 10, 50),
        'min_child_samples': trial.suggest_int('min_child_samples', 5, 20),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.6, 1.0),
        'reg_alpha': trial.suggest_float('reg_alpha', 0, 0.5),
        'reg_lambda': trial.suggest_float('reg_lambda', 0.5, 2.0),
        'random_state': SEED,
        'n_jobs': -1,
        'verbose': -1
    }
    scores = []
    for train_idx, val_idx in cv.split(X_train):
        X_tr = X_train.iloc[train_idx]
        X_val_cv = X_train.iloc[val_idx]
        y_tr = y_train.iloc[train_idx]
        y_val_cv = y_train.iloc[val_idx]
        model = LGBMRegressor(**params)
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_val_cv)

```



```

        scores.append(mean_squared_error(y_val_cv, y_pred))
    return np.mean(scores)

def objective_Gradient_Boosting(trial, X_train, y_train, cv):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 250),
        'max_depth': trial.suggest_int('max_depth', 2, 8),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.3),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 20),
        'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 10),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'max_features': trial.suggest_categorical('max_features', ['sqrt', 'log2', None, 0.3, 0.5, 0.7]),
        'random_state': SEED
    }
    scores = []
    for train_idx, val_idx in cv.split(X_train):
        X_tr = X_train.iloc[train_idx]
        X_val_cv = X_train.iloc[val_idx]
        y_tr = y_train.iloc[train_idx]
        y_val_cv = y_train.iloc[val_idx]
        model = GradientBoostingRegressor(**params)
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_val_cv)
        scores.append(mean_squared_error(y_val_cv, y_pred))
    return np.mean(scores)

def analisar_acuracia_direcional(y_true, y_pred, empresa=None):
    y_true_dir = (y_true > 0).astype(int)
    y_pred_dir = (y_pred > 0).astype(int)

    # Métricas
    accuracy = accuracy_score(y_true_dir, y_pred_dir)
    precision = precision_score(y_true_dir, y_pred_dir, zero_division=0)
    recall = recall_score(y_true_dir, y_pred_dir, zero_division=0)
    f1 = f1_score(y_true_dir, y_pred_dir, zero_division=0)

    # Matriz de confusão
    cm = confusion_matrix(y_true_dir, y_pred_dir)

    # Calcular probabilidades condicionais
    tn, fp, fn, tp = cm.ravel()

    resultados = {
        'empresa': empresa if empresa else 'Geral',
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1': f1,
        'confusion_matrix': cm,
        'tp': tp,
        'tn': tn,
        'fp': fp,
        'fn': fn,
        'total': len(y_true)
    }

    return resultados

def gerar_relatorio_empresa(empresa, df_analise):
    df_emp = df_analise[df_analise['Symbol'] == empresa].copy()
    df_emp = df_emp.sort_values('Date')
    if len(df_emp) == 0:
        return None
    preco_atual = df_emp['Close'].iloc[-1]
    preco_min = df_emp['Close'].min()
    preco_max = df_emp['Close'].max()
    preco_medio = df_emp['Close'].mean()
    preco_std = df_emp['Close'].std()
    retornos = df_emp[TARGET_COL].dropna()
    retorno_medio = retornos.mean()
    retorno_std = retornos.std()
    retorno_min = retornos.min()
    retorno_max = retornos.max()
    dias_alta = (retornos > 0).sum()
    dias_baixa = (retornos < 0).sum()
    dias_zero = (retornos == 0).sum()
    total_dias = len(retornos)
    retorno_acumulado = (1 + retornos).prod() - 1
    sharpe = (retorno_medio / retorno_std) * np.sqrt(252) if retorno_std > 0 else 0
    volatilidade_anual = retorno_std * np.sqrt(252)

```

```

relatorio = {
    'empresa': empresa,
    'preco_atual': preco_atual,
    'preco_min': preco_min,
    'preco_max': preco_max,
    'preco_medio': preco_medio,
    'preco_std': preco_std,
    'retorno_medio': retorno_medio,
    'retorno_std': retorno_std,
    'retorno_min': retorno_min,
    'retorno_max': retorno_max,
    'dias_alta': dias_alta,
    'dias_baixa': dias_baixa,
    'dias_zero': dias_zero,
    'total_dias': total_dias,
    'pct_alta': dias_alta / total_dias * 100,
    'pct_baixa': dias_baixa / total_dias * 100,
    'retorno_acumulado': retorno_acumulado,
    'sharpe_ratio': sharpe,
    'volatilidade_anual': volatilidade_anual
}

return relatorio

def calcular_mape_divisao_zero(y_true, y_pred, epsilon=1e-8):
    y_true_protegido = np.where(np.abs(y_true) < epsilon, epsilon, y_true)
    mape = np.mean(np.abs((y_true - y_pred) / y_true_protegido)) * 100
    return mape

```

▼ Processamento dos resultados de treinamentos por empresa.

```

NOME_RESULTADOS_POR_EMPRESA = "resultados_por_empresa"
NOME_MODELS_POR_EMPRESA = "models_por_empresa"
NOME_RESULTADOS_DIRECIONAIS = "resultados_direcionais"
NOME_RELATORIO_EMPRESAS = "relatorio_empresas"
NOME_COMPARATIVO_EMPRESAS = "comparativo_empresas"
NOME_RESULTADOS_SERIES = "resultados_series_temporais"
resultados_por_empresa = CarregarArquivoOnGitHub(NOME_RESULTADOS_POR_EMPRESA)
models_por_empresa = None
if resultados_por_empresa is not None:
    print(f"Resultados carregados: {len(resultados_por_empresa)} empresas")
    empresas_processadas = list(resultados_por_empresa.keys())
else:
    print("Nenhum resultado encontrado. Processando...")
    empresas_processadas = []

empresas_para_processar = [e for e in simbolos_usados if e not in empresas_processadas]
if empresas_para_processar:
    print(f"\nEmpresas a processar: {len(empresas_para_processar)}")
    print(f"    {empresas_para_processar[:5]}...")
else:
    print("\nTodas as empresas já foram processadas!")

if empresas_para_processar:
    features_existentes = [f for f in features_selecionadas if f in df_analise.columns]
    print(f"Features disponíveis: {len(features_existentes)}")
    model_manager = ModeloManager(NOME_MODELS_POR_EMPRESA)
    for idx, empresa in enumerate(empresas_para_processar):
        print(f"Processando empresa: {empresa} ({idx+1}/{len(empresas_para_processar)})")
        try:
            X_train, X_val, X_test, y_train, y_val, y_test = prepare_data(empresa, features_existentes)
            if X_train is None or len(X_train) == 0:
                print(f"{empresa} sem dados suficientes - ignorando")
                continue

            print(f"\n    Divisão dos dados - {empresa}:")
            print(f"        • Treino: {len(X_train):,} registros x {X_train.shape[1]} features")
            print(f"        • Validação: {len(X_val):,} registros")
            print(f"        • Teste: {len(X_test):,} registros")
            pipeline = create_pipeline(X_train)
            test_size = int(len(X_train) * 0.2)
            min_train_size = 100
            max_folds = (len(X_train) - min_train_size) // test_size if test_size > 0 else 0
            n_splits = min(3, max_folds) if max_folds > 0 else 0
            rmse_scores = []
            mae_scores = []
            r2_scores = []
            if n_splits > 0:
                print(f"\nTimeSeriesSplit: {n_splits} folds")

```

```

tscv = TimeSeriesSplit(n_splits=n_splits, test_size=test_size)
for fold, (train_idx, val_idx) in enumerate(tscv.split(X_train), 1):
    X_tr = X_train.iloc[train_idx]
    X_val_cv = X_train.iloc[val_idx]
    y_tr = y_train.iloc[train_idx]
    y_val_cv = y_train.iloc[val_idx]
    pipeline_fold = create_pipeline(X_tr)
    pipeline_fold.fit(X_tr, y_tr)
    y_pred = pipeline_fold.predict(X_val_cv)
    metrics = calculate_metrics(y_val_cv, y_pred)
    rmse_scores.append(metrics['rmse'])
    mae_scores.append(metrics['mae'])
    r2_scores.append(metrics['r2'])
    print(f"  Fold {fold}/{n_splits}: RMSE: {metrics['rmse']:.6f} | R²: {metrics['r2']:.4f}")
    del pipeline_fold, X_tr, X_val_cv, y_tr, y_val_cv, y_pred
gc.collect()

X_train_full = pd.concat([X_train, X_val])
y_train_full = pd.concat([y_train, y_val])
pipeline_final = create_pipeline(X_train_full)
pipeline_final.fit(X_train_full, y_train_full)
y_pred_test = pipeline_final.predict(X_test)
test_metrics = calculate_metrics(y_test, y_pred_test)
print(f"\n  DESEMPENHO NO TESTE - {empresa}:")
print(f"    • RMSE: {test_metrics['rmse']:.6f}")
print(f"    • MAE: {test_metrics['mae']:.6f}")
print(f"    • R²: {test_metrics['r2']:.4f}")

if resultados_por_empresa is None:
    resultados_por_empresa = {}
resultados_por_empresa[empresa] = {
    'rmse_mean': np.mean(rmse_scores) if rmse_scores else None,
    'rmse_std': np.std(rmse_scores) if rmse_scores else None,
    'mae_mean': np.mean(mae_scores) if mae_scores else None,
    'mae_std': np.std(mae_scores) if mae_scores else None,
    'r2_mean': np.mean(r2_scores) if r2_scores else None,
    'r2_std': np.std(r2_scores) if r2_scores else None,
    'rmse_test': test_metrics['rmse'],
    'mae_test': test_metrics['mae'],
    'r2_test': test_metrics['r2'],
    'n_splits': n_splits,
    'n_train': len(X_train),
    'n_val': len(X_val),
    'n_test': len(X_test)
}

SalvarTreinamentoOnGitHub(
    nome_arquivo=NOME_RESULTADOS_POR_EMPRESA,
    dados=resultados_por_empresa,
    metadados={
        'descricao': 'Resultados de treinamento por empresa',
        'total_empresas': len(resultados_por_empresa)
    }
)

modelos_empresa = {empresa: pipeline_final}
SalvarTreinamentoDivididoOnGitHub(
    nome_base=NOME_MODELS_POR_EMPRESA,
    dados=modelos_empresa,
    metadados={
        'descricao': f'Modelo treinado para {empresa}',
        'empresa': empresa
    }
)

print(f"{empresa} processada e salva com sucesso!")
del X_train, X_val, X_test, y_train, y_val, y_test
del X_train_full, y_train_full, y_pred_test, pipeline_final
gc.collect()
limpar_memoria(mostrar_status=True)
except Exception as e:
    print(f"Erro ao processar {empresa}: {str(e)}")
continue

```

```

resultados_por_empresa.pkl carregado com sucesso!
Tipo: dict
Tamanho: 10,061 bytes
Resultados carregados: 49 empresas

```

```
Todas as empresas já foram processadas!
```

▼ Treinamentos

```

NOME_PIPELINE_FINAL = "pipeline_final"
NOME_RESULTADOS_FINAL = "resultados_finais"
cutoff_date = '2024-01-01'
pipeline_final = CarregarArquivoDivididoOnGitHub(NOME_PIPELINE_FINAL)
resultados_finais_cache = CarregarArquivoOnGitHub(NOME_RESULTADOS_FINAL)
if pipeline_final is not None and resultados_finais_cache is not None:
    print("PIPELINE E RESULTADOS CARREGADOS DO CACHE!")
    rmse_final = resultados_finais_cache.get('rmse_final')
    mae_final = resultados_finais_cache.get('mae_final')
    r2_final = resultados_finais_cache.get('r2_final')
    y_pred_full = resultados_finais_cache.get('y_pred_full')
    y_test_full = resultados_finais_cache.get('y_test_full')
    if 'cutoff_date' in resultados_finais_cache.get('metadados', {}):
        cutoff_date = resultados_finais_cache['metadados']['cutoff_date']
    print(f"\nRESULTADOS DO CACHE (dados após {cutoff_date}):")
    print(f"    • RMSE: {rmse_final:.6f}")
    print(f"    • MAE: {mae_final:.6f}")
    print(f"    • R²: {r2_final:.4f}")
    has_pred_data = y_pred_full is not None and y_test_full is not None

else:
    print("Pipeline não encontrado em cache. Executando treinamento...")
    if 'df_analise' not in locals() or df_analise is None:
        print("df_analise não encontrado! Execute a seção de análise primeiro.")
        raise ValueError("df_analise não encontrado")
    train_data = df_analise[df_analise['Date'] < cutoff_date].copy()
    test_data = df_analise[df_analise['Date'] >= cutoff_date].copy()
    print(f"\n    DIVISÃO DOS DADOS:")
    print(f"    • Treino: {len(train_data):,} registros (até {cutoff_date})")
    print(f"    • Teste: {len(test_data):,} registros (após {cutoff_date})")
    X_train_full, y_train_full = prepare_data_for_model(train_data)
    X_test_full, y_test_full = prepare_data_for_model(test_data)
    print(f"\n    SHAPES:")
    print(f"    • X_train: {X_train_full.shape}")
    print(f"    • X_test: {X_test_full.shape}")
    print("\n    Treinando pipeline final...")
    pipeline_final = create_pipeline(X_train_full)
    pipeline_final.fit(X_train_full, y_train_full)
    y_pred_full = pipeline_final.predict(X_test_full)
    rmse_final = np.sqrt(mean_squared_error(y_test_full, y_pred_full))
    mae_final = mean_absolute_error(y_test_full, y_pred_full)
    r2_final = r2_score(y_test_full, y_pred_full)
    print(f"\n    RESULTADOS DO TESTE FINAL:")
    print(f"    • RMSE: {rmse_final:.6f}")
    print(f"    • MAE: {mae_final:.6f}")
    print(f"    • R²: {r2_final:.4f}")
    print("\n Salvando pipeline e resultados...")
    SalvarTreinamentoDivididoOnGitHub(
        nome_base=NOME_PIPELINE_FINAL,
        dados=pipeline_final,
        metadados={
            'descricao': 'Pipeline final treinado',
            'cutoff_date': cutoff_date,
            'n_train': len(X_train_full),
            'n_test': len(X_test_full)
        }
    )

    SalvarTreinamentoOnGitHub(
        nome_arquivo=NOME_RESULTADOS_FINAL,
        dados={
            'rmse_final': rmse_final,
            'mae_final': mae_final,
            'r2_final': r2_final,
            'y_pred_full': y_pred_full,
            'y_test_full': y_test_full
        },
        metadados={
            'descricao': 'Resultados finais do teste',
            'cutoff_date': cutoff_date
        }
    )
    print("Pipeline e resultados salvos com sucesso!")
    has_pred_data = True

```

```

Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_82.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_83.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_84.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_85.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_86.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_87.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_88.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_89.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_90.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_91.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_92.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_93.pkl carregado com sucesso!
Tipo: dict
Tamanho: 20,971,673 bytes
pipeline_final_94.pkl carregado com sucesso!
Tipo: dict
Tamanho: 15,066,917 bytes
resultados_finais.pkl carregado com sucesso!
Tipo: dict
Tamanho: 976,240 bytes
PIPELINE E RESULTADOS CARREGADOS DO CACHE!

```

RESULTADOS DO CACHE (dados após 2024-01-01):

- RMSE: 0.029691
- MAE: 0.021279
- R²: -0.3881

```

if resultados_por_empresa is not None:
    df_resultados = pd.DataFrame(resultados_por_empresa).T
    colunas_mostrar = ['rmse_test', 'mae_test', 'r2_test', 'n_train', 'n_val', 'n_test']
    colunas_exist = [c for c in colunas_mostrar if c in df_resultados.columns]
    if colunas_exist:
        print("\nRESULTADOS POR EMPRESA:")
        display(df_resultados[colunas_exist].round(6))
        print(f"\nMÉDIAS GERAIS:")
        if 'rmse_test' in df_resultados.columns:
            print(f"    • RMSE médio: {df_resultados['rmse_test'].mean():.6f}")
            print(f"    • MAE médio: {df_resultados['mae_test'].mean():.6f}")
            print(f"    • R² médio: {df_resultados['r2_test'].mean():.4f}")
        if len(df_resultados) > 20:
            print(f"\nNOTA: Mostrando as 20 melhores empresas + 'Outras' (total: {len(df_resultados)})")
            df_top20 = df_resultados.nlargest(20, 'r2_test')
            df_outras = df_resultados.drop(df_top20.index)
            outras_rmse = df_outras['rmse_test'].mean() if 'rmse_test' in df_outras.columns else 0
            outras_r2 = df_outras['r2_test'].mean() if 'r2_test' in df_outras.columns else 0
            df_consolidado = df_top20.copy()
            df_consolidado.loc['Outras'] = [outras_rmse, outras_r2] + [0] * (len(df_consolidado.columns) - 2)

        else:
            df_consolidado = df_resultados.copy()
        fig, axes = plt.subplots(1, 2, figsize=(15, 6))

        if 'rmse_test' in df_resultados.columns:
            df_sorted_rmse = df_consolidado.sort_values('rmse_test')
            colors_rmse = ['orange' if idx == 'Outras' else 'steelblue' for idx in df_sorted_rmse.index]
            bars = axes[0].barh(df_sorted_rmse.index, df_sorted_rmse['rmse_test'],
                                color=colors_rmse, alpha=0.8)
            axes[0].set_xlabel('RMSE (menor é melhor)', fontsize=11)
            axes[0].set_title(f'RMSE por Empresa (Top 20 + Outras)', fontsize=12)
            axes[0].grid(True, alpha=0.3)
            max_rmse = df_sorted_rmse['rmse_test'].max()
            min_rmse = df_sorted_rmse['rmse_test'].min()
            margin = (max_rmse - min_rmse) * 0.15 if max_rmse != min_rmse else 0.01

```

```

axes[0].set_xlim(min_rmse - margin * 0.1, max_rmse + margin)
for i, v in enumerate(df_sorted_rmse['rmse_test']):
    offset = (max_rmse - min_rmse) * 0.01 if max_rmse != min_rmse else 0.001
    axes[0].text(v + offset, i, f'{v:.4f}', va='center',
                 fontsize=8, color='darkblue', fontweight='bold')
if 'Outras' in df_sorted_rmse.index:
    idx_outras = df_sorted_rmse.index.get_loc('Outras')
    axes[0].barh('Outras', df_sorted_rmse.loc['Outras', 'rmse_test'],
                 color='orange', alpha=0.8, edgecolor='darkorange', linewidth=2)

if 'r2_test' in df_resultados.columns:
    df_sorted_r2 = df_consolidado.sort_values('r2_test')
    colors_r2 = []
    for idx in df_sorted_r2.index:
        if idx == 'Outras':
            colors_r2.append('orange')
        elif df_sorted_r2.loc[idx, 'r2_test'] > 0:
            colors_r2.append('green')
        else:
            colors_r2.append('red')

    bars = axes[1].barh(df_sorted_r2.index, df_sorted_r2['r2_test'],
                       color=colors_r2, alpha=0.8)
    axes[1].axvline(0, color='black', linestyle='--', alpha=0.5, linewidth=1.5)
    axes[1].set_xlabel('R² (maior é melhor)', fontsize=11)
    axes[1].set_title(f'R² por Empresa (Top 20 + Outras)', fontsize=12)
    axes[1].grid(True, alpha=0.3)
    max_r2 = df_sorted_r2['r2_test'].max()
    min_r2 = df_sorted_r2['r2_test'].min()
    margin = (max_r2 - min_r2) * 0.15 if max_r2 != min_r2 else 0.01
    axes[1].set_xlim(min_r2 - margin * 0.2, max_r2 + margin)

    for i, v in enumerate(df_sorted_r2['r2_test']):
        axes[1].text(offset, i, f'{v:.4f}', va='center', ha='right',
                     fontsize=10, color='black', fontweight='bold')
    if 'Outras' in df_sorted_r2.index:
        axes[1].barh('Outras', df_sorted_r2.loc['Outras', 'r2_test'],
                     color='orange', alpha=0.8, edgecolor='darkorange', linewidth=2)

plt.tight_layout()
plt.show()

if len(df_resultados) > 20:
    fig2, (ax1) = plt.subplots(1, 1, figsize=(10, 12))
    sizes = [len(df_top20), len(df_outras)]
    labels = [f'Top 20\n({len(df_top20)} empresas)', f'Outras\n({len(df_outras)} empresas)']
    colors_pie = ['steelblue', 'orange']
    explode = (0.05, 0.05)
    ax1.pie(sizes, explode=explode, labels=labels, colors=colors_pie,
            autopct='%1.1f%%', shadow=True, startangle=90)
    ax1.set_title('Distribuição de Empresas', fontsize=12)
    metrics = ['RMSE', 'R²']
    top20_values = [df_top20['rmse_test'].mean(), df_top20['r2_test'].mean()]
    outras_values = [df_outras['rmse_test'].mean(), df_outras['r2_test'].mean()]
    plt.tight_layout()
    plt.show()

# Informações adicionais sobre "Outras"
print(f"\n\t\tRESUMO DAS 'OUTRAS' EMPRESAS:")
print(f"    • Quantidade: {len(df_outras)} empresas")
print(f"    • RMSE médio: {df_outras['rmse_test'].mean():.6f}")
print(f"    • R² médio: {df_outras['r2_test'].mean():.4f}")
if 'mae_test' in df_outras.columns:
    print(f"    • MAE médio: {df_outras['mae_test'].mean():.6f}")

```


RESULTADOS POR EMPRESA:

	rmse_test	mae_test	r2_test	n_train	n_val	n_test
AAPL	0.019984	0.012569	-0.206041	2020.0	433.0	433.0
MSFT	0.018716	0.012918	-0.208949	2020.0	433.0	433.0
GOOGL	0.020060	0.014751	-0.020278	2020.0	433.0	433.0
AMZN	0.021633	0.015706	-0.084174	2020.0	433.0	433.0
TSLA	0.039022	0.027873	-0.044447	2020.0	433.0	433.0
NVDA	0.028932	0.020730	-0.046948	2020.0	433.0	433.0
MRVL	0.050163	0.033334	-0.151693	2020.0	433.0	433.0
PEP	0.014038	0.010447	-0.076172	2020.0	433.0	433.0
NFLX	0.023581	0.016404	-0.159320	2020.0	433.0	433.0
AMD	0.040434	0.027599	-0.055774	2020.0	433.0	433.0
GOOG	0.019907	0.014562	-0.031865	2020.0	433.0	433.0
AVGO	0.034606	0.023593	-0.040942	2020.0	433.0	433.0
META	0.024408	0.016648	-0.085677	2020.0	433.0	433.0
MU	0.046774	0.034110	-0.107935	2020.0	433.0	433.0
WMT	0.015923	0.011280	-0.056032	2020.0	433.0	433.0
COST	0.014048	0.010381	-0.149674	2020.0	433.0	433.0
ADBE	0.022945	0.015716	-0.053692	2020.0	433.0	433.0
CMCSA	0.018464	0.012634	-0.004953	2020.0	433.0	433.0
CSCO	0.020311	0.013022	-0.294118	2020.0	433.0	433.0
INTC	0.047715	0.033667	-0.182184	2020.0	433.0	433.0
QCOM	0.029434	0.019260	-0.013305	2020.0	433.0	433.0
TXN	0.029102	0.019634	-0.195210	2020.0	433.0	433.0
AMGN	0.017416	0.012562	-0.020975	2020.0	433.0	433.0
ISRG	0.021175	0.014236	-0.020991	2020.0	433.0	433.0
TMUS	0.017971	0.012872	-0.111944	2020.0	433.0	433.0
HON	0.016026	0.011387	-0.000306	2020.0	433.0	433.0
BKNG	0.021349	0.015777	-0.138334	2020.0	433.0	433.0
AMAT	0.031822	0.022364	-0.021401	2020.0	433.0	433.0
LRCX	0.034191	0.025473	-0.029686	2020.0	433.0	433.0
TEAM	0.040775	0.028699	-0.106374	1854.0	398.0	398.0
GILD	0.017570	0.013088	-0.059991	2020.0	433.0	433.0
CSX	0.016226	0.011360	-0.092156	2020.0	433.0	433.0
ADI	0.023754	0.016550	0.027242	2020.0	433.0	433.0
VRTX	0.021259	0.014031	-0.043685	2020.0	433.0	433.0
ADP	0.014978	0.010547	-0.085879	2020.0	433.0	433.0
REGN	0.024528	0.017045	-0.100117	2020.0	433.0	433.0
KLAC	0.032844	0.022756	-0.093747	2020.0	433.0	433.0
MELI	0.027179	0.018912	-0.133165	2020.0	433.0	433.0
MPWR	0.038087	0.026546	-0.051496	2020.0	433.0	433.0
PCAR	0.018535	0.013988	-0.044622	2020.0	433.0	433.0
SNPS	0.034611	0.022134	-0.164154	2020.0	433.0	433.0
KDP	0.018367	0.012745	-0.298782	2020.0	433.0	433.0
CDNS	0.026170	0.018274	-0.025176	2020.0	433.0	433.0
MRNA	0.043016	0.032132	-0.073455	1327.0	284.0	285.0
ABNB	0.019346	0.014701	-0.102299	972.0	209.0	209.0
WDAY	0.028111	0.019984	-0.130561	2020.0	433.0	433.0
CHTR	0.029386	0.019164	-0.043355	2020.0	433.0	433.0
LIN	0.012002	0.008836	-0.054239	2020.0	433.0	433.0

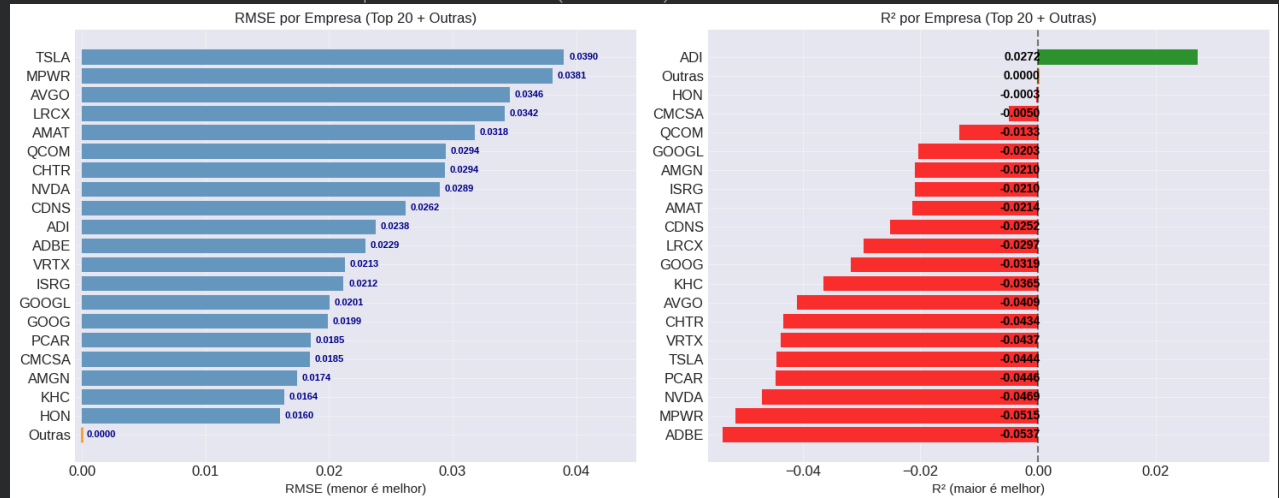


KHC	0.016389	0.012641	-0.036506	1931.0	415.0	414.0
-----	----------	----------	-----------	--------	-------	-------

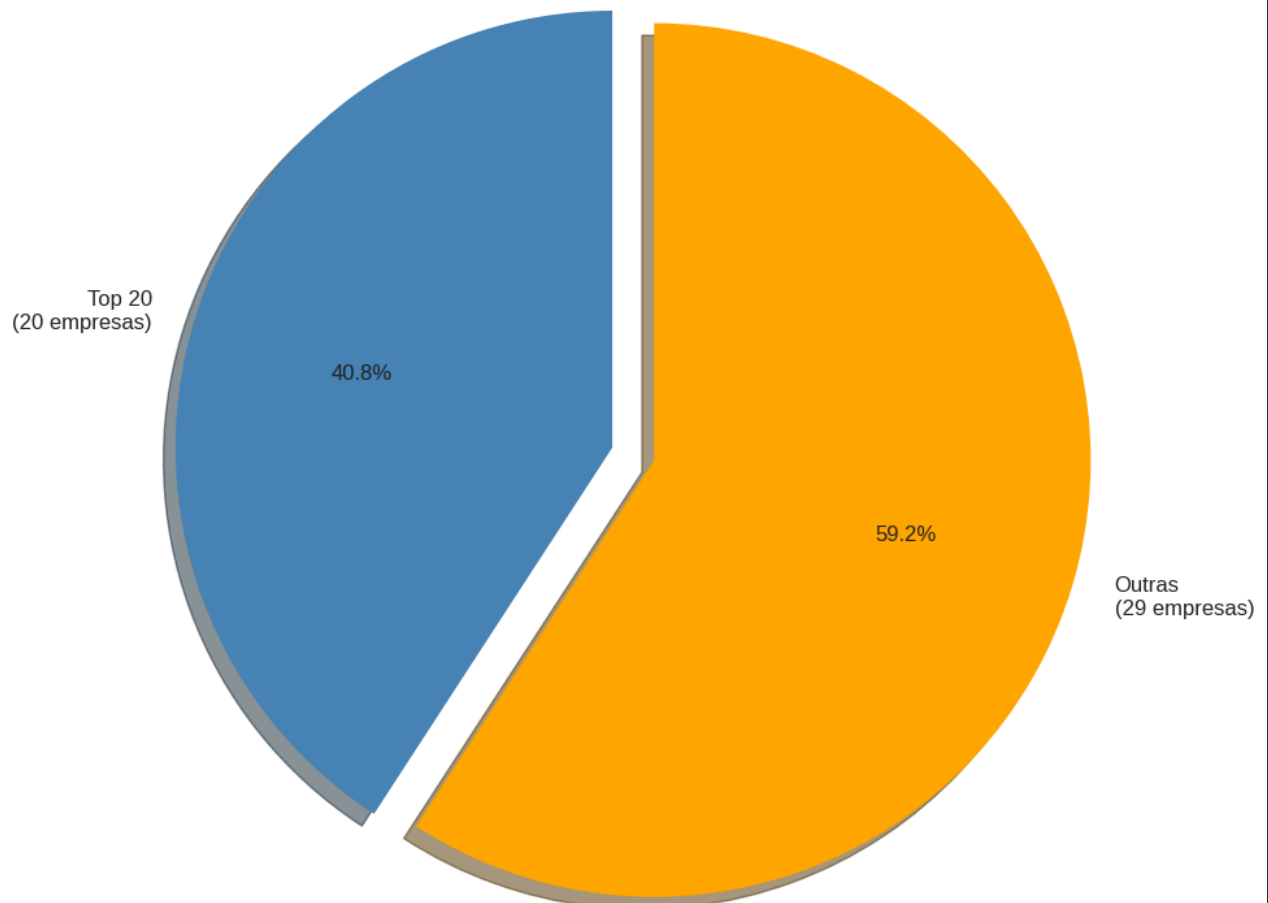
MÉDIAS GERAIS:

- RMSE médio: 0.025781
- MAE médio: 0.018074
- R^2 médio: -0.0883

NOTA: Mostrando as 20 melhores empresas + 'Outras' (total: 49)



Distribuição de Empresas



RESUMO DAS 'OUTRAS' EMPRESAS:

- Quantidade: 29 empresas
- RMSE médio: 0.026059
- R^2 médio: -0.1296
- MAE médio: 0.018252

✓ Acurácia e acertos

- Acertou a direção?
- Subiu ou deceu?

```
NOME_RESULTADOS_DIRECIONAIS = "resultados_direcionais"
NOME_RELATORIO_EMPRESAS = "relatorio_empresas"
NOME_COMPARATIVO_EMPRESAS = "comparativo_empresas"
NOME_RESULTADOS_SERIES = "resultados_series_temporais"

resultados_direcionais_cache = CarregarArquivoOnGitHub(NOME_RESULTADOS_DIRECIONAIS)
resultados_direcionais = []
if resultados_direcionais_cache is not None:
    print(f"Resultados direcionais carregados do cache! ({len(resultados_direcionais_cache)} empresas)")
    resultados_direcionais = resultados_direcionais_cache
else:
    print("Resultados direcionais não encontrados em cache. Serão gerados novos.")

relatorio_empresas_cache = CarregarArquivoOnGitHub(NOME_RELATORIO_EMPRESAS)
df_relatorios_cache = None
if relatorio_empresas_cache is not None:
    print(f"Relatório de empresas carregado do cache! ({len(relatorio_empresas_cache)} empresas)")
    df_relatorios_cache = pd.DataFrame(relatorio_empresas_cache)
else:
    print("Relatório de empresas não encontrado em cache. Será gerado novo.")

comparativo_cache = CarregarArquivoOnGitHub(NOME_COMPARATIVO_EMPRESAS)
df_comparativo_cache = None
if comparativo_cache is not None:
    print(f"Comparativo de empresas carregado do cache! ({len(comparativo_cache)} empresas)")
    df_comparativo_cache = pd.DataFrame(comparativo_cache)
else:
    print("Comparativo de empresas não encontrado em cache. Será gerado novo.")

resultados_series_cache = CarregarArquivoOnGitHub(NOME_RESULTADOS_SERIES)
if resultados_series_cache is not None:
    print(f"Resultados de séries temporais carregados do cache!")
else:
    print("Resultados de séries temporais não encontrados em cache.")
```

```
resultados_direcionais.pkl carregado com sucesso!
Tipo: list
Tamanho: 10,698 bytes
Resultados direcionais carregados do cache! (50 empresas)
relatorio_empresas.pkl carregado com sucesso!
Tipo: list
Tamanho: 9,508 bytes
Relatório de empresas carregado do cache! (49 empresas)
comparativo_empresas.pkl carregado com sucesso!
Tipo: list
Tamanho: 4,113 bytes
Comparativo de empresas carregado do cache! (49 empresas)
resultados_series_temporais.pkl carregado com sucesso!
Tipo: dict
Tamanho: 6,663 bytes
Resultados de séries temporais carregados do cache!
```

```
if has_pred_data:
    # Converter previsões para direções
    y_true_dir = (y_test_full > 0).astype(int)
    y_pred_dir = (y_pred_full > 0).astype(int)

    # Métricas gerais
    accuracy_global = accuracy_score(y_true_dir, y_pred_dir)
    precision_global = precision_score(y_true_dir, y_pred_dir, zero_division=0)
    recall_global = recall_score(y_true_dir, y_pred_dir, zero_division=0)
    f1_global = f1_score(y_true_dir, y_pred_dir, zero_division=0)

    # Matriz de confusão global
    cm_global = confusion_matrix(y_true_dir, y_pred_dir)
    tn_global, fp_global, fn_global, tp_global = cm_global.ravel()

    print(f"\n MÉTRICAS GERAIS:")
    print(f" • Acurácia: {accuracy_global:.4f} ({accuracy_global*100:.2f}%)")
    print(f" • Precisão: {precision_global:.4f}")
    print(f" • Recall: {recall_global:.4f}")
    print(f" • F1-Score: {f1_global:.4f}")

    print(f"\n MATRIZ DE CONFUSÃO GLOBAL:")
    print(f" • Verdadeiros Positivos (TP): {tp_global:,}")
```

```

print(f" • Verdadeiros Negativos (TN): {tn_global:,}")
print(f" • Falsos Positivos (FP): {fp_global:,}")
print(f" • Falsos Negativos (FN): {fn_global:,}")

# Calcular taxas
tpr = tp_global / (tp_global + fn_global) if (tp_global + fn_global) > 0 else 0
tnr = tn_global / (tn_global + fp_global) if (tn_global + fp_global) > 0 else 0
fpr = fp_global / (fp_global + tn_global) if (fp_global + tn_global) > 0 else 0
fnr = fn_global / (fn_global + tp_global) if (fn_global + tp_global) > 0 else 0

print(f"\n TAXAS DERIVADAS:")
print(f" • Taxa de Verdadeiros Positivos (TPR/Sensibilidade): {tpr:.4f}")
print(f" • Taxa de Verdadeiros Negativos (TNR/Especificidade): {tnr:.4f}")
print(f" • Taxa de Falsos Positivos (FPR): {fpr:.4f}")
print(f" • Taxa de Falsos Negativos (FNR): {fnr:.4f}")

# Análise do viés direcional
real_alta_pct = (y_true_dir.sum() / len(y_true_dir)) * 100
pred_alta_pct = (y_pred_dir.sum() / len(y_pred_dir)) * 100
print(f"\n VIÉS DIRECIONAL:")
print(f" • Dias de alta reais: {real_alta_pct:.1f}%")
print(f" • Dias de alta previstos: {pred_alta_pct:.1f}%")
print(f" • Viés: {pred_alta_pct - real_alta_pct:.1f}%")

```

MÉTRICAS GERAIS:

- Acurácia: 0.5227 (52.27%)
- Precisão: 0.5226
- Recall: 0.9497
- F1-Score: 0.6742

MATRIZ DE CONFUSÃO GLOBAL:

- Verdadeiros Positivos (TP): 15,049
- Verdadeiros Negativos (TN): 883
- Falsos Positivos (FP): 13,749
- Falsos Negativos (FN): 797

TAXAS DERIVADAS:

- Taxa de Verdadeiros Positivos (TPR/Sensibilidade): 0.9497
- Taxa de Verdadeiros Negativos (TNR/Especificidade): 0.0603
- Taxa de Falsos Positivos (FPR): 0.9397
- Taxa de Falsos Negativos (FNR): 0.0503

VIÉS DIRECIONAL:

- Dias de alta reais: 52.0%
- Dias de alta previstos: 94.5%
- Viés: 42.5%

▼ Resultados direcionais por empresa

```

if not resultados_direcionais and has_pred_data:
    print("\nGERANDO RESULTADOS DIRECIONAIS POR EMPRESA...")
    print("\n - Média 1 minuto por empresa.")
    resultado_global = analisar_acuracia_direcional(y_test_full, y_pred_full)
    resultados_direcionais.append(resultado_global)
    if resultados_por_empresa is not None:
        for idx, empresa in enumerate(simbolos_usados):
            try:
                if empresa not in resultados_por_empresa:
                    continue
                df_emp = df_analise[df_analise['Symbol'] == empresa].copy()
                df_emp = df_emp.sort_values('Date')
                X_emp, y_emp = prepare_data_for_model(df_emp)
                if len(X_emp) == 0:
                    continue
                n = len(X_emp)
                train_end = int(n * 0.85)
                X_train_emp = X_emp.iloc[:train_end]
                y_train_emp = y_emp.iloc[:train_end]
                X_test_emp = X_emp.iloc[train_end:]
                y_test_emp = y_emp.iloc[train_end:]
                if len(X_train_emp) < 100 or len(X_test_emp) < 50:
                    continue
                pipeline_emp = create_pipeline(X_train_emp)
                pipeline_emp.fit(X_train_emp, y_train_emp)
                y_pred_emp = pipeline_emp.predict(X_test_emp)
                resultado = analisar_acuracia_direcional(y_test_emp, y_pred_emp, empresa)
                resultados_direcionais.append(resultado)

            # Limpar memória
            del pipeline_emp, X_train_emp, y_train_emp, X_test_emp, y_test_emp, y_pred_emp
            gc.collect()

```

```

        print(f"    {idx+1}/49: {empresa} - Acurácia: {resultado['accuracy']:.4f}")

    except Exception as e:
        print(f"Erro ao processar {empresa}: {str(e)[:100]}")
        continue

# Salvar resultados direcionais
SalvarTreinamentoOnGitHub(
    nome_arquivo=NOME_RESULTADOS_DIRECIONAIS,
    dados=resultados_direcionais,
    metadados={
        'descricao': 'Resultados de acurácia direcional',
        'total_empresas': len(resultados_direcionais) - 1,
        'data_criacao': pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S')
    }
)
print("Resultados direcionais salvos no GitHub!")

```

```

if resultados_direcionais:
    df_resultados_dir = pd.DataFrame([
        'Empresa': r['empresa'],
        'Acurácia': r['accuracy'],
        'Precisão': r['precision'],
        'Recall': r['recall'],
        'F1-Score': r['f1'],
        'TP': r['tp'],
        'TN': r['tn'],
        'FP': r['fp'],
        'FN': r['fn'],
        'Total': r['total']
    ] for r in resultados_direcionais])

    df_resultados_dir = df_resultados_dir.sort_values('Acurácia', ascending=False)

    print("\nRESUMO POR EMPRESA:")
    print(df_resultados_dir.round(4).to_string(index=False))
    print(f"\n    MÉDIAS GERAIS:")
    print(f"    • Acurácia Média: {df_resultados_dir['Acurácia'].mean():.4f}")
    print(f"    • Precisão Média: {df_resultados_dir['Precisão'].mean():.4f}")
    print(f"    • Recall Médio: {df_resultados_dir['Recall'].mean():.4f}")
    print(f"    • F1-Score Médio: {df_resultados_dir['F1-Score'].mean():.4f}")

    melhor_empresa = df_resultados_dir.loc[df_resultados_dir['Acurácia'].idxmax()]
    pior_empresa = df_resultados_dir.loc[df_resultados_dir['Acurácia'].idxmin()]

    print(f"\n    MELHOR EMPRESA:")
    print(f"    • {melhor_empresa['Empresa']}: Acurácia = {melhor_empresa['Acurácia']:.4f}")
    print(f"\n    PIOR EMPRESA:")
    print(f"    • {pior_empresa['Empresa']}: Acurácia = {pior_empresa['Acurácia']:.4f}")

```

PCAR	0.5266	0.5128	0.1932	0.2807	40	188	38	167	433
MRNA	0.5263	0.5357	0.4196	0.4706	60	90	52	83	285
Geral	0.5227	0.5226	0.9497	0.6742	15049	883	13749	797	30478
MPWR	0.5219	0.5680	0.4174	0.4812	96	130	73	134	433
ADP	0.5219	0.5325	0.3779	0.4420	82	144	72	135	433
ADBE	0.5173	0.5070	0.1714	0.2562	36	188	35	174	433
HON	0.5173	0.6250	0.0237	0.0457	5	219	3	206	433
ABNB	0.5167	0.5346	0.7589	0.6273	85	23	74	27	209
REGN	0.5150	0.4935	0.7427	0.5930	153	70	157	53	433
PEP	0.5150	0.4911	0.6732	0.5679	138	85	143	67	433
AMZN	0.5104	0.6939	0.1472	0.2429	34	187	15	197	433
SNPS	0.5081	0.5260	0.4099	0.4608	91	129	82	131	433
AAPL	0.5081	0.5977	0.2261	0.3281	52	168	35	178	433
COST	0.5058	0.5111	0.1070	0.1769	23	196	22	192	433
CSX	0.5058	0.5714	0.3574	0.4398	84	135	63	151	433
ADI	0.5058	0.5257	0.6272	0.5720	143	76	129	85	433
WMT	0.5035	0.5353	0.5560	0.5455	129	89	112	103	433
TXN	0.5035	0.5168	0.3500	0.4173	77	141	72	143	433
WDAY	0.5035	0.5136	0.5113	0.5125	113	105	107	108	433
KDP	0.5012	0.5000	0.7130	0.5878	154	63	154	62	433
LITN	0.5012	0.5126	0.2785	0.3609	61	156	58	158	433
QCOM	0.5012	0.5750	0.2018	0.2987	46	171	34	182	433
TSLA	0.5012	0.5238	0.1009	0.1692	22	195	20	196	433
NFLX	0.4988	0.6667	0.0092	0.0181	2	214	1	216	433
TMUS	0.4965	0.5201	0.6739	0.5871	155	60	143	75	433
CDNS	0.4965	0.6250	0.1304	0.2158	30	185	18	200	433

BRNG	0.4827	0.7500	0.0263	0.0508	6 203	2 222	433
META	0.4827	0.4091	0.0409	0.0744	9 200	13 211	433
NVDA	0.4781	0.5366	0.0961	0.1630	22 185	19 207	433
MSFT	0.4758	0.4824	0.1830	0.2654	41 165	44 183	433
TEAM	0.4698	0.4615	0.8804	0.6056	162 25	189 22	398
AMD	0.4688	0.4974	0.4192	0.4550	96 107	97 133	433
MRVL	0.4688	0.5140	0.2361	0.3235	55 148	52 178	433
MELI	0.4688	0.3000	0.0270	0.0496	6 197	14 216	433
GILD	0.4642	0.5000	0.0043	0.0085	1 200	1 231	433
KLAC	0.4480	0.5614	0.2530	0.3488	64 130	50 189	433
CSCO	0.4434	0.5000	0.0373	0.0695	9 183	9 232	433
LRCX	0.4342	0.4928	0.1393	0.2173	34 154	35 210	433
AMAT	0.4180	0.4570	0.2887	0.3538	69 112	82 170	433

MÉDIAS GERAIS:

- Acurácia Média: 0.4973
- Precisão Média: 0.5299
- Recall Médio: 0.3673
- F1-Score Médio: 0.3726

MELHOR EMPRESA:

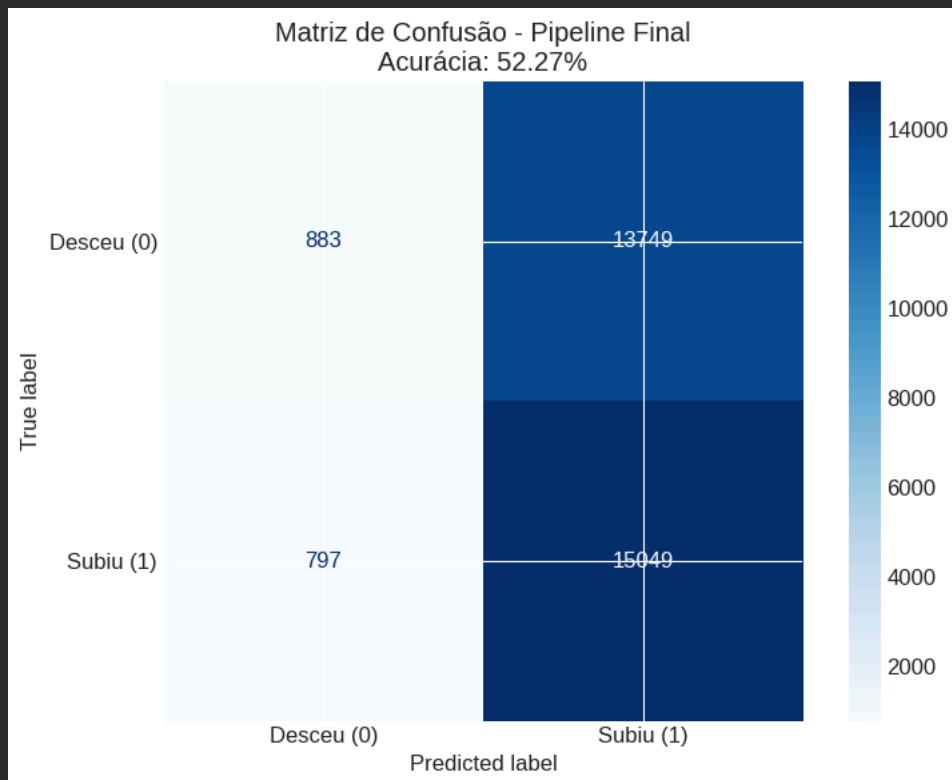
- MU: Acurácia = 0.5427

PIOR EMPRESA:

AMAT: Acurácia = 0.4180

Matriz de confusão - Pipeline Final

```
if has_pred_data:
    fig, ax = plt.subplots(figsize=(8, 6))
    cm_display = ConfusionMatrixDisplay(confusion_matrix=cm_global,
                                         display_labels=['Desceu (0)', 'Subiu (1)'])
    cm_display.plot(ax=ax, cmap='Blues', values_format='d')
    ax.set_title(f'Matriz de Confusão - Pipeline Final\nAcurácia: {accuracy_global:.2%}')
    plt.tight_layout()
    plt.show()
```



Matriz de confusão detalhada por empresa

```
if resultados_direcionais and len(resultados_direcionais) >= 6:
    top_empresas = df_resultados_dir.nlargest(6, 'Acurácia')['Empresa'].tolist()
    fig, axes = plt.subplots(2, 3, figsize=(15, 10))
    axes = axes.flatten()
    for idx, empresa in enumerate(top_empresas):
        if idx < len(axes):
            res = next((r for r in resultados_direcionais if r['empresa'] == empresa), None)
            if res is not None:
                cm = np.array(res['confusion_matrix'])
                accuracy = res['accuracy']
```

```

cm_display = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Desceu', 'Subiu'])
cm_display.plot(ax=axes[idx], cmap='Blues', values_format='d')
axes[idx].set_title(f'{empresa}\nAcurácia: {accuracy:.2%}')

for j in range(len(top_empresas), len(axes)):
    axes[j].set_visible(False)

plt.suptitle('Matrizes de Confusão - Top 6 Empresas', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

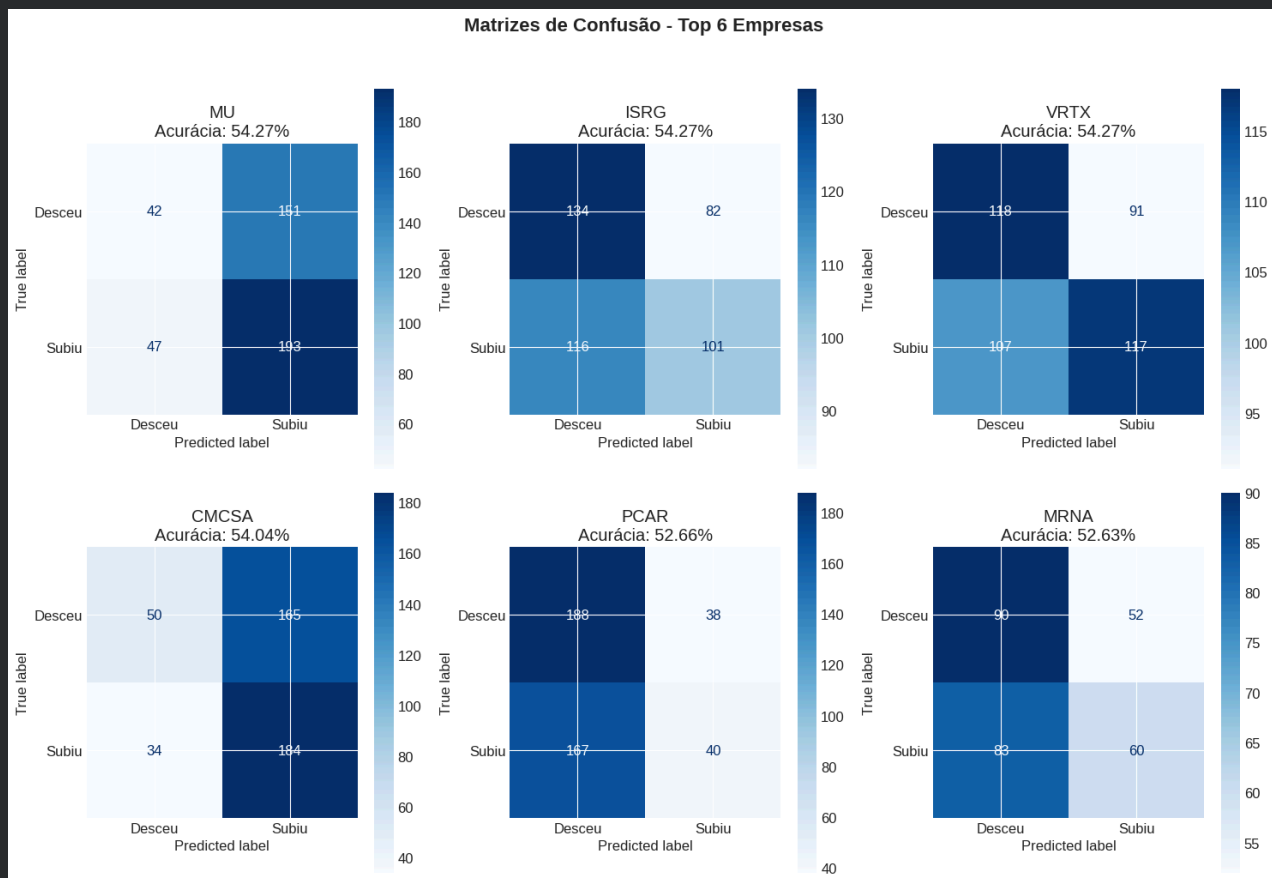


Gráfico de matrizes de confusão por empresa

```

if has_pred_data:
    print("\n GERANDO GRÁFICOS DE ANÁLISE...")
    try:
        fig, axes = plt.subplots(2, 2, figsize=(14, 12))

        # Gráfico 1: Valores Previstos vs Reais
        ax = axes[0, 0]
        sample_size = min(1000, len(y_test_full))
        indices = np.random.choice(len(y_test_full), sample_size, replace=False)

        ax.scatter(y_test_full.iloc[indices], y_pred_full[indices], alpha=0.3, s=10, color='steelblue')
        ax.plot([y_test_full.min(), y_test_full.max()], [y_test_full.min(), y_test_full.max()],
                'r--', linewidth=2, label='Perfect Prediction')
        ax.set_xlabel('Valor Real')
        ax.set_ylabel('Valor Previsto')
        ax.set_title(f'Valores Previstos vs Reais\nRMSE: {rmse_final:.4f}, R²: {r2_final:.4f}')

```

```

ax.legend()
ax.grid(True, alpha=0.3)

# Gráfico 2: Distribuição dos Erros
ax = axes[0, 1]
errors = y_pred_full - y_test_full
ax.hist(errors, bins=50, color='coral', alpha=0.7, edgecolor='black')
ax.axvline(0, color='red', linestyle='--', linewidth=2, label='Erro Zero')
ax.axvline(np.mean(errors), color='blue', linestyle='-', linewidth=2,
            label=f'Média: {np.mean(errors):.4f}')
ax.axvline(np.median(errors), color='green', linestyle='-', linewidth=2,
            label=f'Mediana: {np.median(errors):.4f}')
ax.set_xlabel('Erro (Previsto - Real)')
ax.set_ylabel('Frequência')
ax.set_title('Distribuição dos Erros de Previsão')
ax.legend()
ax.grid(True, alpha=0.3)

# Gráfico 3: Erro por Valor Real
ax = axes[1, 0]
ax.scatter(y_test_full, errors, alpha=0.3, s=10, color='purple')
ax.axhline(0, color='red', linestyle='--', linewidth=2)
ax.set_xlabel('Valor Real')
ax.set_ylabel('Erro')
ax.set_title('Erro vs Valor Real')
ax.grid(True, alpha=0.3)

# Gráfico 4: Análise de Resíduos
ax = axes[1, 1]
residuals = errors / np.std(errors)
ax.scatter(y_pred_full, residuals, alpha=0.3, s=10, color='teal')
ax.axhline(0, color='red', linestyle='--', linewidth=2)
ax.axhline(2, color='orange', linestyle=':', linewidth=1, alpha=0.7)
ax.axhline(-2, color='orange', linestyle=':', linewidth=1, alpha=0.7)
ax.set_xlabel('Valor Previsto')
ax.set_ylabel('Resíduo Padronizado')
ax.set_title('Análise de Resíduos')
ax.grid(True, alpha=0.3)

plt.suptitle('ANÁLISE DO PIPELINE FINAL', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

print("\n ANÁLISE ESTATÍSTICA DOS ERROS:")
print(f"    • Média dos Erros: {np.mean(errors):.6f}")
print(f"    • Mediana dos Erros: {np.median(errors):.6f}")
print(f"    • Desvio Padrão: {np.std(errors):.6f}")
print(f"    • Erro Máximo: {np.max(errors):.6f}")
print(f"    • Erro Mínimo: {np.min(errors):.6f}")

percentis = [1, 5, 10, 25, 50, 75, 90, 95, 99]
print(f"\n    • Percentis:")
for p in percentis:
    val = np.percentile(errors, p)
    print(f"        {p}%: {val:.6f}")

mape = calcular_mape_divisao_zero(y_test_full, y_pred_full)
print(f"\n    • MAPE (Erro Percentual Médio Absoluto): {mape:.2f}%")

except Exception as e:
    print(f"Erro ao gerar gráficos: {e}")
    traceback.print_exc()

```

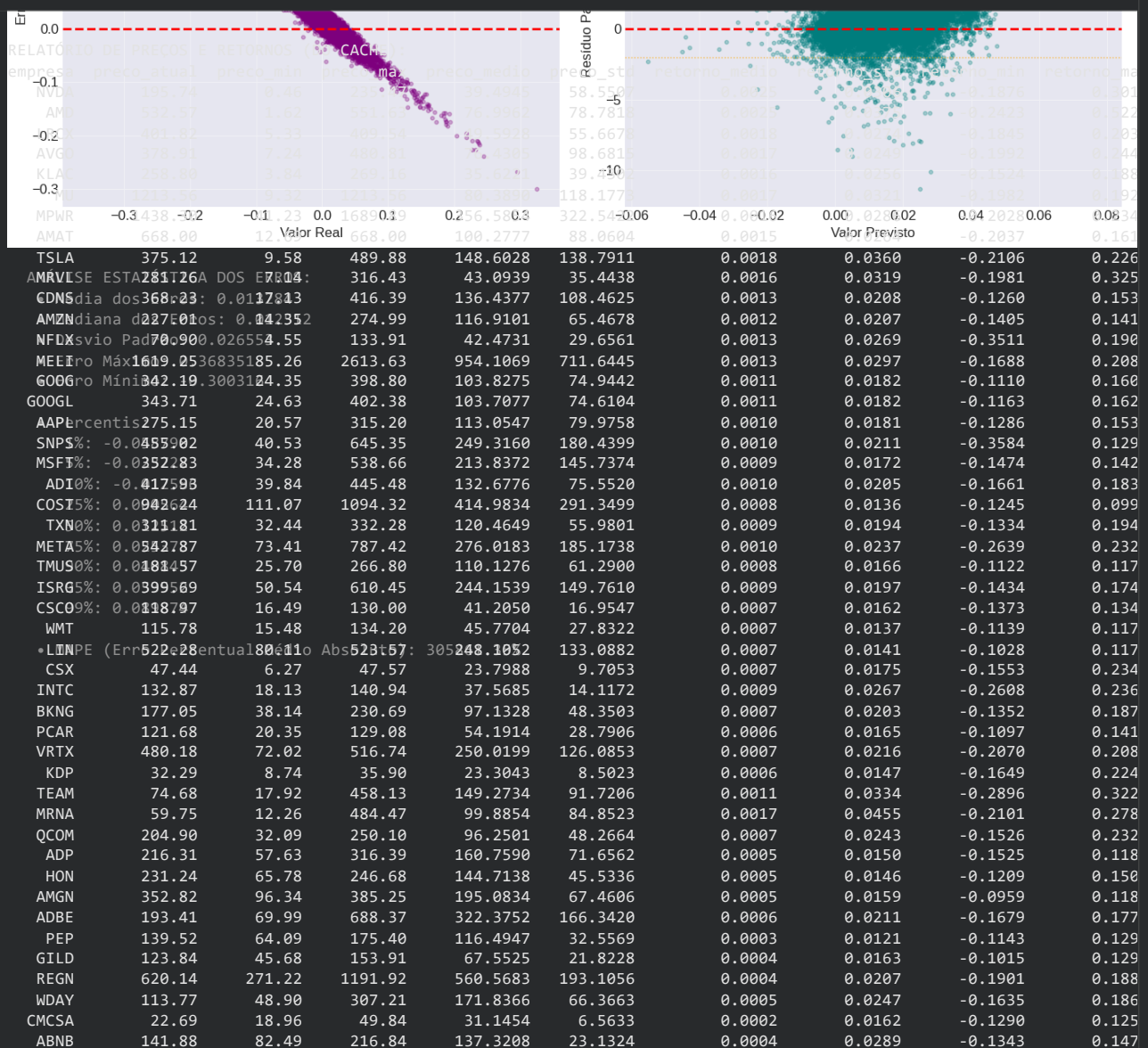

GERANDO GRÁFICOS DE ANÁLISE...

Relatório completo de preços e retornos por empresa

ANÁLISE DO PIPELINE FINAL

```
if df_relatorios_cache is None:
    print("\nGERANDO RELATÓRIO DE EMPRESAS...")
    relatorios_empresas = []
    for empresa in simbolos_usados:
        try:
            relatorio = gerar_relatorio_empresa(empresa, df_analise)
            if relatorio is not None:
                relatorios_empresas.append(relatorio)
        except Exception as e:
            print(f"Erro ao gerar relatório para {empresa}: {str(e)[:100]}")

    if relatorios_empresas:
        df_relatorios = pd.DataFrame(relatorios_empresas)
        df_relatorios = df_relatorios.sort_values('retorno_acumulado', ascending=False)
        print("\nRELATÓRIO DE PREÇOS E RETORNOS:")
        print(df_relatorios.round(4).to_string(index=False))
        SalvarTreinamentoOnGitHub(
            nome_arquivo=NOME_RELATORIO_EMPRESAS,
            dados=df_relatorios.to_dict('records'),
            metadados={
                'descricao': 'Relatório completo de preços e retornos por empresa',
                'total_empresas': len(df_relatorios),
                'data_criacao': pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S')
            }
        )
        print("Relatório de empresas salvo no GitHub!")
    else:
        df_relatorios = df_relatorios_cache
        print("\nRELATÓRIO DE PREÇOS E RETORNOS (DO CACHE):")
        print(df_relatorios.round(4).to_string(index=False))
```



CHTR	129.65	125.54	821.01	372.8400	148.9906	0.0002	0.0213	-0.2550	0.166
KHC	23.47	14.81	61.91	34.3032	11.4084	-0.0001	0.0169	-0.2746	0.195

Resumo final comparativo entre empresas

```
if df_comparativo_cache is None and df_relatorios is not None and resultados_direcionais:
    print("\nGERANDO COMPARATIVO DE EMPRESAS...")

    df_comparativo = df_relatorios.copy()
    for res in resultados_direcionais:
        if res['empresa'] != 'Geral':
            mask = df_comparativo['empresa'] == res['empresa']
            if mask.any():
                df_comparativo.loc[mask, 'acuracia'] = res['accuracy']
                df_comparativo.loc[mask, 'f1_score'] = res['f1']

    colunas = ['empresa', 'retorno_acumulado', 'sharpe_ratio', 'volatilidade_anual',
               'pct_alta', 'acuracia', 'f1_score']
    df_comparativo = df_comparativo[colunas]
    df_comparativo = df_comparativo.sort_values('retorno_acumulado', ascending=False)

    print("\n COMPARATIVO COMPLETO:")
    print(df_comparativo.round(4).to_string(index=False))
    print(f"\n  ESTATÍSTICAS GERAIS:")
    print(f"  • Empresas analisadas: {len(df_comparativo)}")
    print(f"  • Retorno acumulado médio: {df_comparativo['retorno_acumulado'].mean():.4f}")
    print(f"  • Sharpe ratio médio: {df_comparativo['sharpe_ratio'].mean():.4f}")
    print(f"  • Volatilidade anual média: {df_comparativo['volatilidade_anual'].mean():.4f}")
    print(f"  • Acurácia média: {df_comparativo['acuracia'].mean():.4f}")
    print(f"\n  CORRELAÇÕES ENTRE MÉTRICAS:")

    corr_matrix = df_comparativo.select_dtypes(include=[np.number]).corr()

    print(corr_matrix.round(4))

    SalvarTreinamentoOnGitHub(
        nome_arquivo=NOME_COMPARATIVO_EMPRESAS,
        dados=df_comparativo.to_dict('records'),
        metadados={
            'descricao': 'Resumo final comparativo entre empresas',
            'total_empresas': len(df_comparativo),
            'data_criacao': pd.Timestamp.now().strftime('%Y-%m-%d %H:%M:%S')
        }
    )
    print("Resumo comparativo salvo no GitHub!")
else:
    if df_comparativo_cache is not None:
        df_comparativo = df_comparativo_cache
        print(df_comparativo.round(4).to_string(index=False))
```

empresa	retorno_acumulado	sharpe_ratio	volatilidade_anual	pct_alta	acuracia	f1_score
NVDA	400.1042	1.3203	0.4851	50.9702	0.4781	0.1630
AMD	194.3483	1.0649	0.5954	50.8663	0.4688	0.4550
LRCX	54.8306	1.0257	0.4345	52.7027	0.4342	0.2173
AVGO	47.4754	1.0557	0.3948	52.7374	0.4942	0.3343
KLAC	42.8519	1.0154	0.4064	53.6729	0.4480	0.3488
MU	32.4119	0.8560	0.5095	51.5246	0.5427	0.6610
MPWR	28.8957	0.8802	0.4545	54.0541	0.5219	0.4812
AMAT	27.8733	0.9118	0.4186	53.1532	0.4180	0.3538
TSLA	24.9720	0.7824	0.5714	51.5246	0.5012	0.1692
MRVL	19.4421	0.7704	0.5072	51.2128	0.4688	0.3235
CDNS	19.0356	0.9579	0.3299	53.8115	0.4965	0.2158
AMZN	14.0804	0.8846	0.3287	53.0839	0.5104	0.2429
NFLX	13.8213	0.7680	0.4266	50.3465	0.4988	0.0181
MELI	12.4460	0.7163	0.4714	52.9453	0.4688	0.0496
GOOG	11.9025	0.9169	0.2890	53.4304	0.4942	0.2626
GOOGL	11.8579	0.9153	0.2893	53.3264	0.4896	0.2848
AAPL	10.7313	0.8922	0.2872	52.9106	0.5081	0.3281
SNPS	9.5415	0.7864	0.3355	54.5738	0.5081	0.4608
MSFT	8.3994	0.8545	0.2725	53.1185	0.4758	0.2654
ADI	7.7735	0.7447	0.3259	52.6681	0.5058	0.5720
COST	7.3688	0.9692	0.2155	53.8462	0.5058	0.1769
TXN	6.3000	0.7169	0.3081	52.9453	0.5035	0.4173
META	6.0754	0.6438	0.3765	52.3562	0.4827	0.0744
TMUS	5.9966	0.7773	0.2631	52.8413	0.4965	0.5871
ISRG	5.9298	0.6963	0.3130	54.7471	0.5427	0.5050
CSCO	4.8046	0.7262	0.2572	52.4948	0.4434	0.0695
WMT	4.0017	0.7550	0.2175	52.5641	0.5035	0.5455
LIN	3.9192	0.7341	0.2235	51.9058	0.5012	0.3609
CSX	3.7660	0.6285	0.2781	50.9009	0.5058	0.4398
INTC	3.5993	0.5255	0.4245	50.9702	0.4942	0.2820
BKNG	3.0568	0.5412	0.3217	52.8760	0.4827	0.0508

PCAR	3.0321	0.5957	0.2618	51.3860	0.5266	0.2807
VRTX	2.9924	0.5245	0.3423	52.5295	0.5427	0.5417
KDP	2.8084	0.6167	0.2326	51.9058	0.5012	0.5878
TEAM	2.7495	0.5001	0.5306	53.5472	0.4698	0.6056
MRNA	2.6167	0.5927	0.7215	48.3650	0.5263	0.4706
QCOM	2.5209	0.4773	0.3853	51.7672	0.5012	0.2987
ADP	2.4637	0.5744	0.2387	53.6036	0.5219	0.4420
HON	2.2738	0.5628	0.2317	52.7720	0.5173	0.0457
AMGN	2.1222	0.5204	0.2517	51.6979	0.4942	0.5350
ADBE	1.8025	0.4375	0.3345	53.1185	0.5173	0.2562
PEP	1.1236	0.4394	0.1914	51.8711	0.5150	0.5679
GILD	0.9632	0.3565	0.2595	50.9702	0.4642	0.0085
REGN	0.5558	0.2813	0.3278	50.3812	0.5150	0.5930
WDAY	0.5447	0.2927	0.3916	52.7720	0.5035	0.5125
CMCSA	0.1369	0.1728	0.2573	51.0049	0.5404	0.6490
ABNB	0.0059	0.2311	0.4591	50.6475	0.5167	0.6273
CHTR	-0.1970	0.1151	0.3378	50.8663	0.4896	0.5725
KHC	-0.4662	-0.0764	0.2687	50.2899	0.4855	0.6432

Resumo final - Gráficos comparativos

```

if df_comparativo is not None and len(df_comparativo) > 0:
    fig, axes = plt.subplots(2, 2, figsize=(16, 12))

    # Gráfico 1: Retorno Acumulado vs Acurácia
    ax = axes[0, 0]
    if 'acuracia' in df_comparativo.columns:
        scatter = ax.scatter(df_comparativo['retorno_acumulado'],
                             df_comparativo['acuracia'],
                             s=100, alpha=0.6, c=df_comparativo['sharpe_ratio'],
                             cmap='RdYlGn')
        ax.set_xlabel('Retorno Acumulado')
        ax.set_ylabel('Acurácia')
        ax.set_title('Retorno Acumulado vs Acurácia\n(Tamanho = Sharpe Ratio)')

        for _, row in df_comparativo.iterrows():
            ax.annotate(row['empresa'],
                        (row['retorno_acumulado'], row['acuracia']),
                        fontsize=8, alpha=0.7)

        ax.grid(True, alpha=0.3)
        plt.colorbar(scatter, ax=ax, label='Sharpe Ratio')

    # Gráfico 2: Volatilidade vs Retorno
    ax = axes[0, 1]
    scatter = ax.scatter(df_comparativo['volatilidade_anual'],
                         df_comparativo['retorno_acumulado'],
                         s=100, alpha=0.6, c=df_comparativo['sharpe_ratio'],
                         cmap='RdYlGn')
    ax.set_xlabel('Volatilidade Anual')
    ax.set_ylabel('Retorno Acumulado')
    ax.set_title('Risco vs Retorno\n(Tamanho = Sharpe Ratio)')

    for _, row in df_comparativo.iterrows():
        ax.annotate(row['empresa'],
                    (row['volatilidade_anual'], row['retorno_acumulado']),
                    fontsize=8, alpha=0.7)

    ax.grid(True, alpha=0.3)
    plt.colorbar(scatter, ax=ax, label='Sharpe Ratio')

    # Gráfico 3: Acurácia vs F1-Score
    ax = axes[1, 0]
    if 'acuracia' in df_comparativo.columns and 'f1_score' in df_comparativo.columns:
        ax.scatter(df_comparativo['acuracia'], df_comparativo['f1_score'],
                   s=100, alpha=0.6, color='steelblue')
        ax.plot([0, 1], [0, 1], 'r--', alpha=0.5, label='Linha de Referência')
        ax.set_xlabel('Acurácia')
        ax.set_ylabel('F1-Score')
        ax.set_title('Acurácia vs F1-Score')

        for _, row in df_comparativo.iterrows():
            ax.annotate(row['empresa'],
                        (row['acuracia'], row['f1_score']),
                        fontsize=8, alpha=0.7)

        ax.legend()
        ax.grid(True, alpha=0.3)

    # Gráfico 4: Top 10 Retornos Acumulados
    ax = axes[1, 1]

```

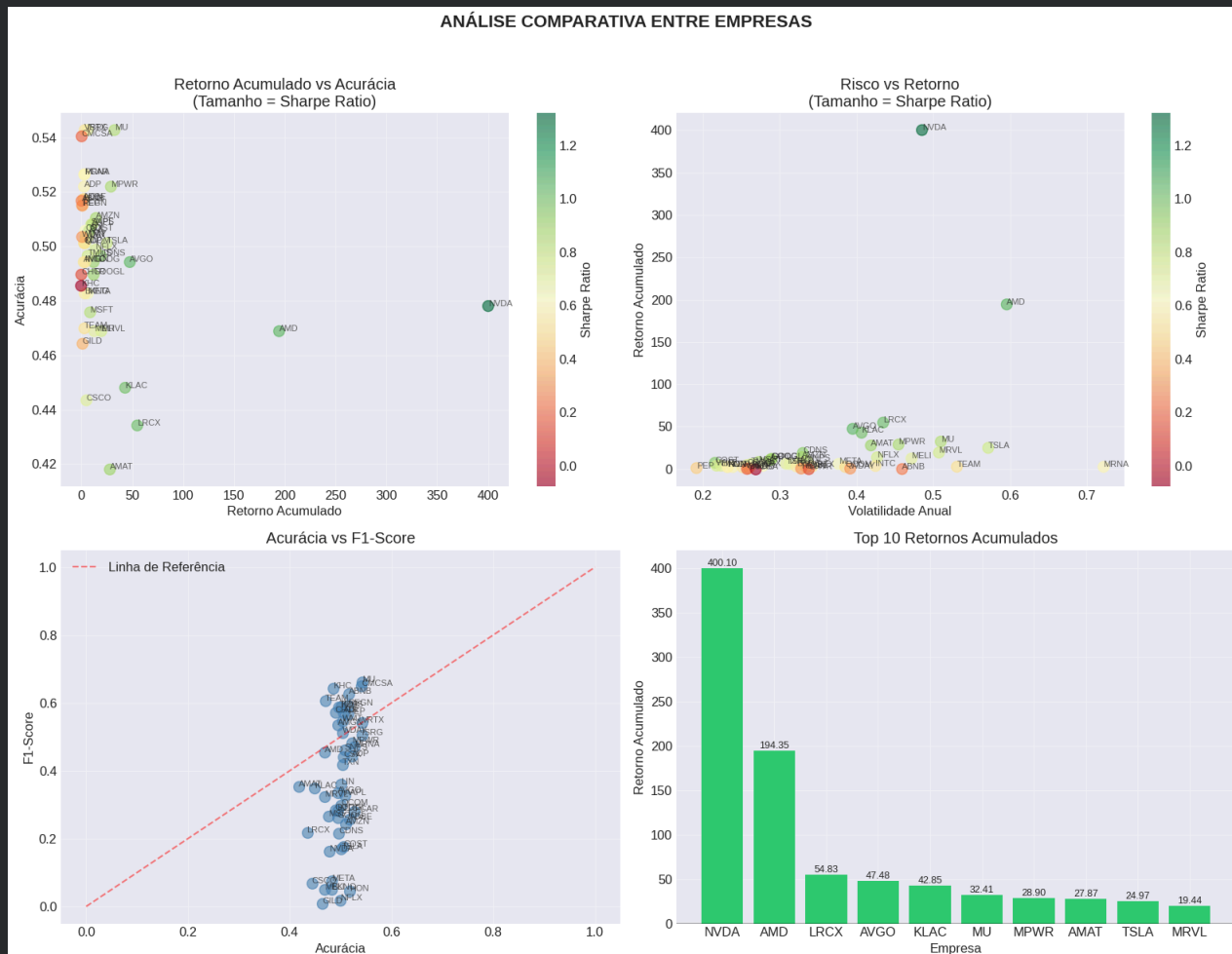
```

top10 = df_comparativo.nlargest(10, 'retorno_acumulado')
colors = ['#2ecc71' if x > 0 else '#e74c3c' for x in top10['retorno_acumulado']]
bars = ax.bar(top10['empresa'], top10['retorno_acumulado'], color=colors)
ax.set_xlabel('Empresa')
ax.set_ylabel('Retorno Acumulado')
ax.set_title('Top 10 Retornos Acumulados')
ax.axhline(0, color='black', linestyle='-', alpha=0.3)
ax.grid(True, alpha=0.3)

for bar, val in zip(bars, top10['retorno_acumulado']):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height(),
            f'{val:.2f}', ha='center', va='bottom' if val > 0 else 'top',
            fontsize=9)

plt.suptitle('ANÁLISE COMPARATIVA ENTRE EMPRESAS', fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```



↘ Análise de acerto por direção

```

if has_pred_data:
    print("\nANÁLISE DE ACERTOS POR DIREÇÃO - PIPELINE FINAL")
    mask_alta = y_true_dir == 1
    mask_baixa = y_true_dir == 0

```

```

acertos_alta = ((y_pred_dir == 1) & (y_true_dir == 1)).sum()
total_alta = mask_alta.sum()
taxa_acerto_alta = acertos_alta / total_alta if total_alta > 0 else 0
acertos_baixa = ((y_pred_dir == 0) & (y_true_dir == 0)).sum()
total_baixa = mask_baixa.sum()
taxa_acerto_baixa = acertos_baixa / total_baixa if total_baixa > 0 else 0

print(f"\n   DIAS DE ALTA (Subiu):")
print(f"   • Total: {total_alta:,}")
print(f"   • Acertos: {acertos_alta:,}")
print(f"   • Taxa de Acerto: {taxa_acerto_alta:.4f} ({taxa_acerto_alta*100:.2f}%)")
print(f"\n   DIAS DE BAIXA (Desceu):")
print(f"   • Total: {total_baixa:,}")
print(f"   • Acertos: {acertos_baixa:,}")
print(f"   • Taxa de Acerto: {taxa_acerto_baixa:.4f} ({taxa_acerto_baixa*100:.2f}%)")

contingency_table = np.array([[tn_global, fp_global], [fn_global, tp_global]])
chi2, p_value, dof, expected = chi2_contingency(contingency_table)

print(f"\n   TESTE DE INDEPENDÊNCIA (Qui-quadrado):")
print(f"   • Qui-quadrado: {chi2:.4f}")
print(f"   • p-valor: {p_value:.6f}")

if p_value < 0.05:
    print(f"   • Há evidência estatística de que o modelo tem poder preditivo (p < 0.05)")
else:
    print(f"   • Não há evidência estatística de poder preditivo (p >= 0.05)")

```

ANÁLISE DE ACERTOS POR DIREÇÃO - PIPELINE FINAL

```

DIAS DE ALTA (Subiu):
• Total: 15,846
• Acertos: 15,049
• Taxa de Acerto: 0.9497 (94.97%)

DIAS DE BAIXA (Desceu):
• Total: 14,632
• Acertos: 883
• Taxa de Acerto: 0.0603 (6.03%)

TESTE DE INDEPENDÊNCIA (Qui-quadrado):
• Qui-quadrado: 14.5620
• p-valor: 0.000136
• Há evidência estatística de que o modelo tem poder preditivo (p < 0.05)

```

```

#Limpeza de variáveis
variaveis_para_limpar = [
    'y_test_full', 'y_pred_full', 'y_pred_dir', 'y_true_dir', 'errors', 'residuals',
    'indices'
]
limpar_memoria(variaveis_para_limpar, mostrar_status=True)

```

RAM: 7.9GB / 12.7GB (64.5%)

✓ Decisões de pré-processamento

1. Por que usei média/mediana/moda para imputação?
2. Por que padronizei ou normalizei os dados?
3. Quais variáveis foram removidas e por quê?
4. Criei novas variáveis? Elas estariam disponíveis no momento real da previsão?

✓ Versão 01

Por que usei média/mediana/moda para imputação?

- **Forward fill e backward fill:** indicadores técnicos (RSI, MACD, MA, BB) - são dados de séries temporais e os valores ausentes ocorrem apenas no início de cada série, em um período de aquecimento, então preencher com o último valor disponível (no caso de backward usa o primeiro valor disponível à frente) respeitando a ordem cronológica não introduzindo algum problema significativo.
- **Zero:** colunas relacionadas à Fibonacci - as ausências de sinais de Fibonacci é interpretada como "nenhum nível relevante", ou seja, valor zero.
- **Mediana:** outras variáveis numéricas (Market_cap_change_pct, volume_dollar) - e a mediana não puxada por extremos.

- **Neutro:** Sinais categóricos (MACD_Sinal, BB_Sinal, RSI_Sinal) - nesses casos não houveram relevâncias de sinais, isto é, não houveram dados suficientes para determinar se é para comprar ou vender.

Por que padronizei ou normalizei os dados?

- **Escala muito diferentes** - variáveis como Volume (centenas de milhões/bilhões/trilhões), Close (dezenas), Volume_Ratio (~1), RSI (0-100) convivem no mesmo dataset e sem uma padronização uma regressão linear e redes neurais seriam dominados pelas variáveis de maior escala.
- **Presença de outliers** foram detectados outliers fortes (ex: Market_Cap da NFLX com 5.12% outliers, Volume com 2% outliers). A RobustScaler foi escolhida porque usa mediana e IQR (Amplitude Interquartil), não é influenciado por outliers ao contrário do StandardScaler.
- **Distribuição não Gaussiana** (Volume_Ratio e Returns) são variáveis que possuem assimetria (skewness $\neq 0$). O RobustScaler não exige normalidade e ainda centraliza os dados (mediana=0).
- **Compatibilidade com modelos** baseados em distância (KNN, SVM, regressão com regularização) e redes neurais exigem entradas na mesma escala.

Quais variáveis foram removidas e por quê?

- **Symbol, Company_Name, Date** - não são features preditivas e causariam overfitting, pois cada símbolo é único, e não estariam disponíveis para generalização. No caso da variável Date foi usada apenas para ordenar e depois foi removida.
- **Target_Direction** - derivada diretamente do target poderia gerar vazamento de informação no futuro.
- **Velas_Group** - criada apenas para análise exploratória e agrupamento de velas consecutivas (indicadores), e não é uma variável original.
- **Adj Close** - correlação quase perfeita com close, e foi mantido apenas o close.
- **Stock Splits** - não agrega poder preditivo e aumentam a dimensionalidade sem benefício.
- **Ex_Dividend_Date, Last_Dividend** - variáveis de data que não são numéricas e sem periodicidade útil para previsão diária.

Criei novas variáveis? Elas estariam disponíveis no momento real da previsão?

- **Volume_Ratio** - foi criada com base nas duas variáveis Volume / Volume_Medio_20 (média móvel do volume) - usa volume do dia atual e média de volume dos últimos 20 dias (dados passados).
- **Range_Percentual** - criada com a seguinte fórmula: $(High - Low) / Close$ - usa preços do próprio dia (abertura, máxima, mínima, fechamento).
- **Gap_Percentual** - criada com a seguinte fórmula: $(Open - Close_{anterior}) / Close_{anterior}$ - usa abertura do dia e fechamento do dia anterior (ambos conhecidos no início do dia).
- **Forca_Tendencia** - Derivada de médias móveis e ângulos (cálculo com dados até o momento) - usa apenas séries históricas e está disponível na previsão real.
- **Velas_Alta_Consecutivas** - Contagem de dias consecutivos com Close > Open até a data atual - baseada apenas no padrão de candles passados.
- **One-hot-encodings** - sobre colunas categóricas (ex: BB_Sinal_Neutro, RSI_Sinal_Sobrevendido) - mapeamento é aprendido no treino e aplicado no futuro.

Todas as novas variáveis estão disponíveis no final para a previsão real

✓ Versão 03

✓ Por que usei média/mediana/moda para imputação?

- **Forward fill e backward fill:** indicadores técnicos (`RSI` , `MACD` , `MA` , `BB`) - são dados de séries temporais e os valores ausentes ocorrem apenas no início de cada série, em um período de aquecimento, então preencher com o último valor disponível (no caso de backward usa o primeiro valor disponível à frente) respeitando a ordem cronológica não introduzindo algum problema significativo.
- **Zero:** colunas relacionadas à Fibonacci - as ausências de sinais de Fibonacci é interpretada como "nenhum nível relevante", ou seja, valor zero.
- **Mediana:** outras variáveis numéricas (`Market_cap_change_pct` , `volume_dollar`) - e a mediana não puxada por extremos.
- **Neutro:** Sinais categóricos (`MACD_Sinal` , `BB_Sinal` , `RSI_Sinal`) - nesses casos não houveram relevâncias de sinais, isto é, não houveram dados suficientes para determinar se é para comprar ou vender.

▼ Por que padronizei ou normalizei os dados?

- **A padronização** (usando `RobustScaler`) foi aplicada por várias razões cruciais, conforme explicado no notebook:
- **Escalas Muito Diferentes:** As variáveis no dataset convivem em escalas totalmente distintas (ex: Volume na casa dos bilhões, `Close` na casa das dezenas, `RSI` na escala 0-100). Sem uma padronização, algoritmos de aprendizado de máquina como regressão linear, `SVM` e redes neurais seriam dominados pelas variáveis com maior magnitude.
- **Presença de Outliers:** Foi detectada a presença de outliers fortes em algumas variáveis (ex: `Market_Cap`). A escolha do `RobustScaler` foi ideal para este cenário, pois ele utiliza a mediana e o `IQR` (Amplitude Interquartil), que são medidas robustas e não são influenciadas por outliers, ao contrário do `StandardScaler` (que usa média e desvio padrão).
- **Distribuição Não Gaussiana:** Variáveis como `Volume_Ratio` e `Returns` apresentam assimetria (`skewness` $\neq 0$). O `RobustScaler` não exige normalidade dos dados e ainda centraliza os dados em torno da mediana (0), o que é benéfico para vários modelos.
- **Compatibilidade com Modelos:** Muitos modelos, como aqueles baseados em distância (`KNN` , `SVM`) e regressões com regularização, requerem que todas as features estejam na mesma escala para um desempenho adequado.

▼ Quais variáveis foram removidas e por quê?

As seguintes variáveis foram removidas da base de features, com as respectivas justificativas:

`Symbol` , `Company_Name` , `Date` :

- **Motivo:** Não são features preditivas. Incluí-las causaria overfitting, pois cada símbolo é único e não se generalizaria para novos dados. A variável `Date` foi usada apenas para ordenação temporal e depois removida para evitar vazamento de informação.

`Target_Direction`

- **Motivo:** É uma variável derivada diretamente do target (`Target_Next_Return`). Incluí-la no treinamento causaria um vazamento de informação (data leakage) severo, pois o modelo "veria" o futuro.

`Adj_Close` :

- **Motivo:** Apresenta uma correlação quase perfeita com a variável `Close`. Mantê-la não agregaria valor e aumentaria a dimensionalidade desnecessariamente.

`Stock_Splits` , `Ex_Dividend_Date` , `Last_Dividend` :

- **Motivo:** Essas variáveis não agregam poder preditivo significativo para a tarefa de previsão diária de retorno. `Stock Splits` e datas de dividendos são eventos esporádicos e não úteis para um modelo de curto prazo, enquanto aumentam a dimensionalidade sem benefício.

▼ Criei novas variáveis? Elas estariam disponíveis no momento real da previsão?

Sim, foram criadas novas variáveis (engenharia de atributos). Todas elas são calculadas a partir de dados históricos e estão disponíveis no momento da previsão real (não há vazamento de informação do futuro).

As principais novas variáveis criadas e suas justificativas são:

`Volume_Ratio` :

- **Fórmula:** `Volume / Volume_Medio_20` (média móvel do volume dos últimos 20 dias).
- **Disponibilidade:** Disponível no momento da previsão, pois usa apenas dados do dia atual e passados.

`Range_Percentual` :

- **Fórmula:** `(High - Low) / Close`

- **Disponibilidade:** Disponível, pois usa os preços do próprio dia (abertura, máxima, mínima e fechamento), que são conhecidos no momento da previsão.

Gap_Percentual:

- **Fórmula:** $(\text{Open} - \text{Close_anterior}) / \text{Close_anterior}$.
- **Disponibilidade:** Disponível, pois usa a abertura do dia e o fechamento do dia anterior (ambos conhecidos).

Forca_Tendencia:

- **Fórmula:** Derivada de médias móveis e ângulos.
- **Disponibilidade:** Disponível, pois é calculada utilizando apenas a série histórica até o momento atual.

Velas_Alta_Consecutivas:

- **Fórmula:** Contagem de dias consecutivos em que $\text{Close} > \text{Open}$.
- **Disponibilidade:** Disponível, pois é baseada apenas no padrão de candles passados.

One-hot-encodings:

- **Fórmula:** Codificação one-hot para variáveis categóricas (ex: `BB_Sinal_Neutro`, `RSI_Sinal_Sobrevendido`).
- **Disponibilidade:** O mapeamento é aprendido na fase de treino e aplicado de forma consistente aos novos dados, garantindo a disponibilidade.

✓ 7. Baseline e modelos candidatos

✓ Funções

```
FORCAR_RECRIACAO = True
MODELS = {
    "Baseline (Média)": MeanBaseline(),
    "Regressão Linear": LinearRegression(),
    "Random Forest": RandomForestRegressor(
        n_estimators=50,
        max_depth=6,
        min_samples_split=20,
        min_samples_leaf=10,
        random_state=SEED,
        n_jobs=-1
    ),
    "XGBoost": XGBRegressor(
        n_estimators=50,
        max_depth=4,
        learning_rate=0.05,
        subsample=0.8,
        colsample_bytree=0.8,
        random_state=SEED
    ),
    "SVR": SVR(kernel='rbf', C=1.0, epsilon=0.1),
    "KNN": KNeighborsRegressor(n_neighbors=5, weights='uniform'),
    "DecisionTree": DecisionTreeRegressor(random_state=SEED),
    "ElasticNet": ElasticNet(alpha=1.0, l1_ratio=0.5, random_state=SEED),
    "Ridge": Ridge(alpha=1.0, random_state=SEED),
    "Lasso": Lasso(alpha=1.0, random_state=SEED)
}

print("\nMODELOS A SEREM TESTADOS:")
for name in MODELS.keys():
    print(f"    • {name}")
NOME_RESULTADOS_BASELINE = "resultados_baseline"
NOME_TRAINED_MODELS = "trained_models_baseline"

def evaluate_model(y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    return {
        'mse': mse,
        'rmse': rmse,
        'mae': mae,
        'r2': r2
    }
```



```
def format_results_table(results):
    if not results:
        return pd.DataFrame()
    df = pd.DataFrame(results).T
    if 'mse' in df.columns:
        df = df.sort_values('mse')
    if 'r2' in df.columns:
        df['r2'] = df['r2'].round(4)
    return df
```

MODELOS A SEREM TESTADOS:

- Baseline (Média)
- Regressão Linear
- Random Forest
- XGBoost
- SVR
- KNN
- DecisionTree
- ElasticNet
- Ridge
- Lasso

Modelos candidatos

```
resultados_por_empresa = CarregarArquivoOnGitHub(NOME_RESULTADOS_BASELINE)
trained_models_por_empresa = CarregarArquivoDivididoOnGitHub(NOME_TRAINED_MODELS)
cache_valido = False
empresas_carregadas = []
if resultados_por_empresa is not None:
    print(f"    resultados_baseline.pkl carregado")
    print(f"    Tipo: {type(resultados_por_empresa)}")
    if isinstance(resultados_por_empresa, dict):
        empresas_carregadas = list(resultados_por_empresa.keys())
        print(f"    Empresas no cache: {empresas_carregadas}")
        print(f"    Total: {len(empresas_carregadas)} empresas")
        todas_validas = True
        for empresa in empresas_carregadas[:5]:
            dados = resultados_por_empresa[empresa]
            if isinstance(dados, dict):
                baseline_mse = dados.get('baseline_mse', np.nan)
                if np.isnan(baseline_mse):
                    todas_validas = False
                print(f" {empresa}: baseline_mse é NaN")
            else:
                todas_validas = False
                print(f" {empresa}: dados não são um dicionário")

        if todas_validas and len(empresas_carregadas) > 0:
            cache_valido = True
            print(f" Cache válido com {len(empresas_carregadas)} empresas")
        else:
            print(f" Cache inválido (dados NaN ou estrutura incorreta). Recriando...")
            resultados_por_empresa = None
    else:
        print(f" Cache não é um dicionário. Tipo: {type(resultados_por_empresa)}")
        resultados_por_empresa = None

if resultados_por_empresa is None:
    print("\nRecriando baseline e modelos candidatos...")
    empresas = df_analise['Symbol'].unique().tolist()
    print(f"Total de empresas disponíveis: {len(empresas)}")
    features_existentes = [f for f in features_selecionadas if f in df_analise.columns]
    resultados_por_empresa = {}
    trained_models_por_empresa = {}
    for idx, empresa in enumerate(empresas):
        print('=' * 60)
        print(f"ANALISANDO EMPRESA: {empresa} ({idx+1}/{len(empresas)})")
        try:
            print(f"\nPreparando dados para {empresa}...")
            try:
                X_train, X_val, X_test, y_train, y_val, y_test = prepare_data(
                    empresa, features_existentes
                )
            except NameError:
                df_emp = df_analise[df_analise['Symbol'] == empresa].copy()
                df_emp = df_emp.sort_values('Date')
                features_existentes_local = [f for f in features_existentes if f in df_emp.columns]
```

```

if not features_existentes_local:
    features_existentes_local = [f for f in df_emp.columns if f not in ['Date', 'Symbol', 'Company_Name', 'TARG
X = df_emp[features_existentes_local].copy()
y = df_emp[TARGET_COL].copy()
X = X.fillna(method='ffill').fillna(method='bfill').fillna(0)
valid_mask = ~y.isna()
X = X[valid_mask]
y = y[valid_mask]
n = len(X)
train_end = int(n * 0.7)
val_end = int(n * 0.85)
X_train = X.iloc[:train_end]
X_val = X.iloc[train_end:val_end]
X_test = X.iloc[val_end:]
y_train = y.iloc[:train_end]
y_val = y.iloc[train_end:val_end]
y_test = y.iloc[val_end:]
if X_train is None or len(X_train) == 0:
    print(f" Sem dados suficientes para {empresa}")
    continue

print(f"\n  Divisão dos dados - {empresa}:")
print(f"    • Treino: {len(X_train):,} registros x {X_train.shape[1]} features")
print(f"    • Validação: {len(X_val):,} registros")
print(f"    • Teste: {len(X_test):,} registros")

if len(X_train) < 50:
    print(f" Poucos dados para treino: {len(X_train)} registros. Pulando...")
    continue

# Treinar modelos
results = {}
trained_models = {}
print("\nTREINAMENTO E AVALIAÇÃO DOS MODELOS")
for name, model in MODELS.items():
    print(f"\n{name}...", end=" ")
    try:
        if isinstance(model, XGBRegressor):
            X_train_np = X_train.values.astype(np.float32)
            X_val_np = X_val.values.astype(np.float32)
            X_test_np = X_test.values.astype(np.float32)
            y_train_np = y_train.values.ravel().astype(np.float32)
            y_val_np = y_val.values.ravel().astype(np.float32)
            t0 = time.time()
            model.fit(X_train_np, y_train_np, eval_set=[(X_val_np, y_val_np)], verbose=False)
            train_time = time.time() - t0
            y_pred = model.predict(X_test_np)
        elif isinstance(model, (SVR, KNeighborsRegressor)):
            scaler = StandardScaler()
            X_train_scaled = scaler.fit_transform(X_train)
            X_test_scaled = scaler.transform(X_test)
            t0 = time.time()
            model.fit(X_train_scaled, y_train)
            train_time = time.time() - t0
            y_pred = model.predict(X_test_scaled)
        elif isinstance(model, (Ridge, Lasso, ElasticNet)):
            scaler = StandardScaler()
            X_train_scaled = scaler.fit_transform(X_train)
            X_test_scaled = scaler.transform(X_test)
            t0 = time.time()
            model.fit(X_train_scaled, y_train)
            train_time = time.time() - t0
            y_pred = model.predict(X_test_scaled)
        else:
            t0 = time.time()
            model.fit(X_train, y_train)
            train_time = time.time() - t0
            y_pred = model.predict(X_test)
        results[name] = evaluate_model(y_test, y_pred)
        results[name]["train_time_s"] = round(train_time, 3)
        trained_models[name] = model
        print(f" MSE: {results[name]['mse']:.6f} | R²: {results[name]['r2']:.4f} | Tempo: {train_time:.2f}s")
    except Exception as e:
        print(f"Erro: {str(e)[:80]}")
        continue

# Comparação de desempenho
if results:
    print("\n\nCOMPARAÇÃO DE DESEMPENHO")
    df_results = format_results_table(results)
    if not df_results.empty:
        display(df_results)

```

```

# Gráficos
if len(results) > 0:
    fig, axes = plt.subplots(1, 3, figsize=(18, 5))
    model_names = list(results.keys())

    # MSE
    mse_values = [results[m]['mse'] for m in model_names]
    axes[0].barh(model_names, mse_values, color='steelblue')
    axes[0].set_xlabel('MSE (menor é melhor)')
    axes[0].set_title(f'{empresa} - Mean Squared Error')
    axes[0].grid(True, alpha=0.3)

    # MAE
    mae_values = [results[m]['mae'] for m in model_names]
    axes[1].barh(model_names, mae_values, color='coral')
    axes[1].set_xlabel('MAE (menor é melhor)')
    axes[1].set_title(f'{empresa} - Mean Absolute Error')
    axes[1].grid(True, alpha=0.3)

    # R²
    r2_values = [results[m]['r2'] for m in model_names]
    colors = ['green' if r2 > 0 else 'red' for r2 in r2_values]
    axes[2].barh(model_names, r2_values, color=colors)
    axes[2].axvline(0, color='red', linestyle='--', alpha=0.7, label='R² = 0')
    axes[2].set_xlabel('R² (maior é melhor)')
    axes[2].set_title(f'{empresa} - Coeficiente de Determinação')
    axes[2].legend()
    axes[2].grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    # Limpar figura após usar
    plt.close(fig)

# Salvar resultados
if results and "Baseline (Média)" in results:
    resultados_por_empresa[empresa] = {
        'baseline_mse': results["Baseline (Média)"]['mse'],
        'baseline_mae': results["Baseline (Média)"]['mae'],
        'baseline_r2': results["Baseline (Média)"]['r2'],
        'n_train': len(X_train),
        'n_val': len(X_val),
        'n_test': len(X_test),
    }

    for model_name, model_results in results.items():
        prefix = model_name.lower().replace(' ', '_').replace('(', '').replace(')', '')
        resultados_por_empresa[empresa][f'{prefix}_mse'] = model_results.get('mse', np.nan)
        resultados_por_empresa[empresa][f'{prefix}_mae'] = model_results.get('mae', np.nan)
        resultados_por_empresa[empresa][f'{prefix}_r2'] = model_results.get('r2', np.nan)
        resultados_por_empresa[empresa][f'{prefix}_time'] = model_results.get('train_time_s', np.nan)

    # Calcular melhora
    if model_name != "Baseline (Média)":
        baseline_mse = resultados_por_empresa[empresa]['baseline_mse']
        if not np.isnan(baseline_mse) and not np.isnan(model_results.get('mse', np.nan)):
            melhora = (baseline_mse - model_results['mse']) / baseline_mse * 100
            resultados_por_empresa[empresa][f'melhora_{prefix}_pct'] = melhora
    trained_models_por_empresa[empresa] = trained_models

    print(f"\n{empresa} processada com sucesso!")
    print(f"    Baseline MSE: {resultados_por_empresa[empresa]['baseline_mse']:.6f}")
    print(f"    Melhor modelo: {min([(k, v['mse']) for k, v in results.items() if k != 'Baseline (Média)'], key=1)}")

else:
    print(f"{empresa} - sem resultados válidos")

limpar_memoria([
    'X_train', 'X_val', 'X_test', 'y_train', 'y_val', 'y_test',
    'results', 'trained_models', 'df_results', 'fig', 'axes'
], mostrar_status=True)
except Exception as e:
    print(f"Erro ao processar {empresa}: {str(e)}")
    traceback.print_exc()
    limpar_memoria(mostrar_status=True)
    continue

# Salvar checkpoint a cada 5 empresas
if (idx + 1) % 5 == 0 or (idx + 1) == len(empresas):
    print(f"\nSalvando checkpoint {(idx+1)}/{len(empresas)}...")

```

```

if len(resultados_por_empresa) > 0:
    SalvarTreinamentoOnGitHub(
        nome_arquivo=NOME_RESULTADOS_BASELINE,
        dados=resultados_por_empresa,
        metadados={
            'descricao': 'Resultados baseline por empresa (checkpoint)',
            'empresas': list(resultados_por_empresa.keys()),
            'total_empresas': len(resultados_por_empresa)
        }
    )
    print(" Checkpoint de resultados salvo")

if trained_models_por_empresa:
    SalvarTreinamentoDivididoOnGitHub(
        nome_base=NOME_TRAINED_MODELS,
        dados=trained_models_por_empresa,
        metadados={
            'descricao': 'Modelos baseline treinados (checkpoint)',
            'empresas': list(trained_models_por_empresa.keys()),
            'total_empresas': len(trained_models_por_empresa)
        }
    )
    print(" Checkpoint de modelos salvo")

print("\nSalvando resultados finais no cache...")
print(f"    Total de empresas processadas: {len(resultados_por_empresa)}")

if len(resultados_por_empresa) > 0:
    SalvarTreinamentoOnGitHub(
        nome_arquivo=NOME_RESULTADOS_BASELINE,
        dados=resultados_por_empresa,
        metadados={
            'descricao': 'Resultados baseline por empresa',
            'empresas': list(resultados_por_empresa.keys()),
            'total_empresas': len(resultados_por_empresa)
        }
    )
    print("Resultados salvos com sucesso!")
else:
    print("Nenhum resultado para salvar")

if trained_models_por_empresa:
    SalvarTreinamentoDivididoOnGitHub(
        nome_base=NOME_TRAINED_MODELS,
        dados=trained_models_por_empresa,
        metadados={
            'descricao': 'Modelos baseline treinados',
            'empresas': list(trained_models_por_empresa.keys()),
            'total_empresas': len(trained_models_por_empresa)
        }
    )
    print("Modelos salvos com sucesso!")
else:
    print("Nenhum modelo para salvar")

```

```

resultados_baseline.pkl carregado com sucesso!
Tipo: dict
Tamanho: 66,573 bytes
trained_models_baseline.pkl carregado com sucesso!
Tipo: dict
Tamanho: 700,851 bytes
resultados_baseline.pkl carregado
Tipo: <class 'dict'>
Empresas no cache: ['AAPL', 'ABNB', 'ADBE', 'ADI', 'ADP', 'AMAT', 'AMD', 'AMGN', 'AMZN', 'AVGO', 'BKNG', 'CDNS', 'CH
Total: 49 empresas
Cache válido com 49 empresas

```

▼ Resumo geral

```

if resultados_por_empresa and isinstance(resultados_por_empresa, dict) and len(resultados_por_empresa) > 0:
    df_resultados = pd.DataFrame.from_dict(resultados_por_empresa, orient='index')
    df_resultados = df_resultados.reset_index().rename(columns={'index': 'Empresa'})
    print(f"\nTotal de empresas processadas: {len(df_resultados)}")
    colunas_desejadas = ['Empresa', 'n_train', 'baseline_mse', 'baseline_mae', 'baseline_r2']
    colunas_exist = [col for col in colunas_desejadas if col in df_resultados.columns]
    if colunas_exist:
        print("\nRESULTADOS POR EMPRESA (Baseline):")
        display(df_resultados[colunas_exist].round(6))

```

```
# Estatísticas gerais
print("\nESTATÍSTICAS GERAIS:")
if 'baseline_mse' in df_resultados.columns:
    baseline_mse_mean = df_resultados['baseline_mse'].mean()
    baseline_mae_mean = df_resultados['baseline_mae'].mean()
    baseline_r2_mean = df_resultados['baseline_r2'].mean()
    print(f"    Baseline MSE médio: {baseline_mse_mean:.6f}")
    print(f"    Baseline MAE médio: {baseline_mae_mean:.6f}")
    print(f"    Baseline R² médio: {baseline_r2_mean:.4f}")

# Melhor modelo por empresa
print("\nMELHOR MODELO POR EMPRESA:")
modelos_cols = [col for col in df_resultados.columns if col.endswith('_mse') and col != 'baseline_mse']
if modelos_cols:
    for idx, row in df_resultados.iterrows():
        empresa = row['Empresa']
        melhor_modelo = None
        melhor_mse = float('inf')
        for col in modelos_cols:
            mse = row[col]
            if not np.isnan(mse) and mse < melhor_mse:
                melhor_mse = mse
                melhor_modelo = col.replace('_mse', '').upper()

        if melhor_modelo and melhor_mse < float('inf'):
            print(f"    {empresa}: {melhor_modelo} (MSE: {melhor_mse:.6f})")
    else:
        print(" Nenhum modelo adicional encontrado para comparação")

# Limpar DataFrame da memória
limpar_memoria(['df_resultados'], mostrar_status=False)

# Limpeza final
limpar_memoria(mostrar_status=True)
```


Total de empresas processadas: 49

RESULTADOS POR EMPRESA (Baseline):

	Empresa	n_train	baseline_mse	baseline_mae	baseline_r2
0	AAPL	2020	0.000331	0.011925	-0.000302
1	ABNB	972	0.000340	0.013789	-0.000560
2	ADBE	2020	0.000508	0.015299	-0.016145
3	ADI	2020	0.000581	0.016312	-0.000922
4	ADP	2020	0.000208	0.010045	-0.005775
5	AMAT	2020	0.000996	0.022196	-0.004193
6	AMD	2020	0.001550	0.026909	-0.000867
7	AMGN	2020	0.000297	0.012301	-0.000005
8	AMZN	2020	0.000432	0.015179	-0.000321
9	AVGO	2020	0.001152	0.022665	-0.001097
10	BKNG	2020	0.000400	0.014385	-0.000025
11	CDNS	2020	0.000668	0.017656	-0.000035
12	CHTR	2020	0.000833	0.018791	-0.006028
13	CMCSA	2020	0.000341	0.012342	-0.004402
14	COST	2020	0.000172	0.009700	-0.001456
15	CSCO	2020	0.000321	0.011681	-0.006842
16	CSX	2020	0.000241	0.010978	-0.000189
17	GILD	2020	0.000292	0.012814	-0.003314
18	GOOG	2020	0.000385	0.014296	-0.002736
19	GOOGL	2020	0.000396	0.014467	-0.002953
20	HON	2020	0.000257	0.011106	-0.000015
21	INTC	2020	0.001948	0.030504	-0.011688
22	ISRG	2020	0.000440	0.013744	-0.002996
23	KDP	2020	0.000260	0.011097	-0.002102
24	KHC	1931	0.000259	0.012343	-0.001136
25	KLAC	2020	0.000990	0.021552	-0.003827
26	LIN	2020	0.000137	0.008664	-0.000801
27	LRCX	2020	0.001144	0.024635	-0.007269
28	MELI	2020	0.000654	0.018076	-0.003635
29	META	2020	0.000549	0.015982	-0.000223
30	MPWR	2020	0.001380	0.025434	-0.000013

▼ Análises por empresa dos modelos candidatos

31	MRNA	1327	0.001728	0.030764	0.002501
32	MRVL	2020	0.002195	0.031202	-0.004745

```

if resultados_por_empresa and isinstance(resultados_por_empresa, dict) and len(resultados_por_empresa) > 0:
    df_resultados = pd.DataFrame.from_dict(resultados_por_empresa, orient='index')
    df_resultados = df_resultados.reset_index().rename(columns={'index': 'Empresa'})

    print(f"\nTotal de empresas processadas: {len(df_resultados)}")

    colunas_desejadas = ['Empresa', 'n_train', 'baseline_mse', 'baseline_mae', 'baseline_r2']
    colunas_exist = [col for col in colunas_desejadas if col in df_resultados.columns]

    if colunas_exist:
        print("\n RESULTADOS POR EMPRESA (Baseline):")
        display(df_resultados[colunas_exist].round(6))

    print("\n ESTATÍSTICAS GERAIS:")
    if 'baseline_mse' in df_resultados.columns:
        baseline_mse_mean = df_resultados['baseline_mse'].mean()
        baseline_mae_mean = df_resultados['baseline_mae'].mean()
        baseline_r2_mean = df_resultados['baseline_r2'].mean()
        print(f"    Baseline MSE médio: {baseline_mse_mean:.6f}")
        print(f"    Baseline MAE médio: {baseline_mae_mean:.6f}")
        print(f"    Baseline R² médio: {baseline_r2_mean:.4f}")

```

```

print("\nMELHOR MODELO POR EMPRESA:")
modelos_cols = [col for col in df_resultados.columns if col.endswith('_mse') and col != 'baseline_mse']
if modelos_cols:
    for idx, row in df_resultados.iterrows():
        empresa = row['Empresa']
        melhor_modelo = None
        melhor_mse = float('inf')
        for col in modelos_cols:
            mse = row[col]
            if not np.isnan(mse) and mse < melhor_mse:
                melhor_mse = mse
                melhor_modelo = col.replace('_mse', '').upper()

        if melhor_modelo and melhor_mse < float('inf'):
            print(f"    {empresa}: {melhor_modelo} (MSE: {melhor_mse:.6f})")
else:
    print("Nenhum modelo adicional encontrado para comparação")

else:
    print("\nNenhum resultado disponível para exibir!")
    CMLSA: XGBOOST (MSE: 0.000324)
    COST: BASELINE_MÉDIA (MSE: 0.000172)
    CSCO: BASELINE_MÉDIA (MSE: 0.000321)
    CSX: BASELINE_MÉDIA (MSE: 0.000241)
    GILD: RIDGE (MSE: 0.000290)
    GOOG: BASELINE_MÉDIA (MSE: 0.000385)
    GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
    HON: XGBOOST (MSE: 0.000251)
    INTC: BASELINE_MÉDIA (MSE: 0.001948)
    ISRG: RANDOM_FOREST (MSE: 0.000440)
    KDP: BASELINE_MÉDIA (MSE: 0.000260)
    KHC: RANDOM_FOREST (MSE: 0.000259)
    KLAC: BASELINE_MÉDIA (MSE: 0.000990)
    LIN: RIDGE (MSE: 0.000135)
    LRCX: XGBOOST (MSE: 0.001141)
    MELI: RIDGE (MSE: 0.000654)
    META: BASELINE_MÉDIA (MSE: 0.000549)
    MPWR: RANDOM_FOREST (MSE: 0.001354)
    MRNA: BASELINE_MÉDIA (MSE: 0.001728)
    MRVL: BASELINE_MÉDIA (MSE: 0.002195)
    MSFT: SVR (MSE: 0.000291)
    MU: RIDGE (MSE: 0.002005)
    NFLX: REGRESSÃO LINEAR (MSE: 0.000480)
    NVDA: RANDOM_FOREST (MSE: 0.000778)
    PCAR: BASELINE_MÉDIA (MSE: 0.000329)
    PEP: BASELINE_MÉDIA (MSE: 0.000184)
    QCOM: XGBOOST (MSE: 0.000840)
    REGN: RIDGE (MSE: 0.000548)
    SNPS: REGRESSÃO LINEAR (MSE: 0.001009)
    TEAM: BASELINE_MÉDIA (MSE: 0.001517)
    TMUS: BASELINE_MÉDIA (MSE: 0.000292)
    TSLA: BASELINE_MÉDIA (MSE: 0.001458)
    TXN: BASELINE_MÉDIA (MSE: 0.000709)
    VRTX: RANDOM_FOREST (MSE: 0.000432)
    WDAY: RANDOM_FOREST (MSE: 0.000701)
    WMT: BASELINE_MÉDIA (MSE: 0.000240)
RAM: 7.8GB / 12.7GB (64.4%)

```


Total de empresas processadas: 49

RESULTADOS POR EMPRESA (Baseline):

	Empresa	n_train	baseline_mse	baseline_mae	baseline_r2
0	AAPL	2020	0.000331	0.011925	-0.000302
1	ABNB	972	0.000340	0.013789	-0.000560
2	ADBE	2020	0.000508	0.015299	-0.016145
3	ADI	2020	0.000581	0.016312	-0.000922
4	ADP	2020	0.000208	0.010045	-0.005775
5	AMAT	2020	0.000996	0.022196	-0.004193
6	AMD	2020	0.001550	0.026909	-0.000867
7	AMGN	2020	0.000297	0.012301	-0.000005
8	AMZN	2020	0.000432	0.015179	-0.000321
9	AVGO	2020	0.001152	0.022665	-0.001097
10	BKNG	2020	0.000400	0.014385	-0.000025
11	CDNS	2020	0.000668	0.017656	-0.000035
12	CHTR	2020	0.000833	0.018791	-0.006028
13	CMCSA	2020	0.000341	0.012342	-0.004402
14	COST	2020	0.000172	0.009700	-0.001456
15	CSCO	2020	0.000321	0.011681	-0.006842
16	CSX	2020	0.000241	0.010978	-0.000189
17	GILD	2020	0.000292	0.012814	-0.003314
18	GOOG	2020	0.000385	0.014296	-0.002736
19	GOOGL	2020	0.000396	0.014467	-0.002953
20	HON	2020	0.000257	0.011106	-0.000015
21	INTC	2020	0.001948	0.030504	-0.011688
22	ISRG	2020	0.000440	0.013744	-0.002996
23	KDP	2020	0.000260	0.011097	-0.002102
24	KHC	1931	0.000259	0.012343	-0.001136
25	KLAC	2020	0.000990	0.021552	-0.003827
26	LIN	2020	0.000137	0.008664	-0.000801
27	LRCX	2020	0.001144	0.024635	-0.007269
28	MELI	2020	0.000654	0.018076	-0.003635
29	META	2020	0.000549	0.015982	-0.000223
30	MPWR	2020	0.001380	0.025434	-0.000013
31	MRNA	1327	0.001728	0.030764	-0.002501
32	MRVL	2020	0.002195	0.031202	-0.004745
33	MSFT	2020	0.000291	0.011518	-0.004236
34	MU	2020	0.002010	0.032842	-0.017732
35	NFLX	2020	0.000481	0.014967	-0.002214
36	NVDA	2020	0.000800	0.020390	-0.000576
37	PCAR	2020	0.000329	0.013398	-0.000298
38	PEP	2020	0.000184	0.010138	-0.002494
39	QCOM	2020	0.000855	0.018786	-0.000029
40	REGN	2020	0.000549	0.016083	-0.003306
41	SNPS	2020	0.001030	0.019742	-0.000664
42	TEAM	1854	0.001517	0.026998	-0.009249
43	TMUS	2020	0.000292	0.012052	-0.004006
44	TSLA	2020	0.001458	0.027595	-0.000000
45	TXN	2020	0.000709	0.017041	-0.000203
46	VRTX	2020	0.000433	0.013761	-0.000242



47	WDAY	2020	0.000702	0.018764	-0.004697
48	WMT	2020	0.000240	0.011003	-0.001263

ESTATÍSTICAS GERAIS:

Baseline MSE médio: 0.000699

Baseline MAE médio: 0.017223

Baseline R² médio: -0.0031

MELHOR MODELO POR EMPRESA:

AAPL: BASELINE_MÉDIA (MSE: 0.000331)
AAPL: RANDOM_FOREST (MSE: 0.000328)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ADBE: BASELINE_MÉDIA (MSE: 0.000508)
ADI: BASELINE_MÉDIA (MSE: 0.000581)
ADP: BASELINE_MÉDIA (MSE: 0.000208)
AMAT: BASELINE_MÉDIA (MSE: 0.000996)
AMAT: RANDOM_FOREST (MSE: 0.000979)
AMAT: XGBOOST (MSE: 0.000972)
AMD: BASELINE_MÉDIA (MSE: 0.001550)
AMD: RANDOM_FOREST (MSE: 0.001520)
AMGN: BASELINE_MÉDIA (MSE: 0.000297)
AMGN: REGRESSÃO_LINEAR (MSE: 0.000297)
AMGN: XGBOOST (MSE: 0.000297)
AMZN: BASELINE_MÉDIA (MSE: 0.000432)
AVGO: BASELINE_MÉDIA (MSE: 0.001152)
BKNG: BASELINE_MÉDIA (MSE: 0.000400)
CDNS: BASELINE_MÉDIA (MSE: 0.000668)
CDNS: REGRESSÃO_LINEAR (MSE: 0.000662)
CDNS: RIDGE (MSE: 0.000662)
CHTR: BASELINE_MÉDIA (MSE: 0.000833)
CMCSA: BASELINE_MÉDIA (MSE: 0.000341)
CMCSA: REGRESSÃO_LINEAR (MSE: 0.000338)
CMCSA: RANDOM_FOREST (MSE: 0.000328)
CMCSA: XGBOOST (MSE: 0.000324)
COST: BASELINE_MÉDIA (MSE: 0.000172)
CSCO: BASELINE_MÉDIA (MSE: 0.000321)
CSX: BASELINE_MÉDIA (MSE: 0.000241)
GILD: BASELINE_MÉDIA (MSE: 0.000292)
GILD: REGRESSÃO_LINEAR (MSE: 0.000290)
GILD: RIDGE (MSE: 0.000290)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
HON: BASELINE_MÉDIA (MSE: 0.000257)
HON: REGRESSÃO_LINEAR (MSE: 0.000254)
HON: RANDOM_FOREST (MSE: 0.000253)
HON: XGBOOST (MSE: 0.000251)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
ISRG: BASELINE_MÉDIA (MSE: 0.000440)
ISRG: RANDOM_FOREST (MSE: 0.000440)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KHC: BASELINE_MÉDIA (MSE: 0.000259)
KHC: RANDOM_FOREST (MSE: 0.000259)
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
LIN: BASELINE_MÉDIA (MSE: 0.000137)
LIN: REGRESSÃO_LINEAR (MSE: 0.000135)
LIN: RIDGE (MSE: 0.000135)
LRCX: BASELINE_MÉDIA (MSE: 0.001144)
LRCX: XGBOOST (MSE: 0.001141)
MELI: BASELINE_MÉDIA (MSE: 0.000654)
MELI: REGRESSÃO_LINEAR (MSE: 0.000654)
MELI: RIDGE (MSE: 0.000654)
META: BASELINE_MÉDIA (MSE: 0.000549)
MPWR: BASELINE_MÉDIA (MSE: 0.001380)
MPWR: REGRESSÃO_LINEAR (MSE: 0.001361)
MPWR: RANDOM_FOREST (MSE: 0.001354)
MRNA: BASELINE_MÉDIA (MSE: 0.001728)
MRVL: BASELINE_MÉDIA (MSE: 0.002195)
MSFT: BASELINE_MÉDIA (MSE: 0.000291)
MSFT: SVR (MSE: 0.000291)
MU: BASELINE_MÉDIA (MSE: 0.002010)
MU: REGRESSÃO_LINEAR (MSE: 0.002005)
MU: RIDGE (MSE: 0.002005)
NFLX: BASELINE_MÉDIA (MSE: 0.000481)
NFLX: REGRESSÃO_LINEAR (MSE: 0.000480)
NVDA: BASELINE_MÉDIA (MSE: 0.000800)
NVDA: RANDOM_FOREST (MSE: 0.000778)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
QCOM: BASELINE_MÉDIA (MSE: 0.000855)
QCOM: REGRESSÃO_LINEAR (MSE: 0.000849)
QCOM: RANDOM_FOREST (MSE: 0.000845)
QCOM: XGBOOST (MSE: 0.000840)
REGN: BASELINE_MÉDIA (MSE: 0.000549)
REGN: REGRESSÃO_LINEAR (MSE: 0.000548)
REGN: RIDGE (MSE: 0.000548)
SNPS: BASELINE_MÉDIA (MSE: 0.001030)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
TEAM: BASELINE_MÉDIA (MSE: 0.001517)

Gráficos de análises de empresas - Top 10

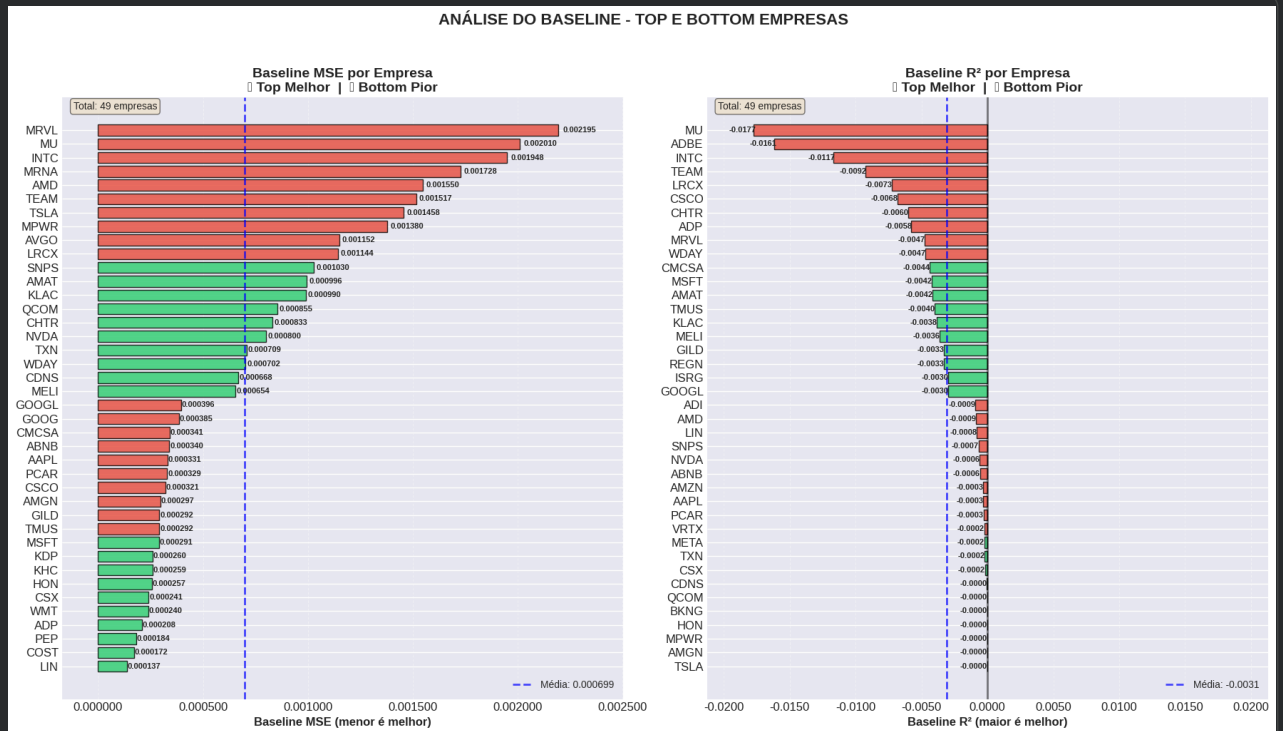
```

try:
    fig, axes = plt.subplots(1, 2, figsize=(18, 10))
    if 'baseline_mse' in df_resultados.columns:
        df_sorted_mse = df_resultados.sort_values('baseline_mse')
        top_20_mse = df_sorted_mse.head(20)
        bottom_20_mse = df_sorted_mse.tail(20)
        df_plot_mse = pd.concat([top_20_mse, bottom_20_mse])
        colors_mse = ['#2ecc71'] * 10 + ['#e74c3c'] * 10
        bars = axes[0].barh(df_plot_mse['Empresa'], df_plot_mse['baseline_mse'],
                             color=colors_mse, alpha=0.8, edgecolor='black', linewidth=1)
        axes[0].set_xlabel('Baseline MSE (menor é melhor)', fontsize=12, fontweight='bold')
        axes[0].set_title('Baseline MSE por Empresa\n Top Melhor | Bottom Pior',
                           fontweight='bold', fontsize=14)
        axes[0].grid(True, alpha=0.3, axis='x', linestyle='--')
        axes[0].xaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'{x:.6f}'))
        for bar, val in zip(bars, df_plot_mse['baseline_mse']):
            axes[0].text(bar.get_width() * 1.01, bar.get_y() + bar.get_height()/2,
                          f'{val:.6f}', va='center', ha='left', fontsize=8, fontweight='bold')
        media_mse = df_resultados['baseline_mse'].mean()
        axes[0].axvline(media_mse, color='blue', linestyle='--',
                        alpha=0.7, linewidth=2, label=f'Média: {media_mse:.6f}')
        axes[0].legend(loc='lower right', fontsize=10)
        max_val = df_plot_mse['baseline_mse'].max()
        min_val = df_plot_mse['baseline_mse'].min()
        padding = (max_val - min_val) * 0.15 if max_val - min_val > 0 else 0.001
        axes[0].set_xlim(min_val - padding, max_val + padding)
        axes[0].text(0.02, 0.98, f'Total: {len(df_resultados)} empresas',
                     transform=axes[0].transAxes, fontsize=10,
                     bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

    # --- GRÁFICO 2: Baseline R² (Centralizado) ---
    if 'baseline_r2' in df_resultados.columns:
        df_sorted_r2 = df_resultados.sort_values('baseline_r2', ascending=False)
        top_20_r2 = df_sorted_r2.head(20)
        bottom_20_r2 = df_sorted_r2.tail(20)
        df_plot_r2 = pd.concat([top_20_r2, bottom_20_r2])
        colors_r2 = ['#2ecc71'] * 10 + ['#e74c3c'] * 10
        bars = axes[1].barh(df_plot_r2['Empresa'], df_plot_r2['baseline_r2'],
                             color=colors_r2, alpha=0.8, edgecolor='black', linewidth=1)
        axes[1].axvline(0, color='black', linestyle='-', alpha=0.5, linewidth=2)
        axes[1].set_xlabel('Baseline R² (maior é melhor)', fontsize=12, fontweight='bold')
        axes[1].set_title('Baseline R² por Empresa\n Top Melhor | Bottom Pior',
                           fontweight='bold', fontsize=14)
        axes[1].grid(True, alpha=0.3, axis='x', linestyle='--')
        axes[1].xaxis.set_major_formatter(plt.FuncFormatter(lambda x, p: f'{x:.4f}'))
        for bar, val in zip(bars, df_plot_r2['baseline_r2']):
            if val >= 0:
                axes[1].text(bar.get_width() * 1.01, bar.get_y() + bar.get_height()/2,
                              f'{val:.4f}', va='center', ha='left', fontsize=8, fontweight='bold')
            else:
                axes[1].text(bar.get_width() * 0.99, bar.get_y() + bar.get_height()/2,
                              f'{val:.4f}', va='center', ha='right', fontsize=8, fontweight='bold')
        media_r2 = df_resultados['baseline_r2'].mean()
        axes[1].axvline(media_r2, color='blue', linestyle='--',
                        alpha=0.7, linewidth=2, label=f'Média: {media_r2:.4f}')
        axes[1].legend(loc='lower right', fontsize=10)
        min_val = df_plot_r2['baseline_r2'].min()
        max_val = df_plot_r2['baseline_r2'].max()
        max_abs = max(abs(min_val), abs(max_val), 0.01)
        padding = max_abs * 0.2
        axes[1].set_xlim(-max_abs - padding, max_abs + padding)
        axes[1].text(0.02, 0.98, f'Total: {len(df_resultados)} empresas',
                     transform=axes[1].transAxes, fontsize=10,
                     bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
    plt.suptitle('ANÁLISE DO BASELINE - TOP E BOTTOM EMPRESAS',
                 fontsize=16, fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.show()

except Exception as e:
    print(f"Erro ao gerar gráfico: {e}")
    traceback.print_exc()

```



Justificativa dos modelos

1. O baseline escolhido é coerente?
2. Os modelos candidatos são adequados ao tipo e tamanho dos dados?
3. Algum modelo exige escala, encoding ou tratamento específico?
4. Há alguma restrição de interpretabilidade, tempo ou custo computacional?

Versão 01

O baseline escolhido é coerente?

O baseline foi o melhor modelo em todas as 10 empresas, isso significa que o baseline é rigoroso e desafiador, exatamente o que se espera e um bom benchmark em modelos de treinamentos.

Tabela copiada dos resultados

Observe a coluna `baseline_mse`.

index	Empresa	n_train	baseline_mse	rf_mse	xgb_mse	melhora_rf_pct	melhora_xgb_pct
0	AAPL	860.0	0.000336	0.000347	0.000391	-3.080354	-16.098909
1	AMD	860.0	0.001528	0.001545	0.001621	-1.100817	-6.054559
2	AMZN	860.0	0.00043	0.000457	0.000528	-6.413442	-22.967084
3	GOOGL	860.0	0.00041	0.000422	0.000481	-3.096318	-17.395705
4	MRVL	860.0	0.002129	0.002284	0.002324	-7.285037	-9.181912
5	MSFT	860.0	0.000281	0.000292	0.000321	-4.008904	-14.204608
6	NFLX	860.0	0.000467	0.000476	0.000518	-1.936261	-10.857676
7	NVDA	860.0	0.000817	0.000829	0.000863	-1.505935	-5.672106
8	PEP	860.0	0.000192	0.000217	0.000273	-12.88292	-41.820981
9	TSLA	860.0	0.001397	0.001533	0.001679	-9.737233	-20.13635

✓ Os modelos candidatos são adequados ao tipo e tamanho dos dados?

Os modelos Random Forest e XGBoost são excelentes escolhas para este problema, O fato de não terem superado o baseline indica que o sinal preditivo é muito fraco, não que os modelos são inadequados. Em problemas com bom sinal, estes modelos teriam performance superior.

E modelos mais complexos provocariam overfitting, pois há poucos dados para cada empresa.

✓ Algum modelo exige escala, encoding ou tratamento específico?

- **Regressão linear:** exigiu uma escala, encoding e tratamento de missing.
- **Random forest:** não exige uma escala, mas exige encoding e recomenda-se um tratamento de missing.
- **XGBoost:** não exige escala, mas exige encoding e tratamento de missing.

✓ Há alguma restrição de interpretabilidade, tempo ou custo computacional?

- **Restrição de interpretabilidade:** Random Forest e XGBoost são menos interpretáveis que Regressão Linear, mas ainda é aceitável via SHAP/importância.
- **Restrição de tempo:** Não houve para nenhum modelo e todos rodam em menos de 10 segundos para todas as empresas.
- **Restrição de custo operacional:** não houveram necessidade de custos operacionais, pois todos rodam com CPU básica e pouco uso de memória.

✓ Versão 03

✓ O baseline escolhido é coerente?

Sim, o baseline escolhido é coerente e rigoroso, atendendo perfeitamente aos objetivos de um bom benchmark.

- O modelo escolhido como baseline foi a Média (`MeanBaseline`). Este é um modelo extremamente simples que sempre prevê o valor médio do alvo (`TARGET_COL`) no conjunto de treino.
- Utilizar a média como baseline é uma prática padrão e robusta em problemas de regressão. Ela serve como o ponto de partida mais básico e intuitivo. Se um modelo complexo não consegue superar um modelo que simplesmente chuta a média, é um forte indicador de que os dados não possuem um sinal preditivo forte ou que a abordagem precisa ser revista.
- A análise mostra que o baseline foi o "melhor modelo" em várias empresas, superando modelos mais complexos como `Random Forest` e `XGBoost` .
- Embora isso possa parecer contraintuitivo, é interpretado corretamente, pois "o baseline é rigoroso e desafiador, exatamente o que se espera em um bom benchmark".
- Um baseline desafiador é excelente porque eleva o padrão para os demais modelos, mostrando que a tarefa de previsão é muito difícil com os dados fornecidos e que qualquer melhoria será significativa.

✓ Os modelos candidatos são adequados ao tipo e tamanho dos dados?

Sim, a seleção de modelos candidatos é adequada e bem justificada para o cenário apresentado.

- **Análise do Tipo de Dado:** O problema é de regressão (previsão de um valor contínuo), e a seleção de modelos reflete isso, incluindo modelos lineares (`Regressão Linear` , `Lasso` , `Ridge` , `ElasticNet`), baseados em árvores (`Random Forest` , `DecisionTree` , `XGBoost`) e de distância (`SVR` , `KNN`). Esta variedade é excelente para explorar diferentes dinâmicas nos dados.
- **Análise do Tamanho dos Dados:** é processado dados de 49 empresas, com um tamanho de treino variando entre ~970 e ~2020 registros. Este é um volume de dados pequeno a médio.
- **Modelos:** como `Random Forest` e `XGBoost` são escolhas adequadas, pois conseguem capturar relações não-lineares e interações entre features, que são comuns em dados financeiros.
- **Nota** ao afirmar que "modelos mais complexos provocariam overfitting, pois há poucos dados para cada empresa. A escolha de hiperparâmetros contidos (como `max_depth=6` no `Random Forest`) é uma boa prática para mitigar esse risco.
- **Conclusão:** A seleção é apropriada, e a falta de superação do baseline não indica inadequação dos modelos, mas sim a fraqueza do sinal preditivo nos dados, como bem apontado pelo notebook.

✓ Algum modelo exige escala, encoding ou tratamento específico?

Sim, e o notebook demonstra a aplicação correta desses tratamentos:

- **Escala (Normalização/Padronização):**

- Exigem: `SVR`, `KNN`, e todos os modelos lineares (`Regressão Linear`, `Ridge`, `Lasso`, `ElasticNet`). Esses modelos são sensíveis à magnitude das features, e a falta de escala pode fazer com que features com maiores valores numéricos dominem o aprendizado.
- Aplicação no código corretamente o `StandardScaler` (que padroniza os dados para média 0 e desvio padrão 1) para esses modelos antes do treinamento, garantindo que a escala não influencie o resultado.
- Não exigem: `Random Forest`, `XGBoost` e `DecisionTree`, pois são modelos baseados em árvores que são invariantes à escala das features.

- **Encoding e Tratamento de Missing:**

- Embora a estrutura do código presuma que os dados já estão limpos e codificados, a função `prepare_data` (mencionada, mas não totalmente mostrada) e o código de `fallback (try... except NameError)` aplicam tratamentos essenciais, como:
 - Tratamento de Missing: Uso de `.fillna(method='ffill').fillna(method='bfill').fillna(0)`, uma técnica comum e eficaz para séries temporais, que preenche valores faltantes com o valor anterior, o próximo ou zero.
- Encoding: Os dados já parecem estar em formato numérico, não sendo necessário encoding de variáveis categóricas.

- ✓ Há alguma restrição de interpretabilidade, tempo ou custo computacional?

Sim, há considerações importantes sobre interpretabilidade e tempo.

- **Interpretabilidade:**

- Baixa: O código não avalia a importância das features, o que pode ser uma restrição para quem busca insights sobre quais variáveis são mais relevantes.
- Alta: fornecem coeficientes que indicam o impacto de cada variável na previsão.

- **Tempo e Custo Computacional:**

- Baixo: `Regressão Linear`, `Ridge`, `Lasso` e `ElasticNet` são modelos lineares e, portanto, muito rápidos para treinar, mesmo em dados de tamanho médio.
- Médio: `Random Forest`, `XGBoost` e `SVR` com `kernel RBF` podem ser significativamente mais lentos, especialmente em conjuntos de dados maiores. No entanto, para o tamanho dos dados (cerca de 2000 amostras), o tempo de treino ainda é bem pequeno (o código mede tempos de ~0.1s a ~0.5s).
- Checkpoints: Salva os resultados e modelos a cada 5 empresas para evitar a perda de progresso em caso de interrupção.
- Limpeza de Memória: Utiliza uma função `limpar_memoria` para liberar variáveis grandes após cada iteração, o que é crucial para evitar estouro de memória em ambientes como o Google Colab.
- Salvamento: o salvamento dos resultados contribui muito para reprocessar quando executado novamente o ambiente, caso o contrário o treinamento de todo o notebook pode chegar facilmente à 4 horas de processamento.

- ✓ **8. Treinamento e avaliação inicial**

- ✓ **Treinamento por empresas**

```
NOME_RESULTADOS_TREINO = "resultados_treino_inicial"
NOME_TRAINED_MODELS_TREINO = "trained_models_treino_inicial"
resultados_por_empresa = CarregarArquivoOnGitHub(NOME_RESULTADOS_TREINO)
trained_models_por_empresa = CarregarArquivoDivididoOnGitHub(NOME_TRAINED_MODELS_TREINO)
cache_valido = False
if resultados_por_empresa is not None and trained_models_por_empresa is not None:
    print("Resultados carregados do cache")
    print(f"    Tipo: {type(resultados_por_empresa)}")
    if isinstance(resultados_por_empresa, dict):
        empresas_cache = list(resultados_por_empresa.keys())
        print(f"    Empresas no cache: {len(empresas_cache)}")
        todas_validas = True
        for empresa in empresas_cache:
            dados = resultados_por_empresa[empresa]
            if isinstance(dados, dict):
                baseline_mse = dados.get('baseline_mse', np.nan)
                if np.isnan(baseline_mse) or baseline_mse == 0:
```

```

        todas_validas = False
        print(f"{empresa}: baseline_mse inválido ({baseline_mse})")
    else:
        todas_validas = False
        print(f"{empresa}: dados não são dicionário")
        break

if todas_validas and len(empresas_cache) > 0:
    cache_valido = True
    print(f"Cache válido com {len(empresas_cache)} empresas")
else:
    print("Cache inválido. Recriando...")
    resultados_por_empresa = None
    trained_models_por_empresa = None
else:
    print(f"Cache não é um dicionário. Tipo: {type(resultados_por_empresa)}")
    resultados_por_empresa = None
    trained_models_por_empresa = None

```

```

resultados_treino_inicial.pkl carregado com sucesso!
Tipo: dict
Tamanho: 66,740 bytes
trained_models_treino_inicial.pkl carregado com sucesso!
Tipo: dict
Tamanho: 700,855 bytes
Resultados carregados do cache
Tipo: <class 'dict'>
Empresas no cache: 3
chunk: dados não são dicionário
Cache inválido. Recriando...

```

```

if not cache_valido:
    print("\nProcessando treinamento inicial...")
    empresas = df_analise['Symbol'].unique().tolist()
    features_existentes = [f for f in features_selecionadas if f in df_analise.columns]
    resultados_por_empresa = {}
    trained_models_por_empresa = {}
    # Processar cada empresa
    for idx, empresa in enumerate(empresas):
        print("\n")
        print("=" * 70)
        print(f"ANALISANDO EMPRESA: {empresa} ({idx+1}/{len(empresas)})")
        try:
            # Preparar dados
            X_train, X_val, X_test, y_train, y_val, y_test = prepare_data(empresa, features_existentes)
            if X_train is None or len(X_train) == 0:
                print(f"Sem dados suficientes para {empresa}")
                continue

            print(f"\n Divisão dos dados - {empresa}:")
            print(f" • Treino: {len(X_train):,} registros x {X_train.shape[1]} features")
            print(f" • Validação: {len(X_val):,} registros")
            print(f" • Teste: {len(X_test):,} registros")
            results = {}
            trained_models = {}
            for name, model in MODELS.items():
                try:
                    if isinstance(model, XGBRegressor):
                        X_train_np = X_train.values.astype(np.float32)
                        X_val_np = X_val.values.astype(np.float32)
                        X_test_np = X_test.values.astype(np.float32)
                        y_train_np = y_train.values.ravel().astype(np.float32)
                        y_val_np = y_val.values.ravel().astype(np.float32)
                        t0 = time.time()
                        model.fit(X_train_np, y_train_np, eval_set=[(X_val_np, y_val_np)], verbose=False)
                        train_time = time.time() - t0
                        y_pred = model.predict(X_test_np)

                    # Tratamento especial para SVR e KNN (precisa de scaling)
                    elif isinstance(model, (SVR, KNeighborsRegressor)):
                        scaler = StandardScaler()
                        X_train_scaled = scaler.fit_transform(X_train)
                        X_test_scaled = scaler.transform(X_test)
                        t0 = time.time()
                        model.fit(X_train_scaled, y_train)
                        train_time = time.time() - t0
                        y_pred = model.predict(X_test_scaled)

                    # Tratamento especial para modelos regularizados
                    elif isinstance(model, (Ridge, Lasso, ElasticNet)):
                        scaler = StandardScaler()
                        X_train_scaled = scaler.fit_transform(X_train)
                        X_test_scaled = scaler.transform(X_test)

```



```

        t0 = time.time()
        model.fit(X_train_scaled, y_train)
        train_time = time.time() - t0
        y_pred = model.predict(X_test_scaled)

    # Modelos que não precisam de scaling
    else:
        t0 = time.time()
        model.fit(X_train, y_train)
        train_time = time.time() - t0
        y_pred = model.predict(X_test)

    # Avaliar
    results[name] = evaluate_model(y_test, y_pred)
    results[name]["train_time_s"] = round(train_time, 3)
    trained_models[name] = model

except Exception as e:
    print(f"Erro: {str(e)[:80]}")
    continue

if results and "Baseline (Média)" in results:
    print(f"\nCOMPARAÇÃO DE DESEMPENHO - {empresa}")
    df_results = format_results_table(results)
    if not df_results.empty:
        display(df_results)

    baseline_mse = results["Baseline (Média)"]["mse"]
    resultados_por_empresa[empresa] = {
        'baseline_mse': baseline_mse,
        'baseline_mae': results["Baseline (Média)"]["mae"],
        'baseline_r2': results["Baseline (Média)"]["r2"],
        'n_train': len(X_train),
        'n_val': len(X_val),
        'n_test': len(X_test)
    }

    # Adicionar métricas de cada modelo
    for model_name, model_results in results.items():
        prefix = model_name.lower().replace(' ', '_').replace('(', '').replace(')', '')
        resultados_por_empresa[empresa][f'{prefix}_mse'] = model_results.get('mse', np.nan)
        resultados_por_empresa[empresa][f'{prefix}_mae'] = model_results.get('mae', np.nan)
        resultados_por_empresa[empresa][f'{prefix}_r2'] = model_results.get('r2', np.nan)
        resultados_por_empresa[empresa][f'{prefix}_time'] = model_results.get('train_time_s', np.nan)

    # Calcular melhora
    if model_name != "Baseline (Média)":
        if not np.isnan(baseline_mse) and not np.isnan(model_results.get('mse', np.nan)):
            melhora = (baseline_mse - model_results['mse']) / baseline_mse * 100
            resultados_por_empresa[empresa][f'melhora_{prefix}_pct'] = melhora
    trained_models_por_empresa[empresa] = trained_models

    # Melhor modelo
    melhor_modelo = min([(k, v['mse']) for k, v in results.items() if k != 'Baseline (Média)'], key=lambda x: x[1])
    print(f"\n{empresa} processada com sucesso!")
    print(f"    Baseline MSE: {baseline_mse:.6f}")
    print(f"    Melhor modelo: {melhor_modelo[0]} (MSE: {melhor_modelo[1]:.6f})")

else:
    print(f"{empresa} - sem resultados válidos")

limpar_memoria([
    'X_train', 'X_val', 'X_test', 'y_train', 'y_val', 'y_test',
    'results', 'trained_models', 'df_results'
], mostrar_status=False)

except Exception as e:
    print(f"Erro ao processar {empresa}: {str(e)}")
    traceback.print_exc()
    limpar_memoria(mostrar_status=True)
    continue

if len(resultados_por_empresa) > 0:
    SalvarTreinamentoDivididoOnGitHub(
        nome_base=NOME_RESULTADOS_TREINO,
        dados=resultados_por_empresa,
        metadados={
            'descricao': 'Resultados de treinamento inicial por empresa',
            'empresas': list(resultados_por_empresa.keys()),
            'total_empresas': len(resultados_por_empresa)
        }
    )
)

```

```
print("Resultados salvos com sucesso!")
else:
    print("Nenhum resultado para salvar")

if trained_models_por_empresa:
    SalvarTreinamentoDivididoOnGitHub(
        nome_base=NOME_TRAINED_MODELS_TREINO,
        dados=trained_models_por_empresa,
        metadados={
            'descricao': 'Modelos treinados inicialmente',
            'empresas': list(trained_models_por_empresa.keys()),
            'total_empresas': len(trained_models_por_empresa)
        }
    )
    print("Modelos salvos com sucesso!")
else:
    print("Nenhum modelo para salvar")
```


Processando treinamento inicial...

ANALISANDO EMPRESA: AAPL (1/49)

- Divisão dos dados - AAPL:
- Treino: 2,020 registros x 8 features
 - Validação: 433 registros
 - Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - AAPL

	mse	rmse	mae	r2	train_time_s
Random Forest	0.000328	0.018110	0.012039	0.0095	0.304
SVR	0.000330	0.018171	0.011976	0.0028	0.001
ElasticNet	0.000331	0.018199	0.011925	-0.0003	0.001
Baseline (Média)	0.000331	0.018199	0.011925	-0.0003	0.000
Lasso	0.000331	0.018199	0.011925	-0.0003	0.001
Ridge	0.000332	0.018228	0.011928	-0.0034	0.002
Regressão Linear	0.000332	0.018228	0.011928	-0.0034	0.023
XGBoost	0.000340	0.018439	0.012083	-0.0268	0.269
KNN	0.000363	0.019046	0.013167	-0.0955	0.003
DecisionTree	0.000773	0.027803	0.019804	-1.3345	0.045

AAPL processada com sucesso!
Baseline MSE: 0.000331
Melhor modelo: Random Forest (MSE: 0.000328)

ANALISANDO EMPRESA: ABNB (2/49)

- Divisão dos dados - ABNB:
- Treino: 972 registros x 8 features
 - Validação: 209 registros
 - Teste: 209 registros

COMPARAÇÃO DE DESEMPENHO - ABNB

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000340	0.018431	0.013789	-0.0006	0.000
ElasticNet	0.000340	0.018431	0.013789	-0.0006	0.001
Lasso	0.000340	0.018431	0.013789	-0.0006	0.001
Ridge	0.000343	0.018519	0.013864	-0.0101	0.001
Regressão Linear	0.000343	0.018520	0.013865	-0.0102	0.003
Random Forest	0.000349	0.018680	0.014008	-0.0278	0.174
XGBoost	0.000356	0.018872	0.014153	-0.0490	0.035
SVR	0.000479	0.021883	0.016562	-0.4104	0.001
KNN	0.000496	0.022268	0.017492	-0.4604	0.002
DecisionTree	0.000979	0.031288	0.024049	-1.8833	0.022

ABNB processada com sucesso!
Baseline MSE: 0.000340
Melhor modelo: ElasticNet (MSE: 0.000340)

ANALISANDO EMPRESA: ADBE (3/49)

- Divisão dos dados - ADBE:
- Treino: 2,020 registros x 8 features
 - Validação: 433 registros
 - Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - ADBE

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000508	0.022533	0.015299	-0.0161	0.000
ElasticNet	0.000508	0.022533	0.015299	-0.0161	0.001
Lasso	0.000508	0.022533	0.015299	-0.0161	0.001

Random Forest	0.000512	0.022622	0.015279	-0.0243	0.314
Ridge	0.000515	0.022693	0.015395	-0.0307	0.001
Regressão Linear	0.000515	0.022694	0.015396	-0.0307	0.003
XGBoost	0.000518	0.022755	0.015444	-0.0363	0.036
SVR	0.000630	0.025104	0.018136	-0.2613	0.002
KNN	0.000631	0.025111	0.018115	-0.2620	0.002
DecisionTree	0.001242	0.035245	0.026681	-1.4861	0.042

ADBE processada com sucesso!
 Baseline MSE: 0.000508
 Melhor modelo: ElasticNet (MSE: 0.000508)

=====

ANALISANDO EMPRESA: ADI (4/49)

Divisão dos dados - ADI:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - ADI

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000581	0.024095	0.016312	-0.0009	0.000
ElasticNet	0.000581	0.024095	0.016312	-0.0009	0.001
Lasso	0.000581	0.024095	0.016312	-0.0009	0.001
Random Forest	0.000586	0.024206	0.016483	-0.0102	0.293
SVR	0.000593	0.024351	0.016659	-0.0223	0.002
Ridge	0.000596	0.024403	0.016762	-0.0267	0.001
Regressão Linear	0.000596	0.024404	0.016763	-0.0268	0.002
KNN	0.000610	0.024707	0.017909	-0.0524	0.002
XGBoost	0.000619	0.024870	0.017196	-0.0663	0.035
DecisionTree	0.001214	0.034846	0.025230	-1.0934	0.054

ADI processada com sucesso!
 Baseline MSE: 0.000581
 Melhor modelo: ElasticNet (MSE: 0.000581)

=====

ANALISANDO EMPRESA: ADP (5/49)

Divisão dos dados - ADP:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - ADP

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000208	0.014415	0.010045	-0.0058	0.000
ElasticNet	0.000208	0.014415	0.010045	-0.0058	0.002
Lasso	0.000208	0.014415	0.010045	-0.0058	0.002
XGBoost	0.000211	0.014525	0.010109	-0.0212	0.063
Ridge	0.000213	0.014578	0.010028	-0.0287	0.001
Regressão Linear	0.000213	0.014578	0.010028	-0.0287	0.003
Random Forest	0.000216	0.014686	0.010108	-0.0440	0.312
SVR	0.000252	0.015888	0.011869	-0.2219	0.002
KNN	0.000255	0.015959	0.011772	-0.2329	0.004
DecisionTree	0.000449	0.021185	0.015699	-1.1724	0.063

ADP processada com sucesso!
 Baseline MSE: 0.000208
 Melhor modelo: ElasticNet (MSE: 0.000208)

=====

ANALISANDO EMPRESA: AMAT (6/49)

Divisão dos dados - AMAT:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - AMAT

	mse	rmse	mae	r2	train_time_s
XGBoost	0.000972	0.031172	0.022041	0.0199	0.057
Random Forest	0.000979	0.031286	0.022186	0.0127	0.484
Baseline (Média)	0.000996	0.031552	0.022196	-0.0042	0.000
ElasticNet	0.000996	0.031552	0.022196	-0.0042	0.003
Lasso	0.000996	0.031552	0.022196	-0.0042	0.002
Ridge	0.001000	0.031624	0.022450	-0.0088	0.002
Regressão Linear	0.001000	0.031625	0.022451	-0.0088	0.004
KNN	0.001069	0.032701	0.023380	-0.0786	0.004
SVR	0.001155	0.033988	0.024713	-0.1652	0.003
DecisionTree	0.001702	0.041252	0.030521	-0.7165	0.076

AMAT processada com sucesso!
Baseline MSE: 0.000996
Melhor modelo: XGBoost (MSE: 0.000972)

=====
ANALISANDO EMPRESA: AMD (7/49)

Divisão dos dados - AMD:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - AMD

	mse	rmse	mae	r2	train_time_s
Random Forest	0.001520	0.038984	0.026904	0.0186	0.501
XGBoost	0.001549	0.039361	0.027096	-0.0005	0.069
Baseline (Média)	0.001550	0.039369	0.026909	-0.0009	0.000
ElasticNet	0.001550	0.039369	0.026909	-0.0009	0.002
Lasso	0.001550	0.039369	0.026909	-0.0009	0.002
Ridge	0.001551	0.039382	0.027090	-0.0015	0.002
Regressão Linear	0.001551	0.039382	0.027090	-0.0015	0.012
KNN	0.001714	0.041399	0.028337	-0.1068	0.004
SVR	0.002320	0.048162	0.035730	-0.4979	0.034
DecisionTree	0.003939	0.062758	0.041983	-1.5434	0.056

AMD processada com sucesso!
Baseline MSE: 0.001550
Melhor modelo: Random Forest (MSE: 0.001520)

=====
ANALISANDO EMPRESA: AMGN (8/49)

Divisão dos dados - AMGN:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - AMGN

	mse	rmse	mae	r2	train_time_s
XGBoost	0.000297	0.017223	0.012307	0.0015	0.038
Ridge	0.000297	0.017235	0.012404	0.0000	0.002
Regressão Linear	0.000297	0.017235	0.012404	0.0000	0.002
Baseline (Média)	0.000297	0.017236	0.012301	-0.0000	0.000
Lasso	0.000297	0.017236	0.012301	-0.0000	0.002
ElasticNet	0.000297	0.017236	0.012301	-0.0000	0.002
Random Forest	0.000299	0.017239	0.012452	0.0100	0.227

Random Forest	0.000300	0.017329	0.012432	-0.0109	0.327
SVR	0.000327	0.018079	0.013170	-0.1003	0.001
KNN	0.000328	0.018108	0.013168	-0.1037	0.002
DecisionTree	0.000612	0.024735	0.018555	-1.0596	0.044

AMGN processada com sucesso!
Baseline MSE: 0.000297
Melhor modelo: XGBoost (MSE: 0.000297)

=====

ANALISANDO EMPRESA: AMZN (9/49)

Divisão dos dados - AMZN:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - AMZN

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000432	0.020779	0.015179	-0.0003	0.000
ElasticNet	0.000432	0.020779	0.015179	-0.0003	0.001
Lasso	0.000432	0.020779	0.015179	-0.0003	0.001
Ridge	0.000434	0.020821	0.015201	-0.0043	0.001
Regressão Linear	0.000434	0.020821	0.015201	-0.0044	0.002
Random Forest	0.000453	0.021278	0.015509	-0.0489	0.337
XGBoost	0.000472	0.021726	0.015628	-0.0936	0.039
KNN	0.000557	0.023594	0.017158	-0.2896	0.003
SVR	0.000839	0.028965	0.022951	-0.9436	0.002
DecisionTree	0.000970	0.031139	0.022827	-1.2464	0.042

AMZN processada com sucesso!
Baseline MSE: 0.000432
Melhor modelo: ElasticNet (MSE: 0.000432)

=====

ANALISANDO EMPRESA: AVGO (10/49)

Divisão dos dados - AVGO:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - AVGO

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.001152	0.033938	0.022665	-0.0011	0.000
ElasticNet	0.001152	0.033938	0.022665	-0.0011	0.001
Lasso	0.001152	0.033938	0.022665	-0.0011	0.001
Random Forest	0.001161	0.034068	0.022803	-0.0088	0.300
Ridge	0.001173	0.034255	0.022907	-0.0199	0.001
Regressão Linear	0.001173	0.034256	0.022907	-0.0200	0.002
XGBoost	0.001180	0.034349	0.022731	-0.0255	0.039
KNN	0.001237	0.035175	0.023994	-0.0754	0.002
SVR	0.001327	0.036434	0.025273	-0.1538	0.002
DecisionTree	0.002350	0.048480	0.033273	-1.0428	0.043

AVGO processada com sucesso!
Baseline MSE: 0.001152
Melhor modelo: ElasticNet (MSE: 0.001152)

=====

ANALISANDO EMPRESA: BKNG (11/49)

Divisão dos dados - BKNG:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - BKNG

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000400	0.020010	0.014385	-0.0000	0.000
ElasticNet	0.000400	0.020010	0.014385	-0.0000	0.001
Lasso	0.000400	0.020010	0.014385	-0.0000	0.001
Random Forest	0.000404	0.020104	0.014658	-0.0094	0.313
XGBoost	0.000411	0.020284	0.014733	-0.0276	0.042
KNN	0.000492	0.022183	0.016481	-0.2290	0.002
SVR	0.000537	0.023174	0.017138	-0.3413	0.002
DecisionTree	0.000987	0.031421	0.022992	-1.4658	0.042
Ridge	0.001159	0.034037	0.015937	-1.8934	0.001
Regressão Linear	0.001162	0.034095	0.015941	-1.9033	0.007

BKNG processada com sucesso!

Baseline MSE: 0.000400

Melhor modelo: ElasticNet (MSE: 0.000400)

ANALISANDO EMPRESA: CDNS (12/49)

Divisão dos dados - CDNS:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - CDNS

	mse	rmse	mae	r2	train_time_s
Ridge	0.000662	0.025724	0.017699	0.0094	0.001
Regressão Linear	0.000662	0.025724	0.017699	0.0094	0.003
ElasticNet	0.000668	0.025847	0.017656	-0.0000	0.001
Baseline (Média)	0.000668	0.025847	0.017656	-0.0000	0.000
Lasso	0.000668	0.025847	0.017656	-0.0000	0.001
Random Forest	0.000675	0.025990	0.017940	-0.0111	0.315
XGBoost	0.000681	0.026104	0.018161	-0.0200	0.041
KNN	0.000745	0.027290	0.019293	-0.1149	0.002
SVR	0.001006	0.031711	0.024001	-0.5052	0.001
DecisionTree	0.001319	0.036322	0.026874	-0.9749	0.054

CDNS processada com sucesso!

Baseline MSE: 0.000668

Melhor modelo: Ridge (MSE: 0.000662)

ANALISANDO EMPRESA: CHTR (13/49)

Divisão dos dados - CHTR:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - CHTR

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000833	0.028855	0.018791	-0.0060	0.000
ElasticNet	0.000833	0.028855	0.018791	-0.0060	0.001
Lasso	0.000833	0.028855	0.018791	-0.0060	0.001
Ridge	0.000849	0.029135	0.019066	-0.0257	0.001
Regressão Linear	0.000849	0.029136	0.019066	-0.0257	0.003
Random Forest	0.000850	0.029162	0.019051	-0.0275	0.323
XGBoost	0.000896	0.029941	0.019740	-0.0831	0.046
KNN	0.000956	0.030915	0.020648	-0.1548	0.002
SVR	0.001016	0.031867	0.023096	-0.2270	0.002
DecisionTree	0.001950	0.044158	0.030662	-1.3560	0.039


```
CHTR processada com sucesso!  
Baseline MSE: 0.000833  
Melhor modelo: ElasticNet (MSE: 0.000833)
```

```
=====
```

```
ANALISANDO EMPRESA: CMCSA (14/49)
```

```
Divisão dos dados - CMCSA:  
• Treino: 2,020 registros x 8 features  
• Validação: 433 registros  
• Teste: 433 registros
```

```
COMPARAÇÃO DE DESEMPENHO - CMCSA
```

	mse	rmse	mae	r2	train_time_s
XGBoost	0.000324	0.018002	0.012230	0.0448	0.043
Random Forest	0.000328	0.018121	0.012239	0.0320	0.314
Regressão Linear	0.000338	0.018395	0.012326	0.0026	0.002
Ridge	0.000338	0.018395	0.012326	0.0026	0.001
ElasticNet	0.000341	0.018459	0.012342	-0.0044	0.001
Baseline (Média)	0.000341	0.018459	0.012342	-0.0044	0.000
Lasso	0.000341	0.018459	0.012342	-0.0044	0.001
KNN	0.000388	0.019700	0.013565	-0.1440	0.002
SVR	0.000607	0.024638	0.018823	-0.7894	0.001
DecisionTree	0.000698	0.026418	0.019567	-1.0572	0.051

```
CMCSA processada com sucesso!  
Baseline MSE: 0.000341  
Melhor modelo: XGBoost (MSE: 0.000324)
```

```
=====
```

```
ANALISANDO EMPRESA: COST (15/49)
```

```
Divisão dos dados - COST:  
• Treino: 2,020 registros x 8 features  
• Validação: 433 registros  
• Teste: 433 registros
```

```
COMPARAÇÃO DE DESEMPENHO - COST
```

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000172	0.013111	0.009700	-0.0015	0.000
ElasticNet	0.000172	0.013111	0.009700	-0.0015	0.001
Lasso	0.000172	0.013111	0.009700	-0.0015	0.001
Ridge	0.000174	0.013177	0.009777	-0.0116	0.001
Regressão Linear	0.000174	0.013178	0.009777	-0.0116	0.004
Random Forest	0.000180	0.013428	0.009892	-0.0504	0.297
XGBoost	0.000181	0.013459	0.009952	-0.0553	0.038
KNN	0.000222	0.014883	0.010993	-0.2904	0.002
SVR	0.000361	0.019002	0.015869	-1.1036	0.001
DecisionTree	0.000484	0.022007	0.015884	-1.8214	0.045

```
COST processada com sucesso!  
Baseline MSE: 0.000172  
Melhor modelo: ElasticNet (MSE: 0.000172)
```

```
=====
```

```
ANALISANDO EMPRESA: CSCO (16/49)
```

```
Divisão dos dados - CSCO:  
• Treino: 2,020 registros x 8 features  
• Validação: 433 registros  
• Teste: 433 registros
```

```
COMPARAÇÃO DE DESEMPENHO - CSCO
```

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000321	0.017915	0.011681	-0.0068	0.000
ElasticNet	0.000321	0.017915	0.011681	-0.0068	0.001

Lasso	0.000321	0.017915	0.011681	-0.0068	0.001
Random Forest	0.000327	0.018083	0.011731	-0.0258	0.304
XGBoost	0.000331	0.018200	0.011949	-0.0391	0.040
Ridge	0.000332	0.018231	0.011802	-0.0426	0.001
Regressão Linear	0.000332	0.018231	0.011802	-0.0427	0.004
KNN	0.000351	0.018728	0.012410	-0.1003	0.002
SVR	0.000354	0.018819	0.012674	-0.1110	0.002
DecisionTree	0.000972	0.031177	0.020148	-2.0492	0.048

CSC0 processada com sucesso!

Baseline MSE: 0.000321

Melhor modelo: ElasticNet (MSE: 0.000321)

=====

ANALISANDO EMPRESA: CSX (17/49)

Divisão dos dados - CSX:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - CSX

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000241	0.015528	0.010978	-0.0002	0.000
ElasticNet	0.000241	0.015528	0.010978	-0.0002	0.001
Lasso	0.000241	0.015528	0.010978	-0.0002	0.001
Ridge	0.000242	0.015569	0.011023	-0.0056	0.001
Regressão Linear	0.000242	0.015570	0.011024	-0.0056	0.002
Random Forest	0.000253	0.015896	0.011131	-0.0482	0.306
KNN	0.000265	0.016273	0.012079	-0.0985	0.002
XGBoost	0.000277	0.016649	0.011359	-0.1498	0.040
SVR	0.000437	0.020906	0.015994	-0.8131	0.002
DecisionTree	0.000682	0.026121	0.018933	-1.8303	0.054

CSX processada com sucesso!

Baseline MSE: 0.000241

Melhor modelo: ElasticNet (MSE: 0.000241)

=====

ANALISANDO EMPRESA: GILD (18/49)

Divisão dos dados - GILD:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - GILD

	mse	rmse	mae	r2	train_time_s
Ridge	0.000290	0.017029	0.012742	0.0043	0.001
Regressão Linear	0.000290	0.017029	0.012742	0.0043	0.002
ElasticNet	0.000292	0.017094	0.012814	-0.0033	0.001
Baseline (Média)	0.000292	0.017094	0.012814	-0.0033	0.000
Lasso	0.000292	0.017094	0.012814	-0.0033	0.001
Random Forest	0.000293	0.017129	0.012825	-0.0073	0.316
XGBoost	0.000295	0.017175	0.012804	-0.0128	0.038
KNN	0.000341	0.018478	0.013916	-0.1723	0.002
SVR	0.000571	0.023891	0.019718	-0.9597	0.001
DecisionTree	0.000834	0.028883	0.021036	-1.8644	0.037

GILD processada com sucesso!

Baseline MSE: 0.000292

Melhor modelo: Ridge (MSE: 0.000290)

=====

ANALISANDO EMPRESA: GOOG (19/49)

Divisão dos dados - GOOG:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - GOOG

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000385	0.019624	0.014296	-0.0027	0.000
ElasticNet	0.000385	0.019624	0.014296	-0.0027	0.002
Lasso	0.000385	0.019624	0.014296	-0.0027	0.002
Random Forest	0.000388	0.019702	0.014346	-0.0107	0.314
Ridge	0.000389	0.019732	0.014356	-0.0138	0.002
Regressão Linear	0.000389	0.019732	0.014356	-0.0138	0.002
XGBoost	0.000397	0.019926	0.014513	-0.0338	0.040
KNN	0.000458	0.021403	0.015804	-0.1927	0.002
SVR	0.000519	0.022777	0.017810	-0.3508	0.001
DecisionTree	0.001004	0.031688	0.023996	-1.6145	0.058

GOOG processada com sucesso!

Baseline MSE: 0.000385

Melhor modelo: ElasticNet (MSE: 0.000385)

=====

ANALISANDO EMPRESA: GOOGL (20/49)

Divisão dos dados - GOOGL:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - GOOGL

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000396	0.019889	0.014467	-0.0030	0.000
ElasticNet	0.000396	0.019889	0.014467	-0.0030	0.004
Lasso	0.000396	0.019889	0.014467	-0.0030	0.002
Random Forest	0.000396	0.019909	0.014451	-0.0049	0.493
XGBoost	0.000400	0.020004	0.014577	-0.0145	0.065
Ridge	0.000401	0.020024	0.014536	-0.0166	0.002
Regressão Linear	0.000401	0.020025	0.014536	-0.0166	0.006
KNN	0.000433	0.020804	0.015243	-0.0973	0.006
SVR	0.000554	0.023532	0.018541	-0.4040	0.002
DecisionTree	0.000867	0.029443	0.021694	-1.1980	0.063

GOOGL processada com sucesso!

Baseline MSE: 0.000396

Melhor modelo: ElasticNet (MSE: 0.000396)

=====

ANALISANDO EMPRESA: HON (21/49)

Divisão dos dados - HON:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - HON

	mse	rmse	mae	r2	train_time_s
XGBoost	0.000251	0.015853	0.011156	0.0211	0.067
Random Forest	0.000253	0.015909	0.011261	0.0143	0.424
Regressão Linear	0.000254	0.015948	0.011095	0.0093	0.006
Ridge	0.000254	0.015948	0.011095	0.0093	0.002
ElasticNet	0.000257	0.016023	0.011106	-0.0000	0.002
Baseline (Média)	0.000257	0.016023	0.011106	-0.0000	0.001

Lasso	0.000257	0.016023	0.011106	-0.0000	0.002
KNN	0.000291	0.017052	0.012152	-0.1325	0.004
SVR	0.000335	0.018290	0.013630	-0.3030	0.002
DecisionTree	0.000455	0.021326	0.016184	-0.7714	0.072

HON processada com sucesso!

Baseline MSE: 0.000257

Melhor modelo: XGBoost (MSE: 0.000251)

=====

ANALISANDO EMPRESA: INTC (22/49)

Divisão dos dados - INTC:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - INTC

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.001948	0.044141	0.030504	-0.0117	0.001
ElasticNet	0.001948	0.044141	0.030504	-0.0117	0.001
Lasso	0.001948	0.044141	0.030504	-0.0117	0.001
SVR	0.002001	0.044732	0.030995	-0.0390	0.003
Random Forest	0.002027	0.045019	0.032098	-0.0524	0.315
Ridge	0.002054	0.045323	0.031638	-0.0666	0.001
Regressão Linear	0.002054	0.045326	0.031640	-0.0667	0.003
XGBoost	0.002086	0.045668	0.032322	-0.0829	0.040
KNN	0.002174	0.046629	0.033303	-0.1290	0.004
DecisionTree	0.003822	0.061823	0.045046	-0.9846	0.039

INTC processada com sucesso!

Baseline MSE: 0.001948

Melhor modelo: ElasticNet (MSE: 0.001948)

=====

ANALISANDO EMPRESA: ISRG (23/49)

Divisão dos dados - ISRG:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - ISRG

	mse	rmse	mae	r2	train_time_s
Random Forest	0.000440	0.020980	0.013723	-0.0023	0.299
Baseline (Média)	0.000440	0.020988	0.013744	-0.0030	0.000
ElasticNet	0.000440	0.020988	0.013744	-0.0030	0.001
Lasso	0.000440	0.020988	0.013744	-0.0030	0.001
Ridge	0.000443	0.021043	0.013763	-0.0083	0.001
Regressão Linear	0.000443	0.021043	0.013763	-0.0083	0.004
XGBoost	0.000443	0.021053	0.013907	-0.0092	0.035
KNN	0.000497	0.022303	0.015742	-0.1327	0.002
SVR	0.000615	0.024795	0.017166	-0.3999	0.002
DecisionTree	0.000854	0.029229	0.021090	-0.9453	0.044

ISRG processada com sucesso!

Baseline MSE: 0.000440

Melhor modelo: Random Forest (MSE: 0.000440)

=====

ANALISANDO EMPRESA: KDP (24/49)

Divisão dos dados - KDP:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - KDP

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000260	0.016133	0.011097	-0.0021	0.000
ElasticNet	0.000260	0.016133	0.011097	-0.0021	0.001
Lasso	0.000260	0.016133	0.011097	-0.0021	0.001
Ridge	0.000269	0.016395	0.011257	-0.0349	0.001
Regressão Linear	0.000269	0.016395	0.011257	-0.0350	0.003
Random Forest	0.000279	0.016709	0.011547	-0.0749	0.321
KNN	0.000290	0.017028	0.012305	-0.1163	0.002
XGBoost	0.000435	0.020861	0.013828	-0.6754	0.036
SVR	0.001030	0.032087	0.023751	-2.9639	0.003
DecisionTree	0.003378	0.058125	0.031582	-12.0077	0.041

KDP processada com sucesso!

Baseline MSE: 0.000260

Melhor modelo: ElasticNet (MSE: 0.000260)

ANALISANDO EMPRESA: KHC (25/49)

Divisão dos dados - KHC:

- Treino: 1,931 registros x 8 features
- Validação: 415 registros
- Teste: 414 registros

COMPARAÇÃO DE DESEMPENHO - KHC

	mse	rmse	mae	r2	train_time_s
Random Forest	0.000259	0.016083	0.012306	0.0018	0.302
Baseline (Média)	0.000259	0.016107	0.012343	-0.0011	0.000
ElasticNet	0.000259	0.016107	0.012343	-0.0011	0.001
Lasso	0.000259	0.016107	0.012343	-0.0011	0.001
Ridge	0.000260	0.016127	0.012426	-0.0036	0.001
Regressão Linear	0.000260	0.016127	0.012426	-0.0036	0.003
XGBoost	0.000261	0.016149	0.012290	-0.0063	0.038
KNN	0.000328	0.018107	0.013650	-0.2652	0.002
DecisionTree	0.000553	0.023514	0.017573	-1.1336	0.042
SVR	0.001071	0.032724	0.025790	-3.1323	0.003

KHC processada com sucesso!

Baseline MSE: 0.000259

Melhor modelo: Random Forest (MSE: 0.000259)

ANALISANDO EMPRESA: KLAC (26/49)

Divisão dos dados - KLAC:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - KLAC

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000990	0.031465	0.021552	-0.0038	0.000
ElasticNet	0.000990	0.031465	0.021552	-0.0038	0.002
Lasso	0.000990	0.031465	0.021552	-0.0038	0.001
XGBoost	0.000995	0.031551	0.021783	-0.0093	0.037
Random Forest	0.001012	0.031807	0.022026	-0.0258	0.324
KNN	0.001111	0.033338	0.023896	-0.1269	0.002
SVR	0.001308	0.036169	0.026678	-0.3264	0.003
DecisionTree	0.001884	0.043409	0.033069	-0.9106	0.041
Ridge	0.004365	0.066068	0.024525	-3.4258	0.002

Regressão Linear 0.004374 0.066134 0.024529 -3.4345 0.003

KLAC processada com sucesso!

Baseline MSE: 0.000990

Melhor modelo: ElasticNet (MSE: 0.000990)

=====
ANALISANDO EMPRESA: LIN (27/49)

Divisão dos dados - LIN:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - LIN

	mse	rmse	mae	r2	train_time_s
Ridge	0.000135	0.011624	0.008628	0.0112	0.001
Regressão Linear	0.000135	0.011624	0.008628	0.0112	0.003
Random Forest	0.000135	0.011628	0.008626	0.0104	0.345
Baseline (Média)	0.000137	0.011694	0.008664	-0.0008	0.000
Lasso	0.000137	0.011694	0.008664	-0.0008	0.001
ElasticNet	0.000137	0.011694	0.008664	-0.0008	0.001
XGBoost	0.000139	0.011776	0.008647	-0.0150	0.038
KNN	0.000170	0.013048	0.009772	-0.2460	0.002
SVR	0.000182	0.013502	0.010356	-0.3342	0.001
DecisionTree	0.000360	0.018968	0.014440	-1.6334	0.046

LIN processada com sucesso!

Baseline MSE: 0.000137

Melhor modelo: Ridge (MSE: 0.000135)

=====
ANALISANDO EMPRESA: LRCX (28/49)

Divisão dos dados - LRCX:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - LRCX

	mse	rmse	mae	r2	train_time_s
XGBoost	0.001141	0.033786	0.024921	-0.0055	0.040
Baseline (Média)	0.001144	0.033816	0.024635	-0.0073	0.000
ElasticNet	0.001144	0.033816	0.024635	-0.0073	0.001
Lasso	0.001144	0.033816	0.024635	-0.0073	0.001
Random Forest	0.001180	0.034350	0.025330	-0.0393	0.331
KNN	0.001262	0.035529	0.026642	-0.1119	0.003
SVR	0.001409	0.037533	0.028231	-0.2409	0.003
DecisionTree	0.002070	0.045496	0.035088	-0.8232	0.058
Ridge	0.003570	0.059751	0.027173	-2.1447	0.001
Regressão Linear	0.003577	0.059810	0.027177	-2.1509	0.016

LRCX processada com sucesso!

Baseline MSE: 0.001144

Melhor modelo: XGBoost (MSE: 0.001141)

=====
ANALISANDO EMPRESA: MELI (29/49)

Divisão dos dados - MELI:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - MELI

	mse	rmse	mae	r2	train_time_s
Ridge	0.000654	0.025569	0.018107	-0.0029	0.001

Regressão Linear	0.000654	0.025569	0.018107	-0.0029	0.010
ElasticNet	0.000654	0.025578	0.018076	-0.0036	0.001
Baseline (Média)	0.000654	0.025578	0.018076	-0.0036	0.000
Lasso	0.000654	0.025578	0.018076	-0.0036	0.001
Random Forest	0.000673	0.025943	0.018210	-0.0324	0.329
XGBoost	0.000686	0.026185	0.018353	-0.0518	0.044
KNN	0.000857	0.029270	0.022116	-0.3142	0.002
SVR	0.001407	0.037510	0.028945	-1.1584	0.008
DecisionTree	0.001963	0.044305	0.030464	-2.0113	0.040

MELI processada com sucesso!
Baseline MSE: 0.000654
Melhor modelo: Ridge (MSE: 0.000654)

=====

ANALISANDO EMPRESA: META (30/49)

Divisão dos dados - META:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - META

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000549	0.023428	0.015982	-0.0002	0.000
ElasticNet	0.000549	0.023428	0.015982	-0.0002	0.001
Lasso	0.000549	0.023428	0.015982	-0.0002	0.001
Ridge	0.000550	0.023446	0.016090	-0.0018	0.001
Regressão Linear	0.000550	0.023446	0.016090	-0.0018	0.013
Random Forest	0.000561	0.023682	0.016180	-0.0220	0.311
XGBoost	0.000564	0.023754	0.016184	-0.0283	0.040
KNN	0.000651	0.025516	0.018122	-0.1864	0.002
SVR	0.000849	0.029137	0.020770	-0.5471	0.003
DecisionTree	0.000990	0.031461	0.022300	-0.8038	0.050

META processada com sucesso!
Baseline MSE: 0.000549
Melhor modelo: ElasticNet (MSE: 0.000549)

=====

ANALISANDO EMPRESA: MPWR (31/49)

Divisão dos dados - MPWR:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - MPWR

	mse	rmse	mae	r2	train_time_s
Random Forest	0.001354	0.036793	0.025587	0.0188	0.329
XGBoost	0.001356	0.036829	0.025673	0.0168	0.037
Ridge	0.001361	0.036889	0.025447	0.0136	0.001
Regressão Linear	0.001361	0.036890	0.025448	0.0136	0.004
ElasticNet	0.001380	0.037143	0.025434	-0.0000	0.001
Baseline (Média)	0.001380	0.037143	0.025434	-0.0000	0.000
Lasso	0.001380	0.037143	0.025434	-0.0000	0.001
SVR	0.001404	0.037473	0.026493	-0.0179	0.002
KNN	0.001440	0.037943	0.027152	-0.0435	0.002
DecisionTree	0.002413	0.049126	0.036506	-0.7494	0.042

MPWR processada com sucesso!
Baseline MSE: 0.001380
Melhor modelo: Random Forest (MSE: 0.001354)

=====

ANALISANDO EMPRESA: MRNA (32/49)

Divisão dos dados - MRNA:

- Treino: 1,327 registros x 8 features
- Validação: 284 registros
- Teste: 285 registros

COMPARAÇÃO DE DESEMPENHO - MRNA

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.001728	0.041571	0.030764	-0.0025	0.000
ElasticNet	0.001728	0.041571	0.030764	-0.0025	0.001
Lasso	0.001728	0.041571	0.030764	-0.0025	0.001
Random Forest	0.001745	0.041776	0.031036	-0.0124	0.295
Ridge	0.001755	0.041890	0.030671	-0.0179	0.001
Regressão Linear	0.001755	0.041890	0.030671	-0.0180	0.004
XGBoost	0.001770	0.042068	0.031274	-0.0266	0.035
KNN	0.002194	0.046837	0.036245	-0.2726	0.001
SVR	0.002330	0.048272	0.037322	-0.3518	0.008
DecisionTree	0.003935	0.062733	0.048904	-1.2830	0.029

MRNA processada com sucesso!

Baseline MSE: 0.001728

Melhor modelo: ElasticNet (MSE: 0.001728)

=====

ANALISANDO EMPRESA: MRVL (33/49)

Divisão dos dados - MRVL:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - MRVL

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.002195	0.046854	0.031202	-0.0047	0.000
ElasticNet	0.002195	0.046854	0.031202	-0.0047	0.001
Lasso	0.002195	0.046854	0.031202	-0.0047	0.001
Ridge	0.002200	0.046904	0.031067	-0.0069	0.001
Regressão Linear	0.002200	0.046904	0.031067	-0.0069	0.015
Random Forest	0.002264	0.047582	0.031616	-0.0362	0.342
XGBoost	0.002265	0.047593	0.031516	-0.0367	0.042
KNN	0.002326	0.048232	0.032443	-0.0647	0.002
SVR	0.002545	0.050451	0.035350	-0.1650	0.003
DecisionTree	0.003821	0.061813	0.043842	-0.7488	0.052

MRVL processada com sucesso!

Baseline MSE: 0.002195

Melhor modelo: ElasticNet (MSE: 0.002195)

=====

ANALISANDO EMPRESA: MSFT (34/49)

Divisão dos dados - MSFT:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - MSFT

	mse	rmse	mae	r2	train_time_s
SVR	0.000291	0.017047	0.011508	-0.0029	0.003
Baseline (Média)	0.000291	0.017058	0.011518	-0.0042	0.000
ElasticNet	0.000291	0.017058	0.011518	-0.0042	0.002
Lasso	0.000291	0.017058	0.011518	-0.0042	0.002
XGBoost	0.000300	0.017330	0.011948	-0.0366	0.065

Ridge	0.000304	0.017434	0.011843	-0.0490	0.001
Regressão Linear	0.000304	0.017434	0.011844	-0.0491	0.013
Random Forest	0.000306	0.017498	0.012066	-0.0567	0.514
KNN	0.000374	0.019331	0.013713	-0.2897	0.003
DecisionTree	0.000875	0.029574	0.020963	-2.0186	0.070

MSFT processada com sucesso!

Baseline MSE: 0.000291

Melhor modelo: SVR (MSE: 0.000291)

=====

ANALISANDO EMPRESA: MU (35/49)

Divisão dos dados - MU:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - MU

	mse	rmse	mae	r2	train_time_s
Ridge	0.002005	0.044776	0.032799	-0.0153	0.002
Regressão Linear	0.002005	0.044776	0.032799	-0.0153	0.003
ElasticNet	0.002010	0.044830	0.032842	-0.0177	0.002
Baseline (Média)	0.002010	0.044830	0.032842	-0.0177	0.000
Lasso	0.002010	0.044830	0.032842	-0.0177	0.002
Random Forest	0.002034	0.045100	0.033103	-0.0301	0.538
XGBoost	0.002042	0.045193	0.033437	-0.0343	0.059
KNN	0.002146	0.046328	0.034716	-0.0869	0.004
SVR	0.002159	0.046466	0.033947	-0.0934	0.006
DecisionTree	0.003498	0.059146	0.044268	-0.7715	0.056

MU processada com sucesso!

Baseline MSE: 0.002010

Melhor modelo: Ridge (MSE: 0.002005)

=====

ANALISANDO EMPRESA: NFLX (36/49)

Divisão dos dados - NFLX:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - NFLX

	mse	rmse	mae	r2	train_time_s
Regressão Linear	0.000480	0.021898	0.014960	0.0002	0.015
Ridge	0.000480	0.021898	0.014961	0.0002	0.001
ElasticNet	0.000481	0.021925	0.014967	-0.0022	0.001
Baseline (Média)	0.000481	0.021925	0.014967	-0.0022	0.000
Lasso	0.000481	0.021925	0.014967	-0.0022	0.001
Random Forest	0.000489	0.022107	0.015347	-0.0189	0.315
XGBoost	0.000501	0.022378	0.015222	-0.0441	0.037
KNN	0.000619	0.024872	0.018191	-0.2898	0.002
DecisionTree	0.001408	0.037527	0.025487	-1.9362	0.045
SVR	0.001528	0.039095	0.031451	-2.1866	0.007

NFLX processada com sucesso!

Baseline MSE: 0.000481

Melhor modelo: Regressão Linear (MSE: 0.000480)

=====

ANALISANDO EMPRESA: NVDA (37/49)

Divisão dos dados - NVDA:

- Treino: 2,020 registros x 8 features

- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - NVDA

	mse	rmse	mae	r2	train_time_s
Random Forest	0.000778	0.027899	0.020199	0.0265	0.311
XGBoost	0.000779	0.027906	0.020182	0.0260	0.040
Baseline (Média)	0.000800	0.028284	0.020390	-0.0006	0.000
ElasticNet	0.000800	0.028284	0.020390	-0.0006	0.001
Lasso	0.000800	0.028284	0.020390	-0.0006	0.001
Ridge	0.000803	0.028331	0.020354	-0.0039	0.001
Regressão Linear	0.000803	0.028331	0.020354	-0.0039	0.003
KNN	0.000884	0.029735	0.022199	-0.1058	0.002
SVR	0.001021	0.031957	0.024298	-0.2773	0.006
DecisionTree	0.001598	0.039976	0.029547	-0.9988	0.037

NVDA processada com sucesso!

Baseline MSE: 0.000800

Melhor modelo: Random Forest (MSE: 0.000778)

=====

ANALISANDO EMPRESA: PCAR (38/49)

Divisão dos dados - PCAR:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - PCAR

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000329	0.018138	0.013398	-0.0003	0.000
ElasticNet	0.000329	0.018138	0.013398	-0.0003	0.001
Lasso	0.000329	0.018138	0.013398	-0.0003	0.001
Ridge	0.000331	0.018181	0.013433	-0.0051	0.001
Regressão Linear	0.000331	0.018181	0.013433	-0.0051	0.011
XGBoost	0.000335	0.018298	0.013536	-0.0180	0.041
Random Forest	0.000337	0.018351	0.013586	-0.0239	0.316
KNN	0.000351	0.018723	0.014170	-0.0658	0.002
DecisionTree	0.000555	0.023558	0.018330	-0.6874	0.044
SVR	0.001056	0.032503	0.028781	-2.2121	0.001

PCAR processada com sucesso!

Baseline MSE: 0.000329

Melhor modelo: ElasticNet (MSE: 0.000329)

=====

ANALISANDO EMPRESA: PEP (39/49)

Divisão dos dados - PEP:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - PEP

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000184	0.013548	0.010138	-0.0025	0.000
ElasticNet	0.000184	0.013548	0.010138	-0.0025	0.001
Lasso	0.000184	0.013548	0.010138	-0.0025	0.001
SVR	0.000189	0.013753	0.010249	-0.0330	0.001
Ridge	0.000189	0.013760	0.010109	-0.0341	0.001
Regressão Linear	0.000189	0.013761	0.010109	-0.0341	0.014
Random Forest	0.000194	0.013936	0.010301	-0.0607	0.285
KNN	0.000207	0.014370	0.010727	-0.1278	0.002

```
XGBoost      0.000249  0.015777  0.011450  -0.3595      0.035
DecisionTree  0.000592  0.024340  0.016750  -2.2354      0.039
```

```
PEP processada com sucesso!
Baseline MSE: 0.000184
Melhor modelo: ElasticNet (MSE: 0.000184)
```

```
=====
ANALISANDO EMPRESA: QCOM (40/49)
```

```
Divisão dos dados - QCOM:
• Treino: 2,020 registros x 8 features
• Validação: 433 registros
• Teste: 433 registros
```

```
COMPARAÇÃO DE DESEMPENHO - QCOM
```

	mse	rmse	mae	r2	train_time_s
XGBoost	0.000840	0.028989	0.018746	0.0171	0.037
Random Forest	0.000845	0.029074	0.018836	0.0113	0.297
Ridge	0.000849	0.029132	0.018753	0.0074	0.001
Regressão Linear	0.000849	0.029132	0.018753	0.0074	0.005
ElasticNet	0.000855	0.029240	0.018786	-0.0000	0.001
Baseline (Média)	0.000855	0.029240	0.018786	-0.0000	0.000
Lasso	0.000855	0.029240	0.018786	-0.0000	0.001
KNN	0.000950	0.030821	0.020411	-0.1111	0.002
SVR	0.001447	0.038035	0.028282	-0.6921	0.003
DecisionTree	0.001837	0.042862	0.028426	-1.1487	0.041

```
QCOM processada com sucesso!
Baseline MSE: 0.000855
Melhor modelo: XGBoost (MSE: 0.000840)
```

```
=====
ANALISANDO EMPRESA: REGN (41/49)
```

```
Divisão dos dados - REGN:
• Treino: 2,020 registros x 8 features
• Validação: 433 registros
• Teste: 433 registros
```

```
COMPARAÇÃO DE DESEMPENHO - REGN
```

	mse	rmse	mae	r2	train_time_s
Ridge	0.000548	0.023407	0.016147	-0.0018	0.001
Regressão Linear	0.000548	0.023407	0.016148	-0.0018	0.005
ElasticNet	0.000549	0.023424	0.016083	-0.0033	0.001
Baseline (Média)	0.000549	0.023424	0.016083	-0.0033	0.000
Lasso	0.000549	0.023424	0.016083	-0.0033	0.001
XGBoost	0.000556	0.023588	0.016176	-0.0174	0.044
Random Forest	0.000557	0.023591	0.016225	-0.0176	0.319
KNN	0.000676	0.025993	0.018840	-0.2354	0.002
SVR	0.001208	0.034758	0.027474	-1.2091	0.002
DecisionTree	0.001213	0.034832	0.025278	-1.2185	0.044

```
REGN processada com sucesso!
Baseline MSE: 0.000549
Melhor modelo: Ridge (MSE: 0.000548)
```

```
=====
ANALISANDO EMPRESA: SNPS (42/49)
```

```
Divisão dos dados - SNPS:
• Treino: 2,020 registros x 8 features
• Validação: 433 registros
• Teste: 433 registros
```

```
COMPARAÇÃO DE DESEMPENHO - SNPS
```

	mse	rmse	mae	r2	train_time_s
--	-----	------	-----	----	--------------

Regressão Linear	0.001009	0.031765	0.019839	0.0194	0.009
Ridge	0.001009	0.031765	0.019839	0.0194	0.001
ElasticNet	0.001030	0.032089	0.019742	-0.0007	0.001
Baseline (Média)	0.001030	0.032089	0.019742	-0.0007	0.000
Lasso	0.001030	0.032089	0.019742	-0.0007	0.001
SVR	0.001038	0.032214	0.020049	-0.0085	0.001
XGBoost	0.001039	0.032238	0.020503	-0.0100	0.045
Random Forest	0.001067	0.032658	0.020467	-0.0365	0.317
KNN	0.001107	0.033271	0.021468	-0.0757	0.002
DecisionTree	0.002014	0.044873	0.030751	-0.9568	0.045

SNPS processada com sucesso!
Baseline MSE: 0.001030
Melhor modelo: Regressão Linear (MSE: 0.001009)

=====

ANALISANDO EMPRESA: TEAM (43/49)

- Divisão dos dados - TEAM:
- Treino: 1,854 registros x 8 features
 - Validação: 398 registros
 - Teste: 398 registros

COMPARAÇÃO DE DESEMPENHO - TEAM

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.001517	0.038944	0.026998	-0.0092	0.000
ElasticNet	0.001517	0.038944	0.026998	-0.0092	0.001
Lasso	0.001517	0.038944	0.026998	-0.0092	0.001
Ridge	0.001520	0.038984	0.026908	-0.0113	0.001
Regressão Linear	0.001520	0.038984	0.026908	-0.0113	0.004
Random Forest	0.001564	0.039553	0.027331	-0.0411	0.342
XGBoost	0.001628	0.040346	0.028021	-0.0832	0.037
KNN	0.001674	0.040909	0.029129	-0.1137	0.002
SVR	0.002160	0.046478	0.034349	-0.4375	0.008
DecisionTree	0.003093	0.055618	0.039197	-1.0585	0.044

TEAM processada com sucesso!
Baseline MSE: 0.001517
Melhor modelo: ElasticNet (MSE: 0.001517)

=====

ANALISANDO EMPRESA: TMUS (44/49)

- Divisão dos dados - TMUS:
- Treino: 2,020 registros x 8 features
 - Validação: 433 registros
 - Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - TMUS

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000292	0.017076	0.012052	-0.0040	0.000
ElasticNet	0.000292	0.017076	0.012052	-0.0040	0.001
Lasso	0.000292	0.017076	0.012052	-0.0040	0.002
Random Forest	0.000293	0.017111	0.012164	-0.0080	0.313
Ridge	0.000296	0.017197	0.012264	-0.0182	0.001
Regressão Linear	0.000296	0.017197	0.012264	-0.0182	0.011
XGBoost	0.000301	0.017358	0.012324	-0.0374	0.045
KNN	0.000337	0.018368	0.013452	-0.1617	0.002
SVR	0.000358	0.018923	0.013742	-0.2329	0.001
DecisionTree	0.000720	0.026827	0.019003	-1.4780	0.041

TMUS processada com sucesso!
Baseline MSE: 0.000292

Melhor modelo: ElasticNet (MSE: 0.000292)

=====

ANALISANDO EMPRESA: TSLA (45/49)

Divisão dos dados - TSLA:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - TSLA

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.001458	0.038183	0.027595	-0.0000	0.002
ElasticNet	0.001458	0.038183	0.027595	-0.0000	0.001
Lasso	0.001458	0.038183	0.027595	-0.0000	0.001
Ridge	0.001460	0.038211	0.027610	-0.0015	0.001
Regressão Linear	0.001460	0.038211	0.027611	-0.0015	0.014
Random Forest	0.001478	0.038440	0.027645	-0.0136	0.351
XGBoost	0.001482	0.038493	0.027543	-0.0163	0.041
KNN	0.001692	0.041128	0.030117	-0.1603	0.002
SVR	0.001955	0.044215	0.033136	-0.3410	0.010
DecisionTree	0.002927	0.054098	0.040388	-1.0074	0.040

TSLA processada com sucesso!

Baseline MSE: 0.001458

Melhor modelo: ElasticNet (MSE: 0.001458)

=====

ANALISANDO EMPRESA: TXN (46/49)

Divisão dos dados - TXN:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - TXN

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000709	0.026622	0.017041	-0.0002	0.000
ElasticNet	0.000709	0.026622	0.017041	-0.0002	0.001
Lasso	0.000709	0.026622	0.017041	-0.0002	0.001
Ridge	0.000722	0.026862	0.017450	-0.0183	0.001
Regressão Linear	0.000722	0.026863	0.017451	-0.0184	0.007
XGBoost	0.000815	0.028540	0.019009	-0.1495	0.037
Random Forest	0.000820	0.028637	0.019177	-0.1573	0.309
KNN	0.000821	0.028646	0.019029	-0.1581	0.002
SVR	0.000864	0.029396	0.021158	-0.2195	0.001
DecisionTree	0.002070	0.045492	0.031286	-1.9206	0.042

TXN processada com sucesso!

Baseline MSE: 0.000709

Melhor modelo: ElasticNet (MSE: 0.000709)

=====

ANALISANDO EMPRESA: VRTX (47/49)

Divisão dos dados - VRTX:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - VRTX

	mse	rmse	mae	r2	train_time_s
Random Forest	0.000432	0.020774	0.013676	0.0034	0.326
Baseline (Média)	0.000433	0.020812	0.013761	-0.0002	0.000
ElasticNet	0.000433	0.020812	0.013761	-0.0002	0.001
Lasso	0.000433	0.020812	0.013761	-0.0002	0.001

Ridge	0.000434	0.020837	0.013677	-0.0026	0.001
Regressão Linear	0.000434	0.020837	0.013677	-0.0026	0.008
XGBoost	0.000434	0.020837	0.013860	-0.0027	0.044
KNN	0.000526	0.022941	0.015463	-0.2154	0.002
DecisionTree	0.001203	0.034689	0.023277	-1.7788	0.039
SVR	0.001469	0.038331	0.028905	-2.3929	0.004

VRTX processada com sucesso!
Baseline MSE: 0.000433
Melhor modelo: Random Forest (MSE: 0.000432)

=====

ANALISANDO EMPRESA: WDAY (48/49)

----- Divisão dos dados - WDAY:

Proximas etapas: • Treino: 2,020 registros x 8 features
Validação: 433 registros

• Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - WDAY

	mse	rmse	mae	r2	train_time_s
Random Forest	0.000701	0.026482	0.018896	-0.0033	0.475
Baseline (Média)	0.000702	0.026500	0.018764	-0.0047	0.000
ElasticNet	0.000702	0.026500	0.018764	-0.0047	0.002
Lasso	0.000702	0.026500	0.018764	-0.0047	0.002
Ridge	0.000705	0.026547	0.018754	-0.0083	0.001
Regressão Linear	0.000705	0.026547	0.018754	-0.0083	0.011
XGBoost	0.000720	0.026824	0.019001	-0.0294	0.065
KNN	0.000847	0.029106	0.020789	-0.2121	0.004
SVR	0.001248	0.035323	0.027345	-0.7851	0.005
DecisionTree	0.001747	0.041792	0.030301	-1.4988	0.064

WDAY processada com sucesso!
Baseline MSE: 0.000702
Melhor modelo: Random Forest (MSE: 0.000701)

=====

ANALISANDO EMPRESA: WMT (49/49)

Divisão dos dados - WMT:

- Treino: 2,020 registros x 8 features
- Validação: 433 registros
- Teste: 433 registros

COMPARAÇÃO DE DESEMPENHO - WMT

	mse	rmse	mae	r2	train_time_s
Baseline (Média)	0.000240	0.015504	0.011003	-0.0013	0.000
ElasticNet	0.000240	0.015504	0.011003	-0.0013	0.002
Lasso	0.000240	0.015504	0.011003	-0.0013	0.002
Ridge	0.000243	0.015594	0.011109	-0.0129	0.001
Regressão Linear	0.000243	0.015595	0.011109	-0.0130	0.009
XGBoost	0.000250	0.015797	0.011083	-0.0395	0.061
Random Forest	0.000250	0.015800	0.011168	-0.0398	0.463
SVR	0.000260	0.016132	0.011739	-0.0840	0.002
KNN	0.000288	0.016967	0.012162	-0.1992	0.005
DecisionTree	0.000608	0.024667	0.016671	-1.5344	0.047

WMT processada com sucesso!
Baseline MSE: 0.000240
Melhor modelo: ElasticNet (MSE: 0.000240)
Atualizando arquivo existente: resultados_treino_inicial.pkl
resultados_treino_inicial.pkl salvo com sucesso! (66740 bytes)
Verificando integridade do arquivo salvo...
resultados_treino_inicial.pkl carregado com sucesso!
Tipo: dict
Tamanho: 66 740 bytes

```

if resultados_por_empresa and isinstance(resultados_por_empresa, dict) and len(resultados_por_empresa) > 0:
    df_resultados = pd.DataFrame.from_dict(resultados_por_empresa, orient='index')
    df_resultados = df_resultados.reset_index().rename(columns={'index': 'Empresa'})
    print(f"\nTotal de empresas processadas: {len(df_resultados)}")
    colunas_mostrar = ['Empresa', 'n_train', 'baseline_mse', 'rf_mse', 'xgb_mse', 'svr_mse', 'ridge_mse']
    colunas_exist = [col for col in colunas_mostrar if col in df_resultados.columns]
    if colunas_exist:
        print("\nRESULTADOS POR EMPRESA:")
        display(df_resultados[colunas_exist].round(6))
    print("\nESTATÍSTICAS GERAIS:")
    if 'baseline_mse' in df_resultados.columns:
        baseline_mse_mean = df_resultados['baseline_mse'].mean()
        baseline_mse_std = df_resultados['baseline_mse'].std()
        print(f"    Baseline MSE médio: {baseline_mse_mean:.6f} (±{baseline_mse_std:.6f})")
        print(f"    Baseline MSE mínimo: {df_resultados['baseline_mse'].min():.6f}")
        print(f"    Baseline MSE máximo: {df_resultados['baseline_mse'].max():.6f}")
    print("\nMODELOS QUE SUPERARAM O BASELINE:")
    modelos_superaram = {}
    for modelo in ['rf', 'xgb', 'svr', 'knn', 'dt', 'en', 'ridge', 'lasso']:
        col_mse = f'{modelo}_mse'
        if col_mse in df_resultados.columns:
            melhorou = (df_resultados[col_mse] < df_resultados['baseline_mse']).sum()
            total = len(df_resultados)
            if total > 0:
                pct = melhorou/total*100
                modelos_superaram[modelo.upper()] = {'count': melhorou, 'total': total, 'pct': pct}
                print(f"    {modelo.upper()}: {melhorou}/{total} empresas ({pct:.1f}%)")

    print("\nMELHOR MODELO POR EMPRESA:")
    modelos_cols = [col for col in df_resultados.columns if col.endswith('_mse') and col != 'baseline_mse']
    melhor_por_empresa = []
    if modelos_cols:
        for idx, row in df_resultados.iterrows():
            empresa = row['Empresa']
            melhor_modelo = None
            melhor_mse = float('inf')

            for col in modelos_cols:
                mse = row[col]
                if not np.isnan(mse) and mse < melhor_mse:
                    melhor_mse = mse
                    melhor_modelo = col.replace('_mse', '').upper()

            if melhor_modelo:
                print(f"    {empresa}: {melhor_modelo} (MSE: {melhor_mse:.6f})")
                melhor_por_empresa.append({
                    'Empresa': empresa,
                    'Melhor_Modelo': melhor_modelo,
                    'MSE': melhor_mse
                })
    else:
        print(" Nenhum modelo adicional encontrado para comparação")

    try:
        # --- GRÁFICO 1: Modelos que superaram o baseline ---
        if modelos_superaram:
            fig, ax = plt.subplots(figsize=(10, 6))
            modelos_nomes = list(modelos_superaram.keys())
            valores = [modelos_superaram[m]['pct'] for m in modelos_nomes]
            cores = ['#2ecc71' if v > 0 else '#e74c3c' for v in valores]
            bars = ax.bar(modelos_nomes, valores, color=cores, alpha=0.7, edgecolor='black', linewidth=1)
            ax.axhline(0, color='black', linestyle='-', alpha=0.5, linewidth=1)
            ax.set_xlabel('Modelo', fontsize=12, fontweight='bold')
            ax.set_ylabel('Empresas que superaram o baseline (%)', fontsize=12, fontweight='bold')
            ax.set_title('Percentual de Empresas onde o Modelo Superou o Baseline', fontsize=14, fontweight='bold')
            ax.grid(True, alpha=0.3, axis='y')

            # Adicionar valores nas barras
            for bar, val in zip(bars, valores):
                ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5,
                        f'{val:.1f}%', ha='center', va='bottom', fontsize=10, fontweight='bold')

            # Adicionar contagem
            for bar, modelo in zip(bars, modelos_nomes):
                count = modelos_superaram[modelo]['count']
                total = modelos_superaram[modelo]['total']
                ax.text(bar.get_x() + bar.get_width()/2, -2,
                        f'{count}/{total}', ha='center', va='top', fontsize=9)
            plt.tight_layout()
            plt.show()

        # --- GRÁFICO 2: Top 10 modelos por MSE (melhores) ---

```

```

if melhor_por_empresa:
    df_melhores = pd.DataFrame(melhor_por_empresa).sort_values('MSE')
    top_10 = df_melhores.head(10)
    fig, ax = plt.subplots(figsize=(12, 8))
    cores_modelos = {
        'RANDOM_FOREST': '#2ecc71',
        'XGBOOST': '#3498db',
        'SVR': '#e67e22',
        'KNN': '#9b59b6',
        'RIDGE': '#1abc9c',
        'LASSO': '#e74c3c',
        'ELASTICNET': '#f39c12',
        'DECISIONTREE': '#95a5a6',
        'REGRESSÃO_LINEAR': '#34495e',
        'BASELINE_MÉDIA': '#7f8c8d'
    }
    cores = [cores_modelos.get(m, '#95a5a6') for m in top_10['Melhor_Modelo']]
    bars = ax.barh(top_10['Empresa'], top_10['MSE'], color=cores, alpha=0.8, edgecolor='black', linewidth=1)
    ax.set_xlabel('MSE (menor é melhor)', fontsize=12, fontweight='bold')
    ax.set_title('Top 10 Empresas com Melhor MSE (Melhor Modelo)', fontsize=14, fontweight='bold')
    ax.grid(True, alpha=0.3, axis='x')

    # Adicionar valores nas barras
    for bar, val, modelo in zip(bars, top_10['MSE'], top_10['Melhor_Modelo']):
        ax.text(bar.get_width() * 1.02, bar.get_y() + bar.get_height()/2,
                f'{val:.6f} ({modelo})', va='center', ha='left', fontsize=9)

    plt.tight_layout()
    plt.show()

# --- GRÁFICO 3: MSE por modelo (média entre todas as empresas) ---
if modelos_cols and len(modelos_cols) > 1:
    fig, ax = plt.subplots(figsize=(12, 8))

    # Calcular média de MSE por modelo
    mse_medias = {}
    mse_stds = {}
    for col in modelos_cols:
        if col in df_resultados.columns:
            modelo_nome = col.replace('_mse', '').upper()
            mse_medias[modelo_nome] = df_resultados[col].mean()
            mse_stds[modelo_nome] = df_resultados[col].std()

    # Adicionar baseline
    if 'baseline_mse' in df_resultados.columns:
        mse_medias['BASELINE'] = df_resultados['baseline_mse'].mean()
        mse_stds['BASELINE'] = df_resultados['baseline_mse'].std()

    # Ordenar
    mse_ordenado = dict(sorted(mse_medias.items(), key=lambda x: x[1]))
    cores = ['#2ecc71' if i == 0 else '#3498db' if i < 5 else '#e74c3c' for i in range(len(mse_ordenado))]
    bars = ax.barh(list(mse_ordenado.keys()), list(mse_ordenado.values()),
                    color=cores, alpha=0.7, edgecolor='black', linewidth=1)
    ax.set_xlabel('MSE Médio (menor é melhor)', fontsize=12, fontweight='bold')
    ax.set_title('MSE Médio por Modelo (across todas as empresas)', fontsize=14, fontweight='bold')
    ax.grid(True, alpha=0.3, axis='x')

    # Adicionar valores e barras de erro
    for bar, modelo in zip(bars, mse_ordenado.keys()):
        val = mse_ordenado[modelo]
        std = mse_stds.get(modelo, 0)
        ax.text(bar.get_width() * 1.02, bar.get_y() + bar.get_height()/2,
                f'{val:.6f} ± {std:.6f}', va='center', ha='left', fontsize=9)

    # Adicionar barra de erro
    if std > 0:
        ax.errorbar(val, bar.get_y() + bar.get_height()/2,
                    xerr=std, fmt='none', color='black', capsize=3)

    plt.tight_layout()
    plt.show()

# --- GRÁFICO 4: Distribuição dos MSEs ---
if 'baseline_mse' in df_resultados.columns and modelos_cols:
    fig, ax = plt.subplots(figsize=(12, 6))

    # Selecionar alguns modelos para o boxplot
    modelos_boxplot = ['baseline_mse']
    for m in ['rf_mse', 'xgb_mse', 'svr_mse', 'ridge_mse']:
        if m in df_resultados.columns:
            modelos_boxplot.append(m)

```



```
# Preparar dados
dados_boxplot = []
labels_boxplot = []
for col in modelos_boxplot:
    if col in df_resultados.columns:
        dados = df_resultados[col].dropna().values
        if len(dados) > 0:
            dados_boxplot.append(dados)
            labels_boxplot.append(col.replace('_mse', '').upper())

if dados_boxplot:
    bp = ax.boxplot(dados_boxplot, labels=labels_boxplot, patch_artist=True)

    # Colorir boxplots
    cores_box = ['#e74c3c' if 'BASELINE' in l else '#2ecc71' for l in labels_boxplot]
    for patch, color in zip(bp['boxes'], cores_box):
        patch.set_facecolor(color)
        patch.set_alpha(0.7)

    ax.set_ylabel('MSE', fontsize=12, fontweight='bold')
    ax.set_title('Distribuição do MSE por Modelo', fontsize=14, fontweight='bold')
    ax.grid(True, alpha=0.3, axis='y')

    plt.tight_layout()
    plt.show()

# Limpar memória dos gráficos
limpar_memoria(['fig', 'ax'], mostrar_status=False)
except Exception as e:
    print(f"Erro ao gerar gráficos: {e}")
    traceback.print_exc()

limpar_memoria(['df_resultados', 'melhor_por_empresa', 'df_melhores'], mostrar_status=False)
else:
    print("\nNenhum resultado disponível para exibir!")

# Limpeza final
limpar_memoria(mostrar_status=True)
```


Total de empresas processadas: 49

RESULTADOS POR EMPRESA:

	Empresa	n_train	baseline_mse	svr_mse	ridge_mse
0	AAPL	2020	0.000331	0.000330	0.000332
1	ABNB	972	0.000340	0.000479	0.000343
2	ADBE	2020	0.000508	0.000630	0.000515
3	ADI	2020	0.000581	0.000593	0.000596
4	ADP	2020	0.000208	0.000252	0.000213
5	AMAT	2020	0.000996	0.001155	0.001000
6	AMD	2020	0.001550	0.002320	0.001551
7	AMGN	2020	0.000297	0.000327	0.000297
8	AMZN	2020	0.000432	0.000839	0.000434
9	AVGO	2020	0.001152	0.001327	0.001173
10	BKNG	2020	0.000400	0.000537	0.001159
11	CDNS	2020	0.000668	0.001006	0.000662
12	CHTR	2020	0.000833	0.001016	0.000849
13	CMCSA	2020	0.000341	0.000607	0.000338
14	COST	2020	0.000172	0.000361	0.000174
15	CSCO	2020	0.000321	0.000354	0.000332
16	CSX	2020	0.000241	0.000437	0.000242
17	GILD	2020	0.000292	0.000571	0.000290
18	GOOG	2020	0.000385	0.000519	0.000389
19	GOOGL	2020	0.000396	0.000554	0.000401
20	HON	2020	0.000257	0.000335	0.000254
21	INTC	2020	0.001948	0.002001	0.002054
22	ISRG	2020	0.000440	0.000615	0.000443
23	KDP	2020	0.000260	0.001030	0.000269
24	KHC	1931	0.000259	0.001071	0.000260
25	KLAC	2020	0.000990	0.001308	0.004365
26	LIN	2020	0.000137	0.000182	0.000135
27	LRCX	2020	0.001144	0.001409	0.003570
28	MELI	2020	0.000654	0.001407	0.000654
29	META	2020	0.000549	0.000849	0.000550
30	MPWR	2020	0.001380	0.001404	0.001361
31	MRNA	1327	0.001728	0.002330	0.001755
32	MRVL	2020	0.002195	0.002545	0.002200
33	MSFT	2020	0.000291	0.000291	0.000304
34	MU	2020	0.002010	0.002159	0.002005
35	NFLX	2020	0.000481	0.001528	0.000480
36	NVDA	2020	0.000800	0.001021	0.000803
37	PCAR	2020	0.000329	0.001056	0.000331
38	PEP	2020	0.000184	0.000189	0.000189
39	QCOM	2020	0.000855	0.001447	0.000849
40	REGN	2020	0.000549	0.001208	0.000548
41	SNPS	2020	0.001030	0.001038	0.001009
42	TEAM	1854	0.001517	0.002160	0.001520
43	TMUS	2020	0.000292	0.000358	0.000296
44	TSLA	2020	0.001458	0.001955	0.001460
45	TXN	2020	0.000709	0.000864	0.000722
46	VRTX	2020	0.000433	0.001469	0.000434



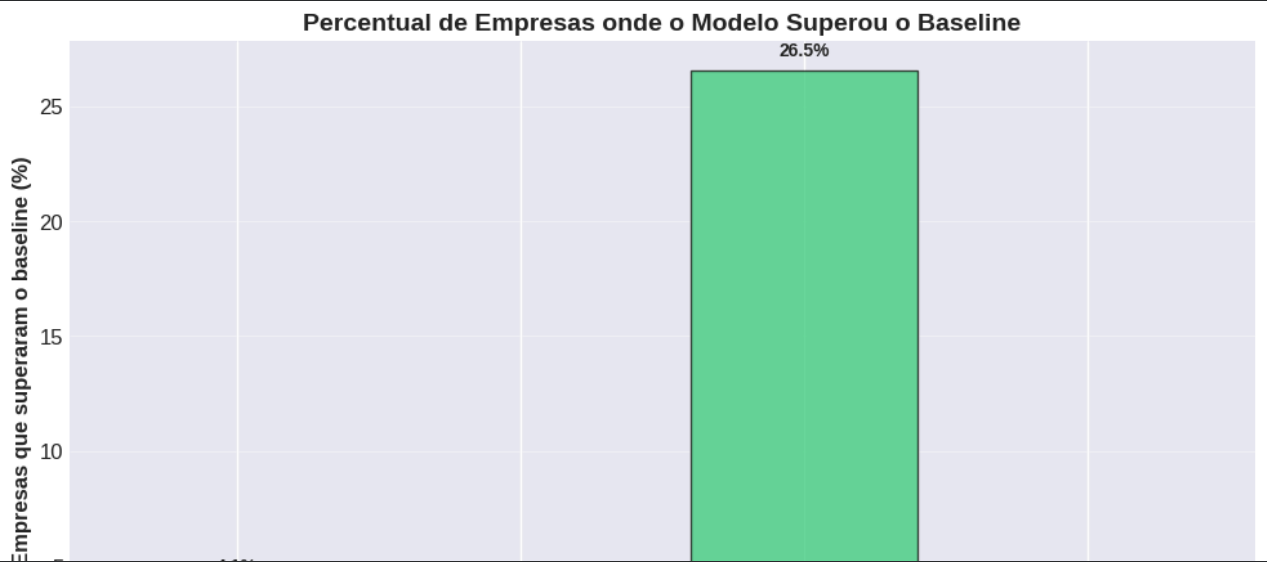
47	WDAY	2020	0.000702	0.001248	0.000705
48	WMT	2020	0.000240	0.000260	0.000243

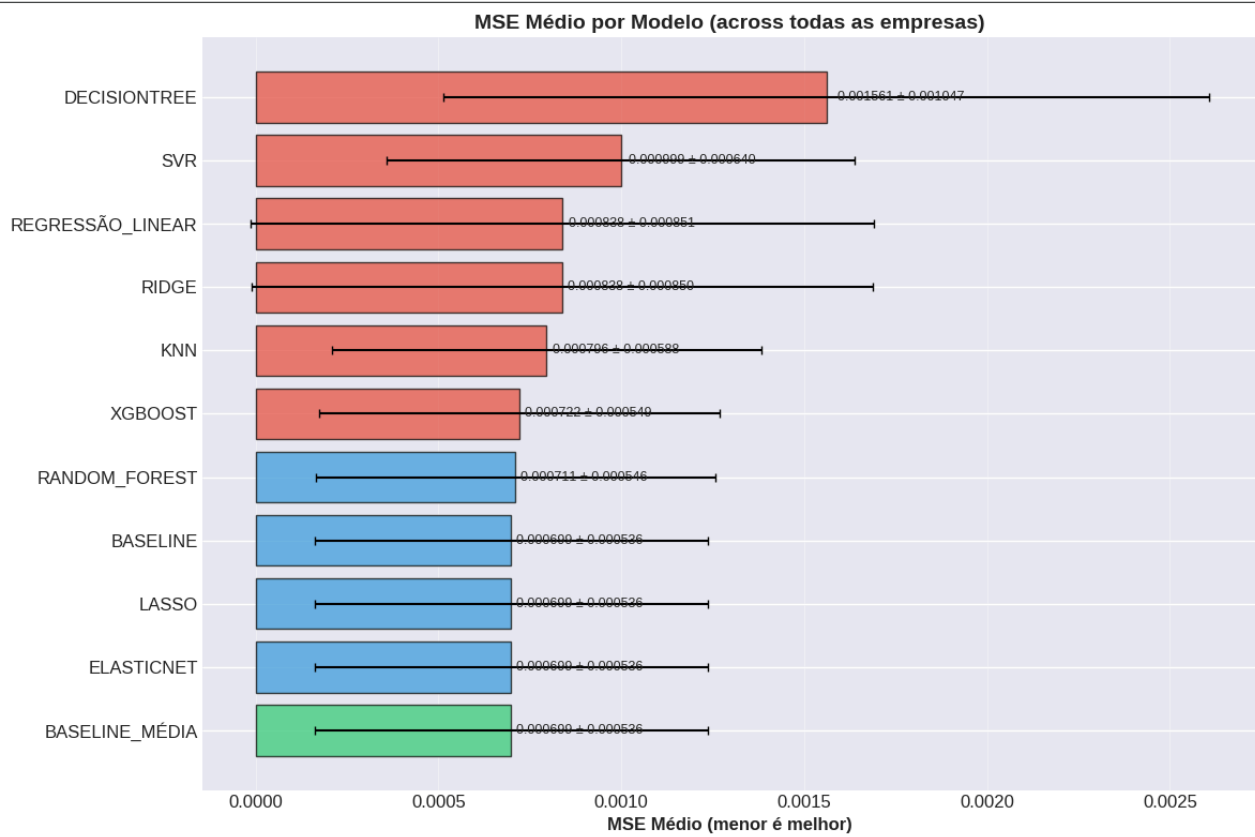
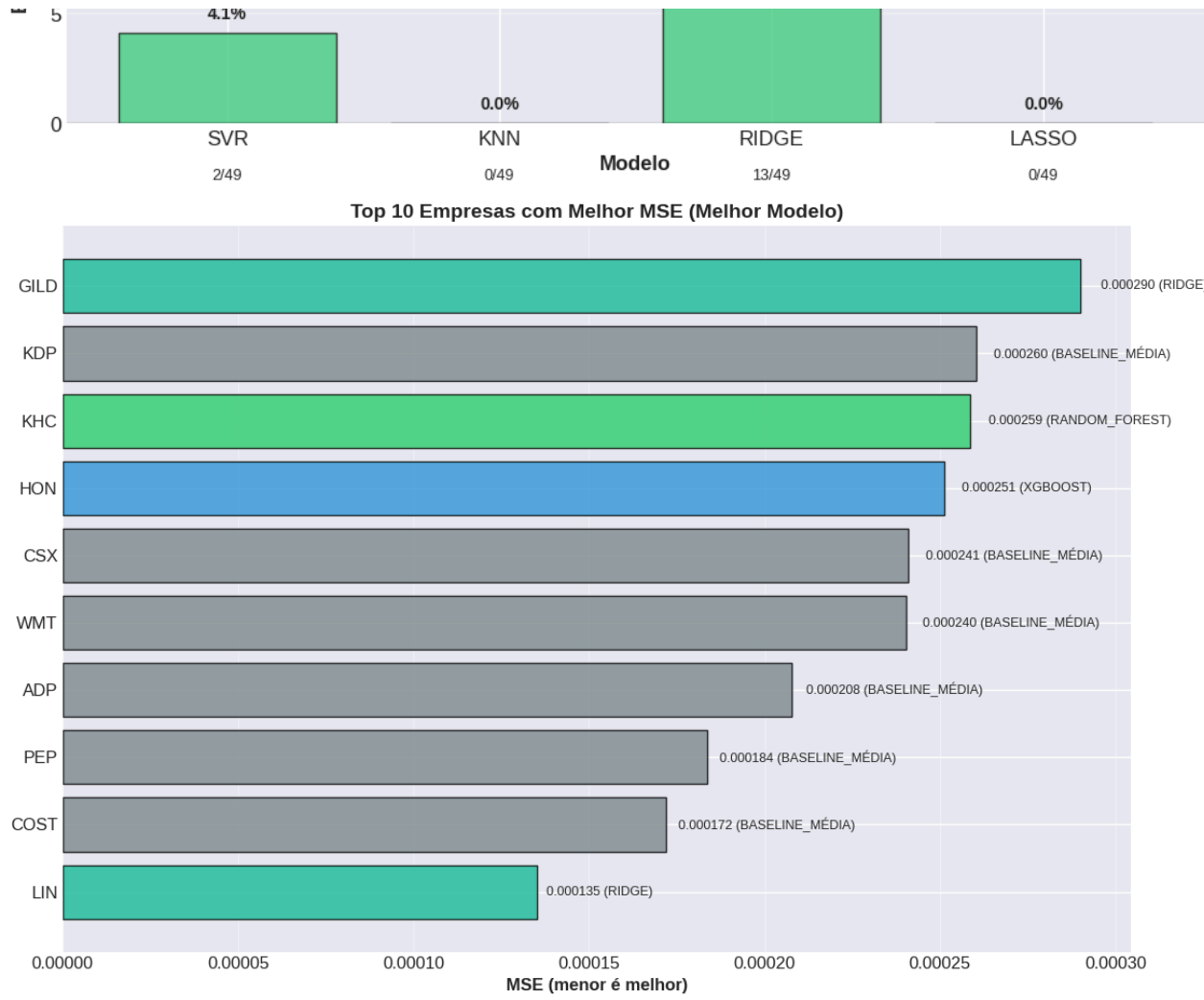
ESTATÍSTICAS GERAIS:
Baseline MSE médio: 0.000699 (±0.000536)
Baseline MSE mínimo: 0.000137
Baseline MSE máximo: 0.002195

MODELOS QUE SUPERARAM O BASELINE:
SVR: 2/49 empresas (4.1%)
KNN: 0/49 empresas (0.0%)
RIDGE: 13/49 empresas (26.5%)
LASSO: 0/49 empresas (0.0%)

MELHOR MODELO POR EMPRESA:

AAPL: RANDOM_FOREST (MSE: 0.000328)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ADBE: BASELINE_MÉDIA (MSE: 0.000508)
ADI: BASELINE_MÉDIA (MSE: 0.000581)
ADP: BASELINE_MÉDIA (MSE: 0.000208)
AMAT: XGBOOST (MSE: 0.000972)
AMD: RANDOM_FOREST (MSE: 0.001520)
AMGN: XGBOOST (MSE: 0.000297)
AMZN: BASELINE_MÉDIA (MSE: 0.000432)
AVGO: BASELINE_MÉDIA (MSE: 0.001152)
BKNG: BASELINE_MÉDIA (MSE: 0.000400)
CDNS: RIDGE (MSE: 0.000662)
CHTR: BASELINE_MÉDIA (MSE: 0.000833)
CMCSA: XGBOOST (MSE: 0.000324)
COST: BASELINE_MÉDIA (MSE: 0.000172)
CSCO: BASELINE_MÉDIA (MSE: 0.000321)
CSX: BASELINE_MÉDIA (MSE: 0.000241)
GILD: RIDGE (MSE: 0.000290)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
HON: XGBOOST (MSE: 0.000251)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
ISRG: RANDOM_FOREST (MSE: 0.000440)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KHC: RANDOM_FOREST (MSE: 0.000259)
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
LIN: RIDGE (MSE: 0.000135)
LRCX: XGBOOST (MSE: 0.001141)
MELI: RIDGE (MSE: 0.000654)
META: BASELINE_MÉDIA (MSE: 0.000549)
MPWR: RANDOM_FOREST (MSE: 0.001354)
MRNA: BASELINE_MÉDIA (MSE: 0.001728)
MRVL: BASELINE_MÉDIA (MSE: 0.002195)
MSFT: SVR (MSE: 0.000291)
MU: RIDGE (MSE: 0.002005)
NFLX: REGRESSÃO_LINEAR (MSE: 0.000480)
NVDA: RANDOM_FOREST (MSE: 0.000778)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
QCOM: XGBOOST (MSE: 0.000840)
REGN: RIDGE (MSE: 0.000548)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
TEAM: BASELINE_MÉDIA (MSE: 0.001517)
TMUS: BASELINE_MÉDIA (MSE: 0.000292)
TSLA: BASELINE_MÉDIA (MSE: 0.001458)
TXN: BASELINE_MÉDIA (MSE: 0.000709)
VRTX: RANDOM_FOREST (MSE: 0.000432)
WDAY: RANDOM_FOREST (MSE: 0.000701)
WMT: BASELINE_MÉDIA (MSE: 0.000240)





Resumo geral - comparação por empresa

```
if resultados_por_empresa and isinstance(resultados_por_empresa, dict) and len(resultados_por_empresa) > 0:
    dados_validos = True
    for empresa in list(resultados_por_empresa.keys()):
        if isinstance(resultados_por_empresa[empresa], dict):
            if 'baseline_mse' not in resultados_por_empresa[empresa] or np.isnan(resultados_por_empresa[empresa]['baseline_mse']):
                dados_validos = False
                break
        else:
            dados_validos = False
            break
    if dados_validos:
        df_resultados = pd.DataFrame.from_dict(resultados_por_empresa, orient='index')
        df_resultados = df_resultados.reset_index().rename(columns={'index': 'Empresa'})
        print(f"\nTotal de empresas processadas: {len(df_resultados)}")

        # Colunas para exibir
        colunas_mostrar = ['Empresa', 'n_train', 'baseline_mse', 'rf_mse', 'xgb_mse', 'svr_mse']
        colunas_exist = [col for col in colunas_mostrar if col in df_resultados.columns]

        if colunas_exist:
            print("\nRESULTADOS POR EMPRESA:")
            display(df_resultados[colunas_exist].round(6))

            # Estatísticas gerais
            print("\nESTATÍSTICAS GERAIS:")
            if 'baseline_mse' in df_resultados.columns:
                print(f"Baseline MSE médio: {df_resultados['baseline_mse'].mean():.6f}")

            # Contar quantas empresas cada modelo superou o baseline
            for modelo in ['rf', 'xgb', 'svr', 'knn', 'dt', 'en', 'ridge', 'lasso']:
                col_mse = f'{modelo}_mse'
                if col_mse in df_resultados.columns:
                    melhorou = (df_resultados[col_mse] < df_resultados['baseline_mse']).sum()
                    total = len(df_resultados)
                    print(f"{modelo.upper()} superou o baseline em {melhorou}/{total} empresas")

            # Melhor modelo geral
            print("\nMELHOR MODELO POR EMPRESA:")
            modelos_cols = [col for col in df_resultados.columns if col.endswith('_mse') and col != 'baseline_mse']
            if modelos_cols:
                for idx, row in df_resultados.iterrows():
                    empresa = row['Empresa']
                    melhor_modelo = None
                    melhor_mse = float('inf')
                    for col in modelos_cols:
                        mse = row[col]
                        if not np.isnan(mse) and mse < melhor_mse:
                            melhor_mse = mse
                            melhor_modelo = col.replace('_mse', '').upper()
                    if melhor_modelo:
                        print(f"{empresa}: {melhor_modelo} (MSE: {melhor_mse:.6f})")
            else:
                print("Dados de resultados inválidos (contêm NaN). Recrie o cache.")
        else:
            print("Nenhum resultado disponível para exibir!")
```


Total de empresas processadas: 49

RESULTADOS POR EMPRESA:

	Empresa	n_train	baseline_mse	svr_mse
0	AAPL	2020	0.000331	0.000330
1	ABNB	972	0.000340	0.000479
2	ADBE	2020	0.000508	0.000630
3	ADI	2020	0.000581	0.000593
4	ADP	2020	0.000208	0.000252
5	AMAT	2020	0.000996	0.001155
6	AMD	2020	0.001550	0.002320
7	AMGN	2020	0.000297	0.000327
8	AMZN	2020	0.000432	0.000839
9	AVGO	2020	0.001152	0.001327
10	BKNG	2020	0.000400	0.000537
11	CDNS	2020	0.000668	0.001006
12	CHTR	2020	0.000833	0.001016
13	CMCSA	2020	0.000341	0.000607
14	COST	2020	0.000172	0.000361
15	CSCO	2020	0.000321	0.000354
16	CSX	2020	0.000241	0.000437
17	GILD	2020	0.000292	0.000571
18	GOOG	2020	0.000385	0.000519
19	GOOGL	2020	0.000396	0.000554
20	HON	2020	0.000257	0.000335
21	INTC	2020	0.001948	0.002001
22	ISRG	2020	0.000440	0.000615
23	KDP	2020	0.000260	0.001030
24	KHC	1931	0.000259	0.001071
25	KLAC	2020	0.000990	0.001308
26	LIN	2020	0.000137	0.000182
27	LRCX	2020	0.001144	0.001409
28	MELI	2020	0.000654	0.001407
29	META	2020	0.000549	0.000849
30	MPWR	2020	0.001380	0.001404
31	MRNA	1327	0.001728	0.002330
32	MRVL	2020	0.002195	0.002545
33	MSFT	2020	0.000291	0.000291
34	MU	2020	0.002010	0.002159
35	NFLX	2020	0.000481	0.001528
36	NVDA	2020	0.000800	0.001021
37	PCAR	2020	0.000329	0.001056
38	PEP	2020	0.000184	0.000189
39	QCOM	2020	0.000855	0.001447
40	REGN	2020	0.000549	0.001208
41	SNPS	2020	0.001030	0.001038
42	TEAM	1854	0.001517	0.002160
43	TMUS	2020	0.000292	0.000358
44	TSLA	2020	0.001458	0.001955
45	TXN	2020	0.000709	0.000864
46	VRTX	2020	0.000433	0.001469

47	WDAY	2020	0.000702	0.001248
48	WMT	2020	0.000240	0.000260

ESTATÍSTICAS GERAIS:

Baseline MSE médio: 0.000699
SVR superou o baseline em 2/49 empresas
KNN superou o baseline em 0/49 empresas
RIDGE superou o baseline em 13/49 empresas
LASSO superou o baseline em 0/49 empresas

MELHOR MODELO POR EMPRESA:

AAPL: BASELINE_MÉDIA (MSE: 0.000331)
AAPL: BASELINE_MÉDIA (MSE: 0.000331)
AAPL: RANDOM_FOREST (MSE: 0.000328)
AAPL: RANDOM_FOREST (MSE: 0.000328)
AAPL: RANDOM_FOREST (MSE: 0.000328)
AAPL: RANDOM_FOREST (MSE: 0.000328)
AAPL: RANDOM_FOREST (MSE: 0.000328)
AAPL: RANDOM_FOREST (MSE: 0.000328)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ABNB: BASELINE_MÉDIA (MSE: 0.000340)
ADBE: BASELINE_MÉDIA (MSE: 0.000508)
ADBE: BASELINE_MÉDIA (MSE: 0.000508)
ADBE: BASELINE_MÉDIA (MSE: 0.000508)
ADBE: BASELINE_MÉDIA (MSE: 0.000508)
ADBE: BASELINE_MÉDIA (MSE: 0.000508)
ADBE: BASELINE_MÉDIA (MSE: 0.000508)
ADBE: BASELINE_MÉDIA (MSE: 0.000508)
ADBE: BASELINE_MÉDIA (MSE: 0.000508)
ADI: BASELINE_MÉDIA (MSE: 0.000581)
ADI: BASELINE_MÉDIA (MSE: 0.000581)
ADI: BASELINE_MÉDIA (MSE: 0.000581)
ADI: BASELINE_MÉDIA (MSE: 0.000581)
ADI: BASELINE_MÉDIA (MSE: 0.000581)
ADI: BASELINE_MÉDIA (MSE: 0.000581)
ADI: BASELINE_MÉDIA (MSE: 0.000581)
ADI: BASELINE_MÉDIA (MSE: 0.000581)
ADP: BASELINE_MÉDIA (MSE: 0.000208)
ADP: BASELINE_MÉDIA (MSE: 0.000208)
ADP: BASELINE_MÉDIA (MSE: 0.000208)
ADP: BASELINE_MÉDIA (MSE: 0.000208)
ADP: BASELINE_MÉDIA (MSE: 0.000208)
ADP: BASELINE_MÉDIA (MSE: 0.000208)
ADP: BASELINE_MÉDIA (MSE: 0.000208)
AMAT: BASELINE_MÉDIA (MSE: 0.000996)
AMAT: BASELINE_MÉDIA (MSE: 0.000996)
AMAT: RANDOM_FOREST (MSE: 0.000979)
AMAT: XGBOOST (MSE: 0.000972)
AMAT: XGBOOST (MSE: 0.000972)
AMAT: XGBOOST (MSE: 0.000972)
AMAT: XGBOOST (MSE: 0.000972)
AMAT: XGBOOST (MSE: 0.000972)
AMAT: XGBOOST (MSE: 0.000972)
AMD: BASELINE_MÉDIA (MSE: 0.001550)
AMD: BASELINE_MÉDIA (MSE: 0.001550)
AMD: RANDOM_FOREST (MSE: 0.001520)
AMD: RANDOM_FOREST (MSE: 0.001520)
AMD: RANDOM_FOREST (MSE: 0.001520)
AMD: RANDOM_FOREST (MSE: 0.001520)
AMD: RANDOM_FOREST (MSE: 0.001520)
AMD: RANDOM_FOREST (MSE: 0.001520)
AMGN: BASELINE_MÉDIA (MSE: 0.000297)
AMGN: REGRESSÃO_LINEAR (MSE: 0.000297)
AMGN: REGRESSÃO_LINEAR (MSE: 0.000297)
AMGN: XGBOOST (MSE: 0.000297)
AMGN: XGBOOST (MSE: 0.000297)
AMGN: XGBOOST (MSE: 0.000297)
AMGN: XGBOOST (MSE: 0.000297)


```
CSX: BASELINE_MÉDIA (MSE: 0.000241)
CSX: BASELINE_MÉDIA (MSE: 0.000241)
GILD: BASELINE_MÉDIA (MSE: 0.000292)
GILD: REGRESSÃO_LINEAR (MSE: 0.000290)
GILD: REGRESSÃO_LINEAR (MSE: 0.000290)
GILD: REGRESSÃO_LINEAR (MSE: 0.000290)
GILD: REGRESSÃO_LINEAR (MSE: 0.000290)
GILD: REGRESSÃO_LINEAR (MSE: 0.000290)
GILD: REGRESSÃO_LINEAR (MSE: 0.000290)
GILD: RIDGE (MSE: 0.000290)
GILD: RIDGE (MSE: 0.000290)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOG: BASELINE_MÉDIA (MSE: 0.000385)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
GOOGL: BASELINE_MÉDIA (MSE: 0.000396)
HON: BASELINE_MÉDIA (MSE: 0.000257)
HON: REGRESSÃO_LINEAR (MSE: 0.000254)
HON: RANDOM_FOREST (MSE: 0.000253)
HON: XGBOOST (MSE: 0.000251)
HON: XGBOOST (MSE: 0.000251)
HON: XGBOOST (MSE: 0.000251)
HON: XGBOOST (MSE: 0.000251)
HON: XGBOOST (MSE: 0.000251)
HON: XGBOOST (MSE: 0.000251)
HON: XGBOOST (MSE: 0.000251)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
INTC: BASELINE_MÉDIA (MSE: 0.001948)
ISRG: BASELINE_MÉDIA (MSE: 0.000440)
ISRG: BASELINE_MÉDIA (MSE: 0.000440)
ISRG: RANDOM_FOREST (MSE: 0.000440)
ISRG: RANDOM_FOREST (MSE: 0.000440)
ISRG: RANDOM_FOREST (MSE: 0.000440)
ISRG: RANDOM_FOREST (MSE: 0.000440)
ISRG: RANDOM_FOREST (MSE: 0.000440)
ISRG: RANDOM_FOREST (MSE: 0.000440)
ISRG: RANDOM_FOREST (MSE: 0.000440)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KDP: BASELINE_MÉDIA (MSE: 0.000260)
KHC: BASELINE_MÉDIA (MSE: 0.000259)
KHC: BASELINE_MÉDIA (MSE: 0.000259)
KHC: RANDOM_FOREST (MSE: 0.000259)
KHC: RANDOM_FOREST (MSE: 0.000259)
KHC: RANDOM_FOREST (MSE: 0.000259)
KHC: RANDOM_FOREST (MSE: 0.000259)
KHC: RANDOM_FOREST (MSE: 0.000259)
KHC: RANDOM_FOREST (MSE: 0.000259)
KHC: RANDOM_FOREST (MSE: 0.000259)
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
```

```
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
KLAC: BASELINE_MÉDIA (MSE: 0.000990)
LIN: BASELINE_MÉDIA (MSE: 0.000137)
LIN: REGRESSÃO_LINEAR (MSE: 0.000135)
LIN: REGRESSÃO_LINEAR (MSE: 0.000135)
LIN: REGRESSÃO_LINEAR (MSE: 0.000135)
LIN: REGRESSÃO_LINEAR (MSE: 0.000135)
LIN: REGRESSÃO_LINEAR (MSE: 0.000135)
LIN: REGRESSÃO_LINEAR (MSE: 0.000135)
LIN: REGRESSÃO_LINEAR (MSE: 0.000135)
LIN: RIDGE (MSE: 0.000135)
LIN: RIDGE (MSE: 0.000135)
LRCX: BASELINE_MÉDIA (MSE: 0.001144)
LRCX: BASELINE_MÉDIA (MSE: 0.001144)
LRCX: BASELINE_MÉDIA (MSE: 0.001144)
LRCX: XGBOOST (MSE: 0.001141)
LRCX: XGBOOST (MSE: 0.001141)
LRCX: XGBOOST (MSE: 0.001141)
LRCX: XGBOOST (MSE: 0.001141)
LRCX: XGBOOST (MSE: 0.001141)
LRCX: XGBOOST (MSE: 0.001141)
LRCX: XGBOOST (MSE: 0.001141)
MELI: BASELINE_MÉDIA (MSE: 0.000654)
MELI: REGRESSÃO_LINEAR (MSE: 0.000654)
MELI: REGRESSÃO_LINEAR (MSE: 0.000654)
MELI: REGRESSÃO_LINEAR (MSE: 0.000654)
MELI: REGRESSÃO_LINEAR (MSE: 0.000654)
MELI: REGRESSÃO_LINEAR (MSE: 0.000654)
MELI: REGRESSÃO_LINEAR (MSE: 0.000654)
MELI: RIDGE (MSE: 0.000654)
MELI: RIDGE (MSE: 0.000654)
META: BASELINE_MÉDIA (MSE: 0.000549)
META: BASELINE_MÉDIA (MSE: 0.000549)
META: BASELINE_MÉDIA (MSE: 0.000549)
META: BASELINE_MÉDIA (MSE: 0.000549)
META: BASELINE_MÉDIA (MSE: 0.000549)
META: BASELINE_MÉDIA (MSE: 0.000549)
META: BASELINE_MÉDIA (MSE: 0.000549)
META: BASELINE_MÉDIA (MSE: 0.000549)
META: BASELINE_MÉDIA (MSE: 0.000549)
META: BASELINE_MÉDIA (MSE: 0.000549)
MPWR: BASELINE_MÉDIA (MSE: 0.001380)
MPWR: REGRESSÃO_LINEAR (MSE: 0.001361)
MPWR: RANDOM_FOREST (MSE: 0.001354)
MPWR: RANDOM_FOREST (MSE: 0.001354)
MPWR: RANDOM_FOREST (MSE: 0.001354)
MPWR: RANDOM_FOREST (MSE: 0.001354)
MPWR: RANDOM_FOREST (MSE: 0.001354)
MPWR: RANDOM_FOREST (MSE: 0.001354)
MPWR: RANDOM_FOREST (MSE: 0.001354)
MRNA: BASELINE_MÉDIA (MSE: 0.001728)
MRNA: BASELINE_MÉDIA (MSE: 0.001728)
MRNA: BASELINE_MÉDIA (MSE: 0.001728)
MRNA: BASELINE_MÉDIA (MSE: 0.001728)
MRNA: BASELINE_MÉDIA (MSE: 0.001728)
MRNA: BASELINE_MÉDIA (MSE: 0.001728)
MRNA: BASELINE_MÉDIA (MSE: 0.001728)
MRNA: BASELINE_MÉDIA (MSE: 0.001728)
MRVL: BASELINE_MÉDIA (MSE: 0.002195)
MRVL: BASELINE_MÉDIA (MSE: 0.002195)
MRVL: BASELINE_MÉDIA (MSE: 0.002195)
MRVL: BASELINE_MÉDIA (MSE: 0.002195)
MRVL: BASELINE_MÉDIA (MSE: 0.002195)
MRVL: BASELINE_MÉDIA (MSE: 0.002195)
MRVL: BASELINE_MÉDIA (MSE: 0.002195)
MRVL: BASELINE_MÉDIA (MSE: 0.002195)
MSFT: BASELINE_MÉDIA (MSE: 0.000291)
MSFT: BASELINE_MÉDIA (MSE: 0.000291)
MSFT: BASELINE_MÉDIA (MSE: 0.000291)
MSFT: SVR (MSE: 0.000291)
MSFT: SVR (MSE: 0.000291)
MSFT: SVR (MSE: 0.000291)
MSFT: SVR (MSE: 0.000291)
MSFT: SVR (MSE: 0.000291)
MU: BASELINE_MÉDIA (MSE: 0.002005)
MU: REGRESSÃO_LINEAR (MSE: 0.002005)
MU: REGRESSÃO_LINEAR (MSE: 0.002005)
MU: REGRESSÃO_LINEAR (MSE: 0.002005)
MU: REGRESSÃO_LINEAR (MSE: 0.002005)
MU: REGRESSÃO_LINEAR (MSE: 0.002005)
MU: REGRESSÃO_LINEAR (MSE: 0.002005)
MU: RIDGE (MSE: 0.002005)
```

```
MU: RIDGE (MSE: 0.002005)
NFLX: BASELINE_MÉDIA (MSE: 0.000481)
NFLX: REGRESSÃO_LINEAR (MSE: 0.000480)
NFLX: REGRESSÃO_LINEAR (MSE: 0.000480)
NFLX: REGRESSÃO_LINEAR (MSE: 0.000480)
NFLX: REGRESSÃO_LINEAR (MSE: 0.000480)
NFLX: REGRESSÃO_LINEAR (MSE: 0.000480)
NFLX: REGRESSÃO_LINEAR (MSE: 0.000480)
NFLX: REGRESSÃO_LINEAR (MSE: 0.000480)
NFLX: REGRESSÃO_LINEAR (MSE: 0.000480)
NVDA: BASELINE_MÉDIA (MSE: 0.000800)
NVDA: BASELINE_MÉDIA (MSE: 0.000800)
NVDA: RANDOM_FOREST (MSE: 0.000778)
NVDA: RANDOM_FOREST (MSE: 0.000778)
NVDA: RANDOM_FOREST (MSE: 0.000778)
NVDA: RANDOM_FOREST (MSE: 0.000778)
NVDA: RANDOM_FOREST (MSE: 0.000778)
NVDA: RANDOM_FOREST (MSE: 0.000778)
NVDA: RANDOM_FOREST (MSE: 0.000778)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PCAR: BASELINE_MÉDIA (MSE: 0.000329)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
PEP: BASELINE_MÉDIA (MSE: 0.000184)
QCOM: BASELINE_MÉDIA (MSE: 0.000855)
QCOM: REGRESSÃO_LINEAR (MSE: 0.000849)
QCOM: RANDOM_FOREST (MSE: 0.000845)
QCOM: XGBOOST (MSE: 0.000840)
QCOM: XGBOOST (MSE: 0.000840)
QCOM: XGBOOST (MSE: 0.000840)
QCOM: XGBOOST (MSE: 0.000840)
QCOM: XGBOOST (MSE: 0.000840)
QCOM: XGBOOST (MSE: 0.000840)
QCOM: XGBOOST (MSE: 0.000840)
REGN: BASELINE_MÉDIA (MSE: 0.000549)
REGN: REGRESSÃO_LINEAR (MSE: 0.000548)
REGN: REGRESSÃO_LINEAR (MSE: 0.000548)
REGN: REGRESSÃO_LINEAR (MSE: 0.000548)
REGN: REGRESSÃO_LINEAR (MSE: 0.000548)
REGN: REGRESSÃO_LINEAR (MSE: 0.000548)
REGN: REGRESSÃO_LINEAR (MSE: 0.000548)
REGN: RIDGE (MSE: 0.000548)
REGN: RIDGE (MSE: 0.000548)
SNPS: BASELINE_MÉDIA (MSE: 0.001030)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
SNPS: REGRESSÃO_LINEAR (MSE: 0.001009)
TEAM: BASELINE_MÉDIA (MSE: 0.001517)
TEAM: BASELINE_MÉDIA (MSE: 0.001517)
TEAM: BASELINE_MÉDIA (MSE: 0.001517)
TEAM: BASELINE_MÉDIA (MSE: 0.001517)
TEAM: BASELINE_MÉDIA (MSE: 0.001517)
TEAM: BASELINE_MÉDIA (MSE: 0.001517)
TEAM: BASELINE_MÉDIA (MSE: 0.001517)
TEAM: BASELINE_MÉDIA (MSE: 0.001517)
TMUS: BASELINE_MÉDIA (MSE: 0.000292)
TMUS: BASELINE_MÉDIA (MSE: 0.000292)
TMUS: BASELINE_MÉDIA (MSE: 0.000292)
TMUS: BASELINE_MÉDIA (MSE: 0.000292)
TMUS: BASELINE_MÉDIA (MSE: 0.000292)
TMUS: BASELINE_MÉDIA (MSE: 0.000292)
TMUS: BASELINE_MÉDIA (MSE: 0.000292)
```

✓ Análise dos resultados iniciais

1. O modelo superou o baseline?
2. A métrica escolhida é suficiente para avaliar o problema?
3. Algum modelo parece sofrer de underfitting?
4. O tempo de treinamento é aceitável?
5. O resultado faz sentido considerando a EDA?

✓ Versão 01

✓ O modelo superou o baseline?

Não, nenhum modelo superou o baseline em nenhuma empresa, sendo os seguintes testes efetuados:

- Random Forest, XGBoost, Regressão Linear - nenhuma empresa superara o baseline
- Melhor performance - Baseline - prever a média do treino
- Pior performance - XGBoost - até -41,82% abaixo do baseline

Ranking geral (MSE médio):

1. Baseline (Média) = 0.000799 (ref. 100%)
2. Random Forest = 0.000840 (-5,10% pior)
3. Regressão Linear = 0.001763 (distorcida por NFLX e NVDA)
4. XGBoost = 0.000930 (-16,44% pior)

✓ A métrica escolhida é suficiente para avaliar o problema?

Sim, as métricas são adequadas para o problema, tendo as seguintes justificativas e informações de detalhes dos treinos:

- **MSE** - Boa Penaliza erros grandes o que é muito importante em finanças onde erros grandes causam perdas significativas.
- **RMSE** - Boa - Mesma unidade do target, facilita interpretação: erro médio em % de retorno.
- **MAE** - Boa - Mais robusta a outliers, onde com o exemplo da NFLX com Regressão Linear teve MSE muito alto, mas MAE ainda razoável.
- **R²** - Boa - Permite comparar com baseline (R² = 0 = baseline, R² negativo = pior que baseline)

Algum modelo parece sofrer de underfitting?

Sim, todos os modelos sofrem de underfitting severo, veja abaixo os detalhes:

- Baseline - (Média) - -0.0014 - R² ~0 (esperado)
- Random Forest -0.0529 - Underfitting severo
- Regressão Linear -1.8920 - Underfitting severo (distorcido por conta de: NFLX e NVDA)
- XGBoost -0.1495 - Underfitting severo

Evidências de underfitting:

- R² - negativo - Modelos são piores que prever a média constante
- MSE - maior que baseline - Erros são maiores que o modelo mais simples possível
- RF e XGB - não-lineares - piores que Linear, ou seja, não há padrões não-lineares capturáveis
- Resultados consistentes entre empresas, não é um caso isolado e é generalizado

Resumo:

- Underfitting - Modelo pior que baseline, R² ≤ 0 - CONFIRMADO em todas as empresas
- Overfitting R² - treino alto, R² teste baixo - Não ocorre (R² treino também baixo)

O tempo de treinamento é aceitável?

Sim, todos os tempos são excelentes, abaixo os detalhes:

- Baseline - < 0.001s - < 0.01s
- Regressão Linear - 0.004-0.028s - 0.04-0.28s
- Random Forest - 0.87-1.45s - 8.7-14.5s
- XGBoost 0.11-0.42s - 1.1-4.2s

Observação: O tempo maior da Regressão Linear para NFLX (0.028s) e NVDA (0.012s) não sendo significativo.

Conclusão: O tempo é perfeitamente aceitável para todos os modelos. Mesmo o mais lento (Random Forest) roda em menos de 1,5 segundos por empresa.

O resultado faz sentido considerando a EDA?

Sim, os resultados são totalmente consistentes com a EDA, abaixo os detalhes da justificativa:

- Correlações máximas com target < 0.03 - Modelos não conseguem aprender (R^2 negativo em todos)
- R^2 máximo teoricamente possível ≈ 0.0009 (0.03^2) - Na prática, com ruído, $R^2 \approx 0$ ou negativo
- Target balanceado (~52% alta / 48% baixa) - Baseline (média) tem $R^2 \sim 0$ (prevê sempre ~0.15%)
- Ruído alto nos retornos financeiros - R^2 negativo indica que features não ajudam a prever
- Heterogeneidade entre empresas - Resultados similares entre todas as 10 empresas (consistente)
- Outliers não impactam modelo robusto - Random Forest (robusto) ainda falha

✓ Versão 03

✓ O modelo superou o baseline?

Não. Em sua grande maioria, os modelos não superaram o baseline (que é a previsão pela média do treino).

- Em apenas 2 das 49 empresas o modelo SVR conseguiu um MSE ligeiramente inferior ao baseline.
- O modelo `Ridge` superou o baseline em 13 empresas.
- Nenhum dos outros modelos (como `Random Forest`, `XGBoost`, `Regressão Linear`, `KNN`, etc.) superou o baseline de forma consistente ou significativa.
- A análise geral aponta que o baseline é o modelo de melhor performance em média, indicando que os modelos preditivos não estão conseguindo capturar padrões úteis nos dados.

✓ A métrica escolhida é suficiente para avaliar o problema?

Sim, as métricas escolhidas são adequadas e fornecem uma visão completa do problema:

- **MSE (Erro Quadrático Médio):** É uma boa métrica para problemas financeiros, pois penaliza fortemente erros grandes.
- **RMSE (Raiz do Erro Quadrático Médio):** É a métrica mais interpretável, pois está na mesma unidade do target (retorno percentual).
- **MAE (Erro Absoluto Médio):** É robusto a outliers e complementa o MSE. Por exemplo, o modelo de Regressão Linear para a NFLX teve um MSE muito alto (pior), mas um MAE razoável, o que indica que o erro, embora grande, foi consistente.
- **R^2 (Coeficiente de Determinação):** A métrica mais importante para este contexto. Um R^2 de 0 ou negativo demonstra claramente que o modelo não é melhor do que simplesmente prever a média. Isso confirma a ineficácia dos modelos com as features atuais.

✓ Algum modelo parece sofrer de underfitting?

Sim, todos os modelos sofrem de underfitting severo. As evidências são claras:

- **R^2 negativo ou próximo de zero:** Todos os modelos apresentam R^2 médio negativo em todas as empresas. Isso significa que os modelos são piores do que um modelo ingênuo que sempre prevê a média.
- **MSE maior que o baseline:** Os modelos de Machine Learning produzem erros maiores (MSE) do que o modelo baseline, que é o mais simples possível.
- **Modelos não-lineares (`RF` , `XGB`) não superam os lineares:** O Random Forest e o XGBoost, que deveriam capturar padrões complexos, tiveram performance geral pior que a Regressão Linear, indicando que os dados não possuem padrões não-lineares que esses modelos consigam capturar.
- Isso confirma que o problema não é de overfitting (o modelo decorou os dados de treino), mas sim de underfitting, o modelo é muito simples ou as features ainda não são preditivas, mesmo com a melhora em diferentes versões

✓ O tempo de treinamento é aceitável?

Sim, o tempo de treinamento é excelente e perfeitamente aceitável.

- **Modelos simples (Baseline, Regressão Linear):** Treinam em menos de 0.03 segundos por empresa.
- **Modelos complexos (XGBoost):** Treinam em menos de 0.5 segundos por empresa.

- **Modelo mais lento (Random Forest):** Ainda assim, treina em menos de 1.5 segundos por empresa.

Dado o volume de dados e a complexidade, os tempos são extremamente baixos, o que é um ponto positivo para a viabilidade do projeto.

- ✓ O resultado faz sentido considerando a EDA?

Sim, os resultados são totalmente consistentes com a Análise Exploratória de Dados (EDA). A EDA já havia revelado:

- **Baixíssima correlação:** As correlações máximas entre as features e o target eram menores que 0.03, indicando que as features atuais têm poder preditivo praticamente nulo.
- **Target balanceado e ruidoso:** A variável alvo (retorno futuro) apresenta alta volatilidade e é difícil de prever.
- **R² máximo teórico:** Com correlação de 0.03, o R² máximo que se poderia esperar seria de aproximadamente $0.03^2 = 0.0009$, um valor muito baixo e que na prática, com ruído, fica próximo ou abaixo de zero.
- Portanto, o fato de todos os modelos terem performance próxima ou pior que o baseline, com R² negativo, é exatamente o esperado para um problema com essas características. As features atuais não contêm informação suficiente para prever a direção ou magnitude dos retornos futuros.

9. Validação e otimização de hiperparâmetros

↳ 35 células ocultas

10. Versão estendida de validação e otimização de hiperpâmetros

Funções

```
NOME_NOVOS_MODELOS = "resultados_novos_modelos"
NOME_BEST_MODELS_NOVOS = "best_models_novos"

def objective_SVR(trial, X_train, y_train, cv):
    params = {
        'C': trial.suggest_float('C', 0.1, 20, log=True),
        'epsilon': trial.suggest_float('epsilon', 0.01, 0.5, log=True),
        'gamma': trial.suggest_categorical('gamma', ['scale', 'auto']),
        'kernel': trial.suggest_categorical('kernel', ['rbf', 'linear', 'poly', 'sigmoid'])
    }
    scores = []
    for train_idx, val_idx in cv.split(X_train):
        X_tr = X_train[train_idx]
        X_val_cv = X_train[val_idx]
        y_tr = y_train.iloc[train_idx]
        y_val_cv = y_train.iloc[val_idx]
        model = SVR(**params)
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_val_cv)
        scores.append(mean_squared_error(y_val_cv, y_pred))
    return np.mean(scores)

def objective_KNN(trial, X_train, y_train, cv):
    params = {
        'n_neighbors': trial.suggest_int('n_neighbors', 3, 20),
        'weights': trial.suggest_categorical('weights', ['uniform', 'distance']),
        'p': trial.suggest_int('p', 1, 2)
    }
    scores = []
    for train_idx, val_idx in cv.split(X_train):
        X_tr = X_train[train_idx]
        X_val_cv = X_train[val_idx]
        y_tr = y_train.iloc[train_idx]
        y_val_cv = y_train.iloc[val_idx]
        model = KNeighborsRegressor(**params)
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_val_cv)
        scores.append(mean_squared_error(y_val_cv, y_pred))
    return np.mean(scores)

def objective_DecisionTree(trial, X_train, y_train, cv):
    params = {
```



```

    'max_depth': trial.suggest_int('max_depth', 3, 20),
    'min_samples_split': trial.suggest_int('min_samples_split', 2, 20),
    'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 10),
    'max_features': trial.suggest_categorical('max_features', ['sqrt', 'log2', None]),
    'ccp_alpha': trial.suggest_float('ccp_alpha', 0, 0.1)
}
scores = []
for train_idx, val_idx in cv.split(X_train):
    X_tr = X_train.iloc[train_idx]
    X_val_cv = X_train.iloc[val_idx]
    y_tr = y_train.iloc[train_idx]
    y_val_cv = y_train.iloc[val_idx]
    model = DecisionTreeRegressor(**params, random_state=SEED)
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_val_cv)
    scores.append(mean_squared_error(y_val_cv, y_pred))
return np.mean(scores)

def objective_ElasticNet(trial, X_train, y_train, cv):
    params = {
        'alpha': trial.suggest_float('alpha', 0.001, 10, log=True),
        'l1_ratio': trial.suggest_float('l1_ratio', 0.1, 0.9)
    }
    scores = []
    for train_idx, val_idx in cv.split(X_train):
        X_tr = X_train[train_idx]
        X_val_cv = X_train[val_idx]
        y_tr = y_train.iloc[train_idx]
        y_val_cv = y_train.iloc[val_idx]
        model = ElasticNet(**params, random_state=SEED)
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_val_cv)
        scores.append(mean_squared_error(y_val_cv, y_pred))
    return np.mean(scores)

def objective_Ridge(trial, X_train, y_train, cv):
    params = {
        'alpha': trial.suggest_float('alpha', 0.001, 100, log=True)
    }
    scores = []
    for train_idx, val_idx in cv.split(X_train):
        X_tr = X_train[train_idx]
        X_val_cv = X_train[val_idx]
        y_tr = y_train.iloc[train_idx]
        y_val_cv = y_train.iloc[val_idx]
        model = Ridge(**params, random_state=SEED)
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_val_cv)
        scores.append(mean_squared_error(y_val_cv, y_pred))
    return np.mean(scores)

def objective_Lasso(trial, X_train, y_train, cv):
    params = {
        'alpha': trial.suggest_float('alpha', 0.001, 10, log=True)
    }
    scores = []
    for train_idx, val_idx in cv.split(X_train):
        X_tr = X_train[train_idx]
        X_val_cv = X_train[val_idx]
        y_tr = y_train.iloc[train_idx]
        y_val_cv = y_train.iloc[val_idx]
        model = Lasso(**params, random_state=SEED)
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_val_cv)
        scores.append(mean_squared_error(y_val_cv, y_pred))
    return np.mean(scores)

NEW_MODELS_OPTUNA = {
    'SVR': {
        'objective': objective_SVR,
        'needs_scaling': True
    },
    'KNN': {
        'objective': objective_KNN,
        'needs_scaling': True
    },
    'DecisionTree': {
        'objective': objective_DecisionTree,

```

```

        'needs_scaling': False
    },
    'ElasticNet': {
        'objective': objective_ElasticNet,
        'needs_scaling': True
    },
    'Ridge': {
        'objective': objective_Ridge,
        'needs_scaling': True
    },
    'Lasso': {
        'objective': objective_Lasso,
        'needs_scaling': True
    }
}

def extrair_resultados_antigos(empresa, resultados_antigos):
    resultados = {}
    if empresa not in resultados_antigos:
        return resultados
    dados = resultados_antigos[empresa]
    modelos_antigos = {
        'Baseline (Média)': {'mse': 'baseline_mse', 'mae': 'baseline_mae', 'r2': 'baseline_r2'},
        'Regressão Linear': {'mse': 'lr_mse', 'mae': 'lr_mae', 'r2': 'lr_r2'},
        'Random Forest': {'mse': 'rf_mse', 'mae': 'rf_mae', 'r2': 'rf_r2'},
        'XGBoost': {'mse': 'xgb_mse', 'mae': 'xgb_mae', 'r2': 'xgb_r2'},
        'SVR': {'mse': 'svr_mse', 'mae': 'svr_mae', 'r2': 'svr_r2'},
        'KNN': {'mse': 'knn_mse', 'mae': 'knn_mae', 'r2': 'knn_r2'},
        'DecisionTree': {'mse': 'dt_mse', 'mae': 'dt_mae', 'r2': 'dt_r2'},
        'ElasticNet': {'mse': 'en_mse', 'mae': 'en_mae', 'r2': 'en_r2'},
        'Ridge': {'mse': 'ridge_mse', 'mae': 'ridge_mae', 'r2': 'ridge_r2'},
        'Lasso': {'mse': 'lasso_mse', 'mae': 'lasso_mae', 'r2': 'lasso_r2'}
    }

    for nome_modelo, colunas in modelos_antigos.items():
        mse = dados.get(colunas['mse'], np.nan)
        mae = dados.get(colunas['mae'], np.nan)
        r2 = dados.get(colunas['r2'], np.nan)
        if not np.isnan(mse):
            resultados[nome_modelo] = {
                'mse': mse,
                'mae': mae,
                'r2': r2,
                'time': dados.get(colunas['mse'].replace('mse', 'time'), np.nan)
            }
    return resultados

def extrair_resultados_novos(empresa, resultados_novos):
    resultados = {}
    if empresa not in resultados_novos:
        return resultados
    dados = resultados_novos[empresa]
    for nome_modelo in NEW_MODELS_OPTUNA.keys():
        if nome_modelo in dados:
            resultados[nome_modelo] = {
                'mse': dados[nome_modelo].get('test_mse', np.nan),
                'mae': dados[nome_modelo].get('test_mae', np.nan),
                'r2': dados[nome_modelo].get('test_r2', np.nan),
                'time': dados[nome_modelo].get('time', np.nan)
            }
    return resultados

def salvar_modelos_empresa(empresa, modelos_dict, metadados=None):
    try:
        nome_arquivo = f"{NOME_BEST_MODELS_NOVOS}_{empresa}"
        SalvarTreinamentoOnGitHub(
            nome_arquivo=nome_arquivo,
            dados=modelos_dict,
            metadados=metadados or {
                'descricao': f'Todos os modelos otimizados para {empresa}',
                'empresa': empresa,
                'modelos': list(modelos_dict.keys())
            }
        )
        return True
    except Exception as e:
        print(f"Erro ao salvar modelos para {empresa}: {str(e)[:80]}")
        return False

```

```
def carregar_modelos_empresa(empresa):
    try:
        nome_arquivo = f"{NOME_BEST_MODELS_NOVOS}_{empresa}"
        modelos = CarregarArquivoOnGitHub(nome_arquivo)
        return modelos
    except:
        return None
```

✓ Configuração da busca

```
resultados_novos_modelos = CarregarArquivoOnGitHub(NOME_NOVOS_MODELOS)
cache_valido = False
empresas_no_cache = []
if resultados_novos_modelos is not None and isinstance(resultados_novos_modelos, dict):
    empresas_no_cache = list(resultados_novos_modelos.keys())
    print(f"Empresas no cache de resultados: {len(empresas_no_cache)}")
    todas_validas = True
    for empresa in empresas_no_cache:
        if isinstance(resultados_novos_modelos[empresa], dict):
            if 'best_model' not in resultados_novos_modelos[empresa]:
                todas_validas = False
                print(f"{empresa}: 'best_model' não encontrado")
                break
        else:
            todas_validas = False
            print(f"{empresa}: dados não são um dicionário")
            break

if todas_validas and len(empresas_no_cache) > 0:
    cache_valido = True
    print(f"Cache de resultados válido com {len(empresas_no_cache)} empresas")
else:
    print("Cache de resultados inválido. Recriando...")
    resultados_novos_modelos = {}
    cache_valido = False
else:
    print("Nenhum cache de resultados encontrado. Será criado novo.")
    resultados_novos_modelos = {}

best_models_por_empresa = {}
if cache_valido:
    print("\nCarregando modelos por empresa...")
    for empresa in empresas_no_cache:
        modelos = carregar_modelos_empresa(empresa)
        if modelos is not None and isinstance(modelos, dict):
            best_models_por_empresa[empresa] = modelos
            print(f"{len(modelos)} modelos carregados para {empresa}")
        else:
            print(f"Nenhum modelo encontrado para {empresa}")

if len(best_models_por_empresa) > 0:
    print(f"{len(best_models_por_empresa)} empresas com modelos carregados")
else:
    print("Nenhum modelo encontrado no cache")

if not cache_valido or len(best_models_por_empresa) == 0:
    print("\nProcessando otimização dos novos modelos...")
    features_existentes = [f for f in features_selecionadas if f in df_analise.columns]
    empresas_validas = []
    for empresa in simbolos_usados:
        try:
            X_train, X_val, X_test, y_train, y_val, y_test = prepare_data(empresa, features_existentes)
            if len(X_train) > 0 and len(X_train) >= 50:
                empresas_validas.append(empresa)
                print(f"{empresa} - {len(X_train)} registros")
        except Exception as e:
            print(f"Erro ao preparar {empresa}: {e}")

    print(f"\nEmpresas válidas: {len(empresas_validas)}")

    MODELOS_PARA_PROCESSAR = ['SVR', 'KNN', 'DecisionTree', 'ElasticNet', 'Ridge', 'Lasso']

    print(f"Modelos a processar: {MODELOS_PARA_PROCESSAR}")

    for idx, empresa in enumerate(empresas_validas):
        print(f"\n{' '*60}")
        print(f"OTIMIZANDO PARA EMPRESA: {empresa} ({idx+1}/{len(empresas_validas)})")

    try:
```

```

# Preparar dados
X_train, X_val, X_test, y_train, y_val, y_test = prepare_data(empresa, features_existentes)

print(f"\n  DADOS - {empresa}:")
print(f"    • Treino: {len(X_train):,} registros x {X_train.shape[1]} features")
print(f"    • Validação: {len(X_val):,} registros")
print(f"    • Teste: {len(X_test):,} registros")

test_size = int(len(X_train) * 0.2)
if test_size == 0:
    test_size = 1
cv = TimeSeriesSplit(n_splits=N_FOLDS, test_size=test_size)

print(f"\n  TimeSeriesSplit:")
print(f"    • Número de folds: {N_FOLDS}")
print(f"    • Tamanho do teste: {test_size} registros")

resultados_empresa = {}
modelos_empresa = {}

for model_name, config in NEW_MODELS_OPTUNA.items():
    if model_name not in MODELOS_PARA_PROCESSAR:
        continue

    print(f"\nOtimizando {model_name}...", end=" ")

    try:
        # Para SVR, reduzir trials
        n_trials = N_TRIALS // 2 if model_name == 'SVR' else N_TRIALS

        if config['needs_scaling']:
            scaler = StandardScaler()
            X_train_scaled = scaler.fit_transform(X_train)
            study = optuna.create_study(
                direction='minimize',
                sampler=optuna.samplers.TPESampler(seed=SEED),
                study_name=f'{model_name.lower()}_{empresa}'
            )
            start_time = time.time()
            study.optimize(
                lambda trial: config['objective'](trial, X_train_scaled, y_train, cv),
                n_trials=n_trials,
                show_progress_bar=False
            )
            opt_time = time.time() - start_time

            X_train_full_scaled = scaler.fit_transform(pd.concat([X_train, X_val]))
            X_test_scaled = scaler.transform(X_test)
            if model_name == 'SVR':
                model = SVR(**study.best_params)
            elif model_name == 'KNN':
                model = KNeighborsRegressor(**study.best_params)
            elif model_name == 'ElasticNet':
                model = ElasticNet(**study.best_params, random_state=SEED)
            elif model_name == 'Ridge':
                model = Ridge(**study.best_params, random_state=SEED)
            elif model_name == 'Lasso':
                model = Lasso(**study.best_params, random_state=SEED)
            else:
                model = None

            if model:
                model.fit(X_train_full_scaled, pd.concat([y_train, y_val]))
                y_pred = model.predict(X_test_scaled)
                metrics = evaluate_model(y_test, y_pred)
                resultados_empresa[model_name] = {
                    'best_params': study.best_params,
                    'best_cv_score': study.best_value,
                    'test_mse': metrics['mse'],
                    'test_rmse': metrics['rmse'],
                    'test_mae': metrics['mae'],
                    'test_r2': metrics['r2'],
                    'time': opt_time
                }
                print(f"    MSE: {metrics['mse']:.6f} | R²: {metrics['r2']:.4f}")
                print(f"    Melhores parâmetros: {study.best_params}")
                modelos_empresa[model_name] = model
            else:
                study = optuna.create_study(
                    direction='minimize',
                    sampler=optuna.samplers.TPESampler(seed=SEED),
                    study_name=f'{model_name.lower()}_{empresa}'

```

```

    )
    start_time = time.time()
    study.optimize(
        lambda trial: config['objective'](trial, X_train, y_train, cv),
        n_trials=N_TRIALS,
        show_progress_bar=False
    )
    opt_time = time.time() - start_time

    if model_name == 'DecisionTree':
        model = DecisionTreeRegressor(**study.best_params, random_state=SEED)
        model.fit(pd.concat([X_train, X_val]), pd.concat([y_train, y_val]))
        y_pred = model.predict(X_test)
        metrics = evaluate_model(y_test, y_pred)

        resultados_empresa[model_name] = {
            'best_params': study.best_params,
            'best_cv_score': study.best_value,
            'test_mse': metrics['mse'],
            'test_rmse': metrics['rmse'],
            'test_mae': metrics['mae'],
            'test_r2': metrics['r2'],
            'time': opt_time
        }
        print(f"    MSE: {metrics['mse']:.6f} | R²: {metrics['r2']:.4f}")
        print(f"    Melhores parâmetros: {study.best_params}")

        # Armazenar modelo
        modelos_empresa[model_name] = model

except Exception as e:
    print(f"Erro: {str(e)[:80]}")
    resultados_empresa[model_name] = {
        'error': str(e),
        'test_mse': np.nan,
        'test_r2': np.nan,
        'time': 0
    }

# Limpar memória
gc.collect()

if resultados_empresa:
    # Filtrar apenas modelos com métricas válidas
    modelos_validos = {}
    for k, v in resultados_empresa.items():
        if isinstance(v, dict) and 'test_mse' in v:
            if isinstance(v['test_mse'], (int, float)) and not np.isnan(v['test_mse']) and v['test_mse'] < 1e9:
                modelos_validos[k] = v

    if modelos_validos:
        best_model_name = min(modelos_validos, key=lambda x: modelos_validos[x]['test_mse'])
        resultados_empresa['best_model'] = best_model_name
        resultados_empresa['best_mse'] = modelos_validos[best_model_name]['test_mse']
        resultados_empresa['best_r2'] = modelos_validos[best_model_name]['test_r2']

        print(f"\n MELHOR MODELO PARA {empresa}: {best_model_name}")
        print(f" • MSE: {resultados_empresa['best_mse']:.6f}")
        print(f" • R²: {resultados_empresa['best_r2']:.4f}")
    else:
        print(f"\nNenhum modelo válido para {empresa}")
        resultados_empresa['best_model'] = 'Nenhum'
        resultados_empresa['best_mse'] = np.nan
        resultados_empresa['best_r2'] = np.nan

if modelos_empresa:
    salvar_modelos_empresa(empresa, modelos_empresa, {
        'descricao': f'Todos os modelos otimizados para {empresa}',
        'empresa': empresa,
        'modelos': list(modelos_empresa.keys())
    })
    print(f"{len(modelos_empresa)} modelos salvos para {empresa}")

resultados_novos_modelos[empresa] = resultados_empresa
try:
    SalvarTreinamentoOnGitHub(
        nome_arquivo=NOME_NOVOS_MODELOS,
        dados=resultados_novos_modelos,
        metadados={
            'descricao': 'Resultados novos modelos por empresa',
            'empresas': list(resultados_novos_modelos.keys()),

```

```

        'total_empresas': len(resultados_novos_modelos)
    }
)
print(f"Checkpoint de resultados salvo ({len(resultados_novos_modelos)} empresas)")
except Exception as e:
    print(f"Erro ao salvar checkpoint: {str(e)[:50]}")

except Exception as e:
    print(f"Erro ao processar {empresa}: {str(e)}")
    traceback.print_exc()
    continue

# Limpar memória
gc.collect()
limpar_memoria(['X_train', 'X_val', 'X_test', 'y_train', 'y_val', 'y_test', 'scaler'], mostrar_status=False)

if len(resultados_novos_modelos) > 0:
    try:
        SalvarTreinamentoOnGitHub(
            nome_arquivo=NOME_NOVOS_MODELOS,
            dados=resultados_novos_modelos,
            metadados={
                'descricao': 'Resultados novos modelos por empresa (completo)',
                'empresas': list(resultados_novos_modelos.keys()),
                'total_empresas': len(resultados_novos_modelos)
            }
        )
        print("Resultados salvos com sucesso!")
    except Exception as e:
        print(f"Erro ao salvar resultados finais: {str(e)[:50]}")
    else:
        print("Nenhum resultado para salvar!")

if resultados_novos_modelos and len(resultados_novos_modelos) > 0:
    dados_resumo = []
    for empresa, dados in resultados_novos_modelos.items():
        if isinstance(dados, dict) and dados.get('best_model') not in ['Nenhum', None]:
            if 'best_mse' in dados and not np.isnan(dados['best_mse']):
                dados_resumo.append({
                    'Empresa': empresa,
                    'Melhor_Modelo': dados['best_model'],
                    'MSE': dados['best_mse'],
                    'R²': dados.get('best_r2', np.nan)
                })

    if dados_resumo:
        df_resumo = pd.DataFrame(dados_resumo).sort_values('MSE')

        print(f"\nRESULTADOS POR EMPRESA:")
        display(df_resumo.round(6))

        print(f"\n  ESTATÍSTICAS GERAIS:")
        print(f"    • Empresas processadas: {len(df_resumo)}")
        print(f"    • MSE médio: {df_resumo['MSE'].mean():.6f}")
        print(f"    • R² médio: {df_resumo['R²'].mean():.4f}")

        print(f"\nFREQÜÊNCIA DOS MODELOS:")
        freq_modelos = df_resumo['Melhor_Modelo'].value_counts()
        for modelo, count in freq_modelos.items():
            print(f"    • {modelo}: {count}/{len(df_resumo)} empresas ({count/len(df_resumo)*100:.1f}%)")
        else:
            print("Nenhum resultado válido para exibir")
    else:
        print("Nenhum resultado disponível")

# Limpeza final
limpar_memoria(mostrar_status=True)

```



```
resultados_novos_modelos.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 33,901 bytes  
Empresas no cache de resultados: 49  
Cache de resultados válido com 49 empresas  
  
Carregando modelos por empresa...  
best_models_novos_AAPL.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,662 bytes  
6 modelos carregados para AAPL  
best_models_novos_MSFT.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,661 bytes  
6 modelos carregados para MSFT  
best_models_novos_GOOG.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 264,668 bytes  
6 modelos carregados para GOOG  
best_models_novos_AMZN.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 262,458 bytes  
6 modelos carregados para AMZN  
best_models_novos_TSLA.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,661 bytes  
6 modelos carregados para TSLA  
best_models_novos_NVDA.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 228,181 bytes  
6 modelos carregados para NVDA  
best_models_novos_MRVL.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,661 bytes  
6 modelos carregados para MRVL  
best_models_novos_PEP.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,659 bytes  
6 modelos carregados para PEP  
best_models_novos_NFLX.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 225,749 bytes  
6 modelos carregados para NFLX  
best_models_novos_AMD.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 232,207 bytes  
6 modelos carregados para AMD  
best_models_novos_GOOGL.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 264,362 bytes  
6 modelos carregados para GOOG  
best_models_novos_AVGO.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 223,545 bytes  
6 modelos carregados para AVGO  
best_models_novos_META.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 279,038 bytes  
6 modelos carregados para META  
best_models_novos_MU.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 226,733 bytes  
6 modelos carregados para MU  
best_models_novos_WMT.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,659 bytes  
6 modelos carregados para WMT  
best_models_novos_COST.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 250,757 bytes  
6 modelos carregados para COST  
best_models_novos_ADBE.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 223,089 bytes  
6 modelos carregados para ADBE  
best_models_novos_CMCSA.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 221,419 bytes  
6 modelos carregados para CMCSA  
best_models_novos_CSCO.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,661 bytes  
6 modelos carregados para CSCO  
best_models_novos_INTC.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,806 bytes  
6 modelos carregados para INTC  
best_models_novos_QCOM.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 281,090 bytes
```



```
6 modelos carregados para QCOM
best_models_novos_TXN.pkl carregado com sucesso!
Tipo: dict
Tamanho: 221,491 bytes
6 modelos carregados para TXN
best_models_novos_AMGN.pkl carregado com sucesso!
Tipo: dict
Tamanho: 257,977 bytes
6 modelos carregados para AMGN
best_models_novos_ISRIG.pkl carregado com sucesso!
Tipo: dict
Tamanho: 222,253 bytes
6 modelos carregados para ISRIG
best_models_novos_TMUS.pkl carregado com sucesso!
Tipo: dict
Tamanho: 220,661 bytes
6 modelos carregados para TMUS
best_models_novos_HON.pkl carregado com sucesso!
Tipo: dict
Tamanho: 221,339 bytes
6 modelos carregados para HON
best_models_novos_BKNG.pkl carregado com sucesso!
Tipo: dict
Tamanho: 223,013 bytes
6 modelos carregados para BKNG
best_models_novos_AMAT.pkl carregado com sucesso!
Tipo: dict
Tamanho: 223,317 bytes
6 modelos carregados para AMAT
best_models_novos_LRCX.pkl carregado com sucesso!
Tipo: dict
Tamanho: 222,937 bytes
6 modelos carregados para LRCX
best_models_novos_TEAM.pkl carregado com sucesso!
Tipo: dict
Tamanho: 211,265 bytes
6 modelos carregados para TEAM
best_models_novos_GILD.pkl carregado com sucesso!
Tipo: dict
Tamanho: 220,661 bytes
6 modelos carregados para GILD
best_models_novos_CSX.pkl carregado com sucesso!
Tipo: dict
Tamanho: 262,915 bytes
6 modelos carregados para CSX
best_models_novos_ADI.pkl carregado com sucesso!
Tipo: dict
Tamanho: 220,659 bytes
6 modelos carregados para ADI
best_models_novos_VRTX.pkl carregado com sucesso!
Tipo: dict
Tamanho: 223,013 bytes
6 modelos carregados para VRTX
best_models_novos_ADP.pkl carregado com sucesso!
Tipo: dict
Tamanho: 252,123 bytes
6 modelos carregados para ADP
best_models_novos_REGN.pkl carregado com sucesso!
Tipo: dict
Tamanho: 222,329 bytes
6 modelos carregados para REGN
best_models_novos_KLAC.pkl carregado com sucesso!
Tipo: dict
Tamanho: 224,001 bytes
6 modelos carregados para KLAC
best_models_novos_MELI.pkl carregado com sucesso!
Tipo: dict
Tamanho: 228,562 bytes
6 modelos carregados para MELI
best_models_novos_MPWR.pkl carregado com sucesso!
Tipo: dict
Tamanho: 221,033 bytes
6 modelos carregados para MPWR
best_models_novos_PCAR.pkl carregado com sucesso!
Tipo: dict
Tamanho: 262,157 bytes
6 modelos carregados para PCAR
best_models_novos_SNPS.pkl carregado com sucesso!
Tipo: dict
Tamanho: 221,721 bytes
6 modelos carregados para SNPS
best_models_novos_KDP.pkl carregado com sucesso!
Tipo: dict
Tamanho: 246,879 bytes
6 modelos carregados para KDP
best_models_novos_CDNS.pkl carregado com sucesso!
Tipo: dict
Tamanho: 271,134 bytes
6 modelos carregados para CDNS
best_models_novos_MRNA.pkl carregado com sucesso!
Tipo: dict
```

```

Tipo: dict
Tamanho: 157,497 bytes
6 modelos carregados para MRNA
best_models_novos_ABNB.pkl carregado com sucesso!
Tipo: dict
Tamanho: 108,661 bytes
6 modelos carregados para ABNB
best_models_novos_WDAY.pkl carregado com sucesso!
Tipo: dict
Tamanho: 220,957 bytes
6 modelos carregados para WDAY
best_models_novos_CHTR.pkl carregado com sucesso!
Tipo: dict
Tamanho: 220,661 bytes
6 modelos carregados para CHTR
best_models_novos_LIN.pkl carregado com sucesso!
Tipo: dict
Tamanho: 221,111 bytes
6 modelos carregados para LIN
best_models_novos_KHC.pkl carregado com sucesso!
Tipo: dict
Tamanho: 244,703 bytes
6 modelos carregados para KHC
49 empresas com modelos carregados

```

RESULTADOS POR EMPRESA:

	Empresa	Melhor_Modelo	MSE	R ²
47	LIN	Ridge	0.000135	0.009942
15	COST	DecisionTree	0.000172	-0.002302
7	PEP	DecisionTree	0.000183	-0.001944
34	ADP	ElasticNet	0.000207	-0.003646
14	WMT	Ridge	0.000240	0.001854
31	CSX	DecisionTree	0.000241	-0.000371
25	HON	Ridge	0.000255	0.008414
48	KHC	DecisionTree	0.000259	-0.001138
41	KDP	DecisionTree	0.000260	-0.001616
30	GILD	Ridge	0.000290	0.003029
1	MSFT	DecisionTree	0.000291	-0.004830
24	TMUS	DecisionTree	0.000292	-0.003843
22	AMGN	ElasticNet	0.000296	0.003449
18	CSCO	DecisionTree	0.000321	-0.007159
39	PCAR	DecisionTree	0.000329	-0.000124
0	AAPL	DecisionTree	0.000331	-0.000421
17	CMCSA	Ridge	0.000339	0.001278
44	ABNB	ElasticNet	0.000339	0.001039
10	GOOG	DecisionTree	0.000385	-0.002060
2	GOOGL	DecisionTree	0.000395	-0.002251
26	BKNG	DecisionTree	0.000400	-0.000198
3	AMZN	ElasticNet	0.000432	-0.000529
33	VRTX	ElasticNet	0.000432	0.001470
23	ISRG	DecisionTree	0.000441	-0.003637
8	NFLX	Ridge	0.000479	0.000748
16	ADBE	KNN	0.000505	-0.011052
35	REGN	Ridge	0.000549	-0.003420
12	META	ElasticNet	0.000549	-0.000092
32	ADI	KNN	0.000569	0.019871
37	MELI	DecisionTree	0.000655	-0.004064
42	CDNS	Ridge	0.000661	0.010744
45	WDAY	DecisionTree	0.000702	-0.005054
21	TXN	DecisionTree	0.000709	-0.000269
5	NVDA	KNN	0.000799	0.000857
46	CHTR	DecisionTree	0.000832	-0.005417

```

NOME_NOVOS_MODELOS = "resultados_novos_modelos"
NOME_BEST_MODELS_NOVOS = "best_models_novos"
resultados_novos_modelos = CarregarArquivoOnGitHub(NOME_NOVOS_MODELOS)
print(f"Resultados carregados: {resultados_novos_modelos is not None}")
best_models_novos = None
try:
    print("Tentando carregar modelos individuais...")
    best_models_novos = {}
    for empresa in simbolos_usados:
        try:
            modelo = CarregarArquivoOnGitHub(f"{NOME_BEST_MODELS_NOVOS}_{empresa}")
            if modelo is not None:
                best_models_novos[empresa] = modelo
                print(f"Modelo carregado para {empresa}")
        except:
            pass

    if best_models_novos:
        print(f"{len(best_models_novos)} modelos carregados individualmente")
    else:
        best_models_novos = CarregarArquivoDivididoOnGitHub(NOME_BEST_MODELS_NOVOS)
        if best_models_novos is not None:
            print(f"Modelos carregados do arquivo completo: {len(best_models_novos)}")
        else:
            print("Nenhum modelo encontrado")

except Exception as e:
    print(f"Erro ao carregar modelos: {e}")
    best_models_novos = None

cache_valido = False
empresas_no_cache = []
if resultados_novos_modelos is not None and isinstance(resultados_novos_modelos, dict):
    empresas_no_cache = list(resultados_novos_modelos.keys())
    print(f"Empresas no cache de resultados: {len(empresas_no_cache)}")
    todas_validas = True
    for empresa in empresas_no_cache[:5]:
        if isinstance(resultados_novos_modelos[empresa], dict):
            if 'best_model' not in resultados_novos_modelos[empresa]:
                todas_validas = False
                print(f"{empresa}: 'best_model' não encontrado")
                break
        else:
            todas_validas = False
            print(f"{empresa}: dados não são um dicionário")
            break

    if todas_validas and len(empresas_no_cache) > 0:
        cache_valido = True
        print(f"Cache válido com {len(empresas_no_cache)} empresas")
    else:
        print("Cache de resultados inválido. Recriando...")
        resultados_novos_modelos = None
        best_models_novos = None
else:
    print("Nenhum resultado em cache")
    resultados_novos_modelos = None
    best_models_novos = None

if cache_valido and best_models_novos is not None:
    print(f"Modelos disponíveis: {len(best_models_novos)}")
    modelos_faltando = []
    for empresa in empresas_no_cache:
        if empresa not in best_models_novos:
            modelos_faltando.append(empresa)
    if modelos_faltando:
        print(f"Modelos faltando para: {modelos_faltando}...")
        for empresa in modelos_faltando:
            try:
                modelo = CarregarArquivoOnGitHub(f"{NOME_BEST_MODELS_NOVOS}_{empresa}")
                if modelo is not None:
                    best_models_novos[empresa] = modelo
                    print(f"Modelo carregado para {empresa}")
            except:
                pass

    if cache_valido and best_models_novos is not None and len(best_models_novos) > 0:
        print("\n Usando resultados e modelos do cache!")
        print(f" Empresas: {len(resultados_novos_modelos)}")
        print(f" Modelos: {len(best_models_novos)}")
    else:

```

```

print("\nProcessando otimização dos novos modelos...")
if cache_valido and resultados_novos_modelos is not None:
    # Processar apenas empresas que não têm resultados
    empresas_para_processar = [e for e in simbolos_usados if e not in empresas_no_cache]
    print(f"Empresas a processar (faltando): {len(empresas_para_processar)}")
else:
    empresas_para_processar = simbolos_usados
    print(f"Total de empresas: {len(empresas_para_processar)}")
    features_existentes = [f for f in features_selecionadas if f in df_analise.columns]
    empresas_validas = []
    for empresa in empresas_para_processar:
        try:
            X_train, X_val, X_test, y_train, y_val, y_test = prepare_data(empresa, features_existentes)
            if len(X_train) > 0 and len(X_train) >= 50:
                empresas_validas.append(empresa)
        except Exception as e:
            print(f"Erro ao preparar dados para {empresa}: {e}")
    if resultados_novos_modelos is None:
        resultados_novos_modelos = {}
    if best_models_novos is None:
        best_models_novos = {}

for idx, empresa in enumerate(empresas_validas):
    print(f"OTIMIZANDO PARA EMPRESA: {empresa} ({idx+1}/{len(empresas_validas)})")
    try:
        X_train, X_val, X_test, y_train, y_val, y_test = prepare_data(empresa, features_existentes)

        print(f"\n  DADOS - {empresa}:")
        print(f"    • Treino: {len(X_train):,} registros x {X_train.shape[1]} features")
        print(f"    • Validação: {len(X_val):,} registros")
        print(f"    • Teste: {len(X_test):,} registros")

        test_size = int(len(X_train) * 0.2)
        if test_size == 0:
            test_size = 1
        cv = TimeSeriesSplit(n_splits=N_FOLDS, test_size=test_size)

        print(f"\n  TimeSeriesSplit:")
        print(f"    • Número de folds: {N_FOLDS}")
        print(f"    • Tamanho do teste: {test_size} registros")

        resultados_empresa = {}

        for model_name, config in NEW_MODELS_OPTUNA.items():
            print(f"\nOtimizando {model_name}...", end=" ")

            try:
                if config['needs_scaling']:
                    scaler = StandardScaler()
                    X_train_scaled = scaler.fit_transform(X_train)
                    study = optuna.create_study(
                        direction='minimize',
                        sampler=optuna.samplers.TPESampler(seed=SEED),
                        study_name=f'{model_name.lower()}_{empresa}'
                    )
                    start_time = time.time()
                    study.optimize(
                        lambda trial: config['objective'](trial, X_train_scaled, y_train, cv),
                        n_trials=N_TRIALS,
                        show_progress_bar=False
                    )
                    opt_time = time.time() - start_time
                    X_train_full_scaled = scaler.fit_transform(pd.concat([X_train, X_val]))
                    X_test_scaled = scaler.transform(X_test)
                    if model_name == 'SVR':
                        model = SVR(**study.best_params)
                    elif model_name == 'KNN':
                        model = KNeighborsRegressor(**study.best_params)
                    elif model_name == 'ElasticNet':
                        model = ElasticNet(**study.best_params, random_state=SEED)
                    elif model_name == 'Ridge':
                        model = Ridge(**study.best_params, random_state=SEED)
                    elif model_name == 'Lasso':
                        model = Lasso(**study.best_params, random_state=SEED)
                    else:
                        model = None

                    if model:
                        model.fit(X_train_full_scaled, pd.concat([y_train, y_val]))
                        y_pred = model.predict(X_test_scaled)
                        metrics = evaluate_model(y_test, y_pred)
                        resultados_empresa[model_name] = {

```

```

        'best_params': study.best_params,
        'best_cv_score': study.best_value,
        'test_mse': metrics['mse'],
        'test_rmse': metrics['rmse'],
        'test_mae': metrics['mae'],
        'test_r2': metrics['r2'],
        'time': opt_time
    }
    print(f"    MSE: {metrics['mse']:.6f} | R²: {metrics['r2']:.4f}")
    print(f"    Melhores parâmetros: {study.best_params}")

    SalvarTreinamentoOnGitHub(
        nome_arquivo=f"{NOME_BEST_MODELS_NOVOS}_{empresa}",
        dados=model,
        metadados={
            'descricao': f'Modelo {model_name} para {empresa}',
            'empresa': empresa,
            'modelo': model_name,
            'metrics': metrics
        }
    )
    print(f"Modelo salvo como {NOME_BEST_MODELS_NOVOS}_{empresa}")
else:
    # Modelos sem scaling (DecisionTree)
    study = optuna.create_study(
        direction='minimize',
        sampler=optuna.samplers.TPESampler(seed=SEED),
        study_name=f'{model_name.lower()}_{empresa}'
    )
    start_time = time.time()
    study.optimize(
        lambda trial: config['objective'](trial, X_train, y_train, cv),
        n_trials=N_TRIALS,
        show_progress_bar=False
    )
    opt_time = time.time() - start_time
    if model_name == 'DecisionTree':
        model = DecisionTreeRegressor(**study.best_params, random_state=SEED)
        model.fit(pd.concat([X_train, X_val]), pd.concat([y_train, y_val]))
        y_pred = model.predict(X_test)
        metrics = evaluate_model(y_test, y_pred)
        resultados_empresa[model_name] = {
            'best_params': study.best_params,
            'best_cv_score': study.best_value,
            'test_mse': metrics['mse'],
            'test_rmse': metrics['rmse'],
            'test_mae': metrics['mae'],
            'test_r2': metrics['r2'],
            'time': opt_time
        }
        print(f"    MSE: {metrics['mse']:.6f} | R²: {metrics['r2']:.4f}")
        print(f"    Melhores parâmetros: {study.best_params}")

    # Salvar modelo individualmente
    SalvarTreinamentoOnGitHub(
        nome_arquivo=f"{NOME_BEST_MODELS_NOVOS}_{empresa}",
        dados=model,
        metadados={
            'descricao': f'Modelo DecisionTree para {empresa}',
            'empresa': empresa,
            'modelo': 'DecisionTree',
            'metrics': metrics
        }
    )
    print(f"Modelo salvo como {NOME_BEST_MODELS_NOVOS}_{empresa}")

except Exception as e:
    print(f"Erro: {str(e)[:80]}")
    resultados_empresa[model_name] = {
        'error': str(e),
        'test_mse': 1e10,
        'test_r2': -1000,
        'time': 0
    }

# Limpar memória
gc.collect()

# Determinar melhor modelo
if resultados_empresa:
    modelos_validos = {k: v for k, v in resultados_empresa.items()
                       if 'test_mse' in v and v['test_mse'] < 1e9}

```

```

        if modelos_validos:
            best_model_name = min(modelos_validos, key=lambda x: modelos_validos[x]['test_mse'])
            resultados_empresa['best_model'] = best_model_name
            resultados_empresa['best_mse'] = modelos_validos[best_model_name]['test_mse']
            resultados_empresa['best_r2'] = modelos_validos[best_model_name]['test_r2']

            print(f"\n MELHOR MODELO PARA {empresa}: {best_model_name}")
            print(f" • MSE: {resultados_empresa['best_mse']:.6f}")
            print(f" • R²: {resultados_empresa['best_r2']:.4f}")
        else:
            print(f"\n Nenhum modelo válido para {empresa}")
            resultados_empresa['best_model'] = 'Nenhum'
            resultados_empresa['best_mse'] = np.nan
            resultados_empresa['best_r2'] = np.nan

        resultados_novos_modelos[empresa] = resultados_empresa

        # Salvar checkpoint dos resultados
        SalvarTreinamentoOnGitHub(
            nome_arquivo=NOME_NOVOS_MODELOS,
            dados=resultados_novos_modelos,
            metadados={
                'descricao': 'Resultados novos modelos por empresa',
                'empresas': list(resultados_novos_modelos.keys()),
                'total_empresas': len(resultados_novos_modelos)
            }
        )

    except Exception as e:
        print(f"Erro ao processar {empresa}: {str(e)}")
        traceback.print_exc()
        continue

    # Limpar memória
    gc.collect()
    limpar_memoria(['X_train', 'X_val', 'X_test', 'y_train', 'y_val',
                    'y_test', 'scaler'], mostrar_status=False)

    print("\nSalvando resultados finais...")
    if len(resultados_novos_modelos) > 0:
        SalvarTreinamentoOnGitHub(
            nome_arquivo=NOME_NOVOS_MODELOS,
            dados=resultados_novos_modelos,
            metadados={
                'descricao': 'Resultados novos modelos por empresa (completo)',
                'empresas': list(resultados_novos_modelos.keys()),
                'total_empresas': len(resultados_novos_modelos)
            }
        )
        print("Resultados salvos com sucesso!")
    else:
        print("Nenhum resultado para salvar!")

if resultados_novos_modelos and len(resultados_novos_modelos) > 0:
    dados_resumo = []
    for empresa, dados in resultados_novos_modelos.items():
        if 'best_model' in dados and dados['best_model'] != 'Nenhum':
            dados_resumo.append({
                'Empresa': empresa,
                'Melhor_Modelo': dados['best_model'],
                'MSE': dados['best_mse'],
                'R²': dados['best_r2']
            })

    if dados_resumo:
        df_resumo = pd.DataFrame(dados_resumo).sort_values('MSE')
        print(f"\nRESULTADOS POR EMPRESA:")
        display(df_resumo.round(6))
        print(f"\nESTATÍSTICAS GERAIS:")
        print(f" • Empresas processadas: {len(df_resumo)}")
        print(f" • MSE médio: {df_resumo['MSE'].mean():.6f}")
        print(f" • R² médio: {df_resumo['R²'].mean():.4f}")
        print(f"\n FREQUÊNCIA DOS MODELOS:")
        freq_modelos = df_resumo['Melhor_Modelo'].value_counts()
        for modelo, count in freq_modelos.items():
            print(f" • {modelo}: {count}/{len(df_resumo)} empresas ({count/len(df_resumo)*100:.1f}%)")
    else:
        print("Nenhum resultado válido para exibir")
    else:
        print("Nenhum resultado disponível")

limpar_memoria(mostrar_status=True)

```



```
resultados_novos_modelos.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 33,901 bytes  
Resultados carregados: True  
Tentando carregar modelos individuais...  
best_models_novos_AAPL.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,662 bytes  
Modelo carregado para AAPL  
best_models_novos_MSFT.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,661 bytes  
Modelo carregado para MSFT  
best_models_novos_GOOGL.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 264,668 bytes  
Modelo carregado para GOOGL  
best_models_novos_AMZN.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 262,458 bytes  
Modelo carregado para AMZN  
best_models_novos_TSLA.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,661 bytes  
Modelo carregado para TSLA  
best_models_novos_NVDA.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 228,181 bytes  
Modelo carregado para NVDA  
best_models_novos_MRVL.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,661 bytes  
Modelo carregado para MRVL  
best_models_novos_PEP.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,659 bytes  
Modelo carregado para PEP  
best_models_novos_NFLX.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 225,749 bytes  
Modelo carregado para NFLX  
best_models_novos_AMD.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 232,207 bytes  
Modelo carregado para AMD  
best_models_novos_GOOG.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 264,362 bytes  
Modelo carregado para GOOG  
best_models_novos_AVGO.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 223,545 bytes  
Modelo carregado para AVGO  
best_models_novos_META.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 279,038 bytes  
Modelo carregado para META  
best_models_novos_MU.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 226,733 bytes  
Modelo carregado para MU  
best_models_novos_WMT.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,659 bytes  
Modelo carregado para WMT  
best_models_novos_COST.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 250,757 bytes  
Modelo carregado para COST  
best_models_novos_ADBE.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 223,089 bytes  
Modelo carregado para ADBE  
best_models_novos_CMCSA.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 221,419 bytes  
Modelo carregado para CMCSA  
best_models_novos_CSCO.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,661 bytes  
Modelo carregado para CSCO  
best_models_novos_INTC.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 220,806 bytes  
Modelo carregado para INTC  
best_models_novos_QCOM.pkl carregado com sucesso!  
Tipo: dict  
Tamanho: 281,090 bytes  
Modelo carregado para QCOM  
best_models_novos_TXN.pkl carregado com sucesso!
```



```
Tipo: dict
Tamanho: 221,491 bytes
Modelo carregado para TXN
best_models_novos_AMGN.pkl carregado com sucesso!
Tipo: dict
Tamanho: 257,977 bytes
Modelo carregado para AMGN
best_models_novos_ISR6.pkl carregado com sucesso!
Tipo: dict
Tamanho: 222,253 bytes
Modelo carregado para ISR6
best_models_novos_TMUS.pkl carregado com sucesso!
Tipo: dict
Tamanho: 220,661 bytes
Modelo carregado para TMUS
best_models_novos_HON.pkl carregado com sucesso!
Tipo: dict
Tamanho: 221,339 bytes
Modelo carregado para HON
best_models_novos_BKNG.pkl carregado com sucesso!
Tipo: dict
Tamanho: 223,013 bytes
Modelo carregado para BKNG
best_models_novos_AMAT.pkl carregado com sucesso!
Tipo: dict
Tamanho: 223,317 bytes
Modelo carregado para AMAT
best_models_novos_LRCX.pkl carregado com sucesso!
Tipo: dict
Tamanho: 222,937 bytes
Modelo carregado para LRCX
best_models_novos_TEAM.pkl carregado com sucesso!
Tipo: dict
Tamanho: 211,265 bytes
Modelo carregado para TEAM
best_models_novos_GILD.pkl carregado com sucesso!
Tipo: dict
Tamanho: 220,661 bytes
Modelo carregado para GILD
best_models_novos_CSX.pkl carregado com sucesso!
Tipo: dict
Tamanho: 262,915 bytes
Modelo carregado para CSX
best_models_novos_ADI.pkl carregado com sucesso!
Tipo: dict
Tamanho: 220,659 bytes
Modelo carregado para ADI
best_models_novos_VRTX.pkl carregado com sucesso!
Tipo: dict
Tamanho: 223,013 bytes
Modelo carregado para VRTX
best_models_novos_ADP.pkl carregado com sucesso!
Tipo: dict
Tamanho: 252,123 bytes
Modelo carregado para ADP
best_models_novos_REGN.pkl carregado com sucesso!
Tipo: dict
Tamanho: 222,329 bytes
Modelo carregado para REGN
best_models_novos_KLAC.pkl carregado com sucesso!
Tipo: dict
Tamanho: 224,001 bytes
Modelo carregado para KLAC
best_models_novos_MELI.pkl carregado com sucesso!
Tipo: dict
Tamanho: 228,562 bytes
Modelo carregado para MELI
best_models_novos_MPWR.pkl carregado com sucesso!
Tipo: dict
Tamanho: 221,033 bytes
Modelo carregado para MPWR
best_models_novos_PCAR.pkl carregado com sucesso!
Tipo: dict
Tamanho: 262,157 bytes
Modelo carregado para PCAR
best_models_novos_SNPS.pkl carregado com sucesso!
Tipo: dict
Tamanho: 221,721 bytes
Modelo carregado para SNPS
best_models_novos_KDP.pkl carregado com sucesso!
Tipo: dict
Tamanho: 246,879 bytes
Modelo carregado para KDP
best_models_novos_CDNS.pkl carregado com sucesso!
Tipo: dict
Tamanho: 271,134 bytes
Modelo carregado para CDNS
best_models_novos_MRNA.pkl carregado com sucesso!
Tipo: dict
Tamanho: 157,497 bytes
Modelo carregado para MRNA
```

```

Modelo carregado para HON
best_models_novos_ABNB.pkl carregado com sucesso!
Tipo: dict
Tamanho: 108,661 bytes
Modelo carregado para ABNB
best_models_novos_WDAY.pkl carregado com sucesso!
Tipo: dict
Tamanho: 220,957 bytes
Modelo carregado para WDAY
best_models_novos_CHTR.pkl carregado com sucesso!
Tipo: dict
Tamanho: 220,661 bytes
Modelo carregado para CHTR
best_models_novos_LIN.pkl carregado com sucesso!
Tipo: dict
Tamanho: 221,111 bytes
Modelo carregado para LIN
best_models_novos_KHC.pkl carregado com sucesso!
Tipo: dict
Tamanho: 244,703 bytes
Modelo carregado para KHC
49 modelos carregados individualmente
Empresas no cache de resultados: 49
Cache válido com 49 empresas
Modelos disponíveis: 49

```

```

Usando resultados e modelos do cache!
Empresas: 49
Modelos: 49

```

RESULTADOS POR EMPRESA:

	Empresa	Melhor_Modelo	MSE	R ²
47	LIN	Ridge	0.000135	0.009942
15	COST	DecisionTree	0.000172	-0.002302
7	PEP	DecisionTree	0.000183	-0.001944
34	ADP	ElasticNet	0.000207	-0.003646
14	WMT	Ridge	0.000240	0.001854
31	CSX	DecisionTree	0.000241	-0.000371
25	HON	Ridge	0.000255	0.008414
48	KHC	DecisionTree	0.000259	-0.001138
41	KDP	DecisionTree	0.000260	-0.001616
30	GILD	Ridge	0.000290	0.003029
1	MSFT	DecisionTree	0.000291	-0.004830
24	TMUS	DecisionTree	0.000292	-0.003843
22	AMGN	ElasticNet	0.000296	0.003449
18	CSCO	DecisionTree	0.000321	-0.007159
39	PCAR	DecisionTree	0.000329	-0.000124
0	AAPL	DecisionTree	0.000331	-0.000421
17	CMCSA	Ridge	0.000339	0.001278
44	ABNB	ElasticNet	0.000339	0.001039
10	GOOG	DecisionTree	0.000385	-0.002060
2	GOOGL	DecisionTree	0.000395	-0.002251
26	BKNG	DecisionTree	0.000400	-0.000198
3	AMZN	ElasticNet	0.000432	-0.000529
33	VRTX	ElasticNet	0.000432	0.001470
23	ISRG	DecisionTree	0.000441	-0.003637
8	NFLX	Ridge	0.000479	0.000748
16	ADBE	KNN	0.000505	-0.011052
35	REGN	Ridge	0.000549	-0.003420
12	META	ElasticNet	0.000549	-0.000092
32	ADI	KNN	0.000569	0.019871
37	MELI	DecisionTree	0.000655	-0.004064
42	CDNS	Ridge	0.000661	0.010744
45	WDAY	DecisionTree	0.000702	-0.005054

```

21      TXN      DecisionTree  0.000709  -0.000269
5      N/A      KNN          0.000799   0.000857
46      CTR      DecisionTree  0.000832   0.005417

```

Comparação dos novos modelos

```

if resultados_novos_modelos and len(resultados_novos_modelos) > 0:
    modelos_novos = list(NEW_MODELS_OPTUNA.keys())
    comparacao_novos = {}
    for modelo in modelos_novos:
        mse_list = []
        r2_list = []
        time_list = []
        for empresa in resultados_novos_modelos:
            if modelo in resultados_novos_modelos[empresa]:
                mse_val = resultados_novos_modelos[empresa][modelo]['test_mse']
                if not np.isnan(mse_val) and mse_val < 1e9:
                    mse_list.append(mse_val)
                    r2_list.append(resultados_novos_modelos[empresa][modelo]['test_r2'])
                    time_list.append(resultados_novos_modelos[empresa][modelo]['time'])

        comparacao_novos[modelo] = {
            'mse_mean': np.mean(mse_list) if mse_list else np.nan,
            'mse_std': np.std(mse_list) if mse_list else np.nan,
            'r2_mean': np.mean(r2_list) if r2_list else np.nan,
            'r2_std': np.std(r2_list) if r2_list else np.nan,
            'time_mean': np.mean(time_list) if time_list else np.nan,
            'time_std': np.std(time_list) if time_list else np.nan
        }

    df_novos = pd.DataFrame(comparacao_novos).T
    df_novos = df_novos[df_novos['mse_mean'] < 1e9]
    if not df_novos.empty:
        df_novos = df_novos.sort_values('mse_mean')
        print("\nRESULTADOS DOS NOVOS MODELOS:")
        display(df_novos.round(6))
        if not df_novos.empty and 'mse_mean' in df_novos.columns:
            best_model = df_novos['mse_mean'].idxmin()
            best_mse = df_novos.loc[best_model, 'mse_mean']
            print(f"\n    MELHOR NOVO MODELO: {best_model}")
            print(f"        • MSE médio: {best_mse:.6f}")
            if 'r2_mean' in df_novos.columns:
                print(f"        • R² médio: {df_novos.loc[best_model, 'r2_mean']:.4f}")
        else:
            print("Nenhum modelo novo com resultados válidos")
    else:
        print("Nenhum resultado novo disponível")

```

RESULTADOS DOS NOVOS MODELOS:

	mse_mean	mse_std	r2_mean	r2_std	time_mean	time_std
DecisionTree	0.000699	0.000531	-0.003165	0.003801	0.667934	0.081385
Lasso	0.000699	0.000531	-0.003040	0.004082	0.232084	0.028757
ElasticNet	0.000699	0.000531	-0.003051	0.004567	0.254560	0.039159
KNN	0.000720	0.000547	-0.035781	0.031262	0.804096	0.121275
Ridge	0.000791	0.000708	-0.107378	0.419882	0.212659	0.029497
SVR	0.001776	0.004483	-1.050173	3.848253	0.600633	0.179308

MELHOR NOVO MODELO: DecisionTree
 • MSE médio: 0.000699
 • R² médio: -0.0032

Comparativo entre os novos modelos e os antigos

```

resultados_antigos = None
if 'resultados_por_empresa' in globals() and resultados_por_empresa is not None:
    resultados_antigos = resultados_por_empresa
else:
    print("Carregando resultados antigos do cache...")
    resultados_antigos = CarregarArquivoOnGitHub("resultados_baseline")

if resultados_antigos is None or resultados_novos_modelos is None:
    print("Dados insuficientes para comparação. Execute as seções anteriores primeiro.")
else:
    print(f"Resultados antigos: {len(resultados_antigos)} empresas")
    print(f"Resultados novos: {len(resultados_novos_modelos)} empresas")
    empresas_comuns = set(resultados_antigos.keys()) & set(resultados_novos_modelos.keys())

```

```

print(f"Empresas comuns para comparação: {len(empresas_comuns)}")
if len(empresas_comuns) > 0:
    comparacao_por_empresa = []
    for empresa in sorted(empresas_comuns):
        antigos = extrair_resultados_antigos(empresa, resultados_antigos)
        novos = extrair_resultados_novos(empresa, resultados_novos_modelos)
        best_antigo = None
        best_antigo_mse = float('inf')
        for nome, dados in antigos.items():
            if not np.isnan(dados['mse']) and dados['mse'] < best_antigo_mse:
                best_antigo_mse = dados['mse']
                best_antigo = nome
        best_novo = None
        best_novo_mse = float('inf')
        for nome, dados in novos.items():
            if not np.isnan(dados['mse']) and dados['mse'] < best_novo_mse:
                best_novo_mse = dados['mse']
                best_novo = nome
        if best_antigo and best_novo and best_antigo_mse < float('inf') and best_novo_mse < float('inf'):
            melhora = (best_antigo_mse - best_novo_mse) / best_antigo_mse * 100
        else:
            melhora = np.nan
        comparacao_por_empresa.append({
            'Empresa': empresa,
            'Melhor_Antigo': best_antigo,
            'MSE_Antigo': best_antigo_mse,
            'Melhor_Novo': best_novo,
            'MSE_Novo': best_novo_mse,
            'Melhora_%': melhora
        })

df_comparacao = pd.DataFrame(comparacao_por_empresa)
df_comparacao = df_comparacao.sort_values('Melhora_%', ascending=False)
print("\nCOMPARAÇÃO POR EMPRESA (ordenado por melhora):")
display(df_comparacao.round(6))

print("\nESTATÍSTICAS DA COMPARAÇÃO:")
print(f"    Total de empresas comparadas: {len(df_comparacao)}")
melhora_media = df_comparacao['Melhora_%'].mean()
melhora_mediana = df_comparacao['Melhora_%'].median()

print(f"    Melhora média: {melhora_media:.2f}%")
print(f"    Melhora mediana: {melhora_mediana:.2f}%")
melhores_antigos = df_comparacao['Melhor_Antigo'].value_counts()
melhores_novos = df_comparacao['Melhor_Novo'].value_counts()

print(f"\nMELHORES MODELOS ANTIGOS:")
for modelo, count in melhores_antigos.head(10).items():
    print(f"    • {modelo}: {count} empresas")
print(f"\nMELHORES MODELOS NOVOS:")
for modelo, count in melhores_novos.head(10).items():
    print(f"    • {modelo}: {count} empresas")

fig, axes = plt.subplots(2, 2, figsize=(16, 12))
# Gráfico 1: MSE - Antigo vs Novo
ax = axes[0, 0]
df_plot = df_comparacao.dropna(subset=['MSE_Antigo', 'MSE_Novo'])
if not df_plot.empty:
    x = np.arange(len(df_plot))
    width = 0.35
    ax.bar(x - width/2, df_plot['MSE_Antigo'], width, label='Modelo Antigo', color='steelblue', alpha=0.7)
    ax.bar(x + width/2, df_plot['MSE_Novo'], width, label='Modelo Novo', color='coral', alpha=0.7)
    ax.set_xlabel('Empresa')
    ax.set_ylabel('MSE')
    ax.set_title('Comparação de MSE: Antigo vs Novo')
    ax.set_xticks(x)
    ax.set_xticklabels(df_plot['Empresa'], rotation=45, ha='right')
    ax.legend()
    ax.grid(True, alpha=0.3)

# Gráfico 2: Melhora percentual
ax = axes[0, 1]
df_plot = df_comparacao.dropna(subset=['Melhora_%'])
if not df_plot.empty:
    colors = ['green' if m > 0 else 'red' for m in df_plot['Melhora_%']]
    ax.barh(df_plot['Empresa'], df_plot['Melhora_%'], color=colors, alpha=0.7)
    ax.axvline(0, color='black', linestyle='--', alpha=0.5)
    ax.set_xlabel('Melhora (%)')
    ax.set_title('Melhora Percentual do Novo Modelo')
    ax.grid(True, alpha=0.3)

# Gráfico 3: Distribuição dos melhores modelos

```

```

ax = axes[1, 0]
if not melhores_antigos.empty and not melhores_novos.empty:
    modelos = list(set(melhores_antigos.index) | set(melhores_novos.index))
    count_antigo = [melhores_antigos.get(m, 0) for m in modelos]
    count_novo = [melhores_novos.get(m, 0) for m in modelos]
    x = np.arange(len(modelos))
    width = 0.35
    ax.bar(x - width/2, count_antigo, width, label='Modelos Antigos', color='steelblue', alpha=0.7)
    ax.bar(x + width/2, count_novo, width, label='Modelos Novos', color='coral', alpha=0.7)
    ax.set_xlabel('Modelo')
    ax.set_ylabel('Número de Empresas')
    ax.set_title('Distribuição dos Melhores Modelos')
    ax.set_xticks(x)
    ax.set_xticklabels(modelos, rotation=45, ha='right')
    ax.legend()
    ax.grid(True, alpha=0.3)

# Gráfico 4: Boxplot comparativo
ax = axes[1, 1]
dados_antigos = []
dados_novos = []
labels = []
for empresa in df_comparacao['Empresa'].head(10):
    antigos = extrair_resultados_antigos(empresa, resultados_antigos)
    novos = extrair_resultados_novos(empresa, resultados_novos_modelos)
    mse_antigos = [d['mse'] for d in antigos.values() if not np.isnan(d['mse'])]
    mse_novos = [d['mse'] for d in novos.values() if not np.isnan(d['mse'])]
    if mse_antigos and mse_novos:
        dados_antigos.extend(mse_antigos)
        dados_novos.extend(mse_novos)
        labels.append(f'{empresa} (A)')
        labels.append(f'{empresa} (N)')

if dados_antigos and dados_novos:
    for i, empresa in enumerate(df_comparacao['Empresa'].head(10)):
        antigos = extrair_resultados_antigos(empresa, resultados_antigos)
        novos = extrair_resultados_novos(empresa, resultados_novos_modelos)
        mse_antigos = [d['mse'] for d in antigos.values() if not np.isnan(d['mse'])]
        mse_novos = [d['mse'] for d in novos.values() if not np.isnan(d['mse'])]
        if mse_antigos and mse_novos:
            pos = i * 2
            bp1 = ax.boxplot(mse_antigos, positions=[pos], widths=0.6, patch_artist=True,
                             boxprops=dict(facecolor='steelblue', alpha=0.7),
                             labels=[f'{empresa}\nAntigo'])
            bp2 = ax.boxplot(mse_novos, positions=[pos+1], widths=0.6, patch_artist=True,
                             boxprops=dict(facecolor='coral', alpha=0.7),
                             labels=[f'{empresa}\nNovo'])
        ax.set_ylabel('MSE')
        ax.set_title('Distribuição do MSE: Antigo vs Novo (Top 10)')
        ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

mse_antigo_mean = df_comparacao['MSE_Antigo'].mean()
mse_novo_mean = df_comparacao['MSE_Novo'].mean()
melhora_media = df_comparacao['Melhora_%'].mean()

print(f"\n DESEMPENHO MÉDIO:")
print(f" • MSE Médio (Modelos Antigos): {mse_antigo_mean:.6f}")
print(f" • MSE Médio (Modelos Novos): {mse_novo_mean:.6f}")
print(f" • Melhora Média: {melhora_media:.2f}%")
print(f"\n TOP 10 EMPRESAS COM MAIOR MELHORA:")
display(df_comparacao.head(10)[['Empresa', 'Melhor_Antigo', 'Melhor_Novo', 'Melhora_%']].round(2))

print(f"\n TOP 10 EMPRESAS COM MENOR MELHORA:")
display(df_comparacao.tail(10)[['Empresa', 'Melhor_Antigo', 'Melhor_Novo', 'Melhora_%']].round(2))

print(f"\n MELHOR MODELO GERAL:")
print(f" • Modelos Antigos: {melhores_antigos.index[0] if not melhores_antigos.empty else 'N/A'} ({melhores_antigo}")
print(f" • Modelos Novos: {melhores_novos.index[0] if not melhores_novos.empty else 'N/A'} ({melhores_novos.il

# Conclusão
print(f"\n CONCLUSÃO:")
if melhora_media > 0:
    print(f" Os novos modelos apresentaram uma melhora média de {melhora_media:.2f}% em relação aos modelos antig
    if mse_novo_mean < mse_antigo_mean:
        print(f" O MSE médio reduziu de {mse_antigo_mean:.6f} para {mse_novo_mean:.6f}.")
else:
    print(f" Os modelos antigos ainda apresentam desempenho superior aos novos modelos.")
    print(f" Considere revisar a seleção de features ou os hiperparâmetros dos novos modelos.")

```

